

PartsIQ implementation guide

Single workflow architecture with Snowflake

Complete 69 feature scope
97 detailed implementation prompts
77 REST operations with typed contracts
16 October 2026

This guide explains what to build,how the components connect,and how to verify the result. It replaces the earlier multi-agent architecture while preserving the complete product scope. The accompanying source specifications and prompts are the implementation inputs.

Prepared for Mohammed Zuhair Hussain. This is a build kit;application implementation,live provider testing and deployment still require execution of its prompts.

Contents

001 README.md 5
002 AGENTS.md 6
003 CLAUDE.md 7
004 MASTER_PROMPT.md 8
005 docs/CONTEXT.md 9
006 docs/PRD.md 10
007 docs/ARCHITECTURE.md 19
008 docs/DATA_OWNERSHIP.md 21
009 docs/WORKFLOW.md 23
010 docs/REST_API.md 25
011 docs/API_EXAMPLES.md 29
012 docs/CONTRACTS.md 30
013 docs/SNOWFLAKE.md 31
014 docs/IMPLEMENTATION_RECIPES.md 33
015 docs/SECURITY.md 35
016 docs/CONSTRAINTS.md 36
017 docs/DECISIONS.md 37
018 docs/DISCREPANCIES.md 38
019 docs/DEPENDENCIES.md 39
020 docs/ENVIRONMENT.md 43
021 docs/TECH_SPEC.md 45
022 docs/UI_SPEC.md 73
023 docs/DEPLOYMENT.md 74
024 docs/ROLLBACK.md 76
025 docs/TEST_CHECKLIST.md 77
026 docs/FIELD_GUIDE_MAP.md 78
027 docs/FEATURE_MATRIX.md 79
028 docs/HANDOVER.md 82
029 docs/SESSION_LOG.md 83
030 docs/RECRUITER_GUIDE.md 84
031 docs/DEMO_SCRIPT.md 85
032 docs/SELF_AUDIT.md 86
033 brand/BRAND.md 87
034 prompts/README.md 88
035 docs/FLOW.md 91
036 docs/REQUIREMENTS.md 93
037 docs/templates/FEATURE_TEMPLATE.md 102
038 docs/templates/PROMPT_TEMPLATE.md 103
039 prompts/00-bootstrap.md 104
040 prompts/01-core.md 107
041 prompts/02-snowflake.md 110
042 prompts/03-database.md 113
043 prompts/04-storage.md 116
044 prompts/05-fixtures.md 119
045 prompts/06-search.md 122
046 prompts/07-auth.md 125
047 prompts/08-intake.md 128
048 prompts/09-workflow.md 131
049 prompts/10-extraction.md 134
050 prompts/11-resolution.md 137
051 prompts/12-verification.md 140
052 prompts/13-drafts.md 143
053 prompts/14-approval.md 146
054 prompts/15-jobs.md 149
055 prompts/16-vision.md 152

056 prompts/17-exports.md 155
057 prompts/18-web.md 158
058 prompts/19-customer_web.md 163
059 prompts/20-staff_web.md 166
060 prompts/21-evaluation.md 169
061 prompts/23-analytics.md 172
062 prompts/24-containers.md 175
063 prompts/25-ci.md 178
064 prompts/F01-branded-landing.md 181
065 prompts/F02-demo-scenarios-and-assets.md 185
066 prompts/F03-demo-role-switching.md 189
067 prompts/F04-demo-isolation.md 193
068 prompts/F05-demo-reset.md 197
069 prompts/F06-responsive-layouts.md 201
070 prompts/F07-arabic-and-english-interface.md 205
071 prompts/F08-light-and-dark-themes.md 209
072 prompts/F09-bilingual-chat.md 213
073 prompts/F10-structured-extraction.md 217
074 prompts/F11-clarifying-questions.md 221
075 prompts/F12-conversation-memory.md 225
076 prompts/F13-private-uploads.md 229
077 prompts/F14-nameplate-ocr.md 233
078 prompts/F15-ocr-confirmation.md 237
079 prompts/F16-photo-quality-feedback.md 241
080 prompts/F17-multi-part-enquiries.md 245
081 prompts/F18-progress-timeline.md 249
082 prompts/F19-enquiry-history.md 253
083 prompts/F20-machine-identification.md 257
084 prompts/F21-machine-mismatch-detection.md 261
085 prompts/F22-compatibility-gate.md 265
086 prompts/F23-aliases-and-typo-matching.md 269
087 prompts/F24-semantic-retrieval.md 273
088 prompts/F25-ranked-candidates.md 277
089 prompts/F26-ambiguity-handling.md 281
090 prompts/F27-match-explanations.md 285
091 prompts/F28-compatibility-references.md 289
092 prompts/F29-evidence-page-viewer.md 293
093 prompts/F30-old-part-numbers.md 297
094 prompts/F31-candidate-comparison.md 301
095 prompts/F32-warehouse-stock.md 305
096 prompts/F33-fulfilment-planning.md 309
097 prompts/F34-transfer-estimates.md 313
098 prompts/F35-stock-freshness.md 317
099 prompts/F36-staff-inbox.md 321
100 prompts/F37-review-workspace.md 325
101 prompts/F38-correct-extracted-details.md 329
102 prompts/F39-review-actions.md 333
103 prompts/F40-approval-invalidation.md 337
104 prompts/F41-internal-notes.md 341
105 prompts/F42-assignments.md 345
106 prompts/F43-overdue-indicators.md 349
107 prompts/F44-conversation-summary.md 353
108 prompts/F45-reply-drafting.md 357
109 prompts/F46-verification-gate.md 361
110 prompts/F47-human-publication-gate.md 365
111 prompts/F48-input-isolation.md 369
112 prompts/F49-audit-trail.md 373

113 prompts/F50-durable-workflows.md 377
114 prompts/F51-background-jobs.md 381
115 prompts/F52-retry-and-deduplication.md 385
116 prompts/F53-graceful-failures.md 389
117 prompts/F54-access-and-permissions.md 393
118 prompts/F55-saved-machines.md 397
119 prompts/F56-repeat-enquiry.md 401
120 prompts/F57-catalogue-browser.md 405
121 prompts/F58-catalogue-imports.md 409
122 prompts/F59-catalogue-versioning.md 414
123 prompts/F60-missing-parts-queue.md 418
124 prompts/F61-enquiry-pdf.md 422
125 prompts/F62-labelled-evaluations.md 426
126 prompts/F63-metrics-dashboard.md 430
127 prompts/F64-latency-and-ai-cost.md 434
128 prompts/F65-correction-feedback.md 438
129 prompts/F66-tests-and-ci.md 442
130 prompts/F67-traces.md 446
131 prompts/F68-recruiter-documentation.md 450
132 prompts/F69-demo-walkthrough.md 454
133 prompts/22-security.md 458
134 prompts/26-deployment.md 461
135 prompts/27-hosted.md 464
136 spec/contracts.py 467
137 spec/api_models.py 472

001 README

README.md

PartsIQ single workflow Snowflake build kit

This kit specifies the complete69-feature PartsIQ application with a conventional Python workflow. It replaces the earlier multi-agent design while retaining the customer, staff, catalogue, analytics, security and recruiter-demo features. Voice and quotation building remain excluded.

This is an implementation kit, not a completed or deployed application. Follow 97 detailed prompts in prompts/README.md: foundations00–21 and23–25,featuresF01–F69,then final gates22 and26–27. Source contracts, SQL and dependency locks are supplied as exact implementation inputs. Test receipts must be produced by the coding agent as it builds the application.

Start here

1. Read AGENTS.md and docs/CONTEXT.md.

2. Read docs/PRD.md, docs/ARCHITECTURE.md and docs/DATA_OWNERSHIP.md.

3. Create your project repository and place this kit at its root. Keep references/legacy-backend.zip unchanged.

4. Open prompts/README.md and execute prompt 00 first. Do not paste all 97 prompts into one coding-agent turn.

5. Give the agent the complete current prompt and permission to inspect the repository. After each step, require real tests, a commit, and an updated handover.

6. Enable live Snowflake/OpenAI only after confirmed account allowance checks. Explicit fixture mode runs synthetic demos without either provider.

What is included

• 97 implementation prompts and69 feature traces.

• PRD, architecture, workflow, REST API, security, Snowflake, data ownership, deployment, rollback and handover documents.

• Typed Python models and a machine-readable OpenAPI3.1 specification.

• Supabase and Snowflake DDL, dependency manifests and resolved reference locks.

• The original backend, logo/brand pack and collaboration field guide.

• A regenerated complete guide and checksums for the archive contents.

Locked product choices

Next.js on Vercel; Supabase Auth, PostgreSQL and Storage; one FastAPI application plus a worker from the same image; Redis for rate limits and wakeups; Snowflake for versioned catalogue source and reporting; OpenAI for extraction, vision and constrained prose; optional BGE-M3 inside the worker search module. No LangGraph or autonomous agent routing.

Snowflake's trial is time/credit limited, not a permanent free plan. AWS eligibility and Vercel plan terms are account specific. The local fixture profile is the no-provider-cost fallback and must always be labelled accurately. The kit does not activate accounts, spend credits or claim production readiness by itself.

002 AGENTS

AGENTS.md

Coding agent instructions

Mission and authority

Build PartsIQ from this single-workflow Snowflake edition. The term agent in this file means the coding assistant executing prompts; it does not authorize an AI-agent architecture in the product. Preserve all 69 features. No supervisor, specialist team, LangGraph, LangChain, CrewAI or LLM tool-calling loop is required or allowed by this design.

Authority order: current user instructions, these rules, docs/CONSTRAINTS.md, approved architecture/ADRs, typed contracts/SQL/API specification, current prompt. If two approved specifications conflict, report the concrete conflict and propose a minimal repair before implementing incompatible behavior. Do not silently reinterpret an unknown business rule.

Work method

Execute one prompt at a time in spec/build-order.json. Read its prerequisites and current docs/HANDOVER.md. State the planned files, interfaces and tests; routine reversible implementation is authorized. Inspect existing code before writing. Preserve the supplied legacy archive and port its tested search logic into the single package instead of recreating it from memory.

Use rg to locate symbols. Prefer small cohesive functions with typed inputs and explicit side effects. Routers handle transport; feature facades handle user cases; domain modules own decisions; repositories handle persistence. Do not put SQL or credentials in React components. Do not copy compatibility, approval or budget rules into feature modules.

For every task: implement the specified behavior, run focused tests, inspect the diff, record real evidence, commit, update handover and feature traces. A file named test does not prove a behavior passed. Never fabricate test counts, accuracy, latency, production use, customer adoption or recorded videos. A blocked provider check is not passed.

Critical invariants

1. Resolve a confirmed machine configuration before ranking its compatible part set.

2. Only activate a Snowflake catalogue version after explicit validation and atomic PostgreSQL serving-cache publication.

3. Missing information and close candidates produce questions or staff review, not guesses.

4. No recommendation is published without a current revision-bound human decision and audit record.

5. Customer identity and staff roles derive from verified authentication and server-side records.

6. Every durable job completion is fenced by lease token, revision and demo generation.

7. AI budget reservation happens before every external attempt; uncertain usage stays reserved until reconciled.

8. No paid fallback, no voice, no quotation builder, no orders/reservations, no external messages.

Tools and dependencies

Use the package pins and locks supplied in spec. Do not install a new library merely because its API is convenient. If a pinned dependency fails on the target runtime, record the failure and propose the smallest justified change with a new lock and tests. Never loosen all pins to make a resolver pass.

Use a disposable local database for tests. Never run schema resets, seed scripts or destructive queries against production. Keep secrets in ignored environment files or provider secret stores. Do not echo keys, JWTs, signed URLs or database passwords into logs. No runtime uses the schema-owner credential.

Completion format

Report changed behavior, paths, commands and actual results, remaining blockers, and commit. Append docs/SESSION_LOG.md. Update docs/HANDOVER.md with the next prompt. Feature traces remain specified until tests and implementation exist. Stop to clarify only when blocked by an actual missing business decision, required credential/account access or destructive operation; continue all other authorized work.

003 CLAUDE

CLAUDE.md

Coding assistant context

Read AGENTS.md and docs/CONTEXT.md first. They apply to every coding assistant. Execute the current prompt from prompts/README.md and update the shared handover. Do not interpret historical references as authorization to restore multi-agent architecture.

004 MASTER PROMPT

MASTER_PROMPT.md

Start or resume implementation

You are implementing the PartsIQ single-workflow Snowflake edition from this repository. Read AGENTS.md, docs/CONTEXT.md, docs/HANDOVER.md and prompts/README.md. Identify the first uncompleted prompt in spec/build-order.json. Inspect its prerequisites and current source before editing.

Complete that prompt in full: typed contracts, domain behavior, REST adapters, UI where specified, permissions, failure handling, tests, documentation and commit. Preserve all 69 features and the fixed compatibility/human approval rules. No agent framework is part of the runtime. Use Snowflake only through the catalogue/reporting adapters and Supabase for operational state/auth/files. Respect the zero-paid-budget gate.

Run the prompt's named checks and record actual outcomes. Do not claim a service was deployed or a live provider was tested without execution evidence. If one external dependency blocks a check, finish independent local implementation and identify the exact missing account/configuration. Update handover and feature trace before returning. Never shorten implementation to a mock UI when the prompt calls for working backend behavior.

005 CONTEXT

docs/CONTEXT.md

Project context

Mohammed Zuhair Hussain is building a recruiter-facing spare-parts enquiry application inspired by industrial equipment distribution in Saudi Arabia. The demonstration uses synthetic equipment, parts, stock and customers; it does not represent a commercial relationship or real TAMGO inventory. The project should demonstrate full-stack engineering, bilingual AI extraction, deterministic data validation, warehouse planning, REST API design, data-platform integration and human review.

The original journey is retained: a Riyadh customer sends Arabic/English text and a nameplate photo; the system extracts details, confirms identifiers, identifies the machine, filters compatible parts, asks about ambiguity, plans available quantities, drafts a reply, and pauses for staff approval. Staff corrections are auditable and can support reviewed evaluation cases.

The earlier multi-agent implementation plan was explicitly replaced on16 October 2026. All 69 feature groups remain. Application orchestration now means ordinary Python functions and a persisted finite-state workflow. There are no specialist agents and no model-selected next steps. AGENTS.md describes how an external coding assistant should work on the repository, not runtime agents.

Snowflake was requested as the data-platform addition. Adopted design: immutable catalogue source versions and redacted analytical events in Snowflake; operational records, auth, files and validated relational serving cache in Supabase. This preserves PostgreSQL compatibility constraints and avoids expensive warehouse queries on every chat poll. Feature behavior must be identical between a validated live snapshot and an explicitly selected synthetic fixture snapshot, apart from provenance and freshness indicators.

Budget remains zero paid spend. The owner says they have free OpenAI keys, but provider allowance and expiry still require confirmation. Snowflake trial availability cannot be assumed permanent. Live services are disabled by default. Fixture mode must never be presented as a successful live Snowflake/OpenAI integration test.

006 PRD

docs/PRD.md

Product requirements document

Purpose and success criteria

PartsIQ helps customers describe equipment spare-part needs and helps staff prepare a defensible response. The system reduces manual interpretation and catalogue searching while keeping compatibility, quantity allocation and final approval under deterministic control. A successful recruiter demonstration shows the full enquiry journey, one ambiguity case, one incompatible request and one warehouse split, including visible provenance and human review.

Acceptance is behavioral rather than a claimed accuracy number: an incompatible exact-number match must never be recommended; unconfirmed photo identifiers must not drive matching; ambiguous candidates must not be silently selected; an unassigned reviewer must not see real enquiries; and a recommendation must not publish without a matching approved revision. Performance and extraction accuracy are measured later using the supplied evaluation plan.

Actors and permissions

Public visitor reads the landing page and architecture. Anonymous demo customer owns an expiring demo session and can switch to reviewer presentation only for that session. Invited customer owns real enquiries when real onboarding is enabled. Reviewer sees assigned real enquiries and can correct, draft, approve or reject. Catalogue administrator stages, validates and publishes source versions. Administrator manages assignments, allowances, evaluation curation and aggregate reporting. Runtime worker identities are technical roles, not human roles.

Main user journey

Customer creates an enquiry with destination warehouse and up to20 requested items. Optional images are privately uploaded, validated and read in the background. Structured extraction preserves the original message and source spans. The customer confirms photo identifiers and answers missing-information questions. Machine identification resolves one configuration. Search retrieves only explicitly compatible parts and supplies evidence. Close scores trigger a choice or review. Fulfilment combines repeated-part demand across items and reports availability, shortage and transfer estimates without reserving stock. A constrained draft is verified and presented to staff. Approval rechecks revision, catalogue and stock before atomically recording the decision and publishing the customer reply. Approved information can be exported as a bilingual enquiry PDF.

Complete feature scope

Every feature below remains mandatory unless its behavior explicitly describes a capability flag, such as optional semantic ranking. Optional capability means implemented and testable with a transparent disabled mode, not silently omitted.

F01 Branded landing

Explain the enquiry-to-approval journey; show synthetic-demo notice; expose Start demo and architecture links; display the supplied bilingual logo without stretching.

Acceptance and failure boundaries: Missing repository link hides that action; logo has meaningful alternative text; no affiliation claim.

F02 Demo scenarios and assets

Seed four deterministic scenarios: mixed-language photo, missing serial, incompatible part, local shortage; derive identifiers from seed 42; validate expected outcomes against generated records.

Acceptance and failure boundaries: Reject a scenario whose expected part lacks compatibility; reject stock claims not supported by fixture quantities.

F03 Demo role switching

Switch the presentation between customer and reviewer using the same anonymous identity and session; authorize review only for enquiries belonging to that active session.

Acceptance and failure boundaries: Forged reviewer role never grants staff membership; another visitor's enquiry returns 404.

F04 Demo isolation

Bind each session to verified anonymous Supabase identity; check generation and expiry on every read, write and worker completion; exclude demos from operational metrics by default.

Acceptance and failure boundaries: Expired, reset and foreign sessions fail; missing identity context returns no rows.

F05 Demo reset

Lock session; increment generation; cancel unfinished jobs from prior generation; enqueue Storage cleanup; keep minimal audit; create fresh scenario enquiries on explicit selection.

Acceptance and failure boundaries: Late OCR from old generation cannot update current data; repeated reset idempotency key returns the same generation.

F06 Responsive layouts

Support 360, 768, 1280 CSS-pixel widths; stack review panels; horizontally scroll comparison tables within their region; preserve sticky primary review actions without covering content.

Acceptance and failure boundaries: No page-wide horizontal overflow; zoom 200 percent remains usable; mobile dialogs retain visible close action.

F07 Arabic and English interface

Use a locale cookie and complete en/ar message catalogs; persist preference; set document lang and dir; isolate serials, numbers and part identifiers with bdi.

Acceptance and failure boundaries: Missing translation fails tests; language switch preserves enquiry ID and unsent draft; mixed Arabic identifiers remain readable.

F08 Light and dark themes

Use CSS semantic tokens; initialize from saved preference then system setting; expose accessible toggle; honor reduced motion and browser color-scheme.

Acceptance and failure boundaries: No hydration mismatch; chart colors and focus rings meet contrast checks; storage-disabled browsers still work.

F09 Bilingual chat

Accept Arabic, English and mixed text; preserve original message; detect preferred reply locale; render customer messages immediately after successful persistence.

Acceptance and failure boundaries: Empty and oversized input returns validation error; HTML is escaped; retry does not duplicate a message.

F10 Structured extraction

Extract item descriptions, quantities, urgency and machine hints with strict structured outputs; preserve source spans; missing quantity becomes a clarification rather than assumed order quantity.

Acceptance and failure boundaries: Unknown keys and invalid quantities reject the response; unsupported model output cannot progress to search.

F11 Clarifying questions

Use workflow reason codes to choose code-owned bilingual questions; persist pending question; associate answer with item; rerun affected resolution only.

Acceptance and failure boundaries: No final recommendation before approval; contradictory answer requires confirmation; repeat event creates one question.

F12 Conversation memory

Persist all messages and typed current state; compact context using retained structured facts; resume by enquiry/revision; never mix customer sessions.

Acceptance and failure boundaries: Restart retains unresolved questions; old revision events are ignored and audited; foreign thread ID denied.

F13 Private uploads

Create pending attachment and unique object path; use signed upload; validate actual bytes and magic type after confirmation; strip EXIF and cap decoded pixels before OCR.

Acceptance and failure boundaries: Wrong MIME, oversized content, image decompression bomb, duplicate confirmation and nonexistent object handled explicitly.

F14 Nameplate OCR

Read validated nameplate via OpenAI vision; return model_text, serial_text, issues and source attachment; persist result only with current lease and revision.

Acceptance and failure boundaries: Unreadable fields remain null; vision instructions in photo are untrusted; timeout is bounded and creates recoverable status.

F15 OCR confirmation

Show cropped or original nameplate beside extracted identifiers; require user confirm or correction; record actor and time; preserve leading zeros in serial text.

Acceptance and failure boundaries: Unconfirmed OCR never authorizes matching; blank corrected serial triggers missing-serial flow; stale confirmation returns 409.

F16 Photo quality feedback

Check decodability, dimensions and blur heuristic before paid call; return actionable retake instructions; mark heuristic as advisory; allow a readable image despite noncritical blur warning.

Acceptance and failure boundaries: Unsupported format rejected; dark or low-resolution sample gets guidance; no fabricated OCR confidence percentage.

F17 Multi-part enquiries

Track up to 20 items with independent quantity, machine, candidate selection and status; clarify individual items; verify aggregate allocation for repeated part requests.

Acceptance and failure boundaries: One unresolved item blocks final all-items approval; allocations across duplicate parts cannot double count available stock; zero quantity rejected.

F18 Progress timeline

Show persisted stages, wait reason and last update; poll with bounded backoff; reconnect using server state; separate waiting for customer from waiting for staff.

Acceptance and failure boundaries: No fake completed progress; failed poll retains last state and offers retry; hidden tabs reduce polling.

F19 Enquiry history

Return cursor-paginated owned history with status and created time; filter active demo generation; link to full conversation; use stable created_at/id ordering.

Acceptance and failure boundaries: Foreign owner ID ignored; invalid cursor returns 422; equal timestamps do not duplicate or skip records.

F20 Machine identification

Preserve identify_machine logic for model, serial and rating; return resolved configuration or explicit missing/ambiguous status; normalize numeric serial only after preserving input.

Acceptance and failure boundaries: Boundary serials match exactly one configuration; overlapping ranges fail Snowflake publication validation and PostgreSQL serving-cache insertion; conflicting hints never silently override.

F21 Machine mismatch detection

Compare model and serial together; return mismatch reason with safe next question; prevent retrieval while identity unresolved.

Acceptance and failure boundaries: Valid serial of a different model does not match requested model; nonnumeric serial handled without crashing.

F22 Compatibility gate

Fetch authoritative fitment set first; rank only members; carry source_reference and catalogue version through every candidate.

Acceptance and failure boundaries: A perfect lexical match outside fitment is absent; empty fitment returns unavailable; blank provenance cannot be published.

F23 Aliases and typo matching

Preserve Arabic normalization, part-number keys, phrase scoring and fuzzy token matching; remove machine terms using resolved model context.

Acceptance and failure boundaries: Arabic part words are not removed as machine words; exact part number outranks unrelated fuzzy alias; normalization parity test matches SQL.

F24 Semantic retrieval

Load one BGE-M3 encoder in the worker search module; use normalized 1024-d vectors; retain legacy weighted fusion; fall back to lexical on unavailable embedding path.

Acceptance and failure boundaries: Wrong dimension rejected; no model loaded in API process; no semantic candidate outside compatible IDs.

F25 Ranked candidates

Return top candidates with lexical/semantic evidence and deterministic tie order; compute runner-up margin before applying response limit.

Acceptance and failure boundaries: Limit one still reports ambiguity; empty set returns no candidate; NaN scores rejected.

F26 Ambiguity handling

Use tested score and margin thresholds in code; ask customer to distinguish fuel filter versus separator; allow staff to select only verified candidates.

Acceptance and failure boundaries: LLM cannot override low confidence; unavailable candidate choice returns 409; thresholds evaluated on labelled cases.

F27 Match explanations

Show actual alias, machine configuration, supersession chain and fitment reference; derive explanations from search evidence rather than invented model prose.

Acceptance and failure boundaries: Absent evidence shown as unavailable; user HTML is escaped; scores never labelled calibrated probability.

F28 Compatibility references

Resolve source_reference to version-specific document and page; retain citation with every candidate and approval snapshot.

Acceptance and failure boundaries: Missing source file blocks evidence viewer and flags review; mismatched version fails verification.

F29 Evidence page viewer

Authorize assigned reviewer or own demo reviewer; request short-lived backend-signed PDF URL; open exact cited page; label synthetic document.

Acceptance and failure boundaries: Customer receives reference only; arbitrary Storage key rejected; expired link can be renewed after authorization.

F30 Old part numbers

Follow supersession chain with cycle protection; display old and current numbers; verify current successor has explicit fitment for resolved configuration.

Acceptance and failure boundaries: Cycles rejected; inactive terminal part returns unavailable; inherited compatibility without source is rejected.

F31 Candidate comparison

Compare two or three candidates from the same item/revision; show number, names, evidence and warehouse availability; selecting a candidate sends its ID, not browser-authored facts.

Acceptance and failure boundaries: Foreign candidate or stale revision rejected; missing attribute shown as unavailable; keyboard navigation works.

F32 Warehouse stock

Show available quantity as on_hand minus reserved; include timestamp and warehouse name; keep display and planner using same snapshot.

Acceptance and failure boundaries: Negative stock blocked; stock unavailable never displayed as zero without explanation; unknown warehouse rejected.

F33 Fulfilment planning
Allocate local stock first then reachable warehouses deterministically; compute shortage; use quantity units; do not reserve stock or create orders.

Acceptance and failure boundaries: Total allocations never exceed request or stock; fractional quantities respect unit policy; unavailable transfer route cannot be invented.

F34 Transfer estimates
Carry integer transfer_days from approved route; show estimate and source warehouse; local availability means zero transfer days.

Acceptance and failure boundaries: No route gives unknown/unavailable plan rather than invented one-day promise; estimate labelled non-guaranteed.

F35 Stock freshness
Compare stock timestamp with configurable freshness limit; mark stale data in review; re-read current stock during approval; invalidate plan when allocation no longer possible.

Acceptance and failure boundaries: Future timestamp flagged; changed stock returns 409 and reruns planning; stale stock never becomes a delivery guarantee.

F36 Staff inbox
List assigned operational enquiries and own active demo enquiries; filter status, priority, overdue and text; paginate stably; show counts in same access scope.

Acceptance and failure boundaries: Unassigned real enquiries hidden from reviewer; admin can see all; filters do not leak global counts.

F37 Review workspace
Assemble current extraction, items, candidates, evidence, stock, draft and timeline; show verification failures; separate staff-only notes from customer messages.

Acceptance and failure boundaries: Missing panels show explicit empty states; stale revision disables approval; unassigned reviewer receives 404.

F38 Correct extracted details
Lock enquiry; compare expected revision; apply typed allowed corrections; increment revision; invalidate derived selections/draft/approval; enqueue recomputation; record before and after.

Acceptance and failure boundaries: Unknown field rejected; old revision returns 409; correction cannot directly set approved status or invent candidate facts.

F39 Review actions
Approve only verified current draft; edit prose through controlled schema then reverify; reject with reason; record actor and idempotency key.

Acceptance and failure boundaries: Double-click publishes once; concurrent edit defeats stale approval; failed verification blocks publication.

F40 Approval invalidation
Bind approval to revision and verified fact hash; any identity, quantity, selection or catalogue-sensitive change requires new draft and approval.

Acceptance and failure boundaries: Old approval cannot authorize new revision; late workflow update cannot overwrite approved current state.

F41 Internal notes
Append reviewer notes with author and timestamp; show only in staff workspace; prohibit customer/PDF exposure; escape rendered content.

Acceptance and failure boundaries: Empty or over-4000-character note rejected; customer route never includes note text; unassigned staff denied.

F42 Assignments

Admin assigns active reviewer or unassigns; persist assignment history atomically; access changes immediately; demo reviewer access remains session-scoped.

Acceptance and failure boundaries: Non-admin assignment denied; disabled/nonreviewer assignee rejected; previous reviewer loses access on next request.

F43 Overdue indicators

Set due_at using configurable urgent/normal thresholds; display overdue only for unresolved states; sort overdue first when requested; store all times UTC.

Acceptance and failure boundaries: Approved/rejected items never overdue; daylight-saving display does not change deadline; thresholds not described as contractual SLA.

F44 Conversation summary

Produce staff-only concise summary from authorized messages and confirmed fields; store message IDs and revision; use shared budget wrapper; regenerate on relevant change.

Acceptance and failure boundaries: Outdated summary labelled stale; instructions inside messages ignored; exhausted budget leaves full conversation available.

F45 Reply drafting

Generate greeting, explanation and closing as structured slots; code inserts all numbers and allocations; choose Arabic or English from confirmed preference.

Acceptance and failure boundaries: Unapproved critical facts never rendered; unsupported slots rejected; provider failure permits staff-authored controlled draft.

F46 Verification gate

Check item/configuration/candidate membership, provenance, catalogue version, selected quantities and allocation bounds; compute canonical fact hash; store named rule results.

Acceptance and failure boundaries: Unknown part, incompatible part, stale catalogue, oversold plan and hidden extra item each fail a specific rule.

F47 Human publication gate

Only approval service inserts recommendation messages; lock enquiry and recheck role, revision and verification; unique publication index prevents duplicates.

Acceptance and failure boundaries: No background job write tool can publish; database rejects recommendation lacking matching approved decision.

F48 Input isolation

Treat customer text, OCR and imported evidence as data; separate trusted instructions; bound inputs, transitions, steps and output size; never execute model-produced templates.

Acceptance and failure boundaries: Prompt injection cannot call privileged functions, change roles or reveal secrets; model result with rogue instructions remains inert.

F49 Audit trail

Append actor, request, entity, revision and minimal before/after metadata for AI suggestions and human changes; redact secrets and full sensitive payloads.

Acceptance and failure boundaries: Application UPDATE, DELETE, TRUNCATE fail; audit write failure rolls back approval; database-owner powers documented honestly.

F50 Durable workflows

Persist workflow_runs and workflow_step_results in PostgreSQL; run a fixed Python transition table with durable jobs; serialize each enquiry and fence completion by revision and lease token; recover after restart without LangGraph.

Acceptance and failure boundaries: Duplicate jobs cannot repeat publication; stale revisions are discarded; interrupted steps resume from committed results; no LLM chooses the next step.

F51 Background jobs

Persist job before Redis dispatch; dispatcher claims by ID; sweeper redispatches pending/expired jobs; worker handles OCR, PDF, imports and cleanup.

Acceptance and failure boundaries: Redis loss does not lose DB job; unavailable Redis rate limiter fails closed; heavy work never blocks HTTP intake.

F52 Retry and deduplication
Claim with FOR UPDATE SKIP LOCKED; assign lease token; heartbeat; retry transient errors with backoff up to max attempts; fence final writes by lease token.

Acceptance and failure boundaries: Expired worker cannot commit; permanent validation failure not retried; provider timeout marked uncertain and budget held conservatively.

F53 Graceful failures
Map stable error codes to bilingual recovery guidance; distinguish validation, quota, service outage and staff-required states; preserve user input.

Acceptance and failure boundaries: No raw stack traces or keys; retry only safe idempotent action; search lexical fallback clearly reported.

F54 Access and permissions
Verify Supabase issuer, audience, signature, expiry; derive identity server-side; enforce assigned-only review; keep credentials server-side; disable public real-client signup.

Acceptance and failure boundaries: Forged role claim ignored; missing context denies rows; preview cannot reach prod database; demo cannot publish shared catalogue.

F55 Saved machines
Save confirmed model and serial with user label; list own active entries; archive entry; prefill a new enquiry but re-identify against current catalogue.

Acceptance and failure boundaries: Duplicate label permitted with distinct ID; foreign ID denied; stale configuration never reused as authoritative.

F56 Repeat enquiry
Copy customer request and confirmed hints into a new enquiry; issue new ID/revision; link origin in audit; run matching, stock and review again.

Acceptance and failure boundaries: Never copy approval, allocation, draft or signed file URL; source ownership required; repeated submission deduplicated.

F57 Catalogue browser
Read published catalogue with model/category/text filters; page results; show part detail and supersession; staff can inspect fitment and sources.

Acceptance and failure boundaries: No unpublished import rows exposed; missing number returns 404; literal query cannot become SQL.

F58 Catalogue imports
Read CSV UTF-8 or XLSX without evaluating formulas; validate row schema, references, serial overlap, supersession cycles and source evidence; show dry-run report; explicit admin publish. Stage a version in Snowflake, validate all relationships explicitly and only then activate a validated PostgreSQL serving snapshot.

Acceptance and failure boundaries: Invalid batch changes no published data; file with macros/formulas rejected; 50000-row and 10MiB limits enforced; public demo cannot publish.

F59 Catalogue versioning
Store immutable versioned catalogue records in Snowflake; verify record counts and canonical SHA256; stage the PostgreSQL serving snapshot; atomically switch the active version in a PostgreSQL transaction; retain published historical records; reconcile partial remote writes.

Acceptance and failure boundaries: Two publishers cannot leave two active versions; failed publish rolls back; old approval retains immutable evidence snapshot.

F60 Missing-parts queue
Record unresolved item once; allow assigned staff to mark investigating, resolved or unavailable with note; link back to enquiry; no automatic catalogue creation.

Acceptance and failure boundaries: Resolved status requires staff explanation; foreign item denied; empty search result is not fabricated stock.

F61 Enquiry PDF

Export approved revision only; render logo, bilingual customer text, verified selections, estimates and references; embed Arabic fonts; store private object and signed download.

Acceptance and failure boundaries: Exclude notes and secrets; stale or unapproved revision rejected; long Arabic names wrap without clipping; no prices or quotation title.

F62 Labelled evaluations

Include four demos plus Arabic, typo, serial-boundary, ambiguity, stock and injection cases; record expected compatible set and clarification outcome; run deterministic suite without API calls.

Acceptance and failure boundaries: No self-reported model confidence as accuracy; evaluation fixture disjoint from examples used in prompts; failures keep actual output.

F63 Metrics dashboard

Show volume, status, review time, clarification and labelled accuracy with denominators; separate demo traffic; filter time window; scope reviewer data to assignments. Export redacted aggregate events to Snowflake through an outbox and show reporting freshness separately from live operational counts.

Acceptance and failure boundaries: Zero denominator displays unavailable; no fabricated percentage; demo traffic never silently improves operational metrics.

F64 Latency and AI cost

Measure stages with monotonic clock; log model/token usage and estimated cost; reserve confirmed free credit before each attempt; expose consumed/reserved allowance.

Acceptance and failure boundaries: Unknown price or expired credit blocks calls; concurrent reservations cannot exceed cap; API key validity never implies free credit.

F65 Correction feedback

List authorized before/after corrections; let admin curate anonymized labelled eval cases; record reviewer and source revision; never auto-train on unreviewed edits.

Acceptance and failure boundaries: Private text excluded from exported fixtures; duplicate case source detected; unreviewed correction cannot become ground truth.

F66 Tests and CI

Run service-isolated Python tests, frontend checks, RLS integration and deterministic evals; build images; require explicit environment approval for deployment; pin action SHAs.

Acceptance and failure boundaries: No prod DB in tests; no secret available to untrusted fork jobs; failed gate blocks deployment.

F67 Traces

Propagate request/enquiry/job IDs; capture stage timing and safe status; export optional Langfuse traces with payload redaction; operate normally when tracing unavailable.

Acceptance and failure boundaries: No raw key or full photo logged; exporter outage never breaks approval; budget accounting remains local and authoritative.

F68 Recruiter documentation

Write concise overview, real architecture, quick start, demo paths, tests, trade-offs and limitations; link measured evidence; explain deterministic decisions and ordinary Python functions.

Acceptance and failure boundaries: No invented accuracy, uptime, client relationship or production adoption; distinguish implementation status from specification.

F69 Demo walkthrough

Create 3-minute and 8-minute scripts covering happy path, missing serial, incompatibility, stock split and human review; include recording checklist and captions.

Acceptance and failure boundaries: Do not claim video exists until recorded; use synthetic data; show failure handling and reset; verify links before sending recruiters.

Nonfunctional requirements

Security: server-verified identity, scope checks, RLS, private files, least-privilege data roles, redacted logging and no model-authorized writes. Durability: database-backed jobs, leases, fencing, idempotent API writes

and recoverable partial Snowflake synchronization. Language: Arabic/English UI and replies, mixed-script identifiers, bidi isolation and bundled Arabic PDF fonts. Accessibility: keyboard operation, visible focus, reduced motion, 200 percent zoom and360px minimum layout. Maintainability: typed models, one domain implementation per rule, versioned migrations and explicit test evidence.

Provisional performance goals are local acceptance budgets, not measured claims: API writes should acknowledge without waiting for AI or warehouse work; provider operations have bounded timeouts; polling uses backoff; optional embeddings cannot prevent lexical readiness. Benchmark the actual host before setting public latency claims. The paid budget is zero; the system requires confirmed free allowance before live provider calls. Real-client retention is an onboarding decision; until it is recorded, real onboarding stays disabled. Demo access expires after24hours and background cleanup follows the documented policy.

Exclusions

No voice input, quotation builder, prices, payments, orders, stock reservations, automatic WhatsApp/email sending, multi-agent routing, autonomous SQL generation or automatic training on staff corrections. No claim of perpetual free cloud hosting. No invented partnership with equipment distributors.

007 ARCHITECTURE

docs/ARCHITECTURE.md

Application architecture

1 What is in the system

The browser runs a Next.js application hosted on Vercel. It authenticates through Supabase and sends business REST requests to a single FastAPI backend behind Caddy. Supabase PostgreSQL stores transactional application state and a validated relational catalogue cache; Supabase Storage holds private attachments, evidence, imports and exports. A worker imports the same Python package as the API and executes durable jobs. Redis provides request-rate counters and optional wakeups. Snowflake stores immutable catalogue source versions, inventory snapshots and redacted analytical events. OpenAI performs bounded language/vision calls. BGE-M3 is an optional worker-local search capability.

```
flowchart TD
 W["Next.js customer and staff UI"] --> A["FastAPI REST application"]
 W --> S["Supabase Auth and Storage"]
 A --> P["Supabase PostgreSQL"]
 A --> R["Redis limits and wakeups"]
 P --> J["Worker using same application package"]
 J --> P
 J --> F["Snowflake catalogue and reporting"]
 J --> O["OpenAI extraction and drafting"]
 J --> S
```

2 Who owns each responsibility

Module	Exact application directory	Responsibility
Transport	apps/api/partsiq/api	Typed REST endpoints, error serialization and dependency injection
Authentication	apps/api/partsiq/auth	JWT verification, scope, roles and demo sessions
Enquiries	apps/api/partsiq/enquiries	Intake, history, corrections, fulfilment, review and publication
Search	apps/api/partsiq/search	Legacy identification, compatibility, lexical ranking and optional embeddings
Workflow	apps/api/partsiq/workflow	Fixed transitions and persisted step results
AI	apps/api/partsiq/ai	Allowance-controlled structured extraction, vision and reply slots
Jobs	apps/api/partsiq/jobs	Claims, leases, dispatch, retry and cleanup
Catalogue	apps/api/partsiq/catalogue	Snowflake source, validation and atomic serving activation
Analytics	apps/api/partsiq/analytics	Redacted outbox, Snowflake export and scoped metrics
Storage and exports	apps/api/partsiq/storage and exports	Private signed transfers and approved PDFs
Features	apps/api/partsiq/features	User-case facades; no duplicated domain safety rules
Web	apps/web/src	Accessible bilingual pages, panels and typed API client

3 Why this structure

The customer journey has a known sequence. Ordinary functions and a fixed state table are sufficient; an LLM does not need to decide which component runs next. One backend avoids multiple internal HTTP hops and duplicated deployment configuration. Background work remains separate because photo reading, imports and PDF rendering can outlive an HTTP request. Snowflake gives the project a real data ingestion/reporting boundary, while the relational serving cache preserves transactional integrity and low-cost interactive access.

4 What each layer may access

Routes call feature facades and inject a verified ActorContext. Facades invoke domain functions and repositories. Domain search never calls an LLM to decide compatibility. AI modules receive minimal data and

return typed results; they have no database write tools. Only the approval service can create recommendation messages. Catalogue synchronization uses a dedicated publisher role; ordinary API routes cannot update published fitment rows. Snowflake connectors execute allowlisted SQL with bound values and task-scoped connections. Frontend has no Snowflake or database-owner credential.

5 Who owns the data

PostgreSQL is authoritative for identity linkage, enquiries, revisions, candidates, drafts, decisions, jobs, budgets, audit and the active catalogue pointer. Snowflake holds the immutable source records for catalogue versions and analytical copies. Activation copies a verified source snapshot into PostgreSQL under a single transaction; that serving copy is authoritative for a particular active version until another validated version is activated. This is an explicit replicated-data contract, not two writable masters. Stock is separately timestamped and never treated as a reservation.

6 Where asynchronous boundaries exist

An API mutation commits the business record and durable job together. The worker claims the job and performs external work outside database locks. It commits only if lease token, revision and demo generation still match. Redis loss does not delete jobs. Snowflake publication is a recoverable two-system process: remote immutable upload first, local validation and activation second. Analytics uses an outbox with idempotent EVENT_ID MERGE and acknowledgement after remote success. No distributed transaction is claimed.

7 How security and failures are contained

Every scoped route checks identity and ownership/assignment. PostgreSQL RLS provides another boundary. Background handlers load trusted job context and fence final writes. Private files require signed access after authorization. Provider errors never bypass compatibility or approval. Wrong/expired allowance blocks live AI; Snowflake outage leaves an explicitly identified last validated snapshot usable only within freshness policy. A new unpublished version cannot replace a working snapshot. If required data is too stale, the workflow pauses.

8 How this is tested and operated

Unit tests cover pure rules; integration tests cover real PostgreSQL constraints, RLS, idempotency and concurrency; feature tests cover all 69 behaviors; browser tests cover both languages and four demos. Live Snowflake and provider checks require separate account receipts. API and worker share one immutable image, with Caddy and Redis on one EC2 host where confirmed credits permit. Supabase dev/prod are separate projects. The host is a single point of failure; backups and recovery are documented without a high-availability claim. Fixture mode remains a clearly labelled local demonstration path.

008 DATA OWNERSHIP

docs/DATA_OWNERSHIP.md

Data ownership and synchronization

Source and serving responsibilities

Data	Authoritative write owner	Read path	Consistency rule
Auth sessions	Supabase Auth	Verified JWT and auth service	Never trust browser role metadata
Enquiries and decisions	PostgreSQL application transactions	Scoped REST resources	Current revision and actor required
Catalogue version source	Snowflake loader	Version-addressed connector read	Immutable once accepted; hash and counts verified
Active catalogue	PostgreSQL activation transaction	Version-filtered relational mirror	Exactly one active pointer; no mixed versions
Stock snapshots	Source snapshot import	Validated serving stock rows	Timestamp and snapshot provenance retained
Private images/evidence/PDF	Supabase Storage	Authorized signed URLs	Bucket/object ID ownership required
AI allowance	PostgreSQL locked budget account	Backend budget wrapper	Reserve before call; no remote reporting dependency
Analytics	PostgreSQL outbox then Snowflake copy	Scoped live counts or admin warehouse reports	At-least-once transport, idempotent event key
Audit	PostgreSQL append-only application role	Authorized staff view	Business change and audit commit together

Catalogue publication protocol

1. Admin creates an import batch and uploads the file privately. Validation parses typed entities without evaluating formulas or macros.

2. Canonicalize each record by entity type, key and stable JSON encoding. Preserve numeric legacy IDs; new records receive IDs through a controlled allocator before the source snapshot is sealed. Compute each row hash and the whole snapshot hash over sorted records.

3. Validate duplicate keys, foreign references, serial-range overlap, nonblank provenance, supersession cycles, valid units and evidence file availability. Imported stock validates nonnegative quantities and timestamps independently.

4. A single publisher job obtains a bounded publication lease. Write the full immutable VERSION_ID to Snowflake. Before retry, read the existing version and require exact hash/count equality; conflicting content under the same ID is an error.

5. Read the version back through SnowflakeCatalogueRepository, validate duplicates before mapping rows, and compare observed counts/hash with the manifest. Standard table PK/FK declarations do not enforce this.

6. Stage a catalogue_syncs receipt. Begin a PostgreSQL transaction, acquire the catalogue advisory lock, and compare the caller's expected active version. Upsert relational rows in dependency order. Keep historical referenced IDs; hide removed rows using the active catalogue_records keyset. Recheck the enforced PostgreSQL constraints.

7. Retire the previous pointer and activate the new version in that same transaction. Commit the sync receipt and audit. If the transaction fails, the old version remains active and the remote version remains unactivated.

8. Pending enquiries detect a changed active version on their next processing/review action and require recomputation. Previously approved replies retain their immutable fact/evidence snapshot and are never rewritten as history.

Snowflake is not queried on every chat message or dashboard poll. The UI displays catalogue source, active version, last activation and stock observed time. Fixture mode uses the same validation contract and a different explicit source label. Never silently fall back from live Snowflake to fixtures after a failure.

Snapshot payload contracts

The CATALOGUE.RECORDS payload uses the exact typed columns of the corresponding legacy PostgreSQL entity, excluding generated search_document and database timestamps where not business data. Supported entities: equipment_families, equipment_models, model_configurations, parts, part_aliases, part_compatibility, part_supersessions, warehouses, warehouse_transfer_routes. Key construction is deterministic: single numeric IDs use their decimal string; composite keys use compact JSON arrays of their primary-key values. A schema version is recorded in the import report. Unknown entity types/fields fail validation rather than being silently dropped.

Stock uses the dedicated Snowflake STOCK_SNAPSHOTS table. Preserve snapshot_id, observed_at and original units during serving import. One snapshot is activated transactionally only after all its rows validate. Approval checks age and availability but does not reserve stock; the final text clearly labels availability as a snapshot and delivery time as an estimate.

Analytics transport

Business code appends an outbox event after redaction in the same PostgreSQL transaction. A single export job takes at most100 events, constructs bound MERGE statements keyed by event_id, and acknowledges only after Snowflake success. Duplicate remote IDs cause a diagnostic failure and repair; Snowflake primary-key metadata is not a uniqueness guarantee. A crash after remote commit replays the same event safely. Only allowlisted analytical columns are exported; no customer text, serial number, email, JWT, notes or signed URL enters Snowflake reporting.

Scoped live queue counts remain PostgreSQL queries. Snowflake reports are administrator-only aggregates and display sync lag. Reviewer dashboards never receive global unassigned counts. An expired trial freezes remote reporting and shows its last timestamp; it does not fabricate current metrics.

Stock activation receipt

stock_snapshot_receipts records the source snapshot ID, observed timestamp, content hash, row count and active flag. Under the publication lock, upsert stock_levels and switch the single active stock receipt in one PostgreSQL transaction. All plan reads return that snapshot ID. Approval compares the current active stock snapshot and freshness against the draft; changed availability triggers a new plan. Do not use a null snapshot ID to hide missing provenance in live mode.

009 WORKFLOW

docs/WORKFLOW.md

Fixed Python workflow

The controller is a deterministic transition table, not an LLM router. Persist state in workflow_runs and completed outputs in workflow_step_results. The unique key is enquiry_id plus revision. A job lease and a separate input hash fence every committed result.

Current step	Condition	Next step	Function owner
intake	Text parsed and no image pending	identify or clarify	ai.extraction.extract_message
intake	Validated image requires reading	await_ocr	jobs.nameplate.read_nameplate
await_ocr	OCR result persisted	confirm	enquiries.extraction.confirm_nameplate
confirm	Customer confirms identifiers	identify	workflow.resolution.resolve_items
identify	Missing or conflicting machine details	clarify	enquiries.questions.clarification
identify	One configuration resolved for each item	search	search.service.SearchService
clarify	New answer accepted in new revision	intake or identify	enquiries.intake.append_message
search	No compatible result	needs_staff	workflow.resolution.resolve_items
search	Close candidates	choose	enquiries.questions.clarification
search	Unique sufficient match	plan	enquiries.fulfilment.plan_enquiry
choose	Current candidate chosen	plan	enquiries.revisions.choose_candidate
plan	Complete or explicitly short plan	draft	ai.reply.draft_slots
plan	Missing/stale inventory	needs_staff	enquiries.fulfilment.plan_enquiry
draft	Structured slots rendered	verify	enquiries.verification.verify_facts
verify	All checks pass	review	workflow.repository.commit_step
verify	Any rule fails	needs_staff	enquiries.verification.verify_facts
review	Current revision approved	complete	enquiries.approval.approve
review	Corrected/rejected	new revision or terminal rejection	enquiries.revisions or approval

Each transition has a typed StepOutcome with reason_code and model-validated step data. The data envelope is not permission to store arbitrary provider output. Define a concrete payload model for each step using the provided IntakeResult, MachineView, CandidateList, PlanView, ReplySlots and VerificationResult contracts. Validate the matching payload before persistence.

Persistence algorithm

Claim a due job with FOR UPDATE SKIP LOCKED and a random lease token. Load enquiry/session and reject obsolete revision/generation. Under a short transaction, claim the matching workflow run. Release locks before calling OpenAI, Storage or Snowflake. Hash the exact step inputs and reuse only an identical completed result. Perform one bounded step. Reacquire row lock, recheck lease/revision/generation, write output and next state, append audit, then commit. Schedule the next step or park at a waiting state. Renew leases while healthy; never let an expired process finalize a result. Stop after20automatic transitions in one job.

No transaction remains open across a provider call. There is no promise of exactly-once external API execution; uncertain provider completions remain conservatively charged/reserved. Business publication is effectively once through idempotency and a unique recommendation-per-revision index.

User changes

New facts or item choice invalidate derived data by creating a new revision. Preserve old outputs for audit but do not reuse them without matching input hash and source version. Assignment and notes need not change the extraction revision unless they alter facts. An assignment has its own audit event and immediate access effect. Reset increments demo generation and makes old jobs obsolete. User questions are code-owned bilingual templates and do not require staff approval because they make no final part recommendation.

Cross-feature dependencies

Search and verification share one source of fitment truth. Draft rendering consumes VerifiedFacts only. Approval rechecks freshness and fact hash. PDF uses ApprovedSnapshot only. Analytics observes committed business events and cannot block approval. AI allowance remains local and authoritative even when traces or Snowflake reporting are unavailable.

Missing quantity before item materialization

IntakeResult permits a null quantity because absence is not permission to assume one. Keep provisional item identifiers and their extracted fields in the typed workflow state until quantity is confirmed. Do not insert a null or invented quantity into enquiry_items, whose quantity constraint is intentionally positive and non-null. Clarification tracks the provisional item identifier in state; once confirmed, materialize that item with the same UUID and current revision. Persist the original extraction separately for audit. Tests must cover multiple items where only one quantity is missing.

010 REST API

docs/REST_API.md

REST API contract

Transport rules

FastAPI is the only business API. Base prefix is /v1. HTTPS is required outside localhost. JSON requests use Content-Type application/json. Authorization uses a Supabase Bearer token; never use a customer-supplied actor field. Every response carries X-Request-ID. Every mutating request, including DELETE, requires an Idempotency-Key UUID. The key is scoped by actor, method and path; reusing it with different payload returns 409. JSON updates carry expected_revision where specified. A POST returning202 has committed a job/resource but has not completed the background action.

Typed models in spec/contracts.py and spec/api_models.py are canonical. spec/openapi.json expands their fields and nested schemas; spec/endpoints.json declares ownership. No unknown keys are accepted. UUIDs are strings on the wire, Decimal quantities/costs are decimal strings, datetimes are ISO8601UTC and serial/part numbers remain strings. For strict Pydantic models use JSON-aware validation at the HTTP boundary; do not reject valid JSON UUID strings because a Python-only strict validator expects a UUID instance.

GET requests with Query models use validated query parameters, not a JSON body. Repeated comparison part_ids are repeated query parameters. Cursor pagination orders by(created_at, id), uses a base64url encoded typed tuple, and rejects invalid cursors. Cursor validity is not authorization; always apply scope filters. limit defaults25 and caps100. Unknown status/filter values are422. A route response is bounded; large histories are paginated and private details are separate resources.

Access labels

public means no account. customer means any verified eligible user including an anonymous demo user with an active session where needed. demo means the subject's own active demo session. owner means the enquiry owner or authorized staff via the same scope rule. review means own-session demo reviewer, assigned real reviewer, or admin. reviewer means reviewer/admin listing only its permitted records, plus own-demo context. catalogue_admin means catalogue_admin/admin and never an anonymous demo. admin means server-side admin membership. scoped means derive ownership through the referenced job/export/resource; never authorize merely because the caller knows a UUID.

Errors

All failures use application/problem+json with Problem fields: type, title, status, code, detail, request_id, retryable.401 means missing/invalid token; 403 means a known disallowed role action; 404 conceals a foreign scoped resource; 409 means stale revision, idempotency conflict, changed catalogue or invalid current state; 413 means size limit; 422 means input/schema validation; 429 means rate limit; 503 means required dependency or live allowance unavailable. Include Retry-After only when retry is appropriate. Never include SQL, provider payloads, secrets or internal file paths. Domain code returns named errors; only route adapters choose HTTP transport.

Complete operation inventory

F02 | GET | /v1/demo/scenarios | None | ScenarioList | demo | 200

F04 | GET | /v1/demo/sessions/{session_id} | None | DemoSession | demo | 200

F05 | POST | /v1/demo/sessions/{session_id}/reset | RevisionOnly | DemoSession | demo | 201

F09 | POST | /v1/enquiries | EnquiryCreate | EnquiryView | customer | 202

F10 | GET | /v1/enquiries/{enquiry_id}/extraction | None | ExtractionView | owner | 200

F11 | POST | /v1/enquiries/{enquiry_id}/messages | MessageCreate | EnquiryView | owner | 202

F12 | GET | /v1/enquiries/{enquiry_id}/messages | PageQuery | MessagePage | owner | 200

F13 | POST | /v1/enquiries/{enquiry_id}/uploads | UploadRequest | UploadTicket | owner | 200

F14 | POST | /v1/attachments/{attachment_id}/confirm-upload | RevisionOnly | JobView | owner | 202

F15 | POST | /v1/enquiries/{enquiry_id}/confirm-nameplate | ConfirmExtraction | EnquiryView | owner | 202

F16 | GET | /v1/attachments/{attachment_id}/quality | None | QualityView | owner | 200

F17 | PATCH | /v1/enquiries/{enquiry_id}/items | ItemsEdit | EnquiryView | owner | 200

F18 | GET | /v1/enquiries/{enquiry_id}/progress | None | ProgressView | owner | 200

F19 | GET | /v1/enquiries | HistoryQuery | EnquiryPage | customer | 200

F20 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/machine | None | MachineView | owner | 200

F21 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/mismatch | None | MismatchView | owner | 200

F22 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/compatibility | None | CompatibilityView | review | 200

F23 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/match-evidence | None | MatchEvidenceView | review | 200

F24 | GET | /v1/search/capabilities | None | SearchCapabilities | customer | 200

F25 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/candidates | None | CandidateList | review | 200

F26 | POST | /v1/enquiries/{enquiry_id}/items/{item_id}/choice | CandidateChoice | EnquiryView | owner | 202

F27 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/explanation | None | ExplanationView | owner | 200

F28 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/references | None | ReferenceList | owner | 200

F29 | POST | /v1/enquiries/{enquiry_id}/evidence/{evidence_id}/download | RevisionOnly | DownloadTicket | review | 200

F30 | GET | /v1/catalogue/parts/{part_number}/supersession | None | SupersessionView | customer | 200

F31 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/comparison | ComparisonQuery | CandidateList | review | 200

F32 | GET | /v1/enquiries/{enquiry_id}/items/{item_id}/stock | None | StockView | review | 200

F33 | GET | /v1/enquiries/{enquiry_id}/plan | None | PlanView | review | 200

F34 | GET | /v1/enquiries/{enquiry_id}/transfers | None | TransferView | review | 200

F35 | GET | /v1/enquiries/{enquiry_id}/stock-freshness | None | FreshnessView | review | 200

F36 | GET | /v1/staff/enquiries | InboxQuery | EnquiryPage | reviewer | 200

F37 | GET | /v1/staff/enquiries/{enquiry_id} | None | ReviewView | review | 200

F38 | PATCH | /v1/staff/enquiries/{enquiry_id}/extraction | RevisionRequest | EnquiryView | review | 200

F39 | POST | /v1/staff/enquiries/{enquiry_id}/approve | ApprovalRequest | ApprovalResult | review | 200

F40 | GET | /v1/staff/enquiries/{enquiry_id}/approval-state | None | ApprovalState | review | 200

F41 | POST | /v1/staff/enquiries/{enquiry_id}/notes | NoteCreate | NoteView | review | 201

F42 | PUT | /v1/staff/enquiries/{enquiry_id}/assignment | AssignRequest | AssignmentView | admin | 200

F43 | GET | /v1/staff/enquiries/{enquiry_id}/deadline | None | DeadlineView | review | 200

F44 | POST | /v1/staff/enquiries/{enquiry_id}/summary | RevisionOnly | JobView | review | 202

F45 | PUT | /v1/staff/enquiries/{enquiry_id}/draft | DraftEdit | DraftView | review | 200

F46 | GET | /v1/staff/enquiries/{enquiry_id}/verification | None | VerificationResult | review | 200

F47 | GET | /v1/enquiries/{enquiry_id}/published-reply | None | PublishedView | owner | 200

F49 | GET | /v1/staff/enquiries/{enquiry_id}/audit | PageQuery | AuditPage | review | 200

F50 | GET | /v1/staff/enquiries/{enquiry_id}/workflow | None | WorkflowView | review | 200

F51 | GET | /v1/jobs/{job_id} | None | JobView | scoped | 200

F52 | POST | /v1/jobs/{job_id}/retry | RetryRequest | JobView | scoped | 202

F54 | GET | /v1/me | None | IdentityView | customer | 200

F55 | POST | /v1/machines | MachineSave | SavedMachineView | customer | 201

F56 | POST | /v1/enquiries/{enquiry_id}/repeat | RepeatRequest | EnquiryView | owner | 202

F57 | GET | /v1/catalogue/parts | CatalogueQuery | CatalogPage | customer | 200

F58 | POST | /v1/admin/imports | ImportCreate | ImportView | catalogue_admin | 201

F59 | POST | /v1/admin/imports/{import_id}/publish | PublishRequest | JobView | catalogue_admin | 202

F60 | PATCH | /v1/staff/missing-parts/{item_id} | MissingUpdate | MissingView | review | 200

F61 | POST | /v1/enquiries/{enquiry_id}/exports | ExportRequest | JobView | owner | 202

F62 | POST | /v1/admin/evaluations | EvalRequest | JobView | admin | 202

F63 | GET | /v1/staff/metrics | MetricsQuery | MetricsView | reviewer | 200

F64 | GET | /v1/admin/ai-allowance | None | AllowanceView | admin | 200

F65 | POST | /v1/admin/corrections/{correction_id}/evaluation-case | CurateRequest | CuratedView | admin | 201

F67 | GET | /v1/staff/enquiries/{enquiry_id}/traces | None | TraceView | review | 200

F04 | POST | /v1/demo/sessions | None | DemoSession | demo | 201

F09 | GET | /v1/enquiries/{enquiry_id} | None | EnquiryView | owner | 200

F13 | GET | /v1/enquiries/{enquiry_id}/attachments | None | AttachmentList | owner | 200

F39 | POST | /v1/staff/enquiries/{enquiry_id}/reject | RejectionRequest | ReviewResult | review | 200

F41 | GET | /v1/staff/enquiries/{enquiry_id}/notes | PageQuery | NotePage | review | 200

F44 | GET | /v1/staff/enquiries/{enquiry_id}/summary | None | SummaryView | review | 200

F55 | GET | /v1/machines | PageQuery | MachinePage | customer | 200

F55 | DELETE | /v1/machines/{machine_id} | None | ArchiveResult | customer | 200

F57 | GET | /v1/catalogue/parts/{part_number} | None | CataloguePart | customer | 200

F58 | GET | /v1/admin/imports/{import_id} | None | ImportView | catalogue_admin | 200

F58 | POST | /v1/admin/imports/{import_id}/upload | ImportUpload | UploadTicket | catalogue_admin | 200

F58 | POST | /v1/admin/imports/{import_id}/validate | None | JobView | catalogue_admin | 202

F59 | GET | /v1/admin/catalogue/versions | None | VersionList | catalogue_admin | 200

F60 | GET | /v1/staff/missing-parts | InboxQuery | MissingPage | reviewer | 200

F61 | GET | /v1/exports/{export_id}/download | None | DownloadTicket | owner | 200

F62 | GET | /v1/admin/evaluations/{run_id} | None | EvaluationView | admin | 200

F63 | GET | /v1/admin/analytics/sync | None | AnalyticsSyncView | admin | 200

F64 | PUT | /v1/admin/ai-allowance | AllowanceSet | AllowanceView | admin | 200

Worked request and lifecycle

POST /v1/enquiries uses EnquiryCreate: text, locale, destination_warehouse_id, and optional session_id. It returns EnquiryView with id, revision, status, messages, item_ids, catalogue_version_id and updated_at. The server creates the first message and a durable workflow job in the same transaction. A client then polls GET /v1/enquiries/{enquiry_id}/progress and GET /v1/enquiries/{enquiry_id}. It does not poll Snowflake directly.

If a nameplate is present, the upload endpoint issues a ticket; the browser uploads bytes and confirms that attachment. The worker validates/reads the image and the client confirms the resulting identifiers. Once a verified draft exists, an assigned reviewer reads the staff workspace and posts ApprovalRequest. The API locks/revalidates and commits approval plus recommendation together. If the request has an old revision, it returns 409 and the UI reloads; it never silently approves a new revision.

Route implementation pattern

Define a separate APIRouter in each owned feature route module. Use Depends for verified actor and typed command dependencies. The handler calls its feature facade exactly once, with path IDs and idempotency context. A facade calls foundation domain functions and returns the declared model. Register the router in api/router.py only after its module exists. No model-generated route, SQL or endpoint name is accepted. Keep responses free from internal notes unless this is an explicitly authorized staff-only model.

Versioning and contract tests

The backend generates OpenAPI during CI. Compare the completed operation subset during incremental work; the final gate requires all 77 specified operations and their response schemas. TypeScript types/Zod schemas must represent the same fields. Additive fields still require a documented contract update because

strict models reject unknown keys. Breaking changes require a new API version or coordinated migration; do not silently change an existing field's type.

011 API EXAMPLES

docs/API_EXAMPLES.md

Synthetic API examples

The UUIDs below are illustrative fixture IDs,not real resources. Obtain actual IDs from API responses during execution. Authenticate with a real test user token without committing it.

create enquiry

```
{
  "text": "ابغى فلتر ديزل للمولد, need 4 pcs urgently",
  "locale": "ar",
  "destination_warehouse_id": 1,
  "session_id": "11111111-1111-4111-8111-111111111111"
}
```

approve

```
{
  "enquiry_id": "22222222-2222-4222-8222-222222222222",
  "expected_revision": 3,
  "draft_id": "33333333-3333-4333-8333-333333333333",
  "edited_slots": null
}
```

problem

```
{
  "type": "urn:partsiq:problem:stale_revision",
  "title": "Enquiry changed",
  "status": 409,
  "code": "stale_revision",
  "detail": "Reload the current enquiry before approving.",
  "request_id": "44444444-4444-4444-8444-444444444444",
  "retryable": false
}
```

012 CONTRACTS

docs/CONTRACTS.md

Typed contracts

The complete executable model definitions are in spec/contracts.py and spec/api_models.py. Their JSON schemas are in spec/contracts.schema.json and the complete HTTP schema is spec/openapi.json. Copy them into partsiq.core in foundation 01. Keep nested fields, limits, enums and nullability unchanged unless a documented correction is required.

The API inventory and access rules are in REST_API.md. Foundation prompts define domain function signatures; feature prompts define facade signatures and exact wire request/response models. Decimal values are serialized as strings. Source record payloads have schema validation keyed by entity type and may never be accepted as arbitrary JSON for publication.

013 SNOWFLAKE

docs/SNOWFLAKE.md

Snowflake implementation guide

What the integration demonstrates

PartsIQ uses Snowflake for immutable catalogue ingestion, inventory snapshots and analytical reporting. This is a data-platform integration, not a replacement for Supabase authentication or a place to run an autonomous SQL agent. Interactive REST reads use the validated serving copy. The demo should visibly show source version, hash, activation time and reporting sync time so the integration is concrete.

Exact client and API choices

Pin snowflake-connector-python4.8.0, released30 September 2026. Its installation receipt is in spec/dependency-receipts.json. Version5.0.0 was released15 October and does not meet this kit's14-day stability window. Current online docs may show5.x renamed connection parameters; the implementation must follow the pinned4.8 signature and smoke-test it. Use connect with account, user, private_key_file, private_key_file_pwd, role, warehouse, database, schema, paramstyle='qmark', login_timeout=10 and network_timeout=30. Set STATEMENT_TIMEOUT_IN_SECONDS=20 and a request-UUID QUERY_TAG through session_parameters.

Use Connection.cursor, Cursor.execute with bound values, Cursor.fetchall or fetchmany for bounded reads, Cursor.sfqid for a query receipt, and close both resources in finally. Use explicit BEGIN/COMMIT/ROLLBACK for a logical remote write; DDL is setup-only and must not be mixed into transactional import publication. Do not share one live connection among unrelated async tasks. The connector is blocking: call it in asyncio.to_thread in the worker with at most2concurrent tasks. Cancellation of the awaiting coroutine is not proof that a remote statement stopped; retain query ID and classify uncertain completion for reconciliation.

Example read query is fixed SQL: SELECT ENTITY_TYPE, ENTITY_KEY, PAYLOAD, ROW_SHA256 FROM PARTSIQ_DEV.CATALOGUE.RECORDS WHERE VERSION_ID=? ORDER BY ENTITY_TYPE, ENTITY_KEY. Bind(str(version_id),). Database/schema identifiers cannot be value placeholders; choose them from the fixed validated environment allowlist. JSON payloads are parsed and checked against entity schemas. No user field becomes an identifier or SQL fragment.

Schema and roles

Apply infra/snowflake/001_schema.sql, 002_roles.sql and003_validation_queries.sql after copying them from spec. Development uses PARTSIQ_DEV and production uses PARTSIQ_PROD through the setup tool's allowlisted substitution. CATALOGUE.VERSIONS stores the version manifest; RECORDS stores typed entity records; STOCK_SNAPSHOTS stores inventory observations. REPORTING.EVENTS stores redacted events and DAILY_METRICS aggregates them. Snowflake standard tables have no application-defined indexes; do not promise PostgreSQL indexes/RLS on these tables. The initial small catalogue does not require clustering or search optimization services.

PARTSIQ_READER can read catalogue tables. PARTSIQ_LOADER has reader access plus controlled writes. PARTSIQ_REPORTER can write/read reporting events and read the report view. Each service identity has only its required role and warehouse usage. No application uses ACCOUNTADMIN, SECURITYADMIN or table-owner credentials. Loader permissions technically allow mutation, but application code rejects changing a sealed version; administrative bypass capability must be documented rather than called tamper-proof.

Validation and publication

Run duplicate-key diagnostics before mapping rows into dictionaries. Validate referential integrity, serial ranges, supersession graph, evidence and stock in Python and again through PostgreSQL constraints on activation. Never use RELY to optimize declared constraints unless the validation pipeline actually guarantees them. The full activation protocol is in DATA_OWNERSHIP.md. An unactivated remote version has no effect on customer search. Failed activation returns an actionable import report with row/field/code and preserves the previous active data.

Reporting and replay

Select at most100 pending analytics events from PostgreSQL. Redact to the defined reporting columns, then MERGE by EVENT_ID under a single exporter lease. Same EVENT_ID with conflicting payload is an integrity error, not a silent update. On retry, identical existing data is success. Acknowledge PostgreSQL only after Snowflake succeeds. Track query ID, row count, last success time and pending backlog. Do not export raw

text or identity data. Dashboard labels distinguish live operational counts from delayed Snowflake aggregates.

Cost and availability

The standard trial lasts30days or until free usage balance is exhausted, whichever happens first. This is not a permanent free tier. Confirm actual balance/expiry before creating or resuming a warehouse. Use one X-Small warehouse with AUTO_SUSPEND=60, AUTO_RESUME=true, INITIALLY_SUSPENDED=true and bounded statement timeouts only when allowance permits. Frequent wakeups still incur minimum resume billing; batch imports/reports and do not run warehouse queries during UI polling. Disable Cortex, Snowpipe, automatic clustering and other unneeded billable features.

A resource monitor is a guardrail, not a hard no-charge guarantee: warehouse suspension may lag and some serverless/cloud-service usage is outside its coverage. The application additionally checks the configured expiry and refuses scheduled remote work after it. Do not automatically attach a payment method or upgrade a trial. After expiry, the operator can select fixture mode explicitly or demonstrate a previously validated labelled snapshot under freshness policy; live Snowflake verification remains unavailable.

Verification checklist

Verify key-pair login with the correct account/role, one bounded SELECT, round-trip of the synthetic catalogue, duplicate-key rejection, serial-overlap rejection, checksum mismatch, remote success/local failure, retry without duplicate reporting, expired allowance, and UI freshness labels. Save query IDs and counts without secrets. Offline fixtures verify domain behavior but are not a substitute for these live checks.

Official references checked16 October 2026

- https://docs.snowflake.com/en/release-notes/clients-drivers/monthly-releases
- https://pypi.org/pypi/snowflake-connector-python/4.8.0/json
- https://docs.snowflake.com/en/developer-guide/python-connector/python-connector-api
- https://docs.snowflake.com/en/developer-guide/python-connector/python-connector-upgrade
- https://docs.snowflake.com/en/sql-reference/constraints
- https://docs.snowflake.com/en/user-guide/admin-trial-account
- https://docs.snowflake.com/en/user-guide/resource-monitors
- https://docs.snowflake.com/en/user-guide/warehouses-considerations

014 IMPLEMENTATION RECIPES

docs/IMPLEMENTATION_RECIPES.md

Implementation recipes

Fixed transition dispatch

Define STEP_HANDLERS as a static mapping from WorkflowStep to an injected async function. A handler receives typed current state and returns StepOutcome. next_step uses explicit conditions from WORKFLOW.md and rejects a transition not in the allowed table. There is no prompt that asks a model which function to call. During early foundation work, inject deterministic test handlers; wire production handlers when their owning foundations exist. Avoid importing future modules merely to satisfy a type annotation.

The runner calculates the input hash before work, checks workflow_step_results for a matching completed value, and performs external calls outside a row lock. On completion, it uses a transaction with SELECT FOR UPDATE on the enquiry and verifies job lease token, current revision and demo generation. Only then can it insert step results and advance state. A failed check returns obsolete and discards output. Tests pause a fake provider call, change the revision, and then release the call to prove stale results cannot commit.

Idempotency transaction

Hash canonical request JSON together with authenticated actor, HTTP method and normalized route path. Insert an idempotency record under its unique scope/key. If the existing hash differs, return409. If the operation is completed, replay its stored status/body. If it is still in progress, return a stable conflict/retry response rather than running it again. The business write, job creation, audit and saved response occur in one transaction. A process crash before commit leaves no partial accepted request. External provider calls occur later in jobs, not inside this transaction.

Catalogue hash and identity

Sort records by(entity_type, entity_key). Serialize each payload as UTF-8 JSON with sorted keys, no insignificant whitespace, decimal strings and normalized timestamp format. Exclude generated search_document and volatile import timestamps. Hash the complete canonical record stream with SHA256. Validate unique keys before constructing a lookup dictionary. Reusing VERSION_ID with different content is an integrity error. Preserve leading zeros in serial text while separately validating numeric ranges. Numeric IDs for new entities are allocated before sealing the source version and retained on every retry.

Compatibility and ranking

Resolve exactly one model configuration. Load only the active version's explicit part_compatibility rows for that configuration. Reject missing source references. Pass that allowed ID set into lexical and semantic candidate retrieval so an incompatible perfect textual match never reaches scoring. Combine scores using the characterized legacy formula and deterministic tie order. Calculate the best-versus-runner-up margin before truncating results. A request for limit1 does not remove evidence of ambiguity. Use code-owned thresholds and labelled evaluation cases; do not call a score a calibrated probability.

Quantity allocation

Build total demand per part across all enquiry items. For each part, read the same stock snapshot, calculate available=max(0, on_hand-reserved) only after validating the source values, and allocate local stock first. Sort reachable warehouses deterministically by approved transfer duration and stable warehouse ID. Never invent a missing route. Subtract each allocation from one shared in-memory availability ledger before allocating another item. Report shortage explicitly. A planner does not mutate stock or create a reservation. Approval reruns the checks against the current snapshot and freshness policy.

Fact-bound reply

The LLM returns only ReplySlots. Code renders part numbers, quantities, warehouses, estimates and references from VerifiedFacts. Reject or replace model slots containing unsupported identifiers or quantitative claims. Serialize and hash the verified facts independently of prose. Store the draft with revision, catalogue version and fact hash. An edit cannot directly set approval. Approval locks and revalidates the current state before inserting decision, recommendation and audit together. A known database number that is incompatible still fails verification.

Redacted reporting outbox

Write a small analytical event in the same business transaction. Export only EVENT_ID, EVENT_TYPE, OCCURRED_AT, IS_DEMO, STATUS, STAGE, DURATION_MS, INPUT_TOKENS, OUTPUT_TOKENS and

ESTIMATED_COST_USD. Reject any extra field before building SQL. MERGE by EVENT_ID under a single exporter lease using bound parameters. Compare identical existing data on replay; conflicting data is an integrity error. Save Snowflake query ID and acknowledge locally only after success. Never export raw customer text, serials, notes, photos, keys or signed URLs.

REST adapter example

A feature router declares response_model and explicit success status. Its verified actor dependency executes before the facade. The facade receives path IDs, typed command and idempotency key; it rejects mismatching enquiry IDs in the body. It invokes one domain function and maps named domain failures through the shared Problem handler. A browser sends only candidate ID or corrected fields, never an authoritative fitment fact. UI state is refreshed after writes. On409, it preserves unsent input and requires a fresh action against the new revision.

Test fixtures and replay

Use a fake clock, isolated database transaction, test actor factory and deterministic provider adapter. Keep at least two customers, two demo sessions, assigned/unassigned reviewers and an admin in permission fixtures. The four demonstration fixtures are not the entire evaluation dataset. Add independent Arabic variants, serial endpoints, wrong part numbers and duplicate-part allocation cases. Store expected compatible IDs and clarification outcomes from reviewed catalogue facts. Never change expected outputs simply because the implementation produced a different answer.

015 SECURITY

docs/SECURITY.md

Security and permission design

Trust boundaries

Browser requests are untrusted. Supabase tokens are verified for issuer, audience, signature, algorithm and expiry. Role membership is loaded server-side. Customer text, OCR text, imported documents and model replies are data, not instructions to privileged code. The AI API has no tools for writes or approval. Snowflake queries use bound values and fixed identifiers.

Access matrix

Resource	Customer or own demo	Assigned reviewer	Catalogue admin	Admin
Own enquiry/ messages/progress	Read and allowed input changes	Assigned real enquiry	No automatic access	Operational access
Candidates and full evidence	Own demo reviewer only	Assigned enquiry	No automatic access	Operational access
Approve/edit/reject	Own demo reviewer only	Assigned enquiry	No automatic access	Operational access
Internal notes/audit	Own demo reviewer only	Assigned enquiry	No automatic access	Operational access
Published catalogue	Read active data	Read	Stage/validate/publish	Stage/validate/publish
Assignments	No	No	No	Manage
AI allowance/ evaluation curation	No	No	No	Manage
Snowflake aggregate reports	No	No	No	Read

An invited user can hold multiple roles explicitly. A catalogue admin is not implicitly a reviewer. API access labels are defined in REST_API.md. PostgreSQL runtime groups are technical permissions and must not be mistaken for human application roles.

Database and file controls

Use RLS on every application table. anon/authenticated cannot access application tables directly except explicitly defined Storage policies. API uses a least-privilege LOGIN role with only the partsiq_api group; worker code selects purpose-specific pools with workflow/search/worker/catalogue groups. Do not use an owner/service-role database connection for ordinary business queries. Trusted jobs authorize by their persisted enquiry/revision and do not accept arbitrary user-selected table IDs.

Storage buckets are private. Use server-chosen object keys and short-lived signed URLs. Check actual image bytes and decoded size, strip metadata, and reject path traversal. Evidence/export download checks assignment or ownership at issuance; already issued URLs have a bounded validity window. PDF rendering blocks external network requests.

Abuse and reliability

Apply per-identity/IP request limits, upload size limits and bounded pagination. Redis limiter failure denies new rate-limited work instead of removing protection. Idempotency keys bind actor, method, path and payload hash. Worker leases prevent expired results overwriting new revisions. API CORS allows only explicit frontend origins. CSP and output escaping limit injected markup. Never display unescaped AI/customer text as HTML.

Audit rows are immutable to runtime roles; schema owners retain administrative capabilities, so describe this honestly. Store minimal before/after fields and redact secrets. Role changes, assignment, approval, publication, imports and allowance changes are auditable. Real retention requires a client decision before onboarding; demo access expires after24hours, with cleanup and minimal audit retention documented separately.

016 CONSTRAINTS

docs/CONSTRAINTS.md

Nonnegotiable constraints

• All 69 feature groups remain; voice and quotation builder stay excluded.

• One application package and fixed Python workflow. No LangGraph, supervisor, specialist-agent team or LLM-selected next steps.

• LLMs extract and draft only. Compatibility, stock allocation, verification and approval are code-controlled.

• Snowflake source versions are immutable and explicitly validated. PostgreSQL owns the active serving pointer and transactional application integrity.

• No incompatible candidate can enter the ranked set. Supersession does not imply fitment inheritance.

• Missing serials, unconfirmed OCR and close scores trigger clarification/review.

• Every final recommendation requires current human approval and audit; clarifying questions do not require approval.

• Real review is assignment scoped; demo review is own-session scoped. Demo generation/expiry are checked at final job commit.

• Private files stay private; customers receive compatibility references, staff receive authorized evidence viewer access.

• BGE-M3 runs only in the worker search module; lexical mode remains usable when semantic capability is disabled.

• No paid fallback. Live OpenAI, Snowflake and hosting require confirmed account-specific allowance. Zero budget is not a promise of free services forever.

• No order/reservation/payment/external message is created. Warehouse planning is an estimate from a timestamped snapshot.

• No production reset or automatic dependency upgrade. Migration-owner credentials never run the application.

• Audit is append-only for runtime roles, not mathematically tamper-proof against database owners.

• No raw prompts, photos, keys, JWTs or signed URLs in telemetry. Analytics exports only allowlisted fields.

• Claims about performance, accuracy, availability, client adoption and hosted status require actual receipts.

017 DECISIONS

docs/DECISIONS.md

Architecture decisions

ADR001 Extend the legacy backend
Accepted. Preserve the supplied backend archive and characterize its identification, lexical ranking and fulfilment behavior. Consolidate those modules under partsiq.search in the new package through explicit import/path adaptation. Do not rewrite working algorithms just to change framework shape. Original source remains available for comparison.

ADR002 Hosting and cost
Amended. Next.js remains on Vercel and Supabase remains managed. One EC2 instance runs Caddy, API, worker and Redis using Docker Compose only when confirmed AWS allowance covers the deployment. Render is a documented alternative, not the chosen deployment. No long-term free-hosting claim. A local fixture deployment remains supported.

ADR003 Replace multi-agent orchestration
Accepted16 October 2026. User requested the complete pre-multi-agent feature set with a simpler runtime. Remove LangGraph, checkpoint-library tables, supervisor routing, specialist agents and internal search/orchestrator HTTP services. Use ordinary Python functions, fixed transitions, PostgreSQL step results and durable leased jobs. This changes implementation architecture, not feature scope.

ADR004 Snowflake source with validated relational serving
Accepted for this revision. Snowflake holds versioned source catalogue/inventory and reporting data. Supabase PostgreSQL stores application records and a validated serving mirror. Standard Snowflake PK/FK declarations do not enforce uniqueness/referential integrity; publication validation and PostgreSQL constraints are retained. Activation is atomic locally; cross-system upload is recoverable and not falsely described as a distributed transaction.

ADR005 Single active source mode
Accepted. DATA_SOURCE is fixture or snowflake and must be explicit. A live provider error never silently changes source. Both modes use identical validators. UI and receipts display provenance and freshness. Fixture results cannot count as live provider evidence.

ADR006 Scope and roles
Accepted. All 69 groups retained with one detailed feature implementation prompt each. No voice or quotation builder. Public demo uses anonymous Supabase users and session isolation; real users are invited and real onboarding stays disabled until retention is configured. Reviewers access assigned enquiries; admins manage assignment.

ADR007 No paid fallback
Accepted. User supplied free keys and a zero-paid-spend budget. The application reserves only confirmed available allowance with expiry and does not infer credit balance from key validity. Snowflake trial and cloud account eligibility are operator preflight items. No new paid provider is selected automatically.

018 DISCREPANCIES

docs/DISCREPANCIES.md

Changes from the previous kit

Previous statement	Revised decision	Files affected
LangGraph and specialist workflow	Fixed Python transition table and durable step results	Workflow, foundations09–16, contracts, dependencies
Separate gateway/search/orchestrator deployments	One FastAPI package and one worker image	Architecture, paths, containers, CI, deployment
PostgreSQL-only source catalogue	Snowflake immutable source plus validated PostgreSQL serving mirror	Catalogue, SQL, source adapters, imports, publication
Checkpointer library schema	workflow_runs and workflow_step_ results	SQL 006; old006/007removed
Some F prompts only verified foundations	Every F prompt now implements its feature surface and tests	All 69 feature prompts and traces
Analytics from only operational records	Redacted outbox to Snowflake; scoped live counts stay PostgreSQL	Analytics, metrics, privacy, roles
Cloud setup described as free	Confirm allowance/expiry; explicit local fixture profile	Cost, deployment, preflight
Agent in AGENTS.md could imply runtime agents	Coding-assistant instructions only	AGENTS.md, CLAUDE.md, context
Smaller MVP suggested in conversation	Rejected; full69 scope retained	PRD, feature matrix, build order

The original backend and original brand archive remain historical references. Their internal paths do not override this edition's application tree. Never execute historical deployment manifests directly without adapting them through the current prompts.

019 DEPENDENCIES

docs/DEPENDENCIES.md

Dependencies and installation

Direct package releases were verified from official registries in the original kit and Snowflake was verified for this revision. Existing pins are deliberately preserved except removed agent/checkpoint libraries and the added Snowflake connector. This is not a vulnerability certification. Python resolution was rerun for this edition; the frozen frontend manifest/lock is retained. Run actual target-runtime builds in CI.

Package	Exact version	Release date	Install owner	Purpose
fastapi	0.135.1	2026-03-01	00	Typed REST routing and generated OpenAPI
uvicorn	0.41.0	2026-02-16	00	ASGI process
pydantic	2.12.5	2025-11-26	00	Strict DTOs and domain contracts
pydantic-settings	2.13.1	2026-02-19	00	Validated environment configuration
httpx	0.28.1	2024-12-06	00	Bounded HTTP clients
asyncpg	0.31.0	2025-11-24	00	PostgreSQL transactions and serving queries
psycopg	3.3.3	2026-02-18	00	Schema/operator utilities
redis	7.3.0	2026-03-06	00	Rate limits and wakeups
openai	3.19.2	2026-09-24	00	Structured text and image calls
structlog	25.5.0	2025-10-27	00	Redacted JSON logs
tenacity	9.1.2	2025-04-02	00	Bounded retry support
PyJWT	2.12.1	2026-03-13	00	JWT verification
cryptography	46.0.5	2026-02-10	00	Key material and cryptographic primitives
sentence-transformers	5.2.3	2026-02-17	00	BGE-M3 wrapper
transformers	4.57.6	2026-01-16	00	Model loading
torch	2.10.0	2026-01-21	00	CPU inference only
numpy	2.4.2	2026-01-31	00	Vector checks and ranking
Jinja2	3.1.6	2025-03-05	00	Escaped export HTML
Pillow	12.1.1	2026-02-11	00	Image verification and sanitation
playwright	1.58.0	2026-01-30	00	Chromium PDF rendering
openpyxl	3.1.5	2024-06-28	00	Nonexecuting XLSX parsing
PyYAML	6.0.3	2025-09-25	00	Configuration tools
pytest	9.0.2	2025-12-06	00	Backend tests
pytest-asyncio	1.3.0	2025-11-10	00	Async backend tests
ruff	0.15.6	2026-03-12	00	Python lint
mypy	1.19.1	2025-12-15	00	Python type checks
uv	0.10.9	2026-03-06	00	Frontend application, styling, types or test tooling as declared in web-package.json
langfuse	3.14.5	2026-02-23	00	Optional redacted traces
python-docx	1.2.0	2025-06-16	00	Frontend application, styling, types or test tooling as declared in

Package	Exact version	Release date	Install owner	Purpose
				web-package.json
pypdf	6.8.0	2026-03-09	00	PDF test extraction
python-multipart	0.0.22	2026-01-25	00	Upload-related form parsing
next	16.2.12	2026-07-25	18	Frontend application, styling, types or test tooling as declared in web-package.json
react	19.2.6	2026-05-06	18	Frontend application, styling, types or test tooling as declared in web-package.json
react-dom	19.2.6	2026-05-06	18	Frontend application, styling, types or test tooling as declared in web-package.json
typescript	5.9.3	2025-09-30	18	Frontend application, styling, types or test tooling as declared in web-package.json
tailwindcss	4.1.18	2025-12-11	18	Frontend application, styling, types or test tooling as declared in web-package.json
@tailwindcss/postcss	4.1.18	2025-12-11	18	Frontend application, styling, types or test tooling as declared in web-package.json
postcss	8.5.6	2025-06-16	18	Frontend application, styling, types or test tooling as declared in web-package.json
@supabase/supabase-js	2.99.1	2026-03-11	18	Frontend application, styling, types or test tooling as declared in web-package.json
@supabase/ssr	0.8.0	2025-11-26	18	Frontend application, styling, types or test tooling as declared in web-package.json
next-intl	4.8.3	2026-02-16	18	Frontend application, styling, types or test tooling as declared in web-package.json
zod	4.6.5	2026-09-13	18	Frontend application, styling, types or test tooling as declared in web-package.json
vitest	4.1.11	2026-08-18	18	Frontend application, styling, types or test tooling as declared in web-package.json
@playwright/test	1.58.2	2026-02-06	18	Frontend application, styling, types or test tooling as declared in web-package.json
eslint	9.39.1	2025-11-03	18	Frontend application, styling, types or test tooling as declared in web-package.json
eslint-config-next	16.2.12	2026-07-25	18	Frontend application, styling, types or test tooling as declared in web-package.json
@testing-library/react	16.3.0	2025-04-02	18	Frontend application, styling, types or test

Package	Exact version	Release date	Install owner	Purpose
				tooling as declared in web-package.json
@testing-library/dom	10.4.1	2025-07-27	18	Frontend application, styling, types or test tooling as declared in web-package.json
@testing-library/jest-dom	6.9.1	2025-10-01	18	Frontend application, styling, types or test tooling as declared in web-package.json
@testing-library/user-event	14.6.1	2025-01-21	18	Frontend application, styling, types or test tooling as declared in web-package.json
jsdom	27.4.0	2025-12-26	18	Frontend application, styling, types or test tooling as declared in web-package.json
@vitejs/plugin-react	5.1.4	2026-02-10	18	Frontend application, styling, types or test tooling as declared in web-package.json
@types/react	19.2.14	2026-02-11	18	Frontend application, styling, types or test tooling as declared in web-package.json
@types/react-dom	19.2.3	2025-11-12	18	Frontend application, styling, types or test tooling as declared in web-package.json
@types/node	22.19.15	2026-03-06	18	Frontend application, styling, types or test tooling as declared in web-package.json
clsx	2.1.1	2024-04-23	18	Frontend application, styling, types or test tooling as declared in web-package.json
tailwind-merge	3.5.0	2026-02-18	18	Frontend application, styling, types or test tooling as declared in web-package.json
class-variance-authority	0.7.1	2024-11-26	18	Frontend application, styling, types or test tooling as declared in web-package.json
lucide-react	0.577.0	2026-03-04	18	Frontend application, styling, types or test tooling as declared in web-package.json
radix-ui	1.4.3	2025-08-13	18	Frontend application, styling, types or test tooling as declared in web-package.json
shadcn	4.20.1	2026-09-02	18	Frontend application, styling, types or test tooling as declared in web-package.json
pnpm	10.30.3	2026-02-26	18	Frontend application, styling, types or test tooling as declared in web-package.json
supabase	2.75.0	2026-02-02	18	Frontend application, styling, types or test tooling as declared in web-package.json

Package	Exact version	Release date	Install owner	Purpose
snowflake-connector-python	4.8.0	2026-09-30	02	Parameterized Snowflake catalogue/ reporting access

Commands

Foundation 00: uv venv --python 3.12, then uv pip sync spec/python-requirements.lock. Copy the selected runtime dependencies into pyproject.toml and generate the project lock without changing pins. The supplied reference lock includes test/tool dependencies for reproducibility; production image may use a derived runtime subset with the same versions. Pin the exact CPU torch wheel URL/hash from the lock; do not install CUDA packages on the CPU host.

Foundation 18: copy spec/web-package.json to apps/web/package.json and spec/pnpm-lock.yaml to apps/web/pnpm-lock.yaml; run corepack pnpm --dir apps/web install --frozen-lockfile. Node 22.22.0 is the target. The earlier frontend resolver ran on Node 24; the target build still needs execution. Use packageManager pnpm10.30.3. PythonPlaywright1.58.0 and webPlaywright1.58.2 are separate installations; install each browser binary with its own CLI and never share an unversioned browser cache between them.

Foundation 02 uses snowflake-connector-python4.8.0 already included in the reference lock. No Snowpark, Pandas, agent framework or Cortex SDK is needed. Langfuse is optional and never on the business-critical path. Do not restore LangGraph through a transitive convenience dependency.

Upgrade policy

Before a dependency change, record the reason, official release date, target-runtime compatibility and security impact. Resolve the complete lock, run focused regression tests and document the new receipts. A package existing on a registry is not proof that the application is compatible or secure. Do not claim all transitives meet the14-day release criterion unless each has been independently verified; the explicit criterion applies to the direct pins in the receipt table.

020 ENVIRONMENT

docs/ENVIRONMENT.md

Environment contract

Empty secret/account values are intentionally required operator inputs, not fabricated credentials. Local fixture mode needs local Supabase values but no Snowflake/OpenAI key. Copy environment.example to ignored .env during implementation.

Variable	Default	Consumer	Meaning
APP_ENV	local	API and worker	local, dev or prod; tests reject prod
DATA_SOURCE	fixture	Worker	fixture or snowflake; never automatic fallback
AI_MODE	fixture	Worker	fixture or live; fixture banner mandatory
REAL_ONBOARDING_ ENABLED	false	API	Remain false until client retention is agreed
DATABASE_URL_API	Required when applicable	API	Least-privilege LOGIN with partsiq_api
DATABASE_URL_WORKFLOW	Required when applicable	Worker	LOGIN with partsiq_workflow
DATABASE_URL_SEARCH	Required when applicable	Worker	LOGIN with partsiq_search
DATABASE_URL_WORKER	Required when applicable	Worker	LOGIN with partsiq_worker
DATABASE_URL_CATALOGUE	Required when applicable	Worker	LOGIN with partsiq_catalogue
DATABASE_URL_MIGRATION	Required when applicable	Migration command only	Schema owner; never runtime
SUPABASE_URL	Required when applicable	API and web	Development or production project URL
SUPABASE_ANON_KEY	Required when applicable	Web	Public browser key; authorization still required
SUPABASE_SERVICE_ROLE_ KEY	Required when applicable	Storage adapter only	Secret; never used for ordinary database queries
SUPABASE_JWT_ISSUER	Required when applicable	API	Exact configured auth issuer
SUPABASE_JWT_AUDIENCE	authenticated	API	Exact JWT audience
REDIS_URL	redis://redis:6379/0	API and worker	Private Compose network
ALLOWED_ORIGINS	http://localhost:3000	API	Comma-separated exact origins
OPENAI_API_KEY	Required when applicable	Worker	Secret live key
OPENAI_TEXT_MODEL	Required when applicable	Worker	Owner-approved available structured-output model
OPENAI_VISION_MODEL	Required when applicable	Worker	Owner-approved available image model
AI_ALLOWANCE_EXPIRES_AT	Required when applicable	Preflight	Owner-confirmed expiry in UTC; no invented value
AI_CONFIRMED_ALLOWANCE_ USD	0	Preflight	Dedicated free allowance; zero blocks live calls
SNOWFLAKE_ACCOUNT	Required when applicable	Worker	Account identifier without domain suffix
SNOWFLAKE_USER_READER	Required when applicable	Worker	Read-only service user
SNOWFLAKE_USER_LOADER	Required when applicable	Publisher	Publisher service user
SNOWFLAKE_USER_ REPORTER	Required when applicable	Analytics worker	Reporting service user
SNOWFLAKE_PRIVATE_KEY_ FILE	Required when applicable	Worker	Mounted secret file; restricted permissions
SNOWFLAKE_PRIVATE_KEY_ PASSWORD	Required when applicable	Worker	Secret optional key passphrase
SNOWFLAKE_DATABASE	PARTSIQ_DEV	Worker	Allowlist PARTSIQ_DEV or PARTSIQ_PROD
SNOWFLAKE_WAREHOUSE	PARTSIQ_WH	Worker	Confirmed allowance required
SNOWFLAKE_CREDIT_	0	Preflight	Zero blocks warehouse

Variable	Default	Consumer	Meaning
ALLOWANCE			creation/resume
SNOWFLAKE_ALLOWANCE_EXPIRES_AT	Required when applicable	Preflight	Confirmed UTC expiry
SEMANTIC_SEARCH_ENABLED	false	Worker	Enable only after memory/capability check
BGE_MODEL_REVISION	Required when applicable	Worker	Copy verified immutable revision from infrastructure receipts
STOCK_MAX_AGE_SECONDS	3600	Domain	Provisional policy; real client chooses
DEMO_SESSION_TTL_SECONDS	86400	API	24-hour access lifetime
DEMO_CLEANUP_GRACE_SECONDS	3600	Worker	Cleanup after access expiry plus one hour
JOB_LEASE_SECONDS	120	Worker	Heartbeat every30seconds
MAX_UPLOAD_BYTES	10485760	API and worker	10MiB
MAX_IMAGE_PIXELS	20000000	Worker	Decoded pixel budget
LANGFUSE_PUBLIC_KEY	Required when applicable	Telemetry	Optional
LANGFUSE_SECRET_KEY	Required when applicable	Telemetry	Optional secret
LANGFUSE_BASE_URL	Required when applicable	Telemetry	Optional approved endpoint
NEXT_PUBLIC_API_ORIGIN	http://localhost:8000	Web	Public backend origin
NEXT_PUBLIC_SUPABASE_URL	Required when applicable	Web	Matching project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Required when applicable	Web	Matching public auth key
API_DOMAIN	Required when applicable	Caddy	Existing owner-controlled domain for hosted HTTPS

Never place private keys in NEXT_PUBLIC variables. Preflight verifies separate dev/prod hosts and reports missing names without values. Environment seeding initializes allowance only when no existing account row exists; restarts must not reset consumed/reserved amounts. Allowance changes require admin audit.

021 TECH SPEC

docs/TECH_SPEC.md

Technical specification

1 Runtime and source tree

The application is a modular monolith: one Python package, one public ASGI application and a worker importing the same domain modules. The web application is a separate Next.js build. spec/file-ownership.json is the exact planned source inventory and names each file's sole creation prompt. Later feature prompts may modify shared router, localization and shell files while retaining their original creator. Package directories require __init__.py generated by bootstrap; they contain no alternate domain implementation.

2 Contracts and REST

Copy spec/contracts.py and spec/api_models.py into partsiq.core. REST_API.md and spec/openapi.json define all 77operations. Foundation function signatures are listed in each prompt. Validation is layered: wire schema, authorization, domain invariants, transactional constraints. No field becomes optional merely because a UI panel is inconvenient to implement.

3 Exact PostgreSQL schema

The following SQL files are the authoritative schema, including tables, columns, types, constraints, indexes, roles, functions, triggers and RLS. Apply in order to a fresh disposable Supabase environment. For an existing deployed database, compare actual schema and migrate additively; never replay CREATE TABLE against live data or drop historical tables without a reviewed migration. LangGraph checkpoint migrations are not part of this edition.

001_initial_schema.sql

```
-- PartsIQ initial schema.
-- Executed automatically by the postgres container on first initialisation (empty data volume).
-- Design principle: the database is the single source of truth for part compatibility.
-- AI components may rank and describe candidates, but never decide whether a part fits a machine.

BEGIN;

CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ------------------------------------------------------------------------------
-- Shared trigger functions
-- ------------------------------------------------------------------------------

CREATE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$;

-- ------------------------------------------------------------------------------
-- Equipment catalogue
-- ------------------------------------------------------------------------------

CREATE TABLE equipment_families (
    id          SMALLSERIAL PRIMARY KEY,
    code        TEXT NOT NULL UNIQUE CHECK (code ~ '^[A-Z0-9_]{2,32}$'),
    name_en     TEXT NOT NULL,
    name_ar     TEXT NOT NULL
);

CREATE TABLE equipment_models (
    id            SERIAL PRIMARY KEY,
    family_id     SMALLINT NOT NULL REFERENCES equipment_families (id),
    brand         TEXT NOT NULL,
    model_code    TEXT NOT NULL,
    display_name  TEXT NOT NULL,
    rated_output  TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (brand, model_code)
);

CREATE INDEX equipment_models_model_code_trgm_idx
    ON equipment_models USING gin (model_code gin_trgm_ops);

-- A configuration is a serial-number range of a model that shares one bill of materials.
```

```
-- Overlapping ranges for the same model are rejected so serial lookup is always unambiguous.
CREATE TABLE model_configurations (
    id            SERIAL PRIMARY KEY,
    model_id      INTEGER NOT NULL REFERENCES equipment_models (id) ON DELETE CASCADE,
    config_code   TEXT NOT NULL,
    description   TEXT NOT NULL,
    serial_start  BIGINT NOT NULL CHECK (serial_start >= 0),
    serial_end    BIGINT NOT NULL,
    CHECK (serial_start <= serial_end),
    UNIQUE (model_id, config_code),
    EXCLUDE USING gist (
        model_id WITH =,
        int8range(serial_start, serial_end, '[]') WITH &&
    )
);

-- ------------------------------------------------------------------------
-- Parts
-- ------------------------------------------------------------------------

CREATE TABLE parts (
    id               SERIAL PRIMARY KEY,
    part_number      TEXT NOT NULL UNIQUE,
    name_en          TEXT NOT NULL,
    name_ar          TEXT NOT NULL,
    description_en   TEXT NOT NULL DEFAULT '',
    category         TEXT NOT NULL,
    unit_of_measure  TEXT NOT NULL DEFAULT 'EA',
    is_active        BOOLEAN NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    search_document  TSVECTOR GENERATED ALWAYS AS (
        to_tsvector(
            'simple'::regconfig,
            part_number || ' ' || name_en || ' ' || name_ar || ' ' || description_en
        )
    ) STORED
);

CREATE INDEX parts_search_document_idx ON parts USING gin (search_document);
CREATE INDEX parts_part_number_trgm_idx ON parts USING gin (part_number gin_trgm_ops);
CREATE INDEX parts_category_idx ON parts (category);

CREATE TRIGGER parts_set_updated_at
    BEFORE UPDATE ON parts
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Informal names used by customers and staff, e.g. "black filter" or Arabic colloquial terms.
CREATE TABLE part_aliases (
    id        SERIAL PRIMARY KEY,
    part_id   INTEGER NOT NULL REFERENCES parts (id) ON DELETE CASCADE,
    alias     TEXT NOT NULL,
    language  TEXT NOT NULL CHECK (language IN ('en', 'ar')),
    UNIQUE (part_id, alias)
);

CREATE INDEX part_aliases_alias_trgm_idx ON part_aliases USING gin (alias gin_trgm_ops);

-- Embeddings are stored separately so the embedding model can be replaced without altering parts.
-- Dimension 1024 matches multilingual models such as BGE-M3, which support Arabic and English.
CREATE TABLE part_embeddings (
    part_id      INTEGER NOT NULL REFERENCES parts (id) ON DELETE CASCADE,
    model_name   TEXT NOT NULL,
    source_text  TEXT NOT NULL,
    embedding    VECTOR(1024) NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (part_id, model_name)
);

CREATE INDEX part_embeddings_hnsw_idx
    ON part_embeddings USING hnsw (embedding vector_cosine_ops);

-- The authoritative compatibility matrix. Every row must cite the document it was derived from.
CREATE TABLE part_compatibility (
    part_id            INTEGER NOT NULL REFERENCES parts (id) ON DELETE CASCADE,
    configuration_id   INTEGER NOT NULL REFERENCES model_configurations (id) ON DELETE CASCADE,
    quantity_per_unit  NUMERIC(10, 3) NOT NULL DEFAULT 1 CHECK (quantity_per_unit > 0),
    source_reference   TEXT NOT NULL,
    notes              TEXT,
    PRIMARY KEY (part_id, configuration_id)
);

CREATE INDEX part_compatibility_configuration_idx ON part_compatibility (configuration_id);

-- Each superseded part has exactly one direct successor; chains are resolved by recursive query.
CREATE TABLE part_supersessions (
    old_part_id     INTEGER PRIMARY KEY REFERENCES parts (id) ON DELETE CASCADE,
    new_part_id     INTEGER NOT NULL REFERENCES parts (id) ON DELETE CASCADE,
    effective_date  DATE NOT NULL,
    reason          TEXT,
    CHECK (old_part_id <> new_part_id)
```

```
);

CREATE INDEX part_supersessions_new_part_idx ON part_supersessions (new_part_id);

CREATE FUNCTION prevent_supersession_cycle() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF EXISTS (
        WITH RECURSIVE chain (part_id) AS (
            SELECT NEW.new_part_id
            UNION
            SELECT s.new_part_id
            FROM part_supersessions AS s
            JOIN chain AS c ON s.old_part_id = c.part_id
            WHERE s.old_part_id <> NEW.old_part_id
        )
        SELECT 1 FROM chain WHERE part_id = NEW.old_part_id
    ) THEN
        RAISE EXCEPTION 'Supersession % -> % would create a cycle', NEW.old_part_id, NEW.new_part_id
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER part_supersessions_no_cycles
    BEFORE INSERT OR UPDATE ON part_supersessions
    FOR EACH ROW EXECUTE FUNCTION prevent_supersession_cycle();

-- ---------------------------------------------------------------------------
-- Inventory
-- ---------------------------------------------------------------------------

CREATE TABLE warehouses (
    id      SMALLSERIAL PRIMARY KEY,
    code    TEXT NOT NULL UNIQUE CHECK (code ~ '^[A-Z0-9_]{2,16}$'),
    name    TEXT NOT NULL,
    region  TEXT NOT NULL
);

CREATE TABLE stock_levels (
    part_id            INTEGER NOT NULL REFERENCES parts (id) ON DELETE CASCADE,
    warehouse_id       SMALLINT NOT NULL REFERENCES warehouses (id) ON DELETE CASCADE,
    quantity_on_hand   INTEGER NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),
    quantity_reserved  INTEGER NOT NULL DEFAULT 0 CHECK (quantity_reserved >= 0),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (part_id, warehouse_id),
    CHECK (quantity_reserved <= quantity_on_hand)
);

CREATE TRIGGER stock_levels_set_updated_at
    BEFORE UPDATE ON stock_levels
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- ---------------------------------------------------------------------------
-- Enquiry workflow
-- ---------------------------------------------------------------------------

CREATE TYPE enquiry_status AS ENUM (
    'received',
    'processing',
    'needs_information',
    'awaiting_review',
    'approved',
    'rejected',
    'failed'
);

CREATE TABLE enquiries (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    channel             TEXT NOT NULL CHECK (channel IN ('chat', 'email', 'staff')),
    customer_reference  TEXT,
    raw_text            TEXT NOT NULL,
    detected_language   TEXT CHECK (detected_language IN ('en', 'ar', 'mixed')),
    extracted           JSONB NOT NULL DEFAULT '{}'::jsonb,
    configuration_id    INTEGER REFERENCES model_configurations (id),
    status              enquiry_status NOT NULL DEFAULT 'received',
    failure_reason      TEXT,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX enquiries_status_created_idx ON enquiries (status, created_at DESC);

CREATE TRIGGER enquiries_set_updated_at
    BEFORE UPDATE ON enquiries
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TABLE enquiry_attachments (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    enquiry_id   UUID NOT NULL REFERENCES enquiries (id) ON DELETE CASCADE,
```

```
    storage_key  TEXT NOT NULL UNIQUE,
    media_type   TEXT NOT NULL CHECK (media_type IN ('image/jpeg', 'image/png', 'image/webp')),
    size_bytes   INTEGER NOT NULL CHECK (size_bytes > 0 AND size_bytes <= 10485760),
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX enquiry_attachments_enquiry_idx ON enquiry_attachments (enquiry_id);

CREATE TABLE enquiry_candidates (
    enquiry_id     UUID NOT NULL REFERENCES enquiries (id) ON DELETE CASCADE,
    rank           SMALLINT NOT NULL CHECK (rank BETWEEN 1 AND 10),
    part_id        INTEGER NOT NULL REFERENCES parts (id),
    score          NUMERIC(6, 5) NOT NULL CHECK (score >= 0 AND score <= 1),
    match_reasons  JSONB NOT NULL DEFAULT '[]'::jsonb,
    PRIMARY KEY (enquiry_id, rank),
    UNIQUE (enquiry_id, part_id)
);

CREATE TABLE review_decisions (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    enquiry_id   UUID NOT NULL REFERENCES enquiries (id) ON DELETE CASCADE,
    reviewer     TEXT NOT NULL,
    decision     TEXT NOT NULL CHECK (decision IN ('approved', 'edited', 'rejected')),
    final_reply  TEXT,
    comment      TEXT,
    decided_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (decision = 'rejected' OR final_reply IS NOT NULL)
);

CREATE INDEX review_decisions_enquiry_idx ON review_decisions (enquiry_id, decided_at DESC);

CREATE TABLE review_approved_parts (
    decision_id  UUID NOT NULL REFERENCES review_decisions (id) ON DELETE CASCADE,
    part_id      INTEGER NOT NULL REFERENCES parts (id),
    quantity     NUMERIC(10, 3) NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (decision_id, part_id)
);

-- ---------------------------------------------------------------------------
-- Append-only audit trail
-- ---------------------------------------------------------------------------

CREATE TABLE audit_log (
    id           BIGSERIAL PRIMARY KEY,
    occurred_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    actor_type   TEXT NOT NULL CHECK (actor_type IN ('system', 'ai', 'user')),
    actor_id     TEXT NOT NULL,
    action       TEXT NOT NULL,
    entity_type  TEXT NOT NULL,
    entity_id    TEXT NOT NULL,
    request_id   TEXT,
    payload      JSONB NOT NULL DEFAULT '{}'::jsonb
);

CREATE INDEX audit_log_entity_idx ON audit_log (entity_type, entity_id, occurred_at DESC);

CREATE FUNCTION prevent_audit_mutation() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'audit_log is append-only' USING ERRCODE = 'insufficient_privilege';
END;
$$;

CREATE TRIGGER audit_log_no_update_delete
    BEFORE UPDATE OR DELETE ON audit_log
    FOR EACH ROW EXECUTE FUNCTION prevent_audit_mutation();

CREATE TRIGGER audit_log_no_truncate
    BEFORE TRUNCATE ON audit_log
    FOR EACH STATEMENT EXECUTE FUNCTION prevent_audit_mutation();

COMMIT;
```

002_search_and_fulfilment.sql

```
-- Search and fulfilment support.

BEGIN;

-- Canonical part-number key so that "vp of 1042", "VP-OF-1042" and "VPOF1042" all match.
-- Must remain consistent with partsiq_common.text.part_number_key.
ALTER TABLE parts
    ADD COLUMN part_number_key TEXT GENERATED ALWAYS AS (
        upper(regexp_replace(part_number, '[^A-Za-z0-9]', '', 'g'))
    ) STORED;

CREATE INDEX parts_part_number_key_idx ON parts (part_number_key);

-- Planned transfer lead times between warehouses, used to recommend where to fulfil from.
CREATE TABLE warehouse_transfer_routes (
```

```
    from_warehouse_id  SMALLINT NOT NULL REFERENCES warehouses (id) ON DELETE CASCADE,
    to_warehouse_id    SMALLINT NOT NULL REFERENCES warehouses (id) ON DELETE CASCADE,
    transfer_days      SMALLINT NOT NULL CHECK (transfer_days BETWEEN 1 AND 60),
    PRIMARY KEY (from_warehouse_id, to_warehouse_id),
    CHECK (from_warehouse_id <> to_warehouse_id)
);

COMMIT;
```

003_application.sql

```
-- Additive application schema. Apply after immutable legacy migrations 001 and 002.
BEGIN;
CREATE SCHEMA IF NOT EXISTS app_private;
REVOKE ALL ON SCHEMA app_private FROM PUBLIC;
CREATE TABLE profiles (
 id uuid PRIMARY KEY REFERENCES auth.users(id),
 display_name text NOT NULL CHECK(length(display_name) BETWEEN 1 AND 120),
 preferred_locale text NOT NULL DEFAULT 'en' CHECK(preferred_locale IN ('en','ar')),
 created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE staff_roles (
 user_id uuid NOT NULL REFERENCES profiles(id),
 role text NOT NULL CHECK(role IN ('reviewer','catalogue_admin','admin')),
 granted_by uuid REFERENCES profiles(id),
 granted_at timestamptz NOT NULL DEFAULT now(),
 PRIMARY KEY(user_id,role)
);
CREATE TABLE demo_sessions (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 owner_id uuid NOT NULL REFERENCES profiles(id),
 generation integer NOT NULL DEFAULT 1 CHECK(generation>0),
 expires_at timestamptz NOT NULL,
 revoked_at timestamptz,
 created_at timestamptz NOT NULL DEFAULT now(),
 CHECK(expires_at>created_at)
);
CREATE INDEX demo_sessions_owner_idx ON demo_sessions(owner_id,expires_at DESC);
CREATE TABLE catalogue_versions (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 label text NOT NULL UNIQUE,
 state text NOT NULL CHECK(state IN ('staging','published','retired')),
 content_sha256 text NOT NULL CHECK(content_sha256 ~ '^[a-f0-9]{64}$'),
 created_by uuid REFERENCES profiles(id),
 created_at timestamptz NOT NULL DEFAULT now(),
 published_at timestamptz,
 CHECK((state='staging' AND published_at IS NULL) OR (state IN ('published','retired') AND published_at
 IS NOT NULL))
);
CREATE UNIQUE INDEX catalogue_one_published_idx ON catalogue_versions((state)) WHERE state='published';
CREATE TABLE catalogue_records (
 version_id uuid NOT NULL REFERENCES catalogue_versions(id),
 entity_type text NOT NULL CHECK(entity_type IN ('equipment_families','equipment_models','model_configu
rations','parts','part_aliases','part_compatibility','part_supersessions','warehouses','warehouse_trans
fer_routes')),
 entity_key text NOT NULL,
 payload jsonb NOT NULL CHECK(jsonb_typeof(payload)='object'),
 PRIMARY KEY(version_id,entity_type,entity_key)
);
CREATE TABLE catalogue_evidence (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 version_id uuid NOT NULL REFERENCES catalogue_versions(id),
 source_reference text NOT NULL CHECK(length(trim(source_reference))>0),
 storage_key text NOT NULL,
 page_number integer NOT NULL CHECK(page_number>0),
 sha256 text NOT NULL CHECK(sha256 ~ '^[a-f0-9]{64}$'),
 synthetic boolean NOT NULL DEFAULT true,
 UNIQUE(version_id,source_reference)
);
ALTER TABLE part_compatibility ADD CONSTRAINT compatibility_nonempty_source
CHECK(length(trim(source_reference))>0);
ALTER TABLE part_embeddings ADD COLUMN model_revision text;
ALTER TABLE part_embeddings ADD COLUMN source_sha256 text;
ALTER TABLE enquiries ADD COLUMN owner_id uuid REFERENCES profiles(id);
ALTER TABLE enquiries ADD COLUMN session_id uuid REFERENCES demo_sessions(id);
ALTER TABLE enquiries ADD COLUMN session_generation integer;
ALTER TABLE enquiries ADD COLUMN revision integer NOT NULL DEFAULT 1 CHECK(revision>0);
ALTER TABLE enquiries ADD COLUMN catalogue_version_id uuid REFERENCES catalogue_versions(id);
ALTER TABLE enquiries ADD COLUMN destination_warehouse_id smallint REFERENCES warehouses(id);
ALTER TABLE enquiries ADD COLUMN assigned_to uuid REFERENCES profiles(id);
ALTER TABLE enquiries ADD COLUMN priority text NOT NULL DEFAULT 'normal' CHECK(priority IN
('normal','urgent'));
ALTER TABLE enquiries ADD COLUMN due_at timestamptz;
ALTER TABLE enquiries ADD COLUMN idempotency_key uuid;
ALTER TABLE enquiries ADD CONSTRAINT enquiry_demo_pair CHECK((session_id IS NULL AND session_generation
 IS NULL) OR (session_id IS NOT NULL AND session_generation>0));
CREATE UNIQUE INDEX enquiries_owner_idempotency_idx ON enquiries(owner_id,idempotency_key) WHERE
idempotency_key IS NOT NULL;
CREATE INDEX enquiries_owner_created_idx ON enquiries(owner_id,created_at DESC,id);
CREATE INDEX enquiries_assigned_status_idx ON enquiries(assigned_to,status,created_at DESC,id);
```

```
CREATE INDEX enquiries_demo_idx ON enquiries(session_id,session_generation);
CREATE TABLE enquiry_items (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 ordinal smallint NOT NULL CHECK(ordinal BETWEEN 1 AND 20),
 requested_text text NOT NULL CHECK(length(trim(requested_text))>0),
 quantity numeric(10,3) NOT NULL CHECK(quantity>0 AND quantity<=100000),
 model_text text,
 serial_text text,
 configuration_id integer REFERENCES model_configurations(id),
 selected_part_id integer REFERENCES parts(id),
 status text NOT NULL DEFAULT 'unresolved' CHECK(status IN
('unresolved','needs_information','ready','unavailable')),
 revision integer NOT NULL CHECK(revision>0),
 UNIQUE(enquiry_id,ordinal),
 UNIQUE(enquiry_id,id)
);
ALTER TABLE enquiry_candidates ADD COLUMN item_id uuid;
ALTER TABLE enquiry_candidates ADD COLUMN revision integer NOT NULL DEFAULT 1;
ALTER TABLE enquiry_candidates ADD COLUMN catalogue_version_id uuid REFERENCES catalogue_versions(id);
ALTER TABLE enquiry_candidates ADD COLUMN evidence_id uuid REFERENCES catalogue_evidence(id);
ALTER TABLE enquiry_candidates DROP CONSTRAINT enquiry_candidates_pkey;
ALTER TABLE enquiry_candidates DROP CONSTRAINT enquiry_candidates_enquiry_id_part_id_key;
-- Historical rows receive a legacy item before the new per-item keys are enforced.
INSERT INTO enquiry_items(enquiry_id,ordinal,requested_text,quantity,revision)
 SELECT id,1,CASE WHEN length(trim(raw_text))>0 THEN raw_text ELSE 'Historical enquiry' END,1,revision
FROM enquiries;
UPDATE enquiry_candidates c SET item_id=i.id FROM enquiry_items i WHERE i.enquiry_id=c.enquiry_id AND
i.ordinal=1;
ALTER TABLE enquiry_candidates ALTER COLUMN item_id SET NOT NULL;
ALTER TABLE enquiry_candidates ADD CONSTRAINT candidate_item_fk FOREIGN KEY(enquiry_id,item_id)
REFERENCES enquiry_items(enquiry_id,id);
ALTER TABLE enquiry_candidates ADD PRIMARY KEY(item_id,revision,rank);
ALTER TABLE enquiry_candidates ADD UNIQUE(item_id,revision,part_id);
ALTER TABLE enquiry_attachments ADD COLUMN status text NOT NULL DEFAULT 'pending' CHECK(status IN
('pending','validated','rejected','deleted'));
ALTER TABLE enquiry_attachments ADD COLUMN sha256 text;
ALTER TABLE enquiry_attachments ADD COLUMN revision integer NOT NULL DEFAULT 1;
CREATE TABLE messages (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL CHECK(revision>0),
 kind text NOT NULL CHECK(kind IN ('customer','clarification','status','recommendation')),
 body text NOT NULL CHECK(length(body) BETWEEN 1 AND 12000),
 author_id uuid REFERENCES profiles(id),
 decision_id uuid REFERENCES review_decisions(id),
 idempotency_key uuid NOT NULL,
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(enquiry_id,idempotency_key),
 CHECK((kind='recommendation' AND decision_id IS NOT NULL) OR (kind<>'recommendation' AND decision_id
IS NULL))
);
CREATE UNIQUE INDEX messages_one_publication_idx ON messages(enquiry_id,revision) WHERE
kind='recommendation';
CREATE INDEX messages_enquiry_created_idx ON messages(enquiry_id,created_at,id);
CREATE TABLE enquiry_revisions (
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL CHECK(revision>0),
 snapshot jsonb NOT NULL CHECK(jsonb_typeof(snapshot)='object'),
 changed_by uuid REFERENCES profiles(id),
 reason text NOT NULL,
 created_at timestamptz NOT NULL DEFAULT now(),
 PRIMARY KEY(enquiry_id,revision)
);
CREATE TABLE extractions (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 attachment_id uuid REFERENCES enquiry_attachments(id),
 revision integer NOT NULL,
 model_text text,
 serial_text text,
 issues jsonb NOT NULL DEFAULT '[]' CHECK(jsonb_typeof(issues)='array'),
 confirmed_by uuid REFERENCES profiles(id),
 confirmed_at timestamptz,
 created_at timestamptz NOT NULL DEFAULT now(),
 CHECK((confirmed_by IS NULL)=(confirmed_at IS NULL))
);
CREATE INDEX extractions_enquiry_idx ON extractions(enquiry_id,revision);
CREATE TABLE fulfilment_plans (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL,
 item_id uuid NOT NULL,
 requested_quantity numeric(10,3) NOT NULL CHECK(requested_quantity>0),
 allocations jsonb NOT NULL CHECK(jsonb_typeof(allocations)='array'),
 shortage numeric(10,3) NOT NULL CHECK(shortage>=0),
 stock_snapshot jsonb NOT NULL CHECK(jsonb_typeof(stock_snapshot)='array'),
 calculated_at timestamptz NOT NULL DEFAULT now(),
 FOREIGN KEY(enquiry_id,item_id) REFERENCES enquiry_items(enquiry_id,id),
 UNIQUE(item_id,revision)
);
```

```
CREATE TABLE drafts (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL,
 locale text NOT NULL CHECK(locale IN ('en','ar')),
 prose jsonb NOT NULL CHECK(jsonb_typeof(prose)='object'),
 verified_facts jsonb NOT NULL CHECK(jsonb_typeof(verified_facts)='object'),
 rendered_text text NOT NULL,
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(enquiry_id,revision)
);
CREATE TABLE verification_results (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL,
 draft_id uuid NOT NULL REFERENCES drafts(id),
 passed boolean NOT NULL,
 failures jsonb NOT NULL CHECK(jsonb_typeof(failures)='array'),
 facts_sha256 text NOT NULL CHECK(facts_sha256 ~ '^[a-f0-9]{64}$'),
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE(enquiry_id,revision)
);
ALTER TABLE review_decisions ADD COLUMN revision integer NOT NULL DEFAULT 1;
ALTER TABLE review_decisions ADD COLUMN reviewer_id uuid REFERENCES profiles(id);
ALTER TABLE review_decisions ADD COLUMN verification_id uuid REFERENCES verification_results(id);
ALTER TABLE review_decisions ADD COLUMN idempotency_key uuid;
CREATE UNIQUE INDEX review_decisions_idempotency_idx ON review_decisions(enquiry_id,idempotency_key)
WHERE idempotency_key IS NOT NULL;
ALTER TABLE review_approved_parts ADD COLUMN item_id uuid REFERENCES enquiry_items(id);
CREATE TABLE internal_notes (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 author_id uuid NOT NULL REFERENCES profiles(id), body text NOT NULL CHECK(length(trim(body)) BETWEEN 1
 AND 4000),
 created_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX internal_notes_enquiry_idx ON internal_notes(enquiry_id,created_at,id);
CREATE TABLE assignment_events (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 assigned_to uuid REFERENCES profiles(id), assigned_by uuid NOT NULL REFERENCES profiles(id),
 created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE saved_machines (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), owner_id uuid NOT NULL REFERENCES profiles(id),
 session_id uuid REFERENCES demo_sessions(id), session_generation integer,
 label text NOT NULL CHECK(length(trim(label)) BETWEEN 1 AND 120),
 model_id integer NOT NULL REFERENCES equipment_models(id), serial_text text NOT NULL,
 created_at timestamptz NOT NULL DEFAULT now(), archived_at timestamptz,
 CHECK((session_id IS NULL AND session_generation IS NULL) OR (session_id IS NOT NULL AND
session_generation>0))
);
CREATE INDEX saved_machines_owner_idx ON saved_machines(owner_id,created_at DESC);
CREATE TABLE missing_part_requests (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 item_id uuid NOT NULL, status text NOT NULL DEFAULT 'open' CHECK(status IN
('open','investigating','resolved','unavailable')),
 staff_note text NOT NULL DEFAULT '', resolved_by uuid REFERENCES profiles(id),
 created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now(),
 FOREIGN KEY(enquiry_id,item_id) REFERENCES enquiry_items(enquiry_id,id), UNIQUE(item_id)
);
CREATE TABLE jobs (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid REFERENCES enquiries(id),
 revision integer, session_generation integer,
 kind text NOT NULL CHECK(kind IN
('nameplate','export_pdf','catalog_import','catalog_publish','storage_cleanup')),
 status text NOT NULL DEFAULT 'pending' CHECK(status IN
('pending','running','succeeded','failed','cancelled')),
 payload jsonb NOT NULL CHECK(jsonb_typeof(payload)='object'),
 result jsonb, dedupe_key text NOT NULL UNIQUE,
 attempts integer NOT NULL DEFAULT 0 CHECK(attempts>=0), max_attempts integer NOT NULL DEFAULT 3
CHECK(max_attempts BETWEEN 1 AND 5),
 available_at timestamptz NOT NULL DEFAULT now(), lease_token uuid, lease_expires_at timestamptz,
 last_error_code text, created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL
DEFAULT now(),
 CHECK((status='running' AND lease_token IS NOT NULL AND lease_expires_at IS NOT NULL) OR
status<>'running')
);
CREATE INDEX jobs_claim_idx ON jobs(status,available_at,lease_expires_at);
CREATE TABLE workflow_events (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid NOT NULL REFERENCES enquiries(id),
revision integer NOT NULL,
 kind text NOT NULL CHECK(kind IN ('start','message','ocr_completed','approved','retry')),
 payload jsonb NOT NULL DEFAULT '{}', dedupe_key text NOT NULL UNIQUE,
 processed_at timestamptz, lease_token uuid, lease_expires_at timestamptz,
 created_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX workflow_events_pending_idx ON workflow_events(created_at) WHERE processed_at IS NULL;
CREATE TABLE exports (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL, decision_id uuid NOT NULL REFERENCES review_decisions(id),
 job_id uuid NOT NULL REFERENCES jobs(id), storage_key text UNIQUE, sha256 text,
 status text NOT NULL CHECK(status IN ('pending','ready','failed','deleted')),
```

```
 created_at timestamptz NOT NULL DEFAULT now(), UNIQUE(enquiry_id,revision)
);
CREATE TABLE import_batches (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), created_by uuid NOT NULL REFERENCES profiles(id),
 version_id uuid REFERENCES catalogue_versions(id), job_id uuid REFERENCES jobs(id),
 storage_key text NOT NULL, sha256 text NOT NULL, status text NOT NULL CHECK(status IN
('uploaded','validating','invalid','valid','published')),
 row_count integer NOT NULL DEFAULT 0 CHECK(row_count BETWEEN 0 AND 50000),
 created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE import_rows (
 batch_id uuid NOT NULL REFERENCES import_batches(id), row_number integer NOT NULL CHECK(row_number>0),
 entity_type text NOT NULL, payload jsonb NOT NULL, errors jsonb NOT NULL DEFAULT '[]',
 PRIMARY KEY(batch_id,row_number)
);
CREATE TABLE ai_budget_accounts (
 id text PRIMARY KEY, cap_usd numeric(14,8) NOT NULL CHECK(cap_usd>=0),
 reserved_usd numeric(14,8) NOT NULL DEFAULT 0 CHECK(reserved_usd>=0),
 consumed_usd numeric(14,8) NOT NULL DEFAULT 0 CHECK(consumed_usd>=0),
 expires_at timestamptz NOT NULL, enabled boolean NOT NULL DEFAULT false,
 CHECK(reserved_usd+consumed_usd<=cap_usd)
);
CREATE TABLE ai_usage (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid REFERENCES enquiries(id),
 request_key uuid NOT NULL UNIQUE, budget_account_id text NOT NULL REFERENCES ai_budget_accounts(id),
 model text NOT NULL, stage text NOT NULL, status text NOT NULL CHECK(status IN
('reserved','completed','uncertain','released')),
 reserved_usd numeric(14,8) NOT NULL CHECK(reserved_usd>=0), actual_usd numeric(14,8)
CHECK(actual_usd>=0),
 input_tokens integer CHECK(input_tokens>=0), output_tokens integer CHECK(output_tokens>=0),
 duration_ms integer CHECK(duration_ms>=0), provider_request_id text,
 created_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ai_usage_created_idx ON ai_usage(created_at,stage);
CREATE TABLE eval_cases (
 id text PRIMARY KEY, scenario text NOT NULL, input jsonb NOT NULL, expected jsonb NOT NULL,
 catalogue_version_id uuid NOT NULL REFERENCES catalogue_versions(id),
 language text NOT NULL CHECK(language IN ('en','ar','mixed')), approved_by uuid REFERENCES
profiles(id)
);
CREATE TABLE eval_runs (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), git_sha text NOT NULL, model text NOT NULL,
 dataset_sha256 text NOT NULL, started_at timestamptz NOT NULL DEFAULT now(), completed_at timestamptz,
 status text NOT NULL CHECK(status IN ('running','passed','failed'))
);
CREATE TABLE eval_results (
 run_id uuid NOT NULL REFERENCES eval_runs(id), case_id text NOT NULL REFERENCES eval_cases(id),
 actual jsonb NOT NULL, passed boolean NOT NULL, duration_ms integer NOT NULL CHECK(duration_ms>=0),
 PRIMARY KEY(run_id,case_id)
);
CREATE TABLE correction_records (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), enquiry_id uuid NOT NULL REFERENCES enquiries(id),
 revision integer NOT NULL, corrected_by uuid NOT NULL REFERENCES profiles(id),
 before_value jsonb NOT NULL, after_value jsonb NOT NULL, reason text NOT NULL,
 eval_case_id text REFERENCES eval_cases(id), created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE enquiry_summaries (
 enquiry_id uuid NOT NULL REFERENCES enquiries(id), revision integer NOT NULL,
 summary text NOT NULL, source_message_ids uuid[] NOT NULL,
 created_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY(enquiry_id,revision)
);
CREATE TABLE retention_policies (
 scope text PRIMARY KEY CHECK(scope IN ('demo','client')),
 enquiry_days integer CHECK(enquiry_days BETWEEN 1 AND 3650),
 upload_days integer CHECK(upload_days BETWEEN 1 AND 3650),
 audit_days integer CHECK(audit_days BETWEEN 1 AND 3650),
 configured_by uuid REFERENCES profiles(id), configured_at timestamptz NOT NULL DEFAULT now()
);
INSERT INTO retention_policies(scope,enquiry_days,upload_days,audit_days) VALUES
('demo',1,1,NULL),('client',NULL,NULL,NULL);
-- NULL client values disable live-client onboarding, not indefinite silent consent.
ALTER TABLE audit_log ADD COLUMN enquiry_id uuid REFERENCES enquiries(id);
ALTER TABLE audit_log ADD COLUMN revision integer;
CREATE INDEX audit_log_enquiry_idx ON audit_log(enquiry_id,occurred_at DESC);
CREATE FUNCTION app_private.lock_supersessions() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN PERFORM pg_advisory_xact_lock(710069); RETURN NEW; END; $$;
CREATE TRIGGER aa_supersessions_lock BEFORE INSERT OR UPDATE ON part_supersessions FOR EACH ROW EXECUTE
 FUNCTION app_private.lock_supersessions();
CREATE FUNCTION app_private.immutable_record() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN RAISE EXCEPTION 'immutable historical record' USING ERRCODE='42501'; END; $$;
CREATE TRIGGER catalogue_records_immutable BEFORE UPDATE OR DELETE ON catalogue_records FOR EACH ROW
EXECUTE FUNCTION app_private.immutable_record();
CREATE TRIGGER revisions_immutable BEFORE UPDATE OR DELETE ON enquiry_revisions FOR EACH ROW EXECUTE
FUNCTION app_private.immutable_record();
CREATE TABLE idempotency_records (
 actor_id uuid NOT NULL REFERENCES profiles(id),
 operation text NOT NULL,
 key uuid NOT NULL,
 request_sha256 text NOT NULL CHECK(request_sha256 ~ '^[a-f0-9]{64}$'),
 response jsonb,
 created_at timestamptz NOT NULL DEFAULT now(),
```

```
 PRIMARY KEY(actor_id,operation,key)
);
COMMIT;
```

004_rls.sql

```
-- Runtime roles are NOLOGIN groups. The operator creates distinct LOGIN roles and grants one group per
 service.
BEGIN;
CREATE ROLE partsiq_api NOLOGIN;
CREATE ROLE partsiq_search NOLOGIN;
CREATE ROLE partsiq_workflow NOLOGIN;
CREATE ROLE partsiq_worker NOLOGIN;
CREATE ROLE partsiq_catalogue NOLOGIN;
GRANT USAGE ON SCHEMA public,app_private TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker,partsiq_catalogue;
CREATE FUNCTION app_private.actor() RETURNS uuid LANGUAGE sql STABLE AS $$
 SELECT nullif(current_setting('app.actor_id',true),'')::uuid;
$$;
CREATE FUNCTION app_private.has_role(wanted text) RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER
SET search_path=public,pg_temp AS $$
 SELECT EXISTS(SELECT 1 FROM staff_roles WHERE user_id=app_private.actor() AND role=wanted);
$$;
CREATE FUNCTION app_private.demo_active(sid uuid, gen integer, uid uuid) RETURNS boolean LANGUAGE sql
STABLE SECURITY DEFINER SET search_path=public,pg_temp AS $$
 SELECT EXISTS(SELECT 1 FROM demo_sessions WHERE id=sid AND owner_id=uid AND generation=gen AND
revoked_at IS NULL AND expires_at>now());
$$;
CREATE FUNCTION app_private.can_read_enquiry(eid uuid) RETURNS boolean LANGUAGE sql STABLE SECURITY
DEFINER SET search_path=public,pg_temp AS $$
 SELECT EXISTS(SELECT 1 FROM enquiries e WHERE e.id=eid AND (
 (e.owner_id=app_private.actor() AND (e.session_id IS NULL OR
app_private.demo_active(e.session_id,e.session_generation,e.owner_id)))
 OR (e.session_id IS NULL AND (app_private.has_role('admin') OR (e.assigned_to=app_private.actor() AND
app_private.has_role('reviewer'))))));
$$;
CREATE FUNCTION app_private.can_review(eid uuid) RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER
SET search_path=public,pg_temp AS $$
 SELECT EXISTS(SELECT 1 FROM enquiries e WHERE e.id=eid AND (
 (e.session_id IS NOT NULL AND e.owner_id=app_private.actor() AND
app_private.demo_active(e.session_id,e.session_generation,e.owner_id))
 OR (e.session_id IS NULL AND (app_private.has_role('admin') OR (e.assigned_to=app_private.actor() AND
app_private.has_role('reviewer'))))));
$$;
REVOKE ALL ON ALL FUNCTIONS IN SCHEMA app_private FROM PUBLIC;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA app_private TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker,partsiq_catalogue;
ALTER TABLE equipment_families ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON equipment_families FROM anon,authenticated;
GRANT SELECT ON equipment_families TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY equipment_families_read ON equipment_families FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON equipment_families TO partsiq_catalogue;
CREATE POLICY equipment_families_publisher ON equipment_families TO partsiq_catalogue USING (true) WITH
 CHECK (true);
ALTER TABLE equipment_models ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON equipment_models FROM anon,authenticated;
GRANT SELECT ON equipment_models TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY equipment_models_read ON equipment_models FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON equipment_models TO partsiq_catalogue;
CREATE POLICY equipment_models_publisher ON equipment_models TO partsiq_catalogue USING (true) WITH
CHECK (true);
ALTER TABLE model_configurations ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON model_configurations FROM anon,authenticated;
GRANT SELECT ON model_configurations TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY model_configurations_read ON model_configurations FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON model_configurations TO partsiq_catalogue;
CREATE POLICY model_configurations_publisher ON model_configurations TO partsiq_catalogue USING (true)
WITH CHECK (true);
ALTER TABLE parts ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON parts FROM anon,authenticated;
GRANT SELECT ON parts TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY parts_read ON parts FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON parts TO partsiq_catalogue;
CREATE POLICY parts_publisher ON parts TO partsiq_catalogue USING (true) WITH CHECK (true);
ALTER TABLE part_aliases ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON part_aliases FROM anon,authenticated;
GRANT SELECT ON part_aliases TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY part_aliases_read ON part_aliases FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON part_aliases TO partsiq_catalogue;
CREATE POLICY part_aliases_publisher ON part_aliases TO partsiq_catalogue USING (true) WITH CHECK
(true);
ALTER TABLE part_embeddings ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON part_embeddings FROM anon,authenticated;
GRANT SELECT ON part_embeddings TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY part_embeddings_read ON part_embeddings FOR SELECT TO
```

```
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON part_embeddings TO partsiq_catalogue;
CREATE POLICY part_embeddings_publisher ON part_embeddings TO partsiq_catalogue USING (true) WITH CHECK
 (true);
GRANT INSERT,UPDATE ON part_embeddings TO partsiq_search;
CREATE POLICY part_embeddings_encoder ON part_embeddings FOR INSERT TO partsiq_search WITH CHECK(true);
CREATE POLICY part_embeddings_encoder_update ON part_embeddings FOR UPDATE TO partsiq_search
USING(true) WITH CHECK(true);
ALTER TABLE part_compatibility ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON part_compatibility FROM anon,authenticated;
GRANT SELECT ON part_compatibility TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY part_compatibility_read ON part_compatibility FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON part_compatibility TO partsiq_catalogue;
CREATE POLICY part_compatibility_publisher ON part_compatibility TO partsiq_catalogue USING (true) WITH
 CHECK (true);
ALTER TABLE part_supersessions ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON part_supersessions FROM anon,authenticated;
GRANT SELECT ON part_supersessions TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY part_supersessions_read ON part_supersessions FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON part_supersessions TO partsiq_catalogue;
CREATE POLICY part_supersessions_publisher ON part_supersessions TO partsiq_catalogue USING (true) WITH
 CHECK (true);
ALTER TABLE warehouses ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON warehouses FROM anon,authenticated;
GRANT SELECT ON warehouses TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY warehouses_read ON warehouses FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON warehouses TO partsiq_catalogue;
CREATE POLICY warehouses_publisher ON warehouses TO partsiq_catalogue USING (true) WITH CHECK (true);
ALTER TABLE stock_levels ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON stock_levels FROM anon,authenticated;
GRANT SELECT ON stock_levels TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY stock_levels_read ON stock_levels FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON stock_levels TO partsiq_catalogue;
CREATE POLICY stock_levels_publisher ON stock_levels TO partsiq_catalogue USING (true) WITH CHECK
(true);
ALTER TABLE enquiries ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON enquiries FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON enquiries TO partsiq_api,partsiq_workflow,partsiq_worker;
CREATE POLICY enquiries_API ON enquiries TO partsiq_api USING(app_private.can_read_enquiry(id)) WITH
CHECK(owner_id=app_private.actor() OR app_private.can_review(id));
CREATE POLICY enquiries_internal ON enquiries TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
ALTER TABLE enquiry_attachments ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON enquiry_attachments FROM anon,authenticated;
GRANT SELECT ON enquiry_attachments TO partsiq_api;
CREATE POLICY enquiry_attachments_API_read ON enquiry_attachments FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT INSERT ON enquiry_attachments TO partsiq_api;
CREATE POLICY enquiry_attachments_API_insert ON enquiry_attachments FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_read_enquiry(enquiry_id));
GRANT UPDATE ON enquiry_attachments TO partsiq_api;
CREATE POLICY enquiry_attachments_API_update ON enquiry_attachments FOR UPDATE TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id)) WITH CHECK(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON enquiry_attachments TO partsiq_workflow,partsiq_worker;
CREATE POLICY enquiry_attachments_internal ON enquiry_attachments TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE enquiry_candidates ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON enquiry_candidates FROM anon,authenticated;
GRANT SELECT ON enquiry_candidates TO partsiq_api;
CREATE POLICY enquiry_candidates_API_read ON enquiry_candidates FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON enquiry_candidates TO partsiq_workflow;
GRANT SELECT ON enquiry_candidates TO partsiq_worker;
CREATE POLICY enquiry_candidates_internal ON enquiry_candidates TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE review_decisions ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON review_decisions FROM anon,authenticated;
GRANT SELECT ON review_decisions TO partsiq_api;
CREATE POLICY review_decisions_API_read ON review_decisions FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON review_decisions TO partsiq_workflow;
GRANT SELECT ON review_decisions TO partsiq_worker;
CREATE POLICY review_decisions_internal ON review_decisions TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE review_approved_parts ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON review_approved_parts FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON review_approved_parts TO partsiq_workflow;
GRANT SELECT ON review_approved_parts TO partsiq_worker;
CREATE POLICY review_approved_parts_internal ON review_approved_parts TO
partsiq_workflow,partsiq_worker USING(true) WITH CHECK(true);
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON audit_log FROM anon,authenticated;
GRANT INSERT ON audit_log TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker,partsiq_catalogue;
CREATE POLICY audit_log_append ON audit_log FOR INSERT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker,partsiq_catalogue WITH CHECK(true);
GRANT SELECT ON audit_log TO partsiq_api;
```

```
CREATE POLICY audit_log_read ON audit_log FOR SELECT TO partsiq_api USING(app_private.has_role('admin')
 OR app_private.can_review(enquiry_id));
ALTER TABLE warehouse_transfer_routes ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON warehouse_transfer_routes FROM anon,authenticated;
GRANT SELECT ON warehouse_transfer_routes TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY warehouse_transfer_routes_read ON warehouse_transfer_routes FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON warehouse_transfer_routes TO partsiq_catalogue;
CREATE POLICY warehouse_transfer_routes_publisher ON warehouse_transfer_routes TO partsiq_catalogue
USING (true) WITH CHECK (true);
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON profiles FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON profiles TO partsiq_api;
CREATE POLICY profiles_owner ON profiles TO partsiq_api USING(id=app_private.actor()) WITH
CHECK(id=app_private.actor());
GRANT SELECT,UPDATE ON profiles TO partsiq_worker;
CREATE POLICY profiles_cleanup ON profiles TO partsiq_worker USING(true) WITH CHECK(true);
ALTER TABLE staff_roles ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON staff_roles FROM anon,authenticated;
GRANT SELECT,INSERT,DELETE ON staff_roles TO partsiq_api;
CREATE POLICY staff_roles_read ON staff_roles FOR SELECT TO partsiq_api
USING(user_id=app_private.actor() OR app_private.has_role('admin'));
CREATE POLICY staff_roles_insert ON staff_roles FOR INSERT TO partsiq_api WITH
CHECK(app_private.has_role('admin'));
CREATE POLICY staff_roles_delete ON staff_roles FOR DELETE TO partsiq_api
USING(app_private.has_role('admin'));
ALTER TABLE demo_sessions ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON demo_sessions FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON demo_sessions TO partsiq_api;
CREATE POLICY demo_sessions_owner ON demo_sessions TO partsiq_api USING(owner_id=app_private.actor())
WITH CHECK(owner_id=app_private.actor());
GRANT SELECT,UPDATE ON demo_sessions TO partsiq_worker;
CREATE POLICY demo_sessions_cleanup ON demo_sessions TO partsiq_worker USING(true) WITH CHECK(true);
ALTER TABLE catalogue_versions ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON catalogue_versions FROM anon,authenticated;
GRANT SELECT ON catalogue_versions TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY catalogue_versions_read ON catalogue_versions FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON catalogue_versions TO partsiq_catalogue;
CREATE POLICY catalogue_versions_publisher ON catalogue_versions TO partsiq_catalogue USING (true) WITH
 CHECK (true);
ALTER TABLE catalogue_records ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON catalogue_records FROM anon,authenticated;
GRANT SELECT ON catalogue_records TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY catalogue_records_read ON catalogue_records FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON catalogue_records TO partsiq_catalogue;
CREATE POLICY catalogue_records_publisher ON catalogue_records TO partsiq_catalogue USING (true) WITH
CHECK (true);
ALTER TABLE catalogue_evidence ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON catalogue_evidence FROM anon,authenticated;
GRANT SELECT ON catalogue_evidence TO partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker;
CREATE POLICY catalogue_evidence_read ON catalogue_evidence FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow,partsiq_worker USING (true);
GRANT SELECT,INSERT,UPDATE ON catalogue_evidence TO partsiq_catalogue;
CREATE POLICY catalogue_evidence_publisher ON catalogue_evidence TO partsiq_catalogue USING (true) WITH
 CHECK (true);
ALTER TABLE enquiry_items ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON enquiry_items FROM anon,authenticated;
GRANT SELECT ON enquiry_items TO partsiq_api;
CREATE POLICY enquiry_items_API_read ON enquiry_items FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT INSERT ON enquiry_items TO partsiq_api;
CREATE POLICY enquiry_items_API_insert ON enquiry_items FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_read_enquiry(enquiry_id));
GRANT UPDATE ON enquiry_items TO partsiq_api;
CREATE POLICY enquiry_items_API_update ON enquiry_items FOR UPDATE TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id)) WITH CHECK(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON enquiry_items TO partsiq_workflow,partsiq_worker;
CREATE POLICY enquiry_items_internal ON enquiry_items TO partsiq_workflow,partsiq_worker USING(true)
WITH CHECK(true);
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON messages FROM anon,authenticated;
GRANT SELECT ON messages TO partsiq_api;
CREATE POLICY messages_API_read ON messages FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT INSERT ON messages TO partsiq_api;
CREATE POLICY messages_API_insert ON messages FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_read_enquiry(enquiry_id) AND kind='customer');
GRANT SELECT,INSERT,UPDATE ON messages TO partsiq_workflow;
GRANT SELECT ON messages TO partsiq_worker;
CREATE POLICY messages_internal ON messages TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
ALTER TABLE enquiry_revisions ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON enquiry_revisions FROM anon,authenticated;
GRANT SELECT ON enquiry_revisions TO partsiq_api;
CREATE POLICY enquiry_revisions_API_read ON enquiry_revisions FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON enquiry_revisions TO partsiq_workflow;
GRANT SELECT ON enquiry_revisions TO partsiq_worker;
```

```
CREATE POLICY enquiry_revisions_internal ON enquiry_revisions TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE extractions ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON extractions FROM anon,authenticated;
GRANT SELECT ON extractions TO partsiq_api;
CREATE POLICY extractions_API_read ON extractions FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON extractions TO partsiq_workflow,partsiq_worker;
CREATE POLICY extractions_internal ON extractions TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
ALTER TABLE fulfilment_plans ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON fulfilment_plans FROM anon,authenticated;
GRANT SELECT ON fulfilment_plans TO partsiq_api;
CREATE POLICY fulfilment_plans_API_read ON fulfilment_plans FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON fulfilment_plans TO partsiq_workflow;
GRANT SELECT ON fulfilment_plans TO partsiq_worker;
CREATE POLICY fulfilment_plans_internal ON fulfilment_plans TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE drafts ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON drafts FROM anon,authenticated;
GRANT SELECT ON drafts TO partsiq_api;
CREATE POLICY drafts_API_read ON drafts FOR SELECT TO partsiq_api
USING(app_private.can_review(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON drafts TO partsiq_workflow;
GRANT SELECT ON drafts TO partsiq_worker;
CREATE POLICY drafts_internal ON drafts TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
ALTER TABLE verification_results ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON verification_results FROM anon,authenticated;
GRANT SELECT ON verification_results TO partsiq_api;
CREATE POLICY verification_results_API_read ON verification_results FOR SELECT TO partsiq_api
USING(app_private.can_review(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON verification_results TO partsiq_workflow;
GRANT SELECT ON verification_results TO partsiq_worker;
CREATE POLICY verification_results_internal ON verification_results TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE internal_notes ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON internal_notes FROM anon,authenticated;
GRANT SELECT ON internal_notes TO partsiq_api;
CREATE POLICY internal_notes_API_read ON internal_notes FOR SELECT TO partsiq_api
USING(app_private.can_review(enquiry_id));
GRANT INSERT ON internal_notes TO partsiq_api;
CREATE POLICY internal_notes_API_insert ON internal_notes FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_review(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON internal_notes TO partsiq_workflow,partsiq_worker;
CREATE POLICY internal_notes_internal ON internal_notes TO partsiq_workflow,partsiq_worker USING(true)
WITH CHECK(true);
ALTER TABLE assignment_events ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON assignment_events FROM anon,authenticated;
GRANT SELECT ON assignment_events TO partsiq_api;
CREATE POLICY assignment_events_API_read ON assignment_events FOR SELECT TO partsiq_api
USING(app_private.can_review(enquiry_id));
GRANT INSERT ON assignment_events TO partsiq_api;
CREATE POLICY assignment_events_API_insert ON assignment_events FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_review(enquiry_id) AND app_private.has_role('admin'));
GRANT SELECT,INSERT,UPDATE ON assignment_events TO partsiq_workflow,partsiq_worker;
CREATE POLICY assignment_events_internal ON assignment_events TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE saved_machines ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON saved_machines FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON saved_machines TO partsiq_api;
CREATE POLICY saved_machines_owner ON saved_machines TO partsiq_api USING(owner_id=app_private.actor()
AND (session_id IS NULL OR app_private.demo_active(session_id,session_generation,owner_id))) WITH
CHECK(owner_id=app_private.actor() AND (session_id IS NULL OR
app_private.demo_active(session_id,session_generation,owner_id)));
GRANT SELECT,UPDATE ON saved_machines TO partsiq_worker;
CREATE POLICY saved_machines_cleanup ON saved_machines TO partsiq_worker USING(true) WITH CHECK(true);
ALTER TABLE missing_part_requests ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON missing_part_requests FROM anon,authenticated;
GRANT SELECT ON missing_part_requests TO partsiq_api;
CREATE POLICY missing_part_requests_API_read ON missing_part_requests FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT INSERT ON missing_part_requests TO partsiq_api;
CREATE POLICY missing_part_requests_API_insert ON missing_part_requests FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_read_enquiry(enquiry_id));
GRANT UPDATE ON missing_part_requests TO partsiq_api;
CREATE POLICY missing_part_requests_API_update ON missing_part_requests FOR UPDATE TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id)) WITH CHECK(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON missing_part_requests TO partsiq_workflow,partsiq_worker;
CREATE POLICY missing_part_requests_internal ON missing_part_requests TO
partsiq_workflow,partsiq_worker USING(true) WITH CHECK(true);
ALTER TABLE jobs ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON jobs FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON jobs TO partsiq_workflow,partsiq_worker;
CREATE POLICY jobs_internal ON jobs TO partsiq_workflow,partsiq_worker USING(true) WITH CHECK(true);
GRANT INSERT,SELECT ON jobs TO partsiq_api;
CREATE POLICY jobs_API ON jobs TO partsiq_api USING(app_private.can_read_enquiry(enquiry_id)) WITH
CHECK(app_private.can_read_enquiry(enquiry_id));
ALTER TABLE workflow_events ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON workflow_events FROM anon,authenticated;
```

```
GRANT SELECT,INSERT,UPDATE ON workflow_events TO partsiq_workflow,partsiq_worker;
CREATE POLICY workflow_events_internal ON workflow_events TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
GRANT INSERT,SELECT ON workflow_events TO partsiq_api;
CREATE POLICY workflow_events_API ON workflow_events TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id)) WITH CHECK(app_private.can_read_enquiry(enquiry_id));
ALTER TABLE exports ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON exports FROM anon,authenticated;
GRANT SELECT ON exports TO partsiq_api;
CREATE POLICY exports_API_read ON exports FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON exports TO partsiq_workflow,partsiq_worker;
CREATE POLICY exports_internal ON exports TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
ALTER TABLE import_batches ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON import_batches FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON import_batches TO partsiq_workflow,partsiq_worker;
CREATE POLICY import_batches_internal ON import_batches TO partsiq_workflow,partsiq_worker USING(true)
WITH CHECK(true);
GRANT SELECT,INSERT,UPDATE ON import_batches TO partsiq_api,partsiq_catalogue;
CREATE POLICY import_batches_admin ON import_batches TO partsiq_api USING(app_private.has_role('admin')
 OR app_private.has_role('catalogue_admin')) WITH CHECK(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin'));
CREATE POLICY import_batches_catalogue ON import_batches TO partsiq_catalogue USING(true) WITH
CHECK(true);
ALTER TABLE import_rows ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON import_rows FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON import_rows TO partsiq_workflow,partsiq_worker;
CREATE POLICY import_rows_internal ON import_rows TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
GRANT SELECT,INSERT,UPDATE ON import_rows TO partsiq_api,partsiq_catalogue;
CREATE POLICY import_rows_admin ON import_rows TO partsiq_api USING(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin')) WITH CHECK(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin'));
CREATE POLICY import_rows_catalogue ON import_rows TO partsiq_catalogue USING(true) WITH CHECK(true);
ALTER TABLE ai_budget_accounts ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON ai_budget_accounts FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON ai_budget_accounts TO partsiq_workflow,partsiq_worker;
CREATE POLICY ai_budget_accounts_internal ON ai_budget_accounts TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE ai_usage ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON ai_usage FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON ai_usage TO partsiq_workflow,partsiq_worker;
CREATE POLICY ai_usage_internal ON ai_usage TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
ALTER TABLE eval_cases ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON eval_cases FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON eval_cases TO partsiq_workflow,partsiq_worker;
CREATE POLICY eval_cases_internal ON eval_cases TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
GRANT SELECT,INSERT,UPDATE ON eval_cases TO partsiq_api,partsiq_catalogue;
CREATE POLICY eval_cases_admin ON eval_cases TO partsiq_api USING(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin')) WITH CHECK(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin'));
CREATE POLICY eval_cases_catalogue ON eval_cases TO partsiq_catalogue USING(true) WITH CHECK(true);
ALTER TABLE eval_runs ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON eval_runs FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON eval_runs TO partsiq_workflow,partsiq_worker;
CREATE POLICY eval_runs_internal ON eval_runs TO partsiq_workflow,partsiq_worker USING(true) WITH
CHECK(true);
GRANT SELECT,INSERT,UPDATE ON eval_runs TO partsiq_api,partsiq_catalogue;
CREATE POLICY eval_runs_admin ON eval_runs TO partsiq_api USING(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin')) WITH CHECK(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin'));
CREATE POLICY eval_runs_catalogue ON eval_runs TO partsiq_catalogue USING(true) WITH CHECK(true);
ALTER TABLE eval_results ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON eval_results FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON eval_results TO partsiq_workflow,partsiq_worker;
CREATE POLICY eval_results_internal ON eval_results TO partsiq_workflow,partsiq_worker USING(true) WITH
 CHECK(true);
GRANT SELECT,INSERT,UPDATE ON eval_results TO partsiq_api,partsiq_catalogue;
CREATE POLICY eval_results_admin ON eval_results TO partsiq_api USING(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin')) WITH CHECK(app_private.has_role('admin') OR
app_private.has_role('catalogue_admin'));
CREATE POLICY eval_results_catalogue ON eval_results TO partsiq_catalogue USING(true) WITH CHECK(true);
ALTER TABLE correction_records ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON correction_records FROM anon,authenticated;
GRANT SELECT ON correction_records TO partsiq_api;
CREATE POLICY correction_records_API_read ON correction_records FOR SELECT TO partsiq_api
USING(app_private.can_review(enquiry_id));
GRANT INSERT ON correction_records TO partsiq_api;
CREATE POLICY correction_records_API_insert ON correction_records FOR INSERT TO partsiq_api WITH
CHECK(app_private.can_review(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON correction_records TO partsiq_workflow,partsiq_worker;
CREATE POLICY correction_records_internal ON correction_records TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE enquiry_summaries ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON enquiry_summaries FROM anon,authenticated;
GRANT SELECT ON enquiry_summaries TO partsiq_api;
CREATE POLICY enquiry_summaries_API_read ON enquiry_summaries FOR SELECT TO partsiq_api
USING(app_private.can_review(enquiry_id));
```

```
GRANT SELECT,INSERT,UPDATE ON enquiry_summaries TO partsiq_workflow;
GRANT SELECT ON enquiry_summaries TO partsiq_worker;
CREATE POLICY enquiry_summaries_internal ON enquiry_summaries TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
ALTER TABLE retention_policies ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON retention_policies FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON retention_policies TO partsiq_workflow,partsiq_worker;
CREATE POLICY retention_policies_internal ON retention_policies TO partsiq_workflow,partsiq_worker
USING(true) WITH CHECK(true);
GRANT SELECT,INSERT,UPDATE ON retention_policies TO partsiq_api,partsiq_catalogue;
CREATE POLICY retention_policies_admin ON retention_policies TO partsiq_api
USING(app_private.has_role('admin') OR app_private.has_role('catalogue_admin')) WITH
CHECK(app_private.has_role('admin') OR app_private.has_role('catalogue_admin'));
CREATE POLICY retention_policies_catalogue ON retention_policies TO partsiq_catalogue USING(true) WITH
CHECK(true);
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO
partsiq_api,partsiq_workflow,partsiq_worker,partsiq_catalogue;
CREATE FUNCTION app_private.validate_publication() RETURNS trigger LANGUAGE plpgsql SECURITY DEFINER
SET search_path=public,pg_temp AS $$
BEGIN
 IF NEW.kind='recommendation' AND NOT EXISTS(
 SELECT 1 FROM review_decisions d JOIN enquiries e ON e.id=d.enquiry_id
 JOIN verification_results v ON v.id=d.verification_id
 WHERE d.id=NEW.decision_id AND d.enquiry_id=NEW.enquiry_id AND d.revision=NEW.revision
 AND e.revision=NEW.revision AND d.decision='approved' AND v.passed AND v.revision=NEW.revision
 ) THEN RAISE EXCEPTION 'approval required for current revision' USING ERRCODE='23514'; END IF;
 RETURN NEW;
END; $$;
REVOKE ALL ON FUNCTION app_private.validate_publication() FROM PUBLIC;
CREATE TRIGGER messages_publication_guard BEFORE INSERT ON messages FOR EACH ROW EXECUTE FUNCTION
app_private.validate_publication();
GRANT USAGE ON SEQUENCE audit_log_id_seq TO partsiq_search;
ALTER TABLE idempotency_records ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON idempotency_records FROM anon,authenticated;
GRANT SELECT,INSERT,UPDATE ON idempotency_records TO
partsiq_api,partsiq_workflow,partsiq_worker,partsiq_catalogue;
CREATE POLICY idempotency_actor ON idempotency_records TO partsiq_api
USING(actor_id=app_private.actor()) WITH CHECK(actor_id=app_private.actor());
CREATE POLICY idempotency_internal ON idempotency_records TO
partsiq_workflow,partsiq_worker,partsiq_catalogue USING(true) WITH CHECK(true);
GRANT SELECT ON ai_usage TO partsiq_api;
CREATE POLICY ai_usage_metrics ON ai_usage FOR SELECT TO partsiq_api
USING(app_private.has_role('admin') OR app_private.can_review(enquiry_id));
COMMIT;
```

005_storage.sql

```
BEGIN;
INSERT INTO storage.buckets(id,name,public,file_size_limit,allowed_mime_types) VALUES
 ('nameplates','nameplates',false,10485760,ARRAY['image/jpeg','image/png','image/webp']),
 ('catalogue-evidence','catalogue-evidence',false,20971520,ARRAY['application/pdf']),
 ('imports','imports',false,10485760,ARRAY['text/csv','application/vnd.openxmlformats-officedocument.sp
readsheetml.sheet']),
 ('exports','exports',false,20971520,ARRAY['application/pdf']);
CREATE FUNCTION public.partsiq_storage_allowed(object_name text, target_bucket text, write_access
boolean) RETURNS boolean
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path=public,pg_temp AS $$
DECLARE actor uuid:=auth.uid(); allowed boolean:=false;
BEGIN
 IF actor IS NULL THEN RETURN false; END IF;
 IF target_bucket='nameplates' THEN
 SELECT EXISTS(SELECT 1 FROM enquiry_attachments a JOIN enquiries e ON e.id=a.enquiry_id
 WHERE a.storage_key=object_name AND e.owner_id=actor
 AND (e.session_id IS NULL OR app_private.demo_active(e.session_id,e.session_generation,actor))
 AND ((write_access AND a.status='pending') OR (NOT write_access AND a.status='validated'))) INTO
allowed;
 END IF;
 RETURN allowed;
END; $$;
REVOKE ALL ON FUNCTION public.partsiq_storage_allowed(text,text,boolean) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.partsiq_storage_allowed(text,text,boolean) TO authenticated;
CREATE POLICY nameplate_insert ON storage.objects FOR INSERT TO authenticated
 WITH CHECK(bucket_id='nameplates' AND public.partsiq_storage_allowed(name,bucket_id,true));
CREATE POLICY nameplate_read ON storage.objects FOR SELECT TO authenticated
 USING(bucket_id='nameplates' AND public.partsiq_storage_allowed(name,bucket_id,false));
-- Other buckets have no end-user policy. Worker signs authorized downloads only after API scope
validation.
COMMIT;
```

006_plain_workflow_and_snowflake.sql

```
BEGIN;
ALTER TABLE jobs DROP CONSTRAINT jobs_kind_check;
ALTER TABLE jobs ADD CONSTRAINT jobs_kind_check CHECK(kind IN ('nameplate','export_pdf','catalog_import
','catalog_publish','storage_cleanup','workflow','analytics_sync','catalog_sync'));
CREATE TABLE workflow_runs (
 enquiry_id uuid NOT NULL REFERENCES enquiries(id), revision integer NOT NULL CHECK(revision>0),
 step text NOT NULL CHECK(step IN ('intake','await_ocr','confirm','identify','clarify','search','choose
','plan','draft','verify','review','complete','needs_staff')),
 state jsonb NOT NULL DEFAULT '{}'::jsonb CHECK(jsonb_typeof(state)='object'),
```

```
 lease_token uuid, lease_expires_at timestamptz, updated_at timestamptz NOT NULL DEFAULT now(),
 PRIMARY KEY(enquiry_id,revision), CHECK((lease_token IS NULL)=(lease_expires_at IS NULL))
);
CREATE TABLE workflow_step_results (
 enquiry_id uuid NOT NULL, revision integer NOT NULL, step text NOT NULL,
 input_sha256 text NOT NULL CHECK(input_sha256 ~ '^[a-f0-9]{64}$'),
 output jsonb NOT NULL CHECK(jsonb_typeof(output)='object'), completed_at timestamptz NOT NULL DEFAULT
now(),
 PRIMARY KEY(enquiry_id,revision,step,input_sha256),
 FOREIGN KEY(enquiry_id,revision) REFERENCES workflow_runs(enquiry_id,revision)
);
CREATE INDEX workflow_runs_lease_idx ON workflow_runs(lease_expires_at) WHERE lease_token IS NOT NULL;
CREATE TABLE catalogue_syncs (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), version_id uuid NOT NULL REFERENCES
catalogue_versions(id),
 source text NOT NULL CHECK(source IN ('snowflake','fixture')),
 state text NOT NULL CHECK(state IN ('requested','remote_written','validated','activated','failed')),
 expected_sha256 text NOT NULL CHECK(expected_sha256 ~ '^[a-f0-9]{64}$'),
 observed_sha256 text, record_count integer CHECK(record_count>=0), query_id text,
 error_code text, created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL
DEFAULT now(),
 UNIQUE(version_id,source)
);
CREATE TABLE analytics_outbox (
 event_id uuid PRIMARY KEY DEFAULT gen_random_uuid(), event_type text NOT NULL,
 enquiry_id uuid REFERENCES enquiries(id), revision integer,
 payload jsonb NOT NULL CHECK(jsonb_typeof(payload)='object'),
 created_at timestamptz NOT NULL DEFAULT now(), delivered_at timestamptz,
 attempts integer NOT NULL DEFAULT 0 CHECK(attempts>=0), last_error_code text
);
CREATE INDEX analytics_outbox_pending_idx ON analytics_outbox(created_at,event_id) WHERE delivered_at
IS NULL;
CREATE TABLE analytics_sync_runs (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(), started_at timestamptz NOT NULL DEFAULT now(),
finished_at timestamptz,
 row_count integer NOT NULL DEFAULT 0 CHECK(row_count>=0),
 state text NOT NULL CHECK(state IN ('running','succeeded','failed')), query_id text, last_error_code
text
);
ALTER TABLE workflow_runs ENABLE ROW LEVEL SECURITY;
ALTER TABLE workflow_step_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE catalogue_syncs ENABLE ROW LEVEL SECURITY;
ALTER TABLE analytics_outbox ENABLE ROW LEVEL SECURITY;
ALTER TABLE analytics_sync_runs ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON workflow_runs,workflow_step_results,catalogue_syncs,analytics_outbox,analytics_sync_runs
FROM PUBLIC,anon,authenticated;
GRANT SELECT ON workflow_runs,workflow_step_results TO partsiq_api;
CREATE POLICY workflow_runs_api_read ON workflow_runs FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
CREATE POLICY workflow_results_api_read ON workflow_step_results FOR SELECT TO partsiq_api
USING(app_private.can_read_enquiry(enquiry_id));
GRANT SELECT,INSERT,UPDATE ON workflow_runs,workflow_step_results TO partsiq_workflow;
CREATE POLICY workflow_runs_job ON workflow_runs TO partsiq_workflow USING(true) WITH CHECK(true);
CREATE POLICY workflow_results_job ON workflow_step_results TO partsiq_workflow USING(true) WITH
CHECK(true);
GRANT SELECT,INSERT,UPDATE ON catalogue_syncs TO partsiq_catalogue;
CREATE POLICY catalogue_syncs_publisher ON catalogue_syncs TO partsiq_catalogue USING(true) WITH
CHECK(true);
GRANT SELECT,INSERT,UPDATE ON analytics_outbox,analytics_sync_runs TO partsiq_worker;
CREATE POLICY analytics_outbox_worker ON analytics_outbox TO partsiq_worker USING(true) WITH
CHECK(true);
CREATE POLICY analytics_runs_worker ON analytics_sync_runs TO partsiq_worker USING(true) WITH
CHECK(true);
GRANT INSERT ON analytics_outbox TO partsiq_api,partsiq_workflow;
CREATE POLICY analytics_outbox_append ON analytics_outbox FOR INSERT TO partsiq_api,partsiq_workflow
WITH CHECK(app_private.can_read_enquiry(enquiry_id));
CREATE TABLE stock_snapshot_receipts (
 id uuid PRIMARY KEY, source text NOT NULL CHECK(source IN ('snowflake','fixture')),
 observed_at timestamptz NOT NULL, activated_at timestamptz NOT NULL DEFAULT now(),
 content_sha256 text NOT NULL CHECK(content_sha256 ~ '^[a-f0-9]{64}$'),
 row_count integer NOT NULL CHECK(row_count>=0), active boolean NOT NULL DEFAULT false
);
CREATE UNIQUE INDEX stock_one_active_idx ON stock_snapshot_receipts((active)) WHERE active;
ALTER TABLE stock_snapshot_receipts ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON stock_snapshot_receipts FROM PUBLIC,anon,authenticated;
GRANT SELECT ON stock_snapshot_receipts TO partsiq_api,partsiq_search,partsiq_workflow;
CREATE POLICY stock_snapshot_read ON stock_snapshot_receipts FOR SELECT TO
partsiq_api,partsiq_search,partsiq_workflow USING(true);
GRANT SELECT,INSERT,UPDATE ON stock_snapshot_receipts TO partsiq_catalogue;
CREATE POLICY stock_snapshot_publish ON stock_snapshot_receipts TO partsiq_catalogue USING(true) WITH
CHECK(true);
COMMIT;
```

4 Exact Snowflake schema

Snowflake standard-table PK metadata does not enforce uniqueness or foreign references. There are no PostgreSQL-style application indexes or RLS policies here. Dedicated database/schema/role grants and application validation define the boundary.

001_schema.sql

```
-- Operator executes in a dedicated development account first. No ACCOUNTADMIN in application secrets.
CREATE DATABASE IF NOT EXISTS PARTSIQ_DEV;
CREATE SCHEMA IF NOT EXISTS PARTSIQ_DEV.CATALOGUE;
CREATE SCHEMA IF NOT EXISTS PARTSIQ_DEV.REPORTING;
CREATE TABLE IF NOT EXISTS PARTSIQ_DEV.CATALOGUE.VERSIONS (
 VERSION_ID VARCHAR(36) NOT NULL, CONTENT_SHA256 VARCHAR(64) NOT NULL,
 RECORD_COUNT NUMBER(10,0) NOT NULL, CREATED_AT TIMESTAMP_TZ NOT NULL,
 PRIMARY KEY(VERSION_ID)
);
CREATE TABLE IF NOT EXISTS PARTSIQ_DEV.CATALOGUE.RECORDS (
 VERSION_ID VARCHAR(36) NOT NULL, ENTITY_TYPE VARCHAR(40) NOT NULL,
 ENTITY_KEY VARCHAR(120) NOT NULL, PAYLOAD VARIANT NOT NULL,
 ROW_SHA256 VARCHAR(64) NOT NULL, PRIMARY KEY(VERSION_ID,ENTITY_TYPE,ENTITY_KEY)
);
CREATE TABLE IF NOT EXISTS PARTSIQ_DEV.CATALOGUE.STOCK_SNAPSHOTS (
 SNAPSHOT_ID VARCHAR(36) NOT NULL, PART_ID NUMBER(10,0) NOT NULL,
 WAREHOUSE_ID NUMBER(5,0) NOT NULL, ON_HAND NUMBER(10,3) NOT NULL,
 RESERVED NUMBER(10,3) NOT NULL, OBSERVED_AT TIMESTAMP_TZ NOT NULL,
 PRIMARY KEY(SNAPSHOT_ID,PART_ID,WAREHOUSE_ID)
);
CREATE TABLE IF NOT EXISTS PARTSIQ_DEV.REPORTING.EVENTS (
 EVENT_ID VARCHAR(36) NOT NULL, EVENT_TYPE VARCHAR(60) NOT NULL,
 OCCURRED_AT TIMESTAMP_TZ NOT NULL, IS_DEMO BOOLEAN NOT NULL,
 STATUS VARCHAR(40), STAGE VARCHAR(40), DURATION_MS NUMBER(12,3),
 INPUT_TOKENS NUMBER(12,0), OUTPUT_TOKENS NUMBER(12,0), ESTIMATED_COST_USD NUMBER(18,8),
 PRIMARY KEY(EVENT_ID)
);
CREATE VIEW IF NOT EXISTS PARTSIQ_DEV.REPORTING.DAILY_METRICS AS
 SELECT DATE_TRUNC('DAY',OCCURRED_AT) AS DAY, IS_DEMO, EVENT_TYPE,
 COUNT(*) AS EVENT_COUNT, AVG(DURATION_MS) AS MEAN_DURATION_MS,
 SUM(ESTIMATED_COST_USD) AS ESTIMATED_COST_USD
 FROM PARTSIQ_DEV.REPORTING.EVENTS GROUP BY 1,2,3;
-- PK declarations are metadata. Publication and outbox writers validate uniqueness themselves.
```

002_roles.sql

```
CREATE ROLE IF NOT EXISTS PARTSIQ_READER;
CREATE ROLE IF NOT EXISTS PARTSIQ_LOADER;
CREATE ROLE IF NOT EXISTS PARTSIQ_REPORTER;
GRANT USAGE ON DATABASE PARTSIQ_DEV TO ROLE PARTSIQ_READER;
GRANT USAGE ON SCHEMA PARTSIQ_DEV.CATALOGUE TO ROLE PARTSIQ_READER;
GRANT SELECT ON ALL TABLES IN SCHEMA PARTSIQ_DEV.CATALOGUE TO ROLE PARTSIQ_READER;
GRANT ROLE PARTSIQ_READER TO ROLE PARTSIQ_LOADER;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA PARTSIQ_DEV.CATALOGUE TO ROLE PARTSIQ_LOADER;
GRANT USAGE ON DATABASE PARTSIQ_DEV TO ROLE PARTSIQ_REPORTER;
GRANT USAGE ON SCHEMA PARTSIQ_DEV.REPORTING TO ROLE PARTSIQ_REPORTER;
GRANT SELECT,INSERT,UPDATE ON TABLE PARTSIQ_DEV.REPORTING.EVENTS TO ROLE PARTSIQ_REPORTER;
GRANT SELECT ON VIEW PARTSIQ_DEV.REPORTING.DAILY_METRICS TO ROLE PARTSIQ_REPORTER;
-- Grant warehouse USAGE to these roles only after the confirmed credit preflight.
-- Create service users and register their public RSA keys through the operator setup prompt.
```

003_validation_queries.sql

```
SELECT VERSION_ID,ENTITY_TYPE,ENTITY_KEY,COUNT(*) AS N
 FROM PARTSIQ_DEV.CATALOGUE.RECORDS GROUP BY 1,2,3 HAVING COUNT(*)<>1;
SELECT VERSION_ID,COUNT(*) AS N FROM PARTSIQ_DEV.CATALOGUE.VERSIONS GROUP BY 1 HAVING COUNT(*)<>1;
SELECT SNAPSHOT_ID,PART_ID,WAREHOUSE_ID,COUNT(*) AS N
 FROM PARTSIQ_DEV.CATALOGUE.STOCK_SNAPSHOTS GROUP BY 1,2,3 HAVING COUNT(*)<>1;
SELECT * FROM PARTSIQ_DEV.CATALOGUE.STOCK_SNAPSHOTS
 WHERE ON_HAND<0 OR RESERVED<0 OR RESERVED>ON_HAND OR OBSERVED_AT>CURRENT_TIMESTAMP();
-- Relationship, serial-range, supersession-cycle and evidence checks run on the typed snapshot before
activation.
```

5 Dependency and environment contracts

DEPENDENCIES.md lists every direct version, release date and installation prompt. spec/python-requirements.lock and spec/pnpm-lock.yaml capture resolved references. ENVIRONMENT.md lists every runtime variable, default, consumer and secret status. No credential values are embedded in this kit.

6 Planned file ownership

Path	Creator
.env.example	00
.github/workflows/ci.yml	25
.github/workflows/deploy.yml	25
.gitignore	00
.nvmrc	00
.python-version	00
apps/api/Dockerfile	24
apps/api/partsiq/__init__.py	00

Path	Creator
apps/api/partsiq/ai/__init__.py	00
apps/api/partsiq/ai/budget.py	10
apps/api/partsiq/ai/client.py	10
apps/api/partsiq/ai/extraction.py	10
apps/api/partsiq/ai/prompts.py	10
apps/api/partsiq/ai/reply.py	13
apps/api/partsiq/ai/vision.py	16
apps/api/partsiq/analytics/__init__.py	00
apps/api/partsiq/analytics/outbox.py	23
apps/api/partsiq/analytics/service.py	23
apps/api/partsiq/analytics/snowflake.py	23
apps/api/partsiq/api/__init__.py	00
apps/api/partsiq/api/f02_demo_scenarios_and_assets.py	F02
apps/api/partsiq/api/f04_demo_isolation.py	F04
apps/api/partsiq/api/f05_demo_reset.py	F05
apps/api/partsiq/api/f09_bilingual_chat.py	F09
apps/api/partsiq/api/f10_structured_extraction.py	F10
apps/api/partsiq/api/f11_clarifying_questions.py	F11
apps/api/partsiq/api/f12_conversation_memory.py	F12
apps/api/partsiq/api/f13_private_uploads.py	F13
apps/api/partsiq/api/f14_nameplate_ocr.py	F14
apps/api/partsiq/api/f15_ocr_confirmation.py	F15
apps/api/partsiq/api/f16_photo_quality_feedback.py	F16
apps/api/partsiq/api/f17_multi_part_enquiries.py	F17
apps/api/partsiq/api/f18_progress_timeline.py	F18
apps/api/partsiq/api/f19_enquiry_history.py	F19
apps/api/partsiq/api/f20_machine_identification.py	F20
apps/api/partsiq/api/f21_machine_mismatch_detection.py	F21
apps/api/partsiq/api/f22_compatibility_gate.py	F22
apps/api/partsiq/api/f23_aliases_and_typo_matching.py	F23
apps/api/partsiq/api/f24_semantic_retrieval.py	F24
apps/api/partsiq/api/f25_ranked_candidates.py	F25
apps/api/partsiq/api/f26_ambiguity_handling.py	F26
apps/api/partsiq/api/f27_match_explanations.py	F27
apps/api/partsiq/api/f28_compatibility_references.py	F28
apps/api/partsiq/api/f29_evidence_page_viewer.py	F29
apps/api/partsiq/api/f30_old_part_numbers.py	F30
apps/api/partsiq/api/f31_candidate_comparison.py	F31
apps/api/partsiq/api/f32_warehouse_stock.py	F32
apps/api/partsiq/api/f33_fulfilment_planning.py	F33
apps/api/partsiq/api/f34_transfer_estimates.py	F34
apps/api/partsiq/api/f35_stock_freshness.py	F35
apps/api/partsiq/api/f36_staff_inbox.py	F36
apps/api/partsiq/api/f37_review_workspace.py	F37
apps/api/partsiq/api/f38_correct_extracted_details.py	F38
apps/api/partsiq/api/f39_review_actions.py	F39
apps/api/partsiq/api/f40_approval_invalidation.py	F40
apps/api/partsiq/api/f41_internal_notes.py	F41
apps/api/partsiq/api/f42_assignments.py	F42
apps/api/partsiq/api/f43_overdue_indicators.py	F43
apps/api/partsiq/api/f44_conversation_summary.py	F44
apps/api/partsiq/api/f45_reply_drafting.py	F45

Path	Creator
apps/api/partsiq/api/f46_verification_gate.py	F46
apps/api/partsiq/api/f47_human_publication_gate.py	F47
apps/api/partsiq/api/f49_audit_trail.py	F49
apps/api/partsiq/api/f50_durable_workflows.py	F50
apps/api/partsiq/api/f51_background_jobs.py	F51
apps/api/partsiq/api/f52_retry_and_deduplication.py	F52
apps/api/partsiq/api/f54_access_and_permissions.py	F54
apps/api/partsiq/api/f55_saved_machines.py	F55
apps/api/partsiq/api/f56_repeat_enquiry.py	F56
apps/api/partsiq/api/f57_catalogue_browser.py	F57
apps/api/partsiq/api/f58_catalogue_imports.py	F58
apps/api/partsiq/api/f59_catalogue_versioning.py	F59
apps/api/partsiq/api/f60_missing_parts_queue.py	F60
apps/api/partsiq/api/f61_enquiry_pdf.py	F61
apps/api/partsiq/api/f62_labelled_evaluations.py	F62
apps/api/partsiq/api/f63_metrics_dashboard.py	F63
apps/api/partsiq/api/f64_latency_and_ai_cost.py	F64
apps/api/partsiq/api/f65_correction_feedback.py	F65
apps/api/partsiq/api/f67_traces.py	F67
apps/api/partsiq/api/router.py	01
apps/api/partsiq/auth/__init__.py	00
apps/api/partsiq/auth/access.py	07
apps/api/partsiq/auth/demo.py	07
apps/api/partsiq/auth/tokens.py	07
apps/api/partsiq/catalogue/__init__.py	00
apps/api/partsiq/catalogue/activation.py	03
apps/api/partsiq/catalogue/fixture_repository.py	02
apps/api/partsiq/catalogue/fixtures.py	05
apps/api/partsiq/catalogue/models.py	02
apps/api/partsiq/catalogue/snowflake_repository.py	02
apps/api/partsiq/catalogue/validation.py	02
apps/api/partsiq/core/__init__.py	00
apps/api/partsiq/core/api_models.py	01
apps/api/partsiq/core/contracts.py	01
apps/api/partsiq/core/db.py	01
apps/api/partsiq/core/dependencies.py	01
apps/api/partsiq/core/errors.py	01
apps/api/partsiq/core/idempotency.py	08
apps/api/partsiq/core/observability.py	23
apps/api/partsiq/core/rate_limit.py	08
apps/api/partsiq/core/settings.py	01
apps/api/partsiq/enquiries/__init__.py	00
apps/api/partsiq/enquiries/approval.py	14
apps/api/partsiq/enquiries/drafts.py	13
apps/api/partsiq/enquiries/extraction.py	16
apps/api/partsiq/enquiries/facts.py	12
apps/api/partsiq/enquiries/fulfilment.py	11
apps/api/partsiq/enquiries/history.py	08
apps/api/partsiq/enquiries/intake.py	08
apps/api/partsiq/enquiries/publication.py	14
apps/api/partsiq/enquiries/questions.py	11
apps/api/partsiq/enquiries/renderer.py	13
apps/api/partsiq/enquiries/review.py	14

Path	Creator
apps/api/partsiq/enquiries/revisions.py	14
apps/api/partsiq/enquiries/verification.py	12
apps/api/partsiq/evaluation/__init__.py	00
apps/api/partsiq/evaluation/curation.py	21
apps/api/partsiq/evaluation/runner.py	21
apps/api/partsiq/exports/__init__.py	00
apps/api/partsiq/exports/render.py	17
apps/api/partsiq/exports/service.py	17
apps/api/partsiq/exports/templates/__init__.py	00
apps/api/partsiq/exports/templates/enquiry.html	17
apps/api/partsiq/features/__init__.py	00
apps/api/partsiq/features/f01_branded_landing.py	F01
apps/api/partsiq/features/f02_demo_scenarios_and_assets.py	F02
apps/api/partsiq/features/f03_demo_role_switching.py	F03
apps/api/partsiq/features/f04_demo_isolation.py	F04
apps/api/partsiq/features/f05_demo_reset.py	F05
apps/api/partsiq/features/f06_responsive_layouts.py	F06
apps/api/partsiq/features/f07_arabic_and_english_interface.py	F07
apps/api/partsiq/features/f08_light_and_dark_themes.py	F08
apps/api/partsiq/features/f09_bilingual_chat.py	F09
apps/api/partsiq/features/f10_structured_extraction.py	F10
apps/api/partsiq/features/f11_clarifying_questions.py	F11
apps/api/partsiq/features/f12_conversation_memory.py	F12
apps/api/partsiq/features/f13_private_uploads.py	F13
apps/api/partsiq/features/f14_nameplate_ocr.py	F14
apps/api/partsiq/features/f15_ocr_confirmation.py	F15
apps/api/partsiq/features/f16_photo_quality_feedback.py	F16
apps/api/partsiq/features/f17_multi_part_enquiries.py	F17
apps/api/partsiq/features/f18_progress_timeline.py	F18
apps/api/partsiq/features/f19_enquiry_history.py	F19
apps/api/partsiq/features/f20_machine_identification.py	F20
apps/api/partsiq/features/f21_machine_mismatch_detection.py	F21
apps/api/partsiq/features/f22_compatibility_gate.py	F22
apps/api/partsiq/features/f23_aliases_and_typo_matching.py	F23
apps/api/partsiq/features/f24_semantic_retrieval.py	F24
apps/api/partsiq/features/f25_ranked_candidates.py	F25
apps/api/partsiq/features/f26_ambiguity_handling.py	F26
apps/api/partsiq/features/f27_match_explanations.py	F27
apps/api/partsiq/features/f28_compatibility_references.py	F28
apps/api/partsiq/features/f29_evidence_page_viewer.py	F29
apps/api/partsiq/features/f30_old_part_numbers.py	F30
apps/api/partsiq/features/f31_candidate_comparison.py	F31
apps/api/partsiq/features/f32_warehouse_stock.py	F32
apps/api/partsiq/features/f33_fulfilment_planning.py	F33
apps/api/partsiq/features/f34_transfer_estimates.py	F34
apps/api/partsiq/features/f35_stock_freshness.py	F35
apps/api/partsiq/features/f36_staff_inbox.py	F36
apps/api/partsiq/features/f37_review_workspace.py	F37
apps/api/partsiq/features/f38_correct_extracted_details.py	F38
apps/api/partsiq/features/f39_review_actions.py	F39
apps/api/partsiq/features/f40_approval_invalidation.py	F40
apps/api/partsiq/features/f41_internal_notes.py	F41
apps/api/partsiq/features/f42_assignments.py	F42

Path	Creator
apps/api/partsiq/features/f43_overdue_indicators.py	F43
apps/api/partsiq/features/f44_conversation_summary.py	F44
apps/api/partsiq/features/f45_reply_drafting.py	F45
apps/api/partsiq/features/f46_verification_gate.py	F46
apps/api/partsiq/features/f47_human_publication_gate.py	F47
apps/api/partsiq/features/f48_input_isolation.py	F48
apps/api/partsiq/features/f49_audit_trail.py	F49
apps/api/partsiq/features/f50_durable_workflows.py	F50
apps/api/partsiq/features/f51_background_jobs.py	F51
apps/api/partsiq/features/f52_retry_and_deduplication.py	F52
apps/api/partsiq/features/f53_graceful_failures.py	F53
apps/api/partsiq/features/f54_access_and_permissions.py	F54
apps/api/partsiq/features/f55_saved_machines.py	F55
apps/api/partsiq/features/f56_repeat_enquiry.py	F56
apps/api/partsiq/features/f57_catalogue_browser.py	F57
apps/api/partsiq/features/f58_catalogue_imports.py	F58
apps/api/partsiq/features/f59_catalogue_versioning.py	F59
apps/api/partsiq/features/f60_missing_parts_queue.py	F60
apps/api/partsiq/features/f61_enquiry_pdf.py	F61
apps/api/partsiq/features/f62_labelled_evaluations.py	F62
apps/api/partsiq/features/f63_metrics_dashboard.py	F63
apps/api/partsiq/features/f64_latency_and_ai_cost.py	F64
apps/api/partsiq/features/f65_correction_feedback.py	F65
apps/api/partsiq/features/f66_tests_and_ci.py	F66
apps/api/partsiq/features/f67_traces.py	F67
apps/api/partsiq/features/f68_recruiter_documentation.py	F68
apps/api/partsiq/features/f69_demo_walkthrough.py	F69
apps/api/partsiq/jobs/__init__.py	00
apps/api/partsiq/jobs/handlers.py	15
apps/api/partsiq/jobs/main.py	15
apps/api/partsiq/jobs/nameplate.py	16
apps/api/partsiq/jobs/repository.py	15
apps/api/partsiq/jobs/sweeper.py	15
apps/api/partsiq/main.py	01
apps/api/partsiq/search/__init__.py	00
apps/api/partsiq/search/embeddings.py	06
apps/api/partsiq/search/fulfilment.py	06
apps/api/partsiq/search/identification.py	06
apps/api/partsiq/search/lexical.py	06
apps/api/partsiq/search/repository.py	06
apps/api/partsiq/search/schemas.py	06
apps/api/partsiq/search/service.py	06
apps/api/partsiq/storage/__init__.py	00
apps/api/partsiq/storage/client.py	04
apps/api/partsiq/storage/images.py	04
apps/api/partsiq/storage/service.py	04
apps/api/partsiq/workflow/__init__.py	00
apps/api/partsiq/workflow/repository.py	09
apps/api/partsiq/workflow/resolution.py	11
apps/api/partsiq/workflow/runner.py	09
apps/api/partsiq/workflow/state.py	09
apps/api/partsiq/workflow/transitions.py	09
apps/api/pyproject.toml	00

Path	Creator
apps/web/next.config.ts	18
apps/web/package.json	18
apps/web/pnpm-lock.yaml	18
apps/web/public/brand/BRAND.md	18
apps/web/public/brand/assets-manifest.json	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/Preview.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-1024.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-128.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-144.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-152.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-16.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-180.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-192.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-24.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-256.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-32.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-384.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-48.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-512.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-64.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-96.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-1024.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-128.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-144.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-152.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-16.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-180.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-192.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-24.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-256.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-32.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-384.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-48.png	18

Path	Creator
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-512.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-64.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-96.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-1280.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-1740.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-320.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-640.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-1280.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-1740.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-320.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-640.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-hdpi-72.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-mdpi-48.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-xhdpi-96.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-xxhdpi-144.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-xxxhdpi-192.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/google-play-512.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/ios-app-store-1024.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/social/partsiq-og-1200x630.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/social/partsiq-profile-1080.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/source/original-logo.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/source/transparent-logo.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/android-chrome-192x192.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/android-chrome-512x512.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/apple-touch-icon.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-16x16.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-32x32.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-48x48.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-96x96.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon.ico	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/	18

Path	Creator
maskable-icon-192x192.png	
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/maskable-icon-512x512.png	18
apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/site.webmanifest	18
apps/web/public/brand/fonts/NotoNaskhArabic-OFL.txt	18
apps/web/public/brand/fonts/NotoNaskhArabic.ttf	18
apps/web/public/brand/fonts/NotoSans-OFL.txt	18
apps/web/public/brand/fonts/NotoSans.ttf	18
apps/web/public/brand/fonts/NotoSansArabic-OFL.txt	18
apps/web/public/brand/fonts/NotoSansArabic.ttf	18
apps/web/public/brand/logo-original.png	18
apps/web/public/brand/tokens.css	18
apps/web/public/brand/tokens.json	18
apps/web/src/app/demo/page.tsx	19
apps/web/src/app/enquiries/[id]/page.tsx	19
apps/web/src/app/globals.css	18
apps/web/src/app/layout.tsx	18
apps/web/src/app/page.tsx	19
apps/web/src/app/providers.tsx	18
apps/web/src/app/staff/enquiries/[id]/page.tsx	20
apps/web/src/app/staff/page.tsx	20
apps/web/src/components/AppShell.tsx	18
apps/web/src/components/CustomerShell.tsx	19
apps/web/src/components/ReviewActions.tsx	20
apps/web/src/components/StaffShell.tsx	20
apps/web/src/features/f01_branded_landing/Panel.tsx	F01
apps/web/src/features/f02_demo_scenarios_and_assets/Panel.tsx	F02
apps/web/src/features/f03_demo_role_switching/Panel.tsx	F03
apps/web/src/features/f04_demo_isolation/Panel.tsx	F04
apps/web/src/features/f05_demo_reset/Panel.tsx	F05
apps/web/src/features/f06_responsive_layouts/Panel.tsx	F06
apps/web/src/features/f07_arabic_and_english_interface/Panel.tsx	F07
apps/web/src/features/f08_light_and_dark_themes/Panel.tsx	F08
apps/web/src/features/f09_bilingual_chat/Panel.tsx	F09
apps/web/src/features/f10_structured_extraction/Panel.tsx	F10
apps/web/src/features/f11_clarifying_questions/Panel.tsx	F11
apps/web/src/features/f12_conversation_memory/Panel.tsx	F12
apps/web/src/features/f13_private_uploads/Panel.tsx	F13
apps/web/src/features/f14_nameplate_ocr/Panel.tsx	F14
apps/web/src/features/f15_ocr_confirmation/Panel.tsx	F15
apps/web/src/features/f16_photo_quality_feedback/Panel.tsx	F16
apps/web/src/features/f17_multi_part_enquiries/Panel.tsx	F17
apps/web/src/features/f18_progress_timeline/Panel.tsx	F18
apps/web/src/features/f19_enquiry_history/Panel.tsx	F19
apps/web/src/features/f20_machine_identification/Panel.tsx	F20
apps/web/src/features/f21_machine_mismatch_detection/Panel.tsx	F21
apps/web/src/features/f22_compatibility_gate/Panel.tsx	F22
apps/web/src/features/f23_aliases_and_typo_matching/Panel.tsx	F23
apps/web/src/features/f24_semantic_retrieval/Panel.tsx	F24
apps/web/src/features/f25_ranked_candidates/Panel.tsx	F25

Path	Creator
apps/web/src/features/f26_ambiguity_handling/Panel.tsx	F26
apps/web/src/features/f27_match_explanations/Panel.tsx	F27
apps/web/src/features/f28_compatibility_references/Panel.tsx	F28
apps/web/src/features/f29_evidence_page_viewer/Panel.tsx	F29
apps/web/src/features/f30_old_part_numbers/Panel.tsx	F30
apps/web/src/features/f31_candidate_comparison/Panel.tsx	F31
apps/web/src/features/f32_warehouse_stock/Panel.tsx	F32
apps/web/src/features/f33_fulfilment_planning/Panel.tsx	F33
apps/web/src/features/f34_transfer_estimates/Panel.tsx	F34
apps/web/src/features/f35_stock_freshness/Panel.tsx	F35
apps/web/src/features/f36_staff_inbox/Panel.tsx	F36
apps/web/src/features/f37_review_workspace/Panel.tsx	F37
apps/web/src/features/f38_correct_extracted_details/Panel.tsx	F38
apps/web/src/features/f39_review_actions/Panel.tsx	F39
apps/web/src/features/f40_approval_invalidation/Panel.tsx	F40
apps/web/src/features/f41_internal_notes/Panel.tsx	F41
apps/web/src/features/f42_assignments/Panel.tsx	F42
apps/web/src/features/f43_overdue_indicators/Panel.tsx	F43
apps/web/src/features/f44_conversation_summary/Panel.tsx	F44
apps/web/src/features/f45_reply_drafting/Panel.tsx	F45
apps/web/src/features/f46_verification_gate/Panel.tsx	F46
apps/web/src/features/f47_human_publication_gate/Panel.tsx	F47
apps/web/src/features/f49_audit_trail/Panel.tsx	F49
apps/web/src/features/f50_durable_workflows/Panel.tsx	F50
apps/web/src/features/f51_background_jobs/Panel.tsx	F51
apps/web/src/features/f52_retry_and_deduplication/Panel.tsx	F52
apps/web/src/features/f54_access_and_permissions/Panel.tsx	F54
apps/web/src/features/f55_saved_machines/Panel.tsx	F55
apps/web/src/features/f56_repeat_enquiry/Panel.tsx	F56
apps/web/src/features/f57_catalogue_browser/Panel.tsx	F57
apps/web/src/features/f58_catalogue_imports/Panel.tsx	F58
apps/web/src/features/f59_catalogue_versioning/Panel.tsx	F59
apps/web/src/features/f60_missing_parts_queue/Panel.tsx	F60
apps/web/src/features/f61_enquiry_pdf/Panel.tsx	F61
apps/web/src/features/f62_labelled_evaluations/Panel.tsx	F62
apps/web/src/features/f63_metrics_dashboard/Panel.tsx	F63
apps/web/src/features/f64_latency_and_ai_cost/Panel.tsx	F64
apps/web/src/features/f65_correction_feedback/Panel.tsx	F65
apps/web/src/features/f67_traces/Panel.tsx	F67
apps/web/src/features/f68_recruiter_documentation/Panel.tsx	F68
apps/web/src/features/f69_demo_walkthrough/Panel.tsx	F69
apps/web/src/hooks/useEnquiry.ts	19
apps/web/src/i18n/ar.json	19
apps/web/src/i18n/en.json	19
apps/web/src/lib/api.ts	18
apps/web/src/lib/auth.ts	18
apps/web/src/lib/contracts.ts	18
apps/web/src/lib/problem.ts	18
apps/web/tests/features/f01.test.tsx	F01
apps/web/tests/features/f02.test.tsx	F02
apps/web/tests/features/f03.test.tsx	F03
apps/web/tests/features/f04.test.tsx	F04

Path	Creator
apps/web/tests/features/f05.test.tsx	F05
apps/web/tests/features/f06.test.tsx	F06
apps/web/tests/features/f07.test.tsx	F07
apps/web/tests/features/f08.test.tsx	F08
apps/web/tests/features/f09.test.tsx	F09
apps/web/tests/features/f10.test.tsx	F10
apps/web/tests/features/f11.test.tsx	F11
apps/web/tests/features/f12.test.tsx	F12
apps/web/tests/features/f13.test.tsx	F13
apps/web/tests/features/f14.test.tsx	F14
apps/web/tests/features/f15.test.tsx	F15
apps/web/tests/features/f16.test.tsx	F16
apps/web/tests/features/f17.test.tsx	F17
apps/web/tests/features/f18.test.tsx	F18
apps/web/tests/features/f19.test.tsx	F19
apps/web/tests/features/f20.test.tsx	F20
apps/web/tests/features/f21.test.tsx	F21
apps/web/tests/features/f22.test.tsx	F22
apps/web/tests/features/f23.test.tsx	F23
apps/web/tests/features/f24.test.tsx	F24
apps/web/tests/features/f25.test.tsx	F25
apps/web/tests/features/f26.test.tsx	F26
apps/web/tests/features/f27.test.tsx	F27
apps/web/tests/features/f28.test.tsx	F28
apps/web/tests/features/f29.test.tsx	F29
apps/web/tests/features/f30.test.tsx	F30
apps/web/tests/features/f31.test.tsx	F31
apps/web/tests/features/f32.test.tsx	F32
apps/web/tests/features/f33.test.tsx	F33
apps/web/tests/features/f34.test.tsx	F34
apps/web/tests/features/f35.test.tsx	F35
apps/web/tests/features/f36.test.tsx	F36
apps/web/tests/features/f37.test.tsx	F37
apps/web/tests/features/f38.test.tsx	F38
apps/web/tests/features/f39.test.tsx	F39
apps/web/tests/features/f40.test.tsx	F40
apps/web/tests/features/f41.test.tsx	F41
apps/web/tests/features/f42.test.tsx	F42
apps/web/tests/features/f43.test.tsx	F43
apps/web/tests/features/f44.test.tsx	F44
apps/web/tests/features/f45.test.tsx	F45
apps/web/tests/features/f46.test.tsx	F46
apps/web/tests/features/f47.test.tsx	F47
apps/web/tests/features/f48.test.tsx	F48
apps/web/tests/features/f49.test.tsx	F49
apps/web/tests/features/f50.test.tsx	F50
apps/web/tests/features/f51.test.tsx	F51
apps/web/tests/features/f52.test.tsx	F52
apps/web/tests/features/f53.test.tsx	F53
apps/web/tests/features/f54.test.tsx	F54
apps/web/tests/features/f55.test.tsx	F55
apps/web/tests/features/f56.test.tsx	F56
apps/web/tests/features/f57.test.tsx	F57

Path	Creator
apps/web/tests/features/f58.test.tsx	F58
apps/web/tests/features/f59.test.tsx	F59
apps/web/tests/features/f60.test.tsx	F60
apps/web/tests/features/f61.test.tsx	F61
apps/web/tests/features/f62.test.tsx	F62
apps/web/tests/features/f63.test.tsx	F63
apps/web/tests/features/f64.test.tsx	F64
apps/web/tests/features/f65.test.tsx	F65
apps/web/tests/features/f66.test.tsx	F66
apps/web/tests/features/f67.test.tsx	F67
apps/web/tests/features/f68.test.tsx	F68
apps/web/tests/features/f69.test.tsx	F69
apps/web/tsconfig.json	18
docker-compose.yml	24
docs/DEMO_SCRIPT.md	27
docs/DEPLOYMENT_RECEIPT.md	26
docs/HOSTED_ACCEPTANCE.md	27
docs/RECRUITER_GUIDE.md	27
evals/README.md	21
evals/cases.json	21
fixtures/README.md	05
fixtures/catalogue.json	05
fixtures/scenarios.json	05
infra/Caddyfile	24
infra/compose.fixture.yml	24
infra/compose.production.yml	24
infra/deploy.sh	26
infra/rollback.sh	26
infra/snowflake/001_schema.sql	02
infra/snowflake/002_roles.sql	02
infra/snowflake/003_validation_queries.sql	02
pyproject.toml	00
supabase/config.toml	03
supabase/migrations/001_initial_schema.sql	03
supabase/migrations/002_search_and_fulfilment.sql	03
supabase/migrations/003_application.sql	03
supabase/migrations/004_rls.sql	03
supabase/migrations/005_storage.sql	03
supabase/migrations/006_plain_workflow_and_snowflake.sql	03
tests/acceptance/test_concurrency.py	22
tests/acceptance/test_recovery.py	22
tests/acceptance/test_security.py	22
tests/features/test_f01.py	F01
tests/features/test_f02.py	F02
tests/features/test_f03.py	F03
tests/features/test_f04.py	F04
tests/features/test_f05.py	F05
tests/features/test_f06.py	F06
tests/features/test_f07.py	F07
tests/features/test_f08.py	F08
tests/features/test_f09.py	F09
tests/features/test_f10.py	F10
tests/features/test_f11.py	F11

Path	Creator
tests/features/test_f12.py	F12
tests/features/test_f13.py	F13
tests/features/test_f14.py	F14
tests/features/test_f15.py	F15
tests/features/test_f16.py	F16
tests/features/test_f17.py	F17
tests/features/test_f18.py	F18
tests/features/test_f19.py	F19
tests/features/test_f20.py	F20
tests/features/test_f21.py	F21
tests/features/test_f22.py	F22
tests/features/test_f23.py	F23
tests/features/test_f24.py	F24
tests/features/test_f25.py	F25
tests/features/test_f26.py	F26
tests/features/test_f27.py	F27
tests/features/test_f28.py	F28
tests/features/test_f29.py	F29
tests/features/test_f30.py	F30
tests/features/test_f31.py	F31
tests/features/test_f32.py	F32
tests/features/test_f33.py	F33
tests/features/test_f34.py	F34
tests/features/test_f35.py	F35
tests/features/test_f36.py	F36
tests/features/test_f37.py	F37
tests/features/test_f38.py	F38
tests/features/test_f39.py	F39
tests/features/test_f40.py	F40
tests/features/test_f41.py	F41
tests/features/test_f42.py	F42
tests/features/test_f43.py	F43
tests/features/test_f44.py	F44
tests/features/test_f45.py	F45
tests/features/test_f46.py	F46
tests/features/test_f47.py	F47
tests/features/test_f48.py	F48
tests/features/test_f49.py	F49
tests/features/test_f50.py	F50
tests/features/test_f51.py	F51
tests/features/test_f52.py	F52
tests/features/test_f53.py	F53
tests/features/test_f54.py	F54
tests/features/test_f55.py	F55
tests/features/test_f56.py	F56
tests/features/test_f57.py	F57
tests/features/test_f58.py	F58
tests/features/test_f59.py	F59
tests/features/test_f60.py	F60
tests/features/test_f61.py	F61
tests/features/test_f62.py	F62
tests/features/test_f63.py	F63
tests/features/test_f64.py	F64

Path	Creator
tests/features/test_f65.py	F65
tests/features/test_f66.py	F66
tests/features/test_f67.py	F67
tests/features/test_f68.py	F68
tests/features/test_f69.py	F69
tests/foundations/test_00.py	00
tests/foundations/test_01.py	01
tests/foundations/test_02.py	02
tests/foundations/test_03.py	03
tests/foundations/test_04.py	04
tests/foundations/test_05.py	05
tests/foundations/test_06.py	06
tests/foundations/test_07.py	07
tests/foundations/test_08.py	08
tests/foundations/test_09.py	09
tests/foundations/test_10.py	10
tests/foundations/test_11.py	11
tests/foundations/test_12.py	12
tests/foundations/test_13.py	13
tests/foundations/test_14.py	14
tests/foundations/test_15.py	15
tests/foundations/test_16.py	16
tests/foundations/test_17.py	17
tests/foundations/test_18.py	18
tests/foundations/test_19.py	19
tests/foundations/test_20.py	20
tests/foundations/test_21.py	21
tests/foundations/test_22.py	22
tests/foundations/test_23.py	23
tests/foundations/test_24.py	24
tests/foundations/test_25.py	25
tests/foundations/test_26.py	26
tests/foundations/test_27.py	27
tests/hosted/test_walkthrough.py	27
tests/integration/test_publication.py	03
tests/integration/test_rls.py	03
tools/account_preflight.py	26
tools/apply_postgres.py	03
tools/apply_snowflake.py	02
tools/check_contract_drift.py	25
tools/check_secret_patterns.py	25
tools/healthcheck.py	24
tools/inspect_legacy.py	00
tools/release_gate.py	22
tools/run_checks.py	00
tools/run_evaluations.py	21
tools/seed_demo.py	05
tools/snowflake_preflight.py	02
tools/verify_kit.py	00

022 UI SPEC

docs/UI_SPEC.md

Interface and interaction specification

Use the supplied PartsIQ bilingual logo and brand tokens. Preserve artwork aspect ratio and use the provided favicon, app and social assets. Navigation includes Home, Demo, My enquiries, Saved machines and authorized Review, Catalogue and Analytics pages. Show a persistent synthetic-demo banner and explicit data/AI mode labels.

The customer page places conversation first, with attachment preview and confirmed machine details. Render persisted progress and the pending question. Multiple items have separate quantities and statuses. Customer references expose citation text, not private evidence URLs. Distinguish approved replies from status messages. Failed submissions retain input and reuse the same idempotency key on retry.

The staff workspace includes queue, extraction, candidates, comparison, evidence, stock plan, verification, draft and approval controls. Internal notes remain staff-only. Show revision and freshness before approval. A 409 response reloads current state and requires a new intentional approval. Assignment is admin-only. Imports show row-level errors, hash, version and an explicit publication action after validation.

Provide complete English and Arabic catalogs. Set html lang/dir and isolate identifiers with bdi. Language/theme changes preserve unsent input. Support keyboard focus, descriptive labels, loading/empty/error states and accessible dialogs. At 360px, stack panels and contain wide tables in scroll regions. At 200 percent zoom, keep primary actions reachable. Honor reduced motion and system color scheme. Never display fake completion percentages or fabricated live inventory.

023 DEPLOYMENT

docs/DEPLOYMENT.md

Deployment runbook

Deployment profiles

Local fixture profile is the default and requires no live OpenAI/Snowflake account. It still runs the real application, local Supabase, private storage, workflow, search and approval; AI fixture outputs are visibly labelled. Live development profile uses Supabase dev plus confirmed OpenAI/Snowflake allowance. Hosted profile uses Next.js/Vercel and one AWS EC2 host for Caddy, API, worker and Redis. Supabase production is a separate project. Render is an alternative only if the owner explicitly selects and verifies an appropriate current plan.

1 Prepare local tooling

Install the target Python 3.12, Node 22.22.0, pnpm10.30.3, Docker Engine/Compose and the pinned Supabase CLI from infrastructure receipts. Create the repository, copy the kit and execute foundation 00. Preserve the original legacy archive. Copy environment.example to an ignored .env and fill only local Supabase configuration. Run supabase start and inspect the actual local endpoints. Never use a production project URL for local tests.

Run uv venv --python 3.12 and uv pip sync spec/python-requirements.lock after the files exist. Install the frontend from the frozen manifest/lock in foundation 18. Implementation prompts create the following runnable commands; they are not shipped as already completed application code in this kit.

2 Initialize local data

Run python tools/apply_postgres.py --environment local after its foundation implementation. It must reject a nonlocal database before applying SQL 001–006. Run python tools/seed_demo.py --source fixture --seed 42. Verify counts, hash, all four scenario expectations, and synthetic evidence assets. Keep real onboarding disabled. Start docker compose -f docker-compose.yml -f infra/compose.fixture.yml up --build and run the feature tests. The API and worker must report fixture provenance; no live model call is made.

3 Configure Supabase dev

Create or select the development project within the owner's allowed plan. Apply migrations through the operator credential, then create least-privilege LOGIN roles mapped to API, workflow, worker, search andcatalogue groups. Store their passwords in secrets, not SQL source. Configure auth issuer/audience and JWKS verification; enable anonymous demo signin and invite named staff. Configure redirect URLs for localhost and the intended Vercel development domain. Set private bucket limits/policies from SQL 005. Verify two-user RLS and private downloads before exposing a demo.

4 Configure Snowflake dev

Confirm actual trial/account allowance and expiry. If unavailable, continue fixture mode and mark live integration blocked. With an authorized dev account, run the setup tool against PARTSIQ_DEV and the supplied schema/role SQL. Generate/register key-pair service-user credentials locally and store only the public key in Snowflake. Private keys remain secret-mounted files with restrictive permissions. Grant each role only the required database/schema/table and warehouse permissions.

Create one initially suspended X-Small warehouse only after allowance confirmation. Set AUTO_SUSPEND60, AUTO_RESUMEtrue and statement timeout20seconds. Attach a resource monitor based on the confirmed credit budget. Do not select an invented budget value or claim the monitor guarantees zero charges. Avoid Cortex/serverless features. Run python tools/snowflake_preflight.py --environment dev and save sanitized role/version/query receipts. Publish the synthetic source version through the same import/sync flow used by the app and compare the serving hash.

5 Enable approved OpenAI usage

Confirm that the owner-provided key has a dedicated free allowance and expiry for the selected text and vision models. Configure the exact accessible model IDs and pricing inputs in the admin allowance record. The key alone is insufficient evidence. Set AI_MODE=live only after preflight succeeds. Test a bounded structured extraction and one synthetic nameplate through the budget wrapper, recording token usage and reservation settlement. Leave fixture mode available as an explicit selection, never an automatic paid fallback.

6 Build and deploy backend

Confirm AWS account eligibility, balance, expiry and intended instance/storage/network costs. Benchmark memory before enabling BGE-M3; small free instances may not fit it. Keep semantic capability disabled transparently when capacity is insufficient, while retaining its implementation/tests. Provision only authorized covered resources. Restrict inbound ports toHTTPS/HTTP and owner-controlled SSH access; Redis and API internal ports remain private. Use the owner-controlled domain in Caddy and verifyDNS/HTTPS.

Build the immutable image with the release commit tag. Run docker compose -f docker-compose.yml -f infra/compose.production.yml config to inspect interpolation without printing secret values. Pull/build approved images, run migrations as a one-off operator job, then start API, worker, Redis and Caddy. Check liveness, readiness, durable queue recovery and restricted ports. No LangGraph/search/orchestrator services are deployed. Keep a previous compatible image tag for rollback.

7 Deploy frontend

Connect the repository to Vercel with apps/web as the project root. Use the exact build command and packageManager from the manifest. Set NEXT_PUBLIC_API_ORIGIN and matching Supabase public values. Preview environments point todev only; production points toprod only. Private Snowflake/OpenAI/database keys never enter frontend variables. Verify CORS contains the exact deployed origin and no wildcard credential policy. Confirm the owner's current Vercel plan permits the intended demo usage.

8 Hosted acceptance

Execute the four demo scenarios inEnglish/Arabic from a clean browser session. Confirm photo extraction, missing serial, incorrect part refusal, warehouse split, review andapprovedPDF. Create a second anonymous visitor and prove it cannot read the first session. Reset a session during a running OCR job and prove the late result cannot reappear. Reassign a real enquiry in dev and prove old reviewer access islost. Compare source/serving version and reporting lag. Record actual timings, not invented targets. Only then mark hosted acceptance passed.

9 Operate within allowance

Monitor allowance expiry, worker backlog, failed jobs, stock freshness, active catalogue hash andreporting delay. Stop live scheduled work before confirmed free access expires. AWS and Snowflake free access may be temporary; local fixture mode is the durable demonstration alternative. Back up PostgreSQL andprivate evidence according to the owner-approved retention policy. Do not claim high availability for a single EC2 host. Deployment doesnot change the human approval gate.

Render alternative

If the owner later chooses Render, run the same API image and an appropriate worker service with privateRedis access and the sameSupabase/Snowflake contracts. Verify current service andworker plan limits before selection. Do not substitute a sleeping web-only process for durable jobs or assume analways-onworker is free. Update ADR002 andrerun hosted acceptance after changing hosting.

024 ROLLBACK

docs/ROLLBACK.md

Rollback and recovery

Before each implementation prompt, commit a clean checkpoint and record the current schema and catalogue version. Revert only the intended commit after checking later dependencies. Never use a hard reset that discards unrelated work. Reverse applied migrations through tested forward compensating migrations; never reset a shared database.

For deployment, select the previous immutable API and worker image pair compatible with the current schema. Pause new jobs, allow or expire active leases, restart the old pair, and verify readiness before resuming. The web release must match the supported API contract. Rollback must preserve compatibility checks and human approval.

If catalogue activation has not committed, the old pointer remains active. A remote-only version can be reconciled by its hash and count. If an activated version must be reverted, validate the previous snapshot and stock freshness, then make a new audited activation. Historical approvals retain their original facts and evidence.

For reporting failures, retain pending PostgreSQL outbox events and retry with the same event IDs. For uncertain OpenAI completion, retain usage reservations until reconciliation. Redis loss does not remove durable jobs; rate-limited intake fails closed. To recover a lost host, restore the committed Compose configuration and secrets, then recover jobs through their leases. Rehearse restores in development before claiming backup recovery works.

025 TEST CHECKLIST

docs/TEST_CHECKLIST.md

Test strategy and acceptance gates

Specification checks versus application tests

The kit validator checks references, schemas, prompt coverage and archive structure. It does not prove that the application works. The implementation agent must execute every named test as source is built. Never mark live Snowflake, OpenAI, RLS or hosted acceptance passed using mocks.

Test layers

Unit tests cover Arabic normalization, serial boundaries, fitment filtering, supersession cycles, rank margins, allocation bounds, fact hashes, fixed transitions and redaction. Integration tests use real disposable PostgreSQL constraints, RLS, actor context reset, Storage policies, idempotency, approval races, job leases and catalogue activation. Contract tests compare the 77 REST operations, strict fields, Problem responses and frontend types.

Each of the 69 features has backend and frontend acceptance paths. Browser tests cover both languages, RTL, keyboard navigation, 360px layouts, themes, long content and four demos. Provider checks separately exercise Snowflake key-pair login, round-trip hashes and duplicate validation, plus OpenAI structured text/vision and allowance accounting. Hosted checks cover CORS, TLS, private ports, worker restart and observed latency.

Required failure cases

Failure	Required behavior
Exact part number is incompatible	Exclude before ranking
Two candidates have close scores	Ask a question or require staff selection
OCR identifiers are unconfirmed	Do not resolve the machine
Quantity changes during processing	Discard old lease output
Two approvals race	Publish once for the current revision
Stock changes before approval	Return 409 and require replanning/review
Source version has duplicate keys	Do not activate
Remote upload succeeds but local commit fails	Keep old version active
Reporting acknowledgement is lost	Replay the same event ID without duplication
Redis is unavailable	Deny rate-limited intake; retain durable jobs
Provider completion is uncertain	Hold the usage reservation
Free allowance expires	Block live work; no paid fallback
Demo is reset during a job	Reject old-generation writes
Reviewer assignment changes	Remove previous reviewer access
Instructions are injected into text/photo	No privileged action occurs

Evidence and final gate

For every check, record command, environment, commit, time, expected result, actual result and sanitized logs. Accuracy requires a denominator; latency requires hardware and runtime context. A skip is not a pass. A resolver result is not a runtime build, and SQL parsing is not live policy validation. Do not remove required tests to obtain a green result.

During incremental development, CI validates completed prompt owners and requires that their named tests exist. The final release gate requires every feature, all 77 operations, and all mandatory tests. Live checks lacking account access remain visibly blocked; they cannot be replaced with fixture evidence.

026 FIELD GUIDE MAP

docs/FIELD_GUIDE_MAP.md

Collaboration practices

The original AI-Collaboration-Field-Guide.pdf remains in references for source review. The following practices apply its collaboration intent; these are not verbatim quotations from the source.

1. Explain product purpose in CONTEXT and PRD.
2. Establish architecture boundaries before feature edits.
3. Execute one sequenced prompt at a time.
4. Read existing legacy code before rewriting it.
5. Define typed contracts before UI assumptions.
6. Specify dependencies and exact installed versions.
7. Name source paths and creation ownership.
8. Make small independently reviewable commits.
9. Test behavior and negative boundaries.
10. Preserve actual failure evidence.
11. Maintain handover and session logs across sessions.
12. Keep security and cost constraints visible.
13. Separate model outputs from deterministic decisions.
14. Require human review before publication.
15. Verify artifacts and deployment rather than trusting generated claims.

ARCHITECTURE.md answers the eight architecture questions. Every implementation prompt includes context, contracts, logic, API flow, tests, commands, documentation, rollback and completion evidence.

027 FEATURE MATRIX

docs/FEATURE_MATRIX.md

Feature ownership matrix

Feature	Purpose	Prompt	Backend test
F01	Branded landing	prompts/F01-branded-landing.md	tests/features/test_f01.py
F02	Demo scenarios and assets	prompts/F02-demo-scenarios-and-assets.md	tests/features/test_f02.py
F03	Demo role switching	prompts/F03-demo-role-switching.md	tests/features/test_f03.py
F04	Demo isolation	prompts/F04-demo-isolation.md	tests/features/test_f04.py
F05	Demo reset	prompts/F05-demo-reset.md	tests/features/test_f05.py
F06	Responsive layouts	prompts/F06-responsive-layouts.md	tests/features/test_f06.py
F07	Arabic and English interface	prompts/F07-arabic-and-english-interface.md	tests/features/test_f07.py
F08	Light and dark themes	prompts/F08-light-and-dark-themes.md	tests/features/test_f08.py
F09	Bilingual chat	prompts/F09-bilingual-chat.md	tests/features/test_f09.py
F10	Structured extraction	prompts/F10-structured-extraction.md	tests/features/test_f10.py
F11	Clarifying questions	prompts/F11-clarifying-questions.md	tests/features/test_f11.py
F12	Conversation memory	prompts/F12-conversation-memory.md	tests/features/test_f12.py
F13	Private uploads	prompts/F13-private-uploads.md	tests/features/test_f13.py
F14	Nameplate OCR	prompts/F14-nameplate-ocr.md	tests/features/test_f14.py
F15	OCR confirmation	prompts/F15-ocr-confirmation.md	tests/features/test_f15.py
F16	Photo quality feedback	prompts/F16-photo-quality-feedback.md	tests/features/test_f16.py
F17	Multi-part enquiries	prompts/F17-multi-part-enquiries.md	tests/features/test_f17.py
F18	Progress timeline	prompts/F18-progress-timeline.md	tests/features/test_f18.py
F19	Enquiry history	prompts/F19-enquiry-history.md	tests/features/test_f19.py
F20	Machine identification	prompts/F20-machine-identification.md	tests/features/test_f20.py
F21	Machine mismatch detection	prompts/F21-machine-mismatch-detection.md	tests/features/test_f21.py
F22	Compatibility gate	prompts/F22-compatibility-gate.md	tests/features/test_f22.py
F23	Aliases and typo matching	prompts/F23-aliases-and-typo-matching.md	tests/features/test_f23.py
F24	Semantic retrieval	prompts/F24-semantic-retrieval.md	tests/features/test_f24.py
F25	Ranked candidates	prompts/F25-ranked-candidates.md	tests/features/test_f25.py
F26	Ambiguity handling	prompts/F26-ambiguity-handling.md	tests/features/test_f26.py
F27	Match explanations	prompts/F27-match-explanations.md	tests/features/test_f27.py
F28	Compatibility references	prompts/F28-compatibility-references.md	tests/features/test_f28.py
F29	Evidence page viewer	prompts/F29-evidence-page-	tests/features/test_f29.py

Feature	Purpose	Prompt	Backend test
		viewer.md	
F30	Old part numbers	prompts/F30-old-part-numbers.md	tests/features/test_f30.py
F31	Candidate comparison	prompts/F31-candidate-comparison.md	tests/features/test_f31.py
F32	Warehouse stock	prompts/F32-warehouse-stock.md	tests/features/test_f32.py
F33	Fulfilment planning	prompts/F33-fulfilment-planning.md	tests/features/test_f33.py
F34	Transfer estimates	prompts/F34-transfer-estimates.md	tests/features/test_f34.py
F35	Stock freshness	prompts/F35-stock-freshness.md	tests/features/test_f35.py
F36	Staff inbox	prompts/F36-staff-inbox.md	tests/features/test_f36.py
F37	Review workspace	prompts/F37-review-workspace.md	tests/features/test_f37.py
F38	Correct extracted details	prompts/F38-correct-extracted-details.md	tests/features/test_f38.py
F39	Review actions	prompts/F39-review-actions.md	tests/features/test_f39.py
F40	Approval invalidation	prompts/F40-approval-invalidation.md	tests/features/test_f40.py
F41	Internal notes	prompts/F41-internal-notes.md	tests/features/test_f41.py
F42	Assignments	prompts/F42-assignments.md	tests/features/test_f42.py
F43	Overdue indicators	prompts/F43-overdue-indicators.md	tests/features/test_f43.py
F44	Conversation summary	prompts/F44-conversation-summary.md	tests/features/test_f44.py
F45	Reply drafting	prompts/F45-reply-drafting.md	tests/features/test_f45.py
F46	Verification gate	prompts/F46-verification-gate.md	tests/features/test_f46.py
F47	Human publication gate	prompts/F47-human-publication-gate.md	tests/features/test_f47.py
F48	Input isolation	prompts/F48-input-isolation.md	tests/features/test_f48.py
F49	Audit trail	prompts/F49-audit-trail.md	tests/features/test_f49.py
F50	Durable workflows	prompts/F50-durable-workflows.md	tests/features/test_f50.py
F51	Background jobs	prompts/F51-background-jobs.md	tests/features/test_f51.py
F52	Retry and deduplication	prompts/F52-retry-and-deduplication.md	tests/features/test_f52.py
F53	Graceful failures	prompts/F53-graceful-failures.md	tests/features/test_f53.py
F54	Access and permissions	prompts/F54-access-and-permissions.md	tests/features/test_f54.py
F55	Saved machines	prompts/F55-saved-machines.md	tests/features/test_f55.py
F56	Repeat enquiry	prompts/F56-repeat-enquiry.md	tests/features/test_f56.py
F57	Catalogue browser	prompts/F57-catalogue-browser.md	tests/features/test_f57.py
F58	Catalogue imports	prompts/F58-catalogue-imports.md	tests/features/test_f58.py
F59	Catalogue versioning	prompts/F59-catalogue-versioning.md	tests/features/test_f59.py
F60	Missing-parts queue	prompts/F60-missing-parts-queue.md	tests/features/test_f60.py
F61	Enquiry PDF	prompts/F61-enquiry-pdf.md	tests/features/test_f61.py

Feature	Purpose	Prompt	Backend test
F62	Labelled evaluations	prompts/F62-labelled-evaluations.md	tests/features/test_f62.py
F63	Metrics dashboard	prompts/F63-metrics-dashboard.md	tests/features/test_f63.py
F64	Latency and AI cost	prompts/F64-latency-and-ai-cost.md	tests/features/test_f64.py
F65	Correction feedback	prompts/F65-correction-feedback.md	tests/features/test_f65.py
F66	Tests and CI	prompts/F66-tests-and-ci.md	tests/features/test_f66.py
F67	Traces	prompts/F67-traces.md	tests/features/test_f67.py
F68	Recruiter documentation	prompts/F68-recruiter-documentation.md	tests/features/test_f68.py
F69	Demo walkthrough	prompts/F69-demo-walkthrough.md	tests/features/test_f69.py

028 HANDOVER

docs/HANDOVER.md

Implementation handover

Current edition: single workflow with Snowflake, 16 October 2026. Status: build kit authored; application implementation not executed. Scope:69 features, 97 prompts and77 REST operations. Start with00 unless the repository contains actual completed source and test receipts.

The previous multi-agent architecture is superseded. Do not restore its checkpoint DDL, agent packages or separate internal services. Legacy backend and brand archives remain references; current documentation and typed contracts govern new work.

Before coding, read AGENTS.md, CONTEXT.md, PRD.md, ARCHITECTURE.md and the current prompt. After each prompt, append changed files, actual test commands/results, commit, blocked live checks and next prompt. Preserve failure history. Missing account access is a deployment input, not permission to invent credentials or successful verification.

029 SESSION LOG

docs/SESSION_LOG.md

Session log

16 October 2026: Revised the build kit from multi-agent orchestration to a fixed Python workflow and Snowflake catalogue/reporting integration while preserving all 69 features. Generated detailed prompts, typed REST contracts and updated specifications. Application implementation, live provider tests and deployment remain future execution steps. Authoring checks are recorded in SELF_AUDIT.md.

030 RECRUITER GUIDE

docs/RECRUITER_GUIDE.md

Recruiter project explanation

PartsIQ is a bilingual spare-parts enquiry platform. OpenAI extracts customer text and nameplate details. Python identifies equipment, filters compatible parts, plans warehouse quantities and validates the reply. Staff approve the final recommendation. Snowflake supplies versioned catalogue data and redacted reporting; Supabase provides transactional records, authentication and private files.

The engineering story is the boundary between probabilistic extraction and deterministic decisions. Demonstrate serial-range constraints, compatibility before ranking, revision-bound approval, job leases, idempotent REST writes and recoverable Snowflake synchronization. Explain why interactive requests use a validated relational cache and why Snowflake source constraints are validated explicitly.

Do not describe this as a multi-agent system or claim real distributor adoption without evidence. Add repository, deployed application, test receipts and recorded walkthrough links only when they exist. A fixture deployment demonstrates application behavior but must be labelled separately from live provider integration evidence.

031 DEMO SCRIPT

docs/DEMO_SCRIPT.md

Recruiter walkthrough script

Three minute version

First 30 seconds: explain the bilingual enquiry problem and human approval. Show the synthetic-data notice and single-backend architecture. Next 60 seconds: run the mixed-language photo scenario, confirm identifiers, and show compatibility evidence and the draft. Next 30 seconds: approve and display the published reply. Next 30 seconds: demonstrate a missing serial or incompatible part and show the system refusing to guess. Final 30 seconds: show the warehouse split, Snowflake source version and test/audit evidence. State which providers are live and which are fixtures.

Eight minute version

Explain the problem and roles. Walk through chat, photo and confirmation. Show the fixed workflow and why agent routing is unnecessary. Demonstrate serial ambiguity and an incompatible exact part number. Compare candidate evidence and stock freshness. Edit an item and demonstrate approval invalidation. Approve and export the result. Show Snowflake source/reporting and the validated serving cache. Finish with measured tests, cost controls and single-host limitations.

Recording checklist

Use synthetic data and clean browser profiles. Hide keys, JWTs, signed URLs and billing identifiers. Verify all four scenarios before recording. Include both languages and one failure path. Do not claim a video exists until recorded. Add captions and check published links. Accuracy claims require denominators; latency claims require actual hardware/runtime context.

032 SELF AUDIT

docs/SELF_AUDIT.md

Build kit self audit

This is a specification audit, not application runtime acceptance. Structural results are stored in spec/kit-validation.json. Dependency resolution, document rendering and archive integrity are recorded separately.

Check	Verification boundary	Correction
69 features retained	Feature, prompt, trace and test mapping	Every feature now has an implementation prompt
No agent runtime	Active architecture and dependency review	Removed LangGraph and checkpoint DDL
REST contracts	77 operations and typed schemas	Explicit request, response, access and status
File ownership	One creator per planned source path	Regenerated for one application package
Build order	No forward declared dependency	All features precede final release gates
Snowflake integrity	Validation and activation protocol	No reliance on unenforced PK/FK metadata
Database specification	Exact PostgreSQL and Snowflake DDL	Plain workflow persistence and stock provenance
Dependencies	Direct release receipts and reference locks	Added Snowflake 4.8.0 and removed agent libraries
Cost constraints	Account allowance must be confirmed	Explicit fixture mode; no permanent-free claim
Application tests	Not executed during kit authoring	Named cases must run during implementation
Live provider checks	Not executed	Owner accounts and allowance required
Production deployment	Not performed	Runbook and hosted acceptance gate included
Original assets/backend	Preserved	Historical archives remain unchanged

033 BRAND

brand/BRAND.md

PartsIQ brand system

Use supplied falcon/gear/magnifier bilingual mark. Preserve aspect ratio; do not redraw the Arabic text. Primary navy#2B2D7F and gold#C59A45 are standardized tokens; the raster artwork also contains gradients and antialiasing. Gold is decorative/accent, not small text on white. Use navy for primary action, neutral surfaces and semantic state colours from tokens.json. Dark-mode muted/border overrides preserve readable contrast.

Typography: Noto Sans for English, Noto Sans Arabic for Arabic UI, Noto Naskh Arabic for Arabic PDF. Licensed font files and OFL notices are bundled. Use font-display: swap; self-host fonts; never depend on external PDF font fetch. Use bdi for part numbers and serials.

Use full lockup at minimum160CSSpx width in header/landing; icon-only at24/32px when space is constrained. Leave clear space equal to one-quarter icon width. Never stretch, crop critical mark elements, place on a busy photo or imply endorsement by a dealer. Small favicon uses supplied icon crop, not illegible full bilingual lockup.

Exports: favicon ICO16/32/48; PNG16/32/48/96; Apple180; PWA192/512; maskable192/512; social1200x630; profile1080; full logo320/640/1280. Existing supplied pack is preserved under assets. Asset manifest records actual dimensions and SHA256; use it rather than guessing file names. No authentic vector source was supplied, so no SVG is falsely presented as original artwork.

Prompt 18 copies every manifest-listed asset and fonts into apps/web/public/brand. PDF worker copies the same font bytes and logo into its image. Brand exports are already bundled; new raster editing is unnecessary.

034 README

prompts/README.md

Build order

Execute one prompt at a time in this order. Numeric filename order is not execution order. Foundations00–21 and23–25 create shared functionality and release tooling; F01–F69 implement the entire product; 22 and26–27 run final gates and deployment.

Order	Prompt	Purpose
1	[00](00-bootstrap.md)	Repository bootstrap and legacy characterization
2	[01](01-core.md)	Typed contracts and application configuration
3	[02](02-snowflake.md)	Snowflake connector and validated catalogue source
4	[03](03-database.md)	Supabase schema permissions and serving cache
5	[04](04-storage.md)	Private uploads evidence and storage access
6	[05](05-fixtures.md)	Synthetic catalogue and four scenario fixtures
7	[06](06-search.md)	Compatibility lexical matching and optional embeddings
8	[07](07-auth.md)	Authentication ownership and demo isolation
9	[08](08-intake.md)	Enquiry intake idempotency and rate limiting
10	[09](09-workflow.md)	Plain Python durable workflow
11	[10](10-extraction.md)	OpenAI extraction and usage allowance
12	[11](11-resolution.md)	Machine search and fulfilment application flow
13	[12](12-verification.md)	Deterministic verification and fact hashing
14	[13](13-drafts.md)	Bilingual reply slots and safe rendering
15	[14](14-approval.md)	Review transactions revisions and publication
16	[15](15-jobs.md)	Durable worker leases retries and cleanup
17	[16](16-vision.md)	Nameplate reading and human confirmation
18	[17](17-exports.md)	Approved bilingual PDF export
19	[18](18-web.md)	Next.js shell branding authentication and API client
20	[19](19-customer_web.md)	Customer pages and demo integration shell
21	[20](20-staff_web.md)	Staff pages and authorized review shell
22	[21](21-evaluation.md)	Deterministic evaluation and evidence receipts
23	[23](23-analytics.md)	Structured logs tracing metrics and Snowflake outbox
24	[24](24-containers.md)	One application image worker and Caddy
25	[25](25-ci.md)	CI reproducible builds and release checks
26	[F01](F01-branded-landing.md)	Branded landing
27	[F02](F02-demo-scenarios-and-assets.md)	Demo scenarios and assets
28	[F03](F03-demo-role-switching.md)	Demo role switching
29	[F04](F04-demo-isolation.md)	Demo isolation
30	[F05](F05-demo-reset.md)	Demo reset

Order	Prompt	Purpose
31	[F06](F06-responsive-layouts.md)	Responsive layouts
32	[F07](F07-arabic-and-english-interface.md)	Arabic and English interface
33	[F08](F08-light-and-dark-themes.md)	Light and dark themes
34	[F09](F09-bilingual-chat.md)	Bilingual chat
35	[F10](F10-structured-extraction.md)	Structured extraction
36	[F11](F11-clarifying-questions.md)	Clarifying questions
37	[F12](F12-conversation-memory.md)	Conversation memory
38	[F13](F13-private-uploads.md)	Private uploads
39	[F14](F14-nameplate-ocr.md)	Nameplate OCR
40	[F15](F15-ocr-confirmation.md)	OCR confirmation
41	[F16](F16-photo-quality-feedback.md)	Photo quality feedback
42	[F17](F17-multi-part-enquiries.md)	Multi-part enquiries
43	[F18](F18-progress-timeline.md)	Progress timeline
44	[F19](F19-enquiry-history.md)	Enquiry history
45	[F20](F20-machine-identification.md)	Machine identification
46	[F21](F21-machine-mismatch-detection.md)	Machine mismatch detection
47	[F22](F22-compatibility-gate.md)	Compatibility gate
48	[F23](F23-aliases-and-typo-matching.md)	Aliases and typo matching
49	[F24](F24-semantic-retrieval.md)	Semantic retrieval
50	[F25](F25-ranked-candidates.md)	Ranked candidates
51	[F26](F26-ambiguity-handling.md)	Ambiguity handling
52	[F27](F27-match-explanations.md)	Match explanations
53	[F28](F28-compatibility-references.md)	Compatibility references
54	[F29](F29-evidence-page-viewer.md)	Evidence page viewer
55	[F30](F30-old-part-numbers.md)	Old part numbers
56	[F31](F31-candidate-comparison.md)	Candidate comparison
57	[F32](F32-warehouse-stock.md)	Warehouse stock
58	[F33](F33-fulfilment-planning.md)	Fulfilment planning
59	[F34](F34-transfer-estimates.md)	Transfer estimates
60	[F35](F35-stock-freshness.md)	Stock freshness
61	[F36](F36-staff-inbox.md)	Staff inbox
62	[F37](F37-review-workspace.md)	Review workspace
63	[F38](F38-correct-extracted-details.md)	Correct extracted details
64	[F39](F39-review-actions.md)	Review actions
65	[F40](F40-approval-invalidation.md)	Approval invalidation
66	[F41](F41-internal-notes.md)	Internal notes
67	[F42](F42-assignments.md)	Assignments
68	[F43](F43-overdue-indicators.md)	Overdue indicators
69	[F44](F44-conversation-summary.md)	Conversation summary
70	[F45](F45-reply-drafting.md)	Reply drafting
71	[F46](F46-verification-gate.md)	Verification gate
72	[F47](F47-human-publication-gate.md)	Human publication gate
73	[F48](F48-input-isolation.md)	Input isolation
74	[F49](F49-audit-trail.md)	Audit trail
75	[F50](F50-durable-workflows.md)	Durable workflows
76	[F51](F51-background-jobs.md)	Background jobs
77	[F52](F52-retry-and-deduplication.md)	Retry and deduplication
78	[F53](F53-graceful-failures.md)	Graceful failures
79	[F54](F54-access-and-permissions.md)	Access and permissions
80	[F55](F55-saved-machines.md)	Saved machines
81	[F56](F56-repeat-enquiry.md)	Repeat enquiry

Order	Prompt	Purpose
82	[F57](F57-catalogue-browser.md)	Catalogue browser
83	[F58](F58-catalogue-imports.md)	Catalogue imports
84	[F59](F59-catalogue-versioning.md)	Catalogue versioning
85	[F60](F60-missing-parts-queue.md)	Missing-parts queue
86	[F61](F61-enquiry-pdf.md)	Enquiry PDF
87	[F62](F62-labelled-evaluations.md)	Labelled evaluations
88	[F63](F63-metrics-dashboard.md)	Metrics dashboard
89	[F64](F64-latency-and-ai-cost.md)	Latency and AI cost
90	[F65](F65-correction-feedback.md)	Correction feedback
91	[F66](F66-tests-and-ci.md)	Tests and CI
92	[F67](F67-traces.md)	Traces
93	[F68](F68-recruiter-documentation.md)	Recruiter documentation
94	[F69](F69-demo-walkthrough.md)	Demo walkthrough
95	[22](22-security.md)	Security concurrency and failure recovery acceptance
96	[26](26-deployment.md)	AWS and Snowflake account setup and deployment
97	[27](27-hosted.md)	Vercel configuration and complete hosted walkthrough

```
flowchart TD
 A["Foundations 00 to 21"] --> B["Analytics and release tooling 23 to 25"]
 B --> C["Features F01 to F69"]
 C --> D["Security gate 22"]
 D --> E["Hosted deployment 26 to 27"]
```

035 FLOW

docs/FLOW.md

Fixed Python workflow

The controller is a deterministic transition table, not an LLM router. Persist state in workflow_runs and completed outputs in workflow_step_results. The unique key is enquiry_id plus revision. A job lease and a separate input hash fence every committed result.

Current step	Condition	Next step	Function owner
intake	Text parsed and no image pending	identify or clarify	ai.extraction.extract_message
intake	Validated image requires reading	await_ocr	jobs.nameplate.read_nameplate
await_ocr	OCR result persisted	confirm	enquiries.extraction.confirm_nameplate
confirm	Customer confirms identifiers	identify	workflow.resolution.resolve_items
identify	Missing or conflicting machine details	clarify	enquiries.questions.clarification
identify	One configuration resolved for each item	search	search.service.SearchService
clarify	New answer accepted in new revision	intake or identify	enquiries.intake.append_message
search	No compatible result	needs_staff	workflow.resolution.resolve_items
search	Close candidates	choose	enquiries.questions.clarification
search	Unique sufficient match	plan	enquiries.fulfilment.plan_enquiry
choose	Current candidate chosen	plan	enquiries.revisions.choose_candidate
plan	Complete or explicitly short plan	draft	ai.reply.draft_slots
plan	Missing/stale inventory	needs_staff	enquiries.fulfilment.plan_enquiry
draft	Structured slots rendered	verify	enquiries.verification.verify_facts
verify	All checks pass	review	workflow.repository.commit_step
verify	Any rule fails	needs_staff	enquiries.verification.verify_facts
review	Current revision approved	complete	enquiries.approval.approve
review	Corrected/rejected	new revision or terminal rejection	enquiries.revisions or approval

Each transition has a typed StepOutcome with reason_code and model-validated step data. The data envelope is not permission to store arbitrary provider output. Define a concrete payload model for each step using the provided IntakeResult, MachineView, CandidateList, PlanView, ReplySlots and VerificationResult contracts. Validate the matching payload before persistence.

Persistence algorithm

Claim a due job with FOR UPDATE SKIP LOCKED and a random lease token. Load enquiry/session and reject obsolete revision/generation. Under a short transaction, claim the matching workflow run. Release locks before calling OpenAI, Storage or Snowflake. Hash the exact step inputs and reuse only an identical completed result. Perform one bounded step. Reacquire row lock, recheck lease/revision/generation, write output and next state, append audit, then commit. Schedule the next step or park at a waiting state. Renew leases while healthy; never let an expired process finalize a result. Stop after20automatic transitions in one job.

No transaction remains open across a provider call. There is no promise of exactly-once external API execution; uncertain provider completions remain conservatively charged/reserved. Business publication is effectively once through idempotency and a unique recommendation-per-revision index.

User changes

New facts or item choice invalidate derived data by creating a new revision. Preserve old outputs for audit but do not reuse them without matching input hash and source version. Assignment and notes need not change the extraction revision unless they alter facts. An assignment has its own audit event and immediate access effect. Reset increments demo generation and makes old jobs obsolete. User questions are code-owned bilingual templates and do not require staff approval because they make no final part recommendation.

Cross-feature dependencies

Search and verification share one source of fitment truth. Draft rendering consumes VerifiedFacts only. Approval rechecks freshness and fact hash. PDF uses ApprovedSnapshot only. Analytics observes committed business events and cannot block approval. AI allowance remains local and authoritative even when traces or Snowflake reporting are unavailable.

036 REQUIREMENTS

docs/REQUIREMENTS.md

Product requirements document

Purpose and success criteria

PartsIQ helps customers describe equipment spare-part needs and helps staff prepare a defensible response. The system reduces manual interpretation and catalogue searching while keeping compatibility, quantity allocation and final approval under deterministic control. A successful recruiter demonstration shows the full enquiry journey, one ambiguity case, one incompatible request and one warehouse split, including visible provenance and human review.

Acceptance is behavioral rather than a claimed accuracy number: an incompatible exact-number match must never be recommended; unconfirmed photo identifiers must not drive matching; ambiguous candidates must not be silently selected; an unassigned reviewer must not see real enquiries; and a recommendation must not publish without a matching approved revision. Performance and extraction accuracy are measured later using the supplied evaluation plan.

Actors and permissions

Public visitor reads the landing page and architecture. Anonymous demo customer owns an expiring demo session and can switch to reviewer presentation only for that session. Invited customer owns real enquiries when real onboarding is enabled. Reviewer sees assigned real enquiries and can correct, draft, approve or reject. Catalogue administrator stages, validates and publishes source versions. Administrator manages assignments, allowances, evaluation curation and aggregate reporting. Runtime worker identities are technical roles, not human roles.

Main user journey

Customer creates an enquiry with destination warehouse and up to20 requested items. Optional images are privately uploaded, validated and read in the background. Structured extraction preserves the original message and source spans. The customer confirms photo identifiers and answers missing-information questions. Machine identification resolves one configuration. Search retrieves only explicitly compatible parts and supplies evidence. Close scores trigger a choice or review. Fulfilment combines repeated-part demand across items and reports availability, shortage and transfer estimates without reserving stock. A constrained draft is verified and presented to staff. Approval rechecks revision, catalogue and stock before atomically recording the decision and publishing the customer reply. Approved information can be exported as a bilingual enquiry PDF.

Complete feature scope

Every feature below remains mandatory unless its behavior explicitly describes a capability flag, such as optional semantic ranking. Optional capability means implemented and testable with a transparent disabled mode, not silently omitted.

F01 Branded landing

Explain the enquiry-to-approval journey; show synthetic-demo notice; expose Start demo and architecture links; display the supplied bilingual logo without stretching.

Acceptance and failure boundaries: Missing repository link hides that action; logo has meaningful alternative text; no affiliation claim.

F02 Demo scenarios and assets

Seed four deterministic scenarios: mixed-language photo, missing serial, incompatible part, local shortage; derive identifiers from seed 42; validate expected outcomes against generated records.

Acceptance and failure boundaries: Reject a scenario whose expected part lacks compatibility; reject stock claims not supported by fixture quantities.

F03 Demo role switching

Switch the presentation between customer and reviewer using the same anonymous identity and session; authorize review only for enquiries belonging to that active session.

Acceptance and failure boundaries: Forged reviewer role never grants staff membership; another visitor's enquiry returns 404.

F04 Demo isolation

Bind each session to verified anonymous Supabase identity; check generation and expiry on every read, write and worker completion; exclude demos from operational metrics by default.

Acceptance and failure boundaries: Expired, reset and foreign sessions fail; missing identity context returns no rows.

F05 Demo reset

Lock session; increment generation; cancel unfinished jobs from prior generation; enqueue Storage cleanup; keep minimal audit; create fresh scenario enquiries on explicit selection.

Acceptance and failure boundaries: Late OCR from old generation cannot update current data; repeated reset idempotency key returns the same generation.

F06 Responsive layouts

Support 360, 768, 1280 CSS-pixel widths; stack review panels; horizontally scroll comparison tables within their region; preserve sticky primary review actions without covering content.

Acceptance and failure boundaries: No page-wide horizontal overflow; zoom 200 percent remains usable; mobile dialogs retain visible close action.

F07 Arabic and English interface

Use a locale cookie and complete en/ar message catalogs; persist preference; set document lang and dir; isolate serials, numbers and part identifiers with bdi.

Acceptance and failure boundaries: Missing translation fails tests; language switch preserves enquiry ID and unsent draft; mixed Arabic identifiers remain readable.

F08 Light and dark themes

Use CSS semantic tokens; initialize from saved preference then system setting; expose accessible toggle; honor reduced motion and browser color-scheme.

Acceptance and failure boundaries: No hydration mismatch; chart colors and focus rings meet contrast checks; storage-disabled browsers still work.

F09 Bilingual chat

Accept Arabic, English and mixed text; preserve original message; detect preferred reply locale; render customer messages immediately after successful persistence.

Acceptance and failure boundaries: Empty and oversized input returns validation error; HTML is escaped; retry does not duplicate a message.

F10 Structured extraction

Extract item descriptions, quantities, urgency and machine hints with strict structured outputs; preserve source spans; missing quantity becomes a clarification rather than assumed order quantity.

Acceptance and failure boundaries: Unknown keys and invalid quantities reject the response; unsupported model output cannot progress to search.

F11 Clarifying questions

Use workflow reason codes to choose code-owned bilingual questions; persist pending question; associate answer with item; rerun affected resolution only.

Acceptance and failure boundaries: No final recommendation before approval; contradictory answer requires confirmation; repeat event creates one question.

F12 Conversation memory

Persist all messages and typed current state; compact context using retained structured facts; resume by enquiry/revision; never mix customer sessions.

Acceptance and failure boundaries: Restart retains unresolved questions; old revision events are ignored and audited; foreign thread ID denied.

F13 Private uploads

Create pending attachment and unique object path; use signed upload; validate actual bytes and magic type after confirmation; strip EXIF and cap decoded pixels before OCR.

Acceptance and failure boundaries: Wrong MIME, oversized content, image decompression bomb, duplicate confirmation and nonexistent object handled explicitly.

F14 Nameplate OCR

Read validated nameplate via OpenAI vision; return model_text, serial_text, issues and source attachment; persist result only with current lease and revision.

Acceptance and failure boundaries: Unreadable fields remain null; vision instructions in photo are untrusted; timeout is bounded and creates recoverable status.

F15 OCR confirmation

Show cropped or original nameplate beside extracted identifiers; require user confirm or correction; record actor and time; preserve leading zeros in serial text.

Acceptance and failure boundaries: Unconfirmed OCR never authorizes matching; blank corrected serial triggers missing-serial flow; stale confirmation returns 409.

F16 Photo quality feedback

Check decodability, dimensions and blur heuristic before paid call; return actionable retake instructions; mark heuristic as advisory; allow a readable image despite noncritical blur warning.

Acceptance and failure boundaries: Unsupported format rejected; dark or low-resolution sample gets guidance; no fabricated OCR confidence percentage.

F17 Multi-part enquiries

Track up to 20 items with independent quantity, machine, candidate selection and status; clarify individual items; verify aggregate allocation for repeated part requests.

Acceptance and failure boundaries: One unresolved item blocks final all-items approval; allocations across duplicate parts cannot double count available stock; zero quantity rejected.

F18 Progress timeline

Show persisted stages, wait reason and last update; poll with bounded backoff; reconnect using server state; separate waiting for customer from waiting for staff.

Acceptance and failure boundaries: No fake completed progress; failed poll retains last state and offers retry; hidden tabs reduce polling.

F19 Enquiry history

Return cursor-paginated owned history with status and created time; filter active demo generation; link to full conversation; use stable created_at/id ordering.

Acceptance and failure boundaries: Foreign owner ID ignored; invalid cursor returns 422; equal timestamps do not duplicate or skip records.

F20 Machine identification

Preserve identify_machine logic for model, serial and rating; return resolved configuration or explicit missing/ambiguous status; normalize numeric serial only after preserving input.

Acceptance and failure boundaries: Boundary serials match exactly one configuration; overlapping ranges fail Snowflake publication validation and PostgreSQL serving-cache insertion; conflicting hints never silently override.

F21 Machine mismatch detection

Compare model and serial together; return mismatch reason with safe next question; prevent retrieval while identity unresolved.

Acceptance and failure boundaries: Valid serial of a different model does not match requested model; nonnumeric serial handled without crashing.

F22 Compatibility gate

Fetch authoritative fitment set first; rank only members; carry source_reference and catalogue version through every candidate.

Acceptance and failure boundaries: A perfect lexical match outside fitment is absent; empty fitment returns unavailable; blank provenance cannot be published.

F23 Aliases and typo matching

Preserve Arabic normalization, part-number keys, phrase scoring and fuzzy token matching; remove machine terms using resolved model context.

Acceptance and failure boundaries: Arabic part words are not removed as machine words; exact part number outranks unrelated fuzzy alias; normalization parity test matches SQL.

F24 Semantic retrieval

Load one BGE-M3 encoder in the worker search module; use normalized 1024-d vectors; retain legacy weighted fusion; fall back to lexical on unavailable embedding path.

Acceptance and failure boundaries: Wrong dimension rejected; no model loaded in API process; no semantic candidate outside compatible IDs.

F25 Ranked candidates

Return top candidates with lexical/semantic evidence and deterministic tie order; compute runner-up margin before applying response limit.

Acceptance and failure boundaries: Limit one still reports ambiguity; empty set returns no candidate; NaN scores rejected.

F26 Ambiguity handling

Use tested score and margin thresholds in code; ask customer to distinguish fuel filter versus separator; allow staff to select only verified candidates.

Acceptance and failure boundaries: LLM cannot override low confidence; unavailable candidate choice returns 409; thresholds evaluated on labelled cases.

F27 Match explanations

Show actual alias, machine configuration, supersession chain and fitment reference; derive explanations from search evidence rather than invented model prose.

Acceptance and failure boundaries: Absent evidence shown as unavailable; user HTML is escaped; scores never labelled calibrated probability.

F28 Compatibility references

Resolve source_reference to version-specific document and page; retain citation with every candidate and approval snapshot.

Acceptance and failure boundaries: Missing source file blocks evidence viewer and flags review; mismatched version fails verification.

F29 Evidence page viewer

Authorize assigned reviewer or own demo reviewer; request short-lived backend-signed PDF URL; open exact cited page; label synthetic document.

Acceptance and failure boundaries: Customer receives reference only; arbitrary Storage key rejected; expired link can be renewed after authorization.

F30 Old part numbers

Follow supersession chain with cycle protection; display old and current numbers; verify current successor has explicit fitment for resolved configuration.

Acceptance and failure boundaries: Cycles rejected; inactive terminal part returns unavailable; inherited compatibility without source is rejected.

F31 Candidate comparison

Compare two or three candidates from the same item/revision; show number, names, evidence and warehouse availability; selecting a candidate sends its ID, not browser-authored facts.

Acceptance and failure boundaries: Foreign candidate or stale revision rejected; missing attribute shown as unavailable; keyboard navigation works.

F32 Warehouse stock

Show available quantity as on_hand minus reserved; include timestamp and warehouse name; keep display and planner using same snapshot.

Acceptance and failure boundaries: Negative stock blocked; stock unavailable never displayed as zero without explanation; unknown warehouse rejected.

F33 Fulfilment planning

Allocate local stock first then reachable warehouses deterministically; compute shortage; use quantity units; do not reserve stock or create orders.

Acceptance and failure boundaries: Total allocations never exceed request or stock; fractional quantities respect unit policy; unavailable transfer route cannot be invented.

F34 Transfer estimates

Carry integer transfer_days from approved route; show estimate and source warehouse; local availability means zero transfer days.

Acceptance and failure boundaries: No route gives unknown/unavailable plan rather than invented one-day promise; estimate labelled non-guaranteed.

F35 Stock freshness

Compare stock timestamp with configurable freshness limit; mark stale data in review; re-read current stock during approval; invalidate plan when allocation no longer possible.

Acceptance and failure boundaries: Future timestamp flagged; changed stock returns 409 and reruns planning; stale stock never becomes a delivery guarantee.

F36 Staff inbox

List assigned operational enquiries and own active demo enquiries; filter status, priority, overdue and text; paginate stably; show counts in same access scope.

Acceptance and failure boundaries: Unassigned real enquiries hidden from reviewer; admin can see all; filters do not leak global counts.

F37 Review workspace

Assemble current extraction, items, candidates, evidence, stock, draft and timeline; show verification failures; separate staff-only notes from customer messages.

Acceptance and failure boundaries: Missing panels show explicit empty states; stale revision disables approval; unassigned reviewer receives 404.

F38 Correct extracted details

Lock enquiry; compare expected revision; apply typed allowed corrections; increment revision; invalidate derived selections/draft/approval; enqueue recomputation; record before and after.

Acceptance and failure boundaries: Unknown field rejected; old revision returns 409; correction cannot directly set approved status or invent candidate facts.

F39 Review actions

Approve only verified current draft; edit prose through controlled schema then reverify; reject with reason; record actor and idempotency key.

Acceptance and failure boundaries: Double-click publishes once; concurrent edit defeats stale approval; failed verification blocks publication.

F40 Approval invalidation

Bind approval to revision and verified fact hash; any identity, quantity, selection or catalogue-sensitive change requires new draft and approval.

Acceptance and failure boundaries: Old approval cannot authorize new revision; late workflow update cannot overwrite approved current state.

F41 Internal notes

Append reviewer notes with author and timestamp; show only in staff workspace; prohibit customer/PDF exposure; escape rendered content.

Acceptance and failure boundaries: Empty or over-4000-character note rejected; customer route never includes note text; unassigned staff denied.

F42 Assignments

Admin assigns active reviewer or unassigns; persist assignment history atomically; access changes immediately; demo reviewer access remains session-scoped.

Acceptance and failure boundaries: Non-admin assignment denied; disabled/nonreviewer assignee rejected; previous reviewer loses access on next request.

F43 Overdue indicators

Set due_at using configurable urgent/normal thresholds; display overdue only for unresolved states; sort overdue first when requested; store all times UTC.

Acceptance and failure boundaries: Approved/rejected items never overdue; daylight-saving display does not change deadline; thresholds not described as contractual SLA.

F44 Conversation summary

Produce staff-only concise summary from authorized messages and confirmed fields; store message IDs and revision; use shared budget wrapper; regenerate on relevant change.

Acceptance and failure boundaries: Outdated summary labelled stale; instructions inside messages ignored; exhausted budget leaves full conversation available.

F45 Reply drafting

Generate greeting, explanation and closing as structured slots; code inserts all numbers and allocations; choose Arabic or English from confirmed preference.

Acceptance and failure boundaries: Unapproved critical facts never rendered; unsupported slots rejected; provider failure permits staff-authored controlled draft.

F46 Verification gate

Check item/configuration/candidate membership, provenance, catalogue version, selected quantities and allocation bounds; compute canonical fact hash; store named rule results.

Acceptance and failure boundaries: Unknown part, incompatible part, stale catalogue, oversold plan and hidden extra item each fail a specific rule.

F47 Human publication gate

Only approval service inserts recommendation messages; lock enquiry and recheck role, revision and verification; unique publication index prevents duplicates.

Acceptance and failure boundaries: No background job write tool can publish; database rejects recommendation lacking matching approved decision.

F48 Input isolation

Treat customer text, OCR and imported evidence as data; separate trusted instructions; bound inputs, transitions, steps and output size; never execute model-produced templates.

Acceptance and failure boundaries: Prompt injection cannot call privileged functions, change roles or reveal secrets; model result with rogue instructions remains inert.

F49 Audit trail

Append actor, request, entity, revision and minimal before/after metadata for AI suggestions and human changes; redact secrets and full sensitive payloads.

Acceptance and failure boundaries: Application UPDATE, DELETE, TRUNCATE fail; audit write failure rolls back approval; database-owner powers documented honestly.

F50 Durable workflows

Persist workflow_runs and workflow_step_results in PostgreSQL; run a fixed Python transition table with durable jobs; serialize each enquiry and fence completion by revision and lease token; recover after restart without LangGraph.

Acceptance and failure boundaries: Duplicate jobs cannot repeat publication; stale revisions are discarded; interrupted steps resume from committed results; no LLM chooses the next step.

F51 Background jobs

Persist job before Redis dispatch; dispatcher claims by ID; sweeper redispatches pending/expired jobs; worker handles OCR, PDF, imports and cleanup.

Acceptance and failure boundaries: Redis loss does not lose DB job; unavailable Redis rate limiter fails closed; heavy work never blocks HTTP intake.

F52 Retry and deduplication
Claim with FOR UPDATE SKIP LOCKED; assign lease token; heartbeat; retry transient errors with backoff up to max attempts; fence final writes by lease token.

Acceptance and failure boundaries: Expired worker cannot commit; permanent validation failure not retried; provider timeout marked uncertain and budget held conservatively.

F53 Graceful failures
Map stable error codes to bilingual recovery guidance; distinguish validation, quota, service outage and staff-required states; preserve user input.

Acceptance and failure boundaries: No raw stack traces or keys; retry only safe idempotent action; search lexical fallback clearly reported.

F54 Access and permissions
Verify Supabase issuer, audience, signature, expiry; derive identity server-side; enforce assigned-only review; keep credentials server-side; disable public real-client signup.

Acceptance and failure boundaries: Forged role claim ignored; missing context denies rows; preview cannot reach prod database; demo cannot publish shared catalogue.

F55 Saved machines
Save confirmed model and serial with user label; list own active entries; archive entry; prefill a new enquiry but re-identify against current catalogue.

Acceptance and failure boundaries: Duplicate label permitted with distinct ID; foreign ID denied; stale configuration never reused as authoritative.

F56 Repeat enquiry
Copy customer request and confirmed hints into a new enquiry; issue new ID/revision; link origin in audit; run matching, stock and review again.

Acceptance and failure boundaries: Never copy approval, allocation, draft or signed file URL; source ownership required; repeated submission deduplicated.

F57 Catalogue browser
Read published catalogue with model/category/text filters; page results; show part detail and supersession; staff can inspect fitment and sources.

Acceptance and failure boundaries: No unpublished import rows exposed; missing number returns 404; literal query cannot become SQL.

F58 Catalogue imports
Read CSV UTF-8 or XLSX without evaluating formulas; validate row schema, references, serial overlap, supersession cycles and source evidence; show dry-run report; explicit admin publish. Stage a version in Snowflake, validate all relationships explicitly and only then activate a validated PostgreSQL serving snapshot.

Acceptance and failure boundaries: Invalid batch changes no published data; file with macros/formulas rejected; 50000-row and 10MiB limits enforced; public demo cannot publish.

F59 Catalogue versioning
Store immutable versioned catalogue records in Snowflake; verify record counts and canonical SHA256; stage the PostgreSQL serving snapshot; atomically switch the active version in a PostgreSQL transaction; retain published historical records; reconcile partial remote writes.

Acceptance and failure boundaries: Two publishers cannot leave two active versions; failed publish rolls back; old approval retains immutable evidence snapshot.

F60 Missing-parts queue
Record unresolved item once; allow assigned staff to mark investigating, resolved or unavailable with note; link back to enquiry; no automatic catalogue creation.

Acceptance and failure boundaries: Resolved status requires staff explanation; foreign item denied; empty search result is not fabricated stock.

F61 Enquiry PDF

Export approved revision only; render logo, bilingual customer text, verified selections, estimates and references; embed Arabic fonts; store private object and signed download.

Acceptance and failure boundaries: Exclude notes and secrets; stale or unapproved revision rejected; long Arabic names wrap without clipping; no prices or quotation title.

F62 Labelled evaluations

Include four demos plus Arabic, typo, serial-boundary, ambiguity, stock and injection cases; record expected compatible set and clarification outcome; run deterministic suite without API calls.

Acceptance and failure boundaries: No self-reported model confidence as accuracy; evaluation fixture disjoint from examples used in prompts; failures keep actual output.

F63 Metrics dashboard

Show volume, status, review time, clarification and labelled accuracy with denominators; separate demo traffic; filter time window; scope reviewer data to assignments. Export redacted aggregate events to Snowflake through an outbox and show reporting freshness separately from live operational counts.

Acceptance and failure boundaries: Zero denominator displays unavailable; no fabricated percentage; demo traffic never silently improves operational metrics.

F64 Latency and AI cost

Measure stages with monotonic clock; log model/token usage and estimated cost; reserve confirmed free credit before each attempt; expose consumed/reserved allowance.

Acceptance and failure boundaries: Unknown price or expired credit blocks calls; concurrent reservations cannot exceed cap; API key validity never implies free credit.

F65 Correction feedback

List authorized before/after corrections; let admin curate anonymized labelled eval cases; record reviewer and source revision; never auto-train on unreviewed edits.

Acceptance and failure boundaries: Private text excluded from exported fixtures; duplicate case source detected; unreviewed correction cannot become ground truth.

F66 Tests and CI

Run service-isolated Python tests, frontend checks, RLS integration and deterministic evals; build images; require explicit environment approval for deployment; pin action SHAs.

Acceptance and failure boundaries: No prod DB in tests; no secret available to untrusted fork jobs; failed gate blocks deployment.

F67 Traces

Propagate request/enquiry/job IDs; capture stage timing and safe status; export optional Langfuse traces with payload redaction; operate normally when tracing unavailable.

Acceptance and failure boundaries: No raw key or full photo logged; exporter outage never breaks approval; budget accounting remains local and authoritative.

F68 Recruiter documentation

Write concise overview, real architecture, quick start, demo paths, tests, trade-offs and limitations; link measured evidence; explain deterministic decisions and ordinary Python functions.

Acceptance and failure boundaries: No invented accuracy, uptime, client relationship or production adoption; distinguish implementation status from specification.

F69 Demo walkthrough

Create 3-minute and 8-minute scripts covering happy path, missing serial, incompatibility, stock split and human review; include recording checklist and captions.

Acceptance and failure boundaries: Do not claim video exists until recorded; use synthetic data; show failure handling and reset; verify links before sending recruiters.

Nonfunctional requirements

Security: server-verified identity, scope checks, RLS, private files, least-privilege data roles, redacted logging and no model-authorized writes. Durability: database-backed jobs, leases, fencing, idempotent API writes

and recoverable partial Snowflake synchronization. Language: Arabic/English UI and replies, mixed-script identifiers, bidi isolation and bundled Arabic PDF fonts. Accessibility: keyboard operation, visible focus, reduced motion, 200 percent zoom and360px minimum layout. Maintainability: typed models, one domain implementation per rule, versioned migrations and explicit test evidence.

Provisional performance goals are local acceptance budgets, not measured claims: API writes should acknowledge without waiting for AI or warehouse work; provider operations have bounded timeouts; polling uses backoff; optional embeddings cannot prevent lexical readiness. Benchmark the actual host before setting public latency claims. The paid budget is zero; the system requires confirmed free allowance before live provider calls. Real-client retention is an onboarding decision; until it is recorded, real onboarding stays disabled. Demo access expires after24hours and background cleanup follows the documented policy.

Exclusions

No voice input, quotation builder, prices, payments, orders, stock reservations, automatic WhatsApp/email sending, multi-agent routing, autonomous SQL generation or automatic training on staff corrections. No claim of perpetual free cloud hosting. No invented partnership with equipment distributors.

037 FEATURE TEMPLATE

docs/templates/FEATURE_TEMPLATE.md

Feature trace

Status: specified, not implemented.

Record feature ID, title, owning prompt, source files, route IDs, test paths, actual commands, results, commit, and remaining blockers. Never replace an execution receipt with a planned test.

038 PROMPT TEMPLATE

docs/templates/PROMPT_TEMPLATE.md

Implementation prompt

GOAL
Describe the concrete requirements for this section.

CONTEXT AND PREREQUISITES
Describe the concrete requirements for this section.

PACKAGES AND INSTALLATION
Describe the concrete requirements for this section.

FILES AND OWNERSHIP
Describe the concrete requirements for this section.

TYPED CONTRACTS
Describe the concrete requirements for this section.

IMPLEMENTATION STEPS
Describe the concrete requirements for this section.

REST API AND DATA FLOW
Describe the concrete requirements for this section.

VALIDATION AND FAILURES
Describe the concrete requirements for this section.

TESTS
Describe the concrete requirements for this section.

COMMANDS AND EXPECTED RESULTS
Describe the concrete requirements for this section.

MANUAL CHECK
Describe the concrete requirements for this section.

DOCUMENTATION UPDATES
Describe the concrete requirements for this section.

ROLLBACK AND COMMIT
Describe the concrete requirements for this section.

COMPLETION EVIDENCE
Describe the concrete requirements for this section.

039 00 bootstrap

prompts/00-bootstrap.md

00 Repository bootstrap and legacy characterization

GOAL

Unpack references/legacy-backend.zip in a temporary directory and calculate SHA256 for every original file.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors none; this is the first step before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `pyproject.toml`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/pyproject.toml`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/__init__.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/run_checks.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/inspect_legacy.py`: implement the responsibility named by this path and the ordered instructions below.
- `.env.example`: implement the responsibility named by this path and the ordered instructions below.
- `.python-version`: implement the responsibility named by this path and the ordered instructions below.
- `.nvmrc`: implement the responsibility named by this path and the ordered instructions below.
- `.gitignore`: implement the responsibility named by this path and the ordered instructions below.
- `tools/verify_kit.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_00.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

Additional owned package markers and assets:

- `apps/api/partsiq/__init__.py`
- `apps/api/partsiq/ai/__init__.py`
- `apps/api/partsiq/analytics/__init__.py`

- apps/api/partsiq/api/__init__.py
- apps/api/partsiq/auth/__init__.py
- apps/api/partsiq/catalogue/__init__.py
- apps/api/partsiq/core/__init__.py
- apps/api/partsiq/enquiries/__init__.py
- apps/api/partsiq/evaluation/__init__.py
- apps/api/partsiq/exports/__init__.py
- apps/api/partsiq/exports/templates/__init__.py
- apps/api/partsiq/features/__init__.py
- apps/api/partsiq/jobs/__init__.py
- apps/api/partsiq/search/__init__.py
- apps/api/partsiq/storage/__init__.py
- apps/api/partsiq/workflow/__init__.py

TYPED CONTRACTS

```
def inspect_legacy(root: Path) -> dict[str, str]
def run_checks(prompt_id: str) -> int
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Unpack references/legacy-backend.zip in a temporary directory and calculate SHA256 for every original file.

2. Read its AGENTS.md before moving code.

3. Keep that archive unchanged.

4. Map identification.py, lexical.py, fulfilment.py, repository.py, service.py and schemas.py into apps/api/partsiq/search; adapt absolute app imports to partsiq.search.

5. Preserve the original function arguments and return semantics behind explicit adapters.

6. Run the legacy tests before changing behavior and record actual failures.

7. Create one Python package named partsiq; API and worker import it.

8. Create test fixtures and a prompt-aware runner that reads spec/build-order.json, finds the exact named tests and fails if a named test file is absent or every case is skipped.

9. Do not create separate gateway/search/orchestrator servers.

10. Configure Python 3.12 and Node 22.22.0.

11. Keep all implementation docs under docs and all generated application files in the ownership manifest.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced

PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_00.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 00`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/00/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step00, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(bootstrap): implement foundation 00 repository bootstrap and legacy characterization`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

040 01 core

prompts/01-core.md

01 Typed contracts and application configuration

GOAL

Copy spec/contracts.py and spec/api_models.py into core; preserve exported type names.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 00 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/main.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/settings.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/contracts.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/api_models.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/errors.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/db.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/dependencies.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/api/router.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_01.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def load_settings() -> Settings
def create_app() -> FastAPI
async def request_scope(actor: ActorContext) -> AsyncIterator[asyncpg.Connection]
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Copy spec/contracts.py and spec/api_models.py into core; preserve exported type names.

2. Pydantic model_validate_json handles incoming JSON and model_dump(mode="json") serializes UUID, Decimal and datetime consistently.

3. FastAPI handlers use typed request models and response_model.

4. Create asyncpg pools during lifespan and close on shutdown.

5. Database URL selection is explicit by role; never use a migration-owner URL at runtime. request_scope begins a transaction and uses SELECT set_config('app.actor_id', $1, true) with the verified subject, ensuring identity disappears on commit or rollback.

6. Define an RFC9457-style Problem response and request ID middleware.

7. Settings reject production database URLs in local/tests and reject live providers with an expired allowance.

8. Route registration is additive as features are implemented; never import unfinished feature modules.

9. Health/liveness checks do not spend Snowflake or OpenAI credits.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_01.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 01`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/01/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step01, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(core): implement foundation 01 typed contracts and application configuration`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

041 02 snowflake

prompts/02-snowflake.md

02 Snowflake connector and validated catalogue source

GOAL

Install snowflake-connector-python4.8.0; this release predates the14-day cutoff while5.0.0 does not.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 01 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/catalogue/snowflake_repository.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/catalogue/models.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/catalogue/validation.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/catalogue/fixture_repository.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/snowflake_preflight.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/apply_snowflake.py`: implement the responsibility named by this path and the ordered instructions below.
- `infra/snowflake/001_schema.sql`: implement the responsibility named by this path and the ordered instructions below.
- `infra/snowflake/002_roles.sql`: implement the responsibility named by this path and the ordered instructions below.
- `infra/snowflake/003_validation_queries.sql`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_02.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
class SnowflakeCatalogueRepository: def read_version(self, version_id: UUID) -> CatalogueSnapshot
def write_version(self, snapshot: CatalogueSnapshot) -> SyncReceipt
def validate_snapshot(snapshot: CatalogueSnapshot) -> list[ValidationIssue]
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Install snowflake-connector-python4.8.0; this release predates the14-day cutoff while5.0.0 does not.

2. Use the4.8 API: connect(account, user, private_key_file, private_key_file_pwd, database, schema, warehouse, role, login_timeout=10, network_timeout=30, paramstyle="qmark", session_parameters={"QUERY_TAG":"partsiq:<request_uuid>", "STATEMENT_TIMEOUT_IN_SECONDS":20}).

3. Load connection parameters only from validated settings.

4. A synchronous cursor executes fixed SQL with ? bindings; identifiers come from a fixed dev/prod allowlist, never request fields.

5. Run blocking connector work in asyncio.to_thread from the worker and use a semaphore of2.

6. Keep one connection per task and close cursor and connection in finally.

7. Store sfqid in SyncReceipt.

8. Read only a requested immutable VERSION_ID; verify declared row count and canonical digest.

9. Validate unique entity keys, foreign references, serial overlaps, evidence references, unit quantities, stock values and supersession cycles in Python.

10. Duplicate rows cause quarantine, never dict overwrite.

11. FixtureRepository implements the same interface with synthetic JSON and a visible fixture mode label.

12. A Snowflake exception never silently changes the data source.

13. Do not use Cortex, Snowpark, Snowpipe, Dynamic Tables or an autonomous SQL agent.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_02.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 02`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/02/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step02, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(snowflake): implement foundation 02 snowflake connector and validated catalogue source`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

042 03 database

prompts/03-database.md

03 Supabase schema permissions and serving cache

GOAL

Apply spec/sql001 through006 to disposable local Supabase first.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 02 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/catalogue/activation.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/apply_postgres.py`: implement the responsibility named by this path and the ordered instructions below.
- `supabase/config.toml`: implement the responsibility named by this path and the ordered instructions below.
- `tests/integration/test_rls.py`: executable acceptance coverage for this step.
- `tests/integration/test_publication.py`: executable acceptance coverage for this step.
- `supabase/migrations/005_storage.sql`: implement the responsibility named by this path and the ordered instructions below.
- `supabase/migrations/003_application.sql`: implement the responsibility named by this path and the ordered instructions below.
- `supabase/migrations/004_rls.sql`: implement the responsibility named by this path and the ordered instructions below.
- `supabase/migrations/001_initial_schema.sql`: implement the responsibility named by this path and the ordered instructions below.
- `supabase/migrations/002_search_and_fulfilment.sql`: implement the responsibility named by this path and the ordered instructions below.
- `supabase/migrations/006_plain_workflow_and_snowflake.sql`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_03.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def activate_version(conn: asyncpg.Connection, snapshot: CatalogueSnapshot, expected_active: UUID
 | None) -> SyncReceipt
async def apply_migrations(dsn: str, files: list[Path]) -> SchemaReceipt
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Apply spec/sql001 through006 to disposable local Supabase first.

2. Copy each SQL file into supabase/migrations preserving exact ordering; do not apply removed checkpoint migrations.

3. Preserve legacy PostgreSQL overlap, cycle, unique, foreign-key and append-only controls.

4. Snowflake versions are remote immutable source records; PostgreSQL owns the single active serving pointer.

5. Read and validate a complete remote version outside a database lock, then acquire the catalogue advisory transaction lock, compare expected_active, upsert stable numeric IDs and reconcile rows in dependency order.

6. Never delete a referenced historical part or configuration.

7. Mark removed records inactive and keep immutable catalogue_records for old decisions.

8. All search reads must filter the active catalogue_records keyset, because historical rows remain in legacy tables.

9. Materialize the relational cache and switch catalogue_versions in one transaction.

10. A remote successful upload followed by local failure remains unactivated and is safe to reconcile.

11. Capture migration checksums and actual constraints; never claim a Snowflake PK protects integrity.

12. Runtime role groups separate API, workflow, worker, search and publisher permissions without requiring different services.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for

diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_03.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 03`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/03/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step03, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(database): implement foundation 03 supabase schema permissions and serving cache`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

043 04 storage

prompts/04-storage.md

04 Private uploads evidence and storage access

GOAL

Use the four private buckets defined in005_storage.sql.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 03 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/storage/service.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/storage/client.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/storage/images.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_04.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def create_upload(actor: ActorContext, command: UploadRequest) -> UploadTicket
async def confirm_upload(actor: ActorContext, attachment_id: UUID, revision: int) -> JobView
async def sign_download(actor: ActorContext, resource_id: UUID, kind: str) -> DownloadTicket
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Use the four private buckets defined in005_storage.sql.

2. API creates an attachment record before obtaining a signed upload URL; use random UUID keys and never accept arbitrary caller paths.

3. Through httpx call Supabase Storage REST with server-side credentials and bounded10-second timeout.

4. Client uploads directly to the signed URL and confirms the attachment ID.

5. Confirmation fetches metadata and actual bytes, checks magic format and10MiB maximum, then queues validation.

6. Pillow Image.verify and an independent decoded-pixel limit protect the worker; reopen, apply EXIF orientation, strip EXIF and save a sanitized copy before vision.

7. The original file is private.

8. Evidence and exports have no public policies; sign a60-second download only after current scope authorization.

9. Reject path traversal before any storage call.

10. A signed URL remains valid until expiry even if assignment changes; document this bounded window.

11. Storage credentials never enter NEXT_PUBLIC variables.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_04.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 04`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/04/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step04, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(storage): implement foundation 04 private uploads evidence and storage access`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

044 05 fixtures

prompts/05-fixtures.md

05 Synthetic catalogue and four scenario fixtures

GOAL

Reuse the legacy generator and its19 models, 181 parts and787 aliases if those counts are verified by execution; do not manufacture a matching count if generation differs.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 04 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/catalogue/fixtures.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/seed_demo.py`: implement the responsibility named by this path and the ordered instructions below.
- `fixtures/scenarios.json`: implement the responsibility named by this path and the ordered instructions below.
- `fixtures/catalogue.json`: implement the responsibility named by this path and the ordered instructions below.
- `fixtures/README.md`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_05.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def build_catalogue(seed: int = 42) -> CatalogueSnapshot
def validate_scenarios(snapshot: CatalogueSnapshot, cases: list[DemoScenario]) -> None
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Reuse the legacy generator and its19 models, 181 parts and787 aliases if those counts are verified by execution; do not manufacture a matching count if generation differs.

2. Seed42 provides stable IDs.

3. Build four scenarios from actual generated records: mixed Arabic/English plus photo; missing serial with two possible configurations; known part number incompatible with the resolved machine; four requested units with one local and at least three remote.

4. Ensure the transfer route explicitly permits the demonstrated estimate.

5. Generate a synthetic evidence PDF and nameplate images with deterministic code and mark them synthetic.

6. Every scenario contains request input, expected clarification and approved-compatible set; it must not imply real TAMGO inventory.

7. Load the exact same logical snapshot into fixture and Snowflake development modes and compare canonical digest.

8. No fixture can directly insert an approved recommendation.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_05.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 05`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not

assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/05/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step05, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(fixtures): implement foundation 05 synthetic catalogue and four scenario fixtures`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

045 06 search

prompts/06-search.md

06 Compatibility lexical matching and optional embeddings

GOAL

Port characterized legacy pure functions without changing scores accidentally.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 05 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/search/identification.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/search/lexical.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/search/fulfilment.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/search/repository.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/search/service.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/search/schemas.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/search/embeddings.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_06.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
class SearchService: async def identify(self, request: IdentifyRequest) -> IdentifyResponse
async def search_parts(self, request: PartSearchRequest) -> PartSearchResponse
def compatible_ids(configuration_id: int, snapshot: CatalogueSnapshot) -> set[int]
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities

use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Port characterized legacy pure functions without changing scores accidentally.
2. Every query pins a catalogue version and uses its validated serving keyset.
3. Identify a single configuration before loading fitment; ambiguous identity yields needs_information.
4. Compute compatible IDs first and filter before lexical or semantic ranking.
5. Normalize Arabic variants, diacritics and tatweel with reviewed mappings; preserve part-number strings.
6. Keep machine words separate from part words.
7. Use existing phrase and typo scoring and explain contributing aliases.
8. BGE-M3 runs only within the worker search module, one CPU encoder per process; API handlers never load it.
9. Normalize1024-dimensional vectors, validate dimension and finite values, and calculate similarity only for compatible parts.
10. Optional embedding unavailability gives a labelled lexical fallback.
11. Compute runner-up margin before top-k truncation.
12. Follow old-number chains with a visited set and require explicit fitment on the final active part.
13. Allocate repeated requested parts together across items so stock cannot be counted twice.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_06.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep

network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 06`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/06/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step06, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(search): implement foundation 06 compatibility lexical matching and optional embeddings`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

046 07 auth

prompts/07-auth.md

07 Authentication ownership and demo isolation

GOAL

Frontend uses Supabase browser auth for anonymous demo sessions and invited staff signin; disable self-service real-client signup.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 06 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/auth/tokens.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/auth/access.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/auth/demo.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_07.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def verify_token(token: str) -> ActorContext
async def require_enquiry(actor: ActorContext, enquiry_id: UUID, action: AccessAction) -> EnquiryView
async def reset_session(actor: ActorContext, session_id: UUID, key: UUID) -> DemoSession
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Frontend uses Supabase browser auth for anonymous demo sessions and invited staff signin; disable self-service real-client signup.

2. Verify JWT signature with an allowlisted algorithm and cached JWKS, issuer, audience and expiry.

3. A failed kid lookup refreshes once, not on every request.

4. Read staff roles from the database rather than trusting editable metadata.

5. Customer identity never comes from JSON actor_id.

6. Enforce assigned-only real review and admin assignment; own-session demo role switching is presentation only.

7. Create sessions with24-hour access expiry and generation1.

8. Reset locks the session, increments generation, cancels old jobs and schedules cleanup.

9. Every background final write checks current generation.

10. Return404 for foreign scoped resources so their existence is not exposed.

11. Use transaction-local actor context for RLS and tests with two users, two sessions and reassignment.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_07.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 07`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/07/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step07, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(auth): implement foundation 07 authentication ownership and demo isolation`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

047 08 intake

prompts/08-intake.md

08 Enquiry intake idempotency and rate limiting

GOAL

Validate UTF-8 text length1..12000, locale and destination warehouse against active snapshot.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 07 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/enquiries/intake.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/history.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/idempotency.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/rate_limit.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_08.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def create_enquiry(actor: ActorContext, command: EnquiryCreate, key: UUID) -> EnquiryView
async def append_message(actor: ActorContext, enquiry_id: UUID, command: MessageCreate, key: UUID) ->
EnquiryView
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Validate UTF-8 text length1..12000, locale and destination warehouse against active snapshot.

2. Hash normalized request JSON together with method, path and actor for idempotency.

3. Same key and same hash replay the saved result; same key and different hash returns 409.

4. In one PostgreSQL transaction persist enquiry, message, revision snapshot and workflow job.

5. Commit before returning202; never wait for vision or Snowflake.

6. Redis Lua implements bounded atomic request counters with TTL; API fails closed503 if its configured limiter is unavailable.

7. A worker queue wake-up is optional acceleration, not durable truth.

8. History uses keyset pagination(created_at, id), and message append locks the enquiry, checks expected_revision and creates a new revision that invalidates previous derived results.

9. Do not retry a non-idempotent insert outside the transaction wrapper.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_08.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 08`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/08/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step08, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(intake): implement foundation 08 enquiry intake idempotency and rate limiting`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

048 09 workflow

prompts/09-workflow.md

09 Plain Python durable workflow

GOAL

Implement the fixed transition table in docs/WORKFLOW.md as ordinary Python; use no LangGraph, supervisor, LLM tool calls or agent abstraction.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 08 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/workflow/runner.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/workflow/transitions.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/workflow/state.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/workflow/repository.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_09.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def process_enquiry(job: JobLease) -> JobResult
def next_step(state: EnquiryState, outcome: StepOutcome) -> WorkflowStep
async def commit_step(lease: JobLease, step: WorkflowStep, input_hash: str, result: StepOutcome) ->
bool
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Implement the fixed transition table in docs/WORKFLOW.md as ordinary Python; use no LangGraph, supervisor, LLM tool calls or agent abstraction.

2. Load the current revision and session generation under a short row lock, claim the job and release the transaction before external calls.

3. Compute canonical input SHA256.

4. Reuse a completed step only when its input hash and revision match.

5. Call the appropriate domain function directly.

6. Reacquire lock and commit output only if the lease token, revision and session generation still match; stale output is discarded and audited.

7. Persist workflow_step_results and next step atomically.

8. A process crash after an external call but before commit may require another attempt; preserve uncertain AI usage reservations rather than claiming exactly-once provider invocation.

9. Missing facts pause in clarify; unreadable photo pauses in confirm; ambiguous ranking pauses in choose; verified draft pauses in review.

10. Only approval publishes.

11. Limit automatic transitions to20 per job and send unexpected loops to needs_staff.

12. Resume requires a new message, confirmation, choice or authorized retry event.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_09.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 09`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/09/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step 09, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(workflow): implement foundation 09 plain python durable workflow`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

049 10 extraction

prompts/10-extraction.md

10 OpenAI extraction and usage allowance

GOAL

Use AsyncOpenAI with max_retries=0 and a bounded timeout; the application owns retries and allowance accounting.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 09 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/ai/client.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/ai/budget.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/ai/extraction.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/ai/prompts.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_10.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def extract_message(context: ExtractionContext) -> IntakeResult
async def reserve_ai(request: AIRequest) -> BudgetReservation
async def call_structured(request: AIRequest, schema: type[BaseModel]) -> AIResult
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Use AsyncOpenAI with max_retries=0 and a bounded timeout; the application owns retries and allowance accounting.

2. Use Responses API structured output with a JSON schema or responses.parse supported by the pinned SDK; validate again through Pydantic.

3. No tool definitions are sent.

4. Separate system instructions from untrusted user content.

5. Intake extracts only descriptions, quantity, urgency, locale and machine hints with source offsets.

6. Do not resolve a part ID or configuration through model knowledge.

7. Unknown quantity remains null and causes a question.

8. Reserve conservative input/output cost in a locked budget row before every attempt.

9. Free key validity is not free-credit evidence; owner supplies a confirmed dedicated allowance and expiry.

10. Unknown pricing, expired credit or insufficient allowance blocks live AI.

11. Release only demonstrably unused reservation; ambiguous timeouts remain held until reconciled.

12. Fixture responses require explicit fixture mode and a visible demo label.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_10.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 10`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests`

and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/10/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step10, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(extraction): implement foundation 10 openai extraction and usage allowance`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

050 11 resolution

prompts/11-resolution.md

11 Machine search and fulfilment application flow

GOAL

For each confirmed item call SearchService.identify, then search_parts only when identification resolves.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 10 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/workflow/resolution.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/fulfilment.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/questions.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_11.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def resolve_items(state: EnquiryState) -> ResolutionOutcome
async def plan_enquiry(state: EnquiryState) -> PlanView
def clarification(reason: str, locale: Locale) -> str
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. For each confirmed item call SearchService.identify, then search_parts only when identification resolves.
2. Unconfirmed OCR hints cannot trigger matching.
3. Apply code-owned score and margin rules; do not ask the model to choose between close candidates.

4. Store all candidate evidence with item_id, revision and catalogue_version_id.

5. A customer choice sends only a part ID from that current candidate set.

6. For multiple items consolidate demand by part, allocate local stock then approved reachable warehouses, then distribute allocations back to item order.

7. Keep shortage explicit; estimates use only stored transfer routes.

8. Read stock observed_at and enforce configured freshness; unknown stock remains unknown.

9. Resolve conflicts between typed text and OCR by asking a question, never by silently preferring one.

10. Every result is a typed value passed directly to the workflow, not a specialist-agent message.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_11.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 11`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/11/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is

blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step11, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(resolution): implement foundation 11 machine search and fulfilment application flow`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

051 12 verification

prompts/12-verification.md

12 Deterministic verification and fact hashing

GOAL

Check every requested item has exactly one current selected candidate and positive quantity.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 11 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/enquiries/verification.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/facts.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_12.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def verify_facts(facts: VerifiedFacts, evidence: VerificationEvidence) -> VerificationResult
def canonical_fact_hash(facts: VerifiedFacts) -> str
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Check every requested item has exactly one current selected candidate and positive quantity.
2. Verify candidate membership, configuration fitment, source evidence, active catalogue version, terminal supersession validity and stock allocation bounds.
3. Aggregate allocations by warehouse and part across all items.
4. Reject unknown or duplicate hidden items and unsupported transfer estimates.

5. Serialize canonical facts with sorted keys, stable item ordering, decimal strings and UTC timestamps, then SHA256 hash.

6. Store each named rule result and its evidence identifiers.

7. Validation failure goes to staff, not a best-effort published answer.

8. No LLM is used here.

9. A known database part number alone is insufficient: it must fit the resolved configuration and requested item.

10. Reuse this exact verifier for automatic drafts, staff edits and approval.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_12.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 12`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/12/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is

blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step12, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(verification): implement foundation 12 deterministic verification and fact hashing`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

052 13 drafts

prompts/13-drafts.md

13 Bilingual reply slots and safe rendering

GOAL

Give the model minimal verified facts and ask for bounded greeting, explanation and closing fields.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 12 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/ai/reply.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/drafts.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/renderer.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_13.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def draft_slots(facts: VerifiedFacts, locale: Locale) -> ReplySlots
def render_reply(slots: ReplySlots, facts: VerifiedFacts, locale: Locale) -> str
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Give the model minimal verified facts and ask for bounded greeting, explanation and closing fields.
2. Treat reply prose as untrusted too.
3. Code inserts part identifiers, quantities, warehouse allocations, source references and estimates in controlled blocks.

4. Enforce forbidden identifier/number checks on model slots; if a slot claims unsupported quantities or part numbers, use a safe template instead.

5. Rendering does not call the model and does not publish.

6. Staff edits use the same ReplySlots schema and verification.

7. Provider failure permits an authorized manual controlled draft while preserving approval requirements.

8. Use locale-aware templates and bidi isolation; escape HTML before dashboard/PDF rendering.

9. Persist the draft against revision, fact hash and catalogue version so any material change invalidates it.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_13.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 13`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/13/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step13, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(drafts): implement foundation 13 bilingual reply slots and safe rendering`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

053 14 approval

prompts/14-approval.md

14 Review transactions revisions and publication

GOAL

Lock enquiry, check scope and expected revision, then ensure every item is ready and the draft matches verified facts.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 13 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/enquiries/approval.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/revisions.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/publication.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/review.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_14.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def approve(actor: ActorContext, enquiry_id: UUID, command: ApprovalRequest, key: UUID) ->
ApprovalResult
async def correct_extraction(actor: ActorContext, enquiry_id: UUID, command: RevisionRequest) ->
EnquiryView
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Lock enquiry, check scope and expected revision, then ensure every item is ready and the draft matches verified facts.

2. Read current active catalogue pointer and stock in the same transaction boundary; run deterministic verification again.

3. If stock/catalogue changed invalidate the draft and enqueue recomputation with 409, never mutate the old approved snapshot.

4. Insert review_decision, approved part rows, recommendation message and audit atomically.

5. Unique(enquiry_id, revision) publication prevents double clicks.

6. A rejection records its reason without sending a final recommendation.

7. Corrections increment revision, save before/after values and invalidate selection, plan, draft and approval.

8. Assignment changes use an admin-only transaction and take effect immediately.

9. Audit failure rolls back the business transaction.

10. Approval does not reserve inventory or send WhatsApp/email.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_14.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 14`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not

assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/14/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step14, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(approval): implement foundation 14 review transactions revisions and publication`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

054 15 jobs

prompts/15-jobs.md

15 Durable worker leases retries and cleanup

GOAL

Run one worker process from the same application image.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 14 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/jobs/main.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/jobs/repository.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/jobs/handlers.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/jobs/sweeper.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_15.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def claim_job(worker_id: str) -> JobLease | None
async def complete_job(lease: JobLease, result: JobResult) -> bool
async def sweep_expired(now: datetime) -> int
async def worker_main() -> None
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Run one worker process from the same application image.

2. PostgreSQL jobs are the durable queue; use FOR UPDATE SKIP LOCKED and a new random lease token on claim.

3. Lease120seconds, heartbeat every30seconds, max attempts3; image/PDF jobs may renew while progress is healthy.

4. A sweeper in the worker recovers expired leases and performs cleanup every30seconds.

5. Redis notifications may wake a poller, but missed notifications do not lose jobs.

6. Job handlers cover workflow, nameplate, export_pdf, catalog_import, catalog_publish, storage_cleanup, catalog_sync and analytics_sync.

7. Summary/evaluation requests use typed workflow job payload operations with explicit dispatch allowlists.

8. Final writes require matching lease token and current enquiry revision/generation.

9. Retry only transient timeout/429/5xx with jittered backoff and honor Retry-After.

10. Invalid input and authorization failures are terminal.

11. Never put complete photos, credentials or raw prompts in Redis messages.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_15.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 15`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not

assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/15/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step15, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(jobs): implement foundation 15 durable worker leases retries and cleanup`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

055 16 vision

prompts/16-vision.md

16 Nameplate reading and human confirmation

GOAL

Read only the sanitized private image produced by storage validation.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 15 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/ai/vision.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/jobs/nameplate.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/enquiries/extraction.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_16.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def read_nameplate(lease: JobLease, image: ImageInfo) -> NameplateResult
async def confirm_nameplate(actor: ActorContext, enquiry_id: UUID, command: ConfirmExtraction) ->
EnquiryView
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Read only the sanitized private image produced by storage validation.

2. Send it to the approved OpenAI vision model through the shared budget wrapper and Responses API input_image.

3. Limit image bytes, pixels and output fields.

4. Text printed on the image is data, including any attempted instructions.

5. Parse model_text, serial_text and explicit issues; preserve leading zeros and do not fabricate a confidence percentage.

6. Keep the attachment ID code-owned.

7. Store OCR output only with current lease and revision, then pause for user confirmation.

8. Confirmation may correct either identifier and records the actor, time and source attachment.

9. If serial is not visible, allow a missing serial value and ask the normal identification clarification rather than blocking confirmation forever.

10. Reject contradictory or stale confirmation with 409.

11. Never match an unconfirmed machine from the photo.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_16.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 16`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/16/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step16, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(vision): implement foundation 16 nameplate reading and human confirmation`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

056 17 exports

prompts/17-exports.md

17 Approved bilingual PDF export

GOAL

Export only an immutable approved snapshot.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 16 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

• `apps/api/partsiq/exports/service.py`: implement the responsibility named by this path and the ordered instructions below.

• `apps/api/partsiq/exports/render.py`: implement the responsibility named by this path and the ordered instructions below.

• `apps/api/partsiq/exports/templates/enquiry.html`: implement the responsibility named by this path and the ordered instructions below.

• `tests/foundations/test_17.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def request_pdf(actor: ActorContext, enquiry_id: UUID, command: ExportRequest) -> JobView
async def render_pdf(snapshot: ApprovedSnapshot) -> bytes
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Export only an immutable approved snapshot.

2. Jinja2 autoescape renders a local HTML template with bundled licensed Arabic fonts, logo, customer request, selected parts, quantities, availability estimates and source references.

3. Use Playwright Chromium page.pdf with print backgrounds and A4 sizing; deny external network navigation and URL fetches.

4. Never include internal notes, private traces, API keys or prices.

5. An export is an enquiry summary, not a quotation.

6. Store the PDF privately and return a scoped short-lived link only after the job succeeds.

7. Idempotency binds export to decision ID and locale.

8. Test long mixed Arabic/English names, multipage lists and bidi part numbers.

9. Compare extracted PDF text against approved facts and visually inspect at least one Arabic and one English fixture.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_17.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 17`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/17/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable

dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step17, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(exports): implement foundation 17 approved bilingual pdf export`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

057 18 web

prompts/18-web.md

18 Next.js shell branding authentication and API client

GOAL

Copy the exact package pins and brand tokens.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 17 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/web/package.json`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/tsconfig.json`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/next.config.ts`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/app/layout.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/app/globals.css`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/lib/api.ts`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/lib/auth.ts`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/lib/contracts.ts`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/components/AppShell.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/pnpm-lock.yaml`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/app/providers.tsx`: implement the responsibility named by this path and the ordered instructions below.

- `apps/web/src/lib/problem.ts`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_18.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

Additional owned package markers and assets:

- `apps/web/public/brand/assets-manifest.json`
- `apps/web/public/brand/BRAND.md`
- `apps/web/public/brand/logo-original.png`
- `apps/web/public/brand/tokens.json`
- `apps/web/public/brand/tokens.css`
- `apps/web/public/brand/fonts/NotoSansArabic.ttf`
- `apps/web/public/brand/fonts/NotoSans.ttf`
- `apps/web/public/brand/fonts/NotoNaskhArabic.ttf`
- `apps/web/public/brand/fonts/NotoSansArabic-OFL.txt`
- `apps/web/public/brand/fonts/NotoSans-OFL.txt`
- `apps/web/public/brand/fonts/NotoNaskhArabic-OFL.txt`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/Preview.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-48x48.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/site.webmanifest`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/maskable-icon-512x512.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/apple-touch-icon.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/android-chrome-192x192.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/maskable-icon-192x192.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-32x32.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/android-chrome-512x512.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-16x16.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon-96x96.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/web/favicon.ico`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/source/original-logo.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/source/transparent-logo.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-180.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-384.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-1024.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-152.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-256.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-24.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-32.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-48.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-144.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-512.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-384.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-144.png`
- `apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-128.png`

- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-64.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-1024.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-180.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-48.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-16.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-512.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-96.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-64.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-24.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-256.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-128.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-192.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-192.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-32.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-16.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-white-152.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/icons/partsiq-icon-transparent-96.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-320.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-1280.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-1280.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-640.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-1740.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-320.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-white-640.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/logos/partsiq-logo-transparent-1740.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-xhdpi-96.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-hdpi-72.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-xxxhdpi-192.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-xxhdpi-144.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/google-play-512.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/android-launcher-mdpi-48.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/mobile/ios-app-store-1024.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/social/partsiq-og-1200x630.png
- apps/web/public/brand/assets/PartsIQ-Asset-Pack/social/partsiq-profile-1080.png

TYPED CONTRACTS

```
export function apiFetch<T>(path: string, init: RequestInit, schema: z.ZodType<T>): Promise<T>
export function AppShell(props: {children: React.ReactNode}): React.JSX.Element
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Copy the exact package pins and brand tokens.
2. Use Next.js App Router, React 19, TypeScript, Tailwind 4, next-intl, Zod and Supabase SSR.

3. Keep locale in a validated cookie and set html lang/dir; avoid parallel locale-route trees.

4. Server code uses request-scoped auth clients; browser code obtains a Supabase access token and calls the single FastAPI origin with Authorization.

5. Use explicit CORS origin configuration and no wildcard credentials. apiFetch validates success and Problem payloads, passes Idempotency-Key on writes, and never retries a write without that same key.

6. Decode decimal values as strings until formatting.

7. Build loading, error, empty and unauthorized UI primitives.

8. Copy asset files by the brand manifest; do not recreate the supplied logo.

9. Theme uses CSS variables with persistent system fallback and accessible focus rings.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_18.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 18`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/18/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable

dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step18, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(web): implement foundation 18 next.js shell branding authentication and api client`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

058 19 customer web

prompts/19-customer_web.md

19 Customer pages and demo integration shell

GOAL

Build pages around typed server resources, not local invented workflow state.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 18 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/web/src/app/page.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/app/demo/page.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/app/enquiries/[id]/page.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/components/CustomerShell.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/hooks/useEnquiry.ts`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/i18n/en.json`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/i18n/ar.json`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_19.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
export function EnquiryPage(props: {enquiryId: string}): React.JSX.Element
export function useEnquiry(id: string): EnquiryStateView
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities

use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Build pages around typed server resources, not local invented workflow state.

2. Poll progress and current enquiry every2seconds while actively processing, then back off to10seconds while waiting; pause in hidden tabs and abort on navigation.

3. Display a visible synthetic-data banner and source mode.

4. Demo selection creates ordinary enquiries and follows the same safety gates.

5. Feature prompts later add chat, attachments, confirmation, history and saved-machine panels to this shell.

6. Preserve unsent text across language/theme changes.

7. A failed POST retains the text and the same idempotency key for an explicit retry.

8. Render customer-safe fields only; never fetch the staff workspace endpoint for customer convenience.

9. Accessible dialogs trap focus and return it to the opener.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_19.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 19`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests`

and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/19/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step19, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(customer_web): implement foundation 19 customer pages and demo integration shell`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

059 20 staff web

prompts/20-staff_web.md

20 Staff pages and authorized review shell

GOAL

Staff shell checks authenticated identity but relies on API authorization for every resource.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 19 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/web/src/app/staff/page.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/app/staff/enquiries/[id]/page.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/components/StaffShell.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `apps/web/src/components/ReviewActions.tsx`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_20.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
export function ReviewPage(props: {enquiryId: string}): React.JSX.Element
export function ReviewActions(props: {revision: number;canApprove: boolean}): React.JSX.Element
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Staff shell checks authenticated identity but relies on API authorization for every resource.

2. Show assigned queue, details, candidates, evidence, plan, verification and draft areas without granting access through the UI role toggle.

3. Disable approval until the latest workspace revision is loaded and verification passes.

4. On409 refetch current workspace and show what changed; never silently retry approval with a new revision.

5. Preserve internal notes in separate controls and exclude them from customer/PDF DTOs.

6. Support side-by-side candidate comparison on desktop and stacked sections on mobile.

7. Real assignments are admin controlled; demo reviewer sees only its own session.

8. Feature prompts connect individual panels and their tests.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_20.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 20`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/20/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable

dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step20, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(staff_web): implement foundation 20 staff pages and authorized review shell`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

060 21 evaluation

prompts/21-evaluation.md

21 Deterministic evaluation and evidence receipts

GOAL

Create labelled cases for all four demos plus boundaries, Arabic variants, typos, ambiguity, incompatible exact-number matches, duplicate parts and unavailable transfer routes.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 20 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/evaluation/runner.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/evaluation/curation.py`: implement the responsibility named by this path and the ordered instructions below.
- `evals/cases.json`: implement the responsibility named by this path and the ordered instructions below.
- `evals/README.md`: implement the responsibility named by this path and the ordered instructions below.
- `tools/run_evaluations.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_21.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def evaluate_case(case: EvalCase, service: SearchService) -> EvalResult
def summarize_results(results: list[EvalResult]) -> EvalRun
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Create labelled cases for all four demos plus boundaries, Arabic variants, typos, ambiguity, incompatible exact-number matches, duplicate parts and unavailable transfer routes.

2. The deterministic suite runs without external model or Snowflake calls against a validated synthetic snapshot.

3. Live extraction evaluation is opt-in and allowance-gated.

4. Separate extraction accuracy, compatible-candidate recall, clarification correctness and publication safety; include numerator, denominator and dataset version.

5. Do not summarize simulated model responses as measured model accuracy.

6. Human corrections become evaluation ground truth only after admin review and anonymization.

7. Write actual failed outputs and source revisions into receipts; never update expected answers just to make tests pass.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_21.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 21`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/21/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step21, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(evaluation): implement foundation 21 deterministic evaluation and evidence receipts`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

061 23 analytics

prompts/23-analytics.md

23 Structured logs tracing metrics and Snowflake outbox

GOAL

Write a redacted analytics_outbox row in the same business transaction as an event.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 21 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/partsiq/analytics/outbox.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/analytics/snowflake.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/analytics/service.py`: implement the responsibility named by this path and the ordered instructions below.
- `apps/api/partsiq/core/observability.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_23.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def export_events(batch_size: int = 100) -> AnalyticsSyncReceipt
async def metrics(actor: ActorContext, query: MetricsQuery) -> MetricsView
def redact_event(event: Mapping[str, object]) -> dict[str, object]
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Write a redacted analytics_outbox row in the same business transaction as an event.

2. Export by fixed EVENT_ID using a single writer and MERGE with bound parameters; standard Snowflake PKs do not deduplicate rows.

3. Record delivery only after remote success.

4. A crash after MERGE before acknowledgement safely repeats the same event.

5. Reject duplicate source IDs before MERGE and periodically run duplicate diagnostics.

6. Allow only event type, time, demo flag, status, stage, duration, tokens and estimated cost in Snowflake; exclude identity, messages, serials and notes.

7. Live staff counts remain scoped PostgreSQL queries.

8. Cross-account aggregate reporting comes from Snowflake for admins and displays last_success_at.

9. Optional Langfuse traces redact payloads and cannot block business transactions.

10. Monotonic stage timers measure durations; UTC timestamps describe events.

11. Budget accounting remains local even if reporting fails.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_23.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 23`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/23/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step23, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(analytics): implement foundation 23 structured logs tracing metrics and snowflake outbox`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

062 24 containers

prompts/24-containers.md

24 One application image worker and Caddy

GOAL

Build one CPU-only application image with frozen Python dependencies.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 23 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `apps/api/Dockerfile`: implement the responsibility named by this path and the ordered instructions below.
- `docker-compose.yml`: implement the responsibility named by this path and the ordered instructions below.
- `infra/Caddyfile`: implement the responsibility named by this path and the ordered instructions below.
- `infra/compose.production.yml`: implement the responsibility named by this path and the ordered instructions below.
- `infra/compose.fixture.yml`: implement the responsibility named by this path and the ordered instructions below.
- `tools/healthcheck.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_24.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def readiness(settings: Settings) -> ReadinessReport
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Build one CPU-only application image with frozen Python dependencies.

2. API command is uvicorn partsiq.main: create_app --factory --host 0.0.0.0 --port 8000 --workers 1; worker command is python -m partsiq.jobs.main.

3. API never imports the optional encoder on startup.

4. Caddy exposes HTTPS and routes /v1 and health endpoints to API.

5. Redis is private and persistent only for rate limits/notifications; PostgreSQL owns jobs.

6. Supabase stays managed and is not redeployed on EC2.

7. Bind local development ports to127.0.0.1.

8. Do not expose Redis or internal metrics.

9. Add non-root user, read-only root filesystem where possible, temporary scratch mount and explicit CPU/memory limits.

10. Include a separate optional semantic profile with measured RAM requirements; do not promise BGE-M3 fits a free micro instance.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_24.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 24`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/24/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step24, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(containers): implement foundation 24 one application image worker and caddy`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

063 25 ci

prompts/25-ci.md

25 CI reproducible builds and release checks

GOAL

Pin GitHub actions to the recorded SHAs.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 24 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `.github/workflows/ci.yml`: implement the responsibility named by this path and the ordered instructions below.
- `.github/workflows/deploy.yml`: implement the responsibility named by this path and the ordered instructions below.
- `tools/check_contract_drift.py`: implement the responsibility named by this path and the ordered instructions below.
- `tools/check_secret_patterns.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_25.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def check_contract_drift(openapi_path: Path, expected_path: Path) -> list[str]
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Pin GitHub actions to the recorded SHAs.

2. CI installs frozen Python/JS dependencies, checks lint/types, starts disposable Supabase, runs unit/RLS/feature tests and deterministic evaluation, and builds API/web artifacts.

3. Runtime OpenAPI must match spec/openapi.json structurally after canonical sorting.

4. Missing feature test files fail the completeness gate.

5. Untrusted fork jobs have no service keys.

6. Live Snowflake smoke tests require a dedicated development account and explicit manual dispatch; not running them must remain visible.

7. Build immutable image tags by commit and publish only after successful gates.

8. Production deploy uses environment approval and the documented rollback tag.

9. Do not auto-upgrade dependencies during feature implementation.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_25.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 25`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/25/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is

blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step25, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(ci): implement foundation 25 ci reproducible builds and release checks`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

During incremental development,validate only completed prompt owners and their existing operations. Activate the all-feature/all-operation gate in prompt22 after F69. CI itself must be usable before feature implementation;missing future features are not mistaken for completed work.

064 F01 branded landing

prompts/F01-branded-landing.md

F01 Branded landing

GOAL

Implement the complete branded landing behavior in the single-workflow Snowflake edition. Explain the enquiry-to-approval journey; show synthetic-demo notice; expose Start demo and architecture links; display the supplied bilingual logo without stretching.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows foundation 25. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f01_branded_landing.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f01_branded_landing/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f01.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f01.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f01`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f01_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F01; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Explain the enquiry-to-approval journey.

4. Show synthetic-demo notice.

5. Expose Start demo and architecture links.

6. Display the supplied bilingual logo without stretching.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Missing repository link hides that action. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: logo has meaningful alternative text. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: no affiliation claim. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f01.py`. Frontend path: `apps/web/tests/features/f01.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Missing repository link hides that action. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: logo has meaningful alternative text. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: no affiliation claim. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F01` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f01.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f01.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F01/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F01-branded-landing.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f01): implement branded landing`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F01 Branded landing

Status: specified, not implemented.

Owning prompt: `prompts/F01-branded-landing.md`.

Behavior: Explain the enquiry-to-approval journey; show synthetic-demo notice; expose Start demo and architecture links; display the supplied bilingual logo without stretching.

Failure criteria: Missing repository link hides that action; logo has meaningful alternative text; no affiliation claim.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f01_branded_landing.py`
- `apps/web/src/features/f01_branded_landing/Panel.tsx`
- `tests/features/test_f01.py`
- `apps/web/tests/features/f01.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

065 F02 demo scenarios and assets

prompts/F02-demo-scenarios-and-assets.md

F02 Demo scenarios and assets

GOAL

Implement the complete demo scenarios and assets behavior in the single-workflow Snowflake edition. Seed four deterministic scenarios: mixed-language photo, missing serial, incompatible part, local shortage; derive identifiers from seed 42; validate expected outcomes against generated records.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F01. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `catalog` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f02_demo_scenarios_and_assets.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f02_demo_scenarios_and_assets/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f02.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f02.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f02_demo_scenarios_and_assets.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f02`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f02_list_scenarios(actor: ActorContext) -> ScenarioList
class ScenarioList(Contract):
    items:list[DemoScenario]
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code,

details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F02; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Seed four deterministic scenarios: mixed-language photo, missing serial, incompatible part, local shortage.

4. Derive identifiers from seed 42.

5. Validate expected outcomes against generated records.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/demo/scenarios` → `f02_list_scenarios`; request `None` (no body); response `ScenarioList`, HTTP 200; access `demo`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Reject a scenario whose expected part lacks compatibility. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: reject stock claims not supported by fixture quantities. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f02.py`. Frontend path: `apps/web/tests/features/f02.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Reject a scenario whose expected part lacks compatibility. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: reject stock claims not supported by fixture quantities. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F02` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f02.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f02.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F02/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F02-demo-scenarios-and-assets.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f02): implement demo scenarios and assets`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F02 Demo scenarios and assets

Status: specified, not implemented.

Owning prompt: `prompts/F02-demo-scenarios-and-assets.md`.

Behavior: Seed four deterministic scenarios: mixed-language photo, missing serial, incompatible part, local shortage; derive identifiers from seed 42; validate expected outcomes against generated records.

Failure criteria: Reject a scenario whose expected part lacks compatibility; reject stock claims not supported by fixture quantities.

API operations: f02_list_scenarios

Files:

- `apps/api/partsiq/features/f02_demo_scenarios_and_assets.py`
- `apps/web/src/features/f02_demo_scenarios_and_assets/Panel.tsx`
- `tests/features/test_f02.py`
- `apps/web/tests/features/f02.test.tsx`
- `apps/api/partsiq/api/f02_demo_scenarios_and_assets.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

066 F03 demo role switching

prompts/F03-demo-role-switching.md

F03 Demo role switching

GOAL

Implement the complete demo role switching behavior in the single-workflow Snowflake edition. Switch the presentation between customer and reviewer using the same anonymous identity and session; authorize review only for enquiries belonging to that active session.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F02. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f03_demo_role_switching.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f03_demo_role_switching/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f03.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f03.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f03`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f03_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F03; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Switch the presentation between customer and reviewer using the same anonymous identity and session.

4. Authorize review only for enquiries belonging to that active session.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Forged reviewer role never grants staff membership. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: another visitor's enquiry returns 404. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f03.py`. Frontend path: `apps/web/tests/features/f03.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Forged reviewer role never grants staff membership. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: another visitor's enquiry returns 404. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F03` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f03.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f03.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F03/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F03-demo-role-switching.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f03): implement demo role switching`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F03 Demo role switching

Status: specified, not implemented.

Owning prompt: `prompts/F03-demo-role-switching.md`.

Behavior: Switch the presentation between customer and reviewer using the same anonymous identity and session; authorize review only for enquiries belonging to that active session.

Failure criteria: Forged reviewer role never grants staff membership; another visitor's enquiry returns 404.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f03_demo_role_switching.py`
- `apps/web/src/features/f03_demo_role_switching/Panel.tsx`
- `tests/features/test_f03.py`
- `apps/web/tests/features/f03.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

067 F04 demo isolation

prompts/F04-demo-isolation.md

F04 Demo isolation

GOAL

Implement the complete demo isolation behavior in the single-workflow Snowflake edition. Bind each session to verified anonymous Supabase identity; check generation and expiry on every read, write and worker completion; exclude demos from operational metrics by default.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F03. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f04_demo_isolation.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f04_demo_isolation/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f04.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f04.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f04_demo_isolation.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f04`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f04_require_demo_session(actor: ActorContext, session_id: UUID) -> DemoSession
async def f04_create_session(actor: ActorContext, idempotency_key: UUID) -> DemoSession
class DemoSession(Contract):
    id: UUID
    generation: int = Field(ge=1)
    expires_at: datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal

feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F04; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Bind each session to verified anonymous Supabase identity.

4. Check generation and expiry on every read, write and worker completion.

5. Exclude demos from operational metrics by default.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/demo/sessions/{session_id}` → `f04_require_demo_session`; request `None` (no body); response `DemoSession`, HTTP 200; access `demo`.

• `POST /v1/demo/sessions` → `f04_create_session`; request `None` (no body); response `DemoSession`, HTTP 201; access `demo`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Expired, reset and foreign sessions fail. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing identity context returns no rows. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f04.py`. Frontend path: `apps/web/tests/features/f04.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Expired, reset and foreign sessions fail. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing identity context returns no rows. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F04` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f04.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f04.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F04/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F04-demo-isolation.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f04): implement demo isolation`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F04 Demo isolation

Status: specified, not implemented.

Owning prompt: `prompts/F04-demo-isolation.md`.

Behavior: Bind each session to verified anonymous Supabase identity; check generation and expiry on every read, write and worker completion; exclude demos from operational metrics by default.

Failure criteria: Expired, reset and foreign sessions fail; missing identity context returns no rows.

API operations: f04_require_demo_session, f04_create_session

Files:

- `apps/api/partsiq/features/f04_demo_isolation.py`
- `apps/web/src/features/f04_demo_isolation/Panel.tsx`
- `tests/features/test_f04.py`
- `apps/web/tests/features/f04.test.tsx`
- `apps/api/partsiq/api/f04_demo_isolation.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

068 F05 demo reset

prompts/F05-demo-reset.md

F05 Demo reset

GOAL

Implement the complete demo reset behavior in the single-workflow Snowflake edition. Lock session; increment generation; cancel unfinished jobs from prior generation; enqueue Storage cleanup; keep minimal audit; create fresh scenario enquiries on explicit selection.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F04. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f05_demo_reset.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f05_demo_reset/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f05.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f05.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f05_demo_reset.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f05`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f05_reset_session(actor: ActorContext, session_id: UUID, command: RevisionOnly,
idempotency_key: UUID) -> DemoSession
class DemoSession(Contract):
    id: UUID
    generation: int = Field(ge=1)
    expires_at: datetime

class RevisionOnly(Contract):
    expected_revision:int = Field(ge=1)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F05; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Lock session.

4. Increment generation.

5. Cancel unfinished jobs from prior generation.

6. Enqueue Storage cleanup.

7. Keep minimal audit.

8. Create fresh scenario enquiries on explicit selection.

9. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

10. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

11. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

12. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/demo/sessions/{session_id}/reset` → `f05_reset_session`; request `RevisionOnly` as JSON; response `DemoSession`, HTTP 201; access `demo`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Late OCR from old generation cannot update current data. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: repeated reset idempotency key returns the same generation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f05.py`. Frontend path: `apps/web/tests/features/f05.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Late OCR from old generation cannot update current data. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: repeated reset idempotency key returns the same generation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F05` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f05.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f05.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F05/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F05-demo-reset.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f05): implement demo reset`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval

snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F05 Demo reset

Status: specified, not implemented.

Owning prompt: `prompts/F05-demo-reset.md`.

Behavior: Lock session; increment generation; cancel unfinished jobs from prior generation; enqueue Storage cleanup; keep minimal audit; create fresh scenario enquiries on explicit selection.

Failure criteria: Late OCR from old generation cannot update current data; repeated reset idempotency key returns the same generation.

API operations: f05_reset_session

Files:

- `apps/api/partsiq/features/f05_demo_reset.py`
- `apps/web/src/features/f05_demo_reset/Panel.tsx`
- `tests/features/test_f05.py`
- `apps/web/tests/features/f05.test.tsx`
- `apps/api/partsiq/api/f05_demo_reset.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

069 F06 responsive layouts

prompts/F06-responsive-layouts.md

F06 Responsive layouts

GOAL

Implement the complete responsive layouts behavior in the single-workflow Snowflake edition. Support 360, 768, 1280 CSS-pixel widths; stack review panels; horizontally scroll comparison tables within their region; preserve sticky primary review actions without covering content.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F05. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f06_responsive_layouts.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f06_responsive_layouts/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f06.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f06.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f06`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f06_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F06; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Support 360, 768, 1280 CSS-pixel widths.

4. Stack review panels.

5. Horizontally scroll comparison tables within their region.

6. Preserve sticky primary review actions without covering content.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No page-wide horizontal overflow. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: zoom 200 percent remains usable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: mobile dialogs retain visible close action. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f06.py`. Frontend path: `apps/web/tests/features/f06.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No page-wide horizontal overflow. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: zoom 200 percent remains usable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: mobile dialogs retain visible close action. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F06` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f06.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f06.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F06/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F06-responsive-layouts.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f06): implement responsive layouts`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F06 Responsive layouts

Status: specified, not implemented.

Owning prompt: `prompts/F06-responsive-layouts.md`.

Behavior: Support 360, 768, 1280 CSS-pixel widths; stack review panels; horizontally scroll comparison tables within their region; preserve sticky primary review actions without covering content.

Failure criteria: No page-wide horizontal overflow; zoom 200 percent remains usable; mobile dialogs retain visible close action.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f06_responsive_layouts.py`
- `apps/web/src/features/f06_responsive_layouts/Panel.tsx`
- `tests/features/test_f06.py`
- `apps/web/tests/features/f06.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

070 F07 arabic and english interface

prompts/F07-arabic-and-english-interface.md

F07 Arabic and English interface

GOAL

Implement the complete arabic and english interface behavior in the single-workflow Snowflake edition. Use a locale cookie and complete en/ar message catalogs; persist preference; set document lang and dir; isolate serials, numbers and part identifiers with bdi.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F06. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f07_arabic_and_english_interface.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f07_arabic_and_english_interface/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f07.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f07.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f07`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f07_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F07; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Use a locale cookie and complete en/ar message catalogs.

4. Persist preference.

5. Set document lang and dir.

6. Isolate serials, numbers and part identifiers with bdi.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Missing translation fails tests. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: language switch preserves enquiry ID and unsent draft. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: mixed Arabic identifiers remain readable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f07.py`. Frontend path: `apps/web/tests/features/f07.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Missing translation fails tests. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: language switch preserves enquiry ID and unsent draft. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: mixed Arabic identifiers remain readable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F07` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f07.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f07.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F07/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F07-arabic-and-english-interface.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f07): implement arabic and english interface`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case

above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F07 Arabic and English interface

Status: specified, not implemented.

Owning prompt: `prompts/F07-arabic-and-english-interface.md`.

Behavior: Use a locale cookie and complete en/ar message catalogs; persist preference; set document lang and dir; isolate serials, numbers and part identifiers with bdi.

Failure criteria: Missing translation fails tests; language switch preserves enquiry ID and unsent draft; mixed Arabic identifiers remain readable.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f07_arabic_and_english_interface.py`
- `apps/web/src/features/f07_arabic_and_english_interface/Panel.tsx`
- `tests/features/test_f07.py`
- `apps/web/tests/features/f07.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

071 F08 light and dark themes

prompts/F08-light-and-dark-themes.md

F08 Light and dark themes

GOAL

Implement the complete light and dark themes behavior in the single-workflow Snowflake edition. Use CSS semantic tokens; initialize from saved preference then system setting; expose accessible toggle; honor reduced motion and browser color-scheme.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F07. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f08_light_and_dark_themes.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f08_light_and_dark_themes/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f08.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f08.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f08`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f08_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F08; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Use CSS semantic tokens.

4. Initialize from saved preference then system setting.

5. Expose accessible toggle.

6. Honor reduced motion and browser color-scheme.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No hydration mismatch. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: chart colors and focus rings meet contrast checks. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: storage-disabled browsers still work. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f08.py`. Frontend path: `apps/web/tests/features/f08.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No hydration mismatch. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: chart colors and focus rings meet contrast checks. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: storage-disabled browsers still work. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F08` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f08.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f08.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F08/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F08-light-and-dark-themes.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f08): implement light and dark themes`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F08 Light and dark themes

Status: specified, not implemented.

Owning prompt: `prompts/F08-light-and-dark-themes.md`.

Behavior: Use CSS semantic tokens; initialize from saved preference then system setting; expose accessible toggle; honor reduced motion and browser color-scheme.

Failure criteria: No hydration mismatch; chart colors and focus rings meet contrast checks; storage-disabled browsers still work.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f08_light_and_dark_themes.py`
- `apps/web/src/features/f08_light_and_dark_themes/Panel.tsx`
- `tests/features/test_f08.py`
- `apps/web/tests/features/f08.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

072 F09 bilingual chat

prompts/F09-bilingual-chat.md

F09 Bilingual chat

GOAL

Implement the complete bilingual chat behavior in the single-workflow Snowflake edition. Accept Arabic, English and mixed text; preserve original message; detect preferred reply locale; render customer messages immediately after successful persistence.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F08. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f09_bilingual_chat.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f09_bilingual_chat/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f09.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f09.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f09_bilingual_chat.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f09`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f09_create_enquiry(actor: ActorContext, command: EnquiryCreate, idempotency_key: UUID) ->
EnquiryView
async def f09_get_enquiry(actor: ActorContext, enquiry_id: UUID) -> EnquiryView
class EnquiryCreate(Contract):
    text: str = Field(min_length=1,max_length=12000)
    locale: Locale
    destination_warehouse_id: int = Field(gt=0)
    session_id: UUID | None = None
```

```
class EnquiryView(Contract):
    id: UUID
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
    catalogue_version_id: UUID | None
    updated_at: datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F09; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Accept Arabic, English and mixed text.

4. Preserve original message.

5. Detect preferred reply locale.

6. Render customer messages immediately after successful persistence.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries` → `f09_create_enquiry`; request `EnquiryCreate` as JSON; response `EnquiryView`, HTTP 202; access `customer`.

• `GET /v1/enquiries/{enquiry_id}` → `f09_get_enquiry`; request `None` (no body); response `EnquiryView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not

proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Empty and oversized input returns validation error. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: HTML is escaped. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: retry does not duplicate a message. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f09.py`. Frontend path: `apps/web/tests/features/f09.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Empty and oversized input returns validation error. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: HTML is escaped. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: retry does not duplicate a message. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F09` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f09.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f09.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F09/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F09-bilingual-chat.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together;

never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f09): implement bilingual chat`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F09 Bilingual chat

Status: specified, not implemented.

Owning prompt: `prompts/F09-bilingual-chat.md`.

Behavior: Accept Arabic, English and mixed text; preserve original message; detect preferred reply locale; render customer messages immediately after successful persistence.

Failure criteria: Empty and oversized input returns validation error; HTML is escaped; retry does not duplicate a message.

API operations: f09_create_enquiry, f09_get_enquiry

Files:

- `apps/api/partsiq/features/f09_bilingual_chat.py`
- `apps/web/src/features/f09_bilingual_chat/Panel.tsx`
- `tests/features/test_f09.py`
- `apps/web/tests/features/f09.test.tsx`
- `apps/api/partsiq/api/f09_bilingual_chat.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

073 F10 structured extraction

prompts/F10-structured-extraction.md

F10 Structured extraction

GOAL

Implement the complete structured extraction behavior in the single-workflow Snowflake edition. Extract item descriptions, quantities, urgency and machine hints with strict structured outputs; preserve source spans; missing quantity becomes a clarification rather than assumed order quantity.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F09. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f10_structured_extraction.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f10_structured_extraction/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f10.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f10.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f10_structured_extraction.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f10`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f10_read_current(actor: ActorContext, enquiry_id: UUID) -> ExtractionView
class ExtractionView(Contract):
    revision:int
    intake:IntakeResult | None
    nameplate:NameplateResult | None
    confirmed:bool
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F10; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Extract item descriptions, quantities, urgency and machine hints with strict structured outputs.

4. Preserve source spans.

5. Missing quantity becomes a clarification rather than assumed order quantity.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/extraction` → `f10_read_current`; request `None` (no body); response `ExtractionView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unknown keys and invalid quantities reject the response. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unsupported model output cannot progress to search. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f10.py`. Frontend path: `apps/web/tests/features/f10.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unknown keys and invalid quantities reject the response. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unsupported model output cannot progress to search. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F10` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f10.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f10.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F10/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F10-structured-extraction.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f10): implement structured extraction`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case

above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F10 Structured extraction

Status: specified, not implemented.

Owning prompt: `prompts/F10-structured-extraction.md`.

Behavior: Extract item descriptions, quantities, urgency and machine hints with strict structured outputs; preserve source spans; missing quantity becomes a clarification rather than assumed order quantity.

Failure criteria: Unknown keys and invalid quantities reject the response; unsupported model output cannot progress to search.

API operations: f10_read_current

Files:

- `apps/api/partsiq/features/f10_structured_extraction.py`
- `apps/web/src/features/f10_structured_extraction/Panel.tsx`
- `tests/features/test_f10.py`
- `apps/web/tests/features/f10.test.tsx`
- `apps/api/partsiq/api/f10_structured_extraction.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

074 F11 clarifying questions

prompts/F11-clarifying-questions.md

F11 Clarifying questions

GOAL

Implement the complete clarifying questions behavior in the single-workflow Snowflake edition. Use workflow reason codes to choose code-owned bilingual questions; persist pending question; associate answer with item; rerun affected resolution only.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F10. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f11_clarifying_questions.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f11_clarifying_questions/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f11.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f11.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f11_clarifying_questions.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f11`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f11_append_message(actor: ActorContext, enquiry_id: UUID, command: MessageCreate,
idempotency_key: UUID) -> EnquiryView
class EnquiryView(Contract):
    id: UUID
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
```

```
    catalogue_version_id: UUID | None
    updated_at: datetime

class MessageCreate(Contract):
    text: str = Field(min_length=1,max_length=12000)
    expected_revision: int = Field(ge=1)
    item_id: UUID | None = None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F11; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Use workflow reason codes to choose code-owned bilingual questions.

4. Persist pending question.

5. Associate answer with item.

6. Rerun affected resolution only.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/messages` → `f11_append_message`; request `MessageCreate` as JSON; response `EnquiryView`, HTTP 202; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No final recommendation before approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: contradictory answer requires confirmation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: repeat event creates one question. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f11.py`. Frontend path: `apps/web/tests/features/f11.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No final recommendation before approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: contradictory answer requires confirmation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: repeat event creates one question. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F11` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f11.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f11.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F11/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F11-clarifying-questions.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f11): implement clarifying questions`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F11 Clarifying questions

Status: specified, not implemented.

Owning prompt: `prompts/F11-clarifying-questions.md`.

Behavior: Use workflow reason codes to choose code-owned bilingual questions; persist pending question; associate answer with item; rerun affected resolution only.

Failure criteria: No final recommendation before approval; contradictory answer requires confirmation; repeat event creates one question.

API operations: f11_append_message

Files:

- `apps/api/partsiq/features/f11_clarifying_questions.py`
- `apps/web/src/features/f11_clarifying_questions/Panel.tsx`
- `tests/features/test_f11.py`
- `apps/web/tests/features/f11.test.tsx`
- `apps/api/partsiq/api/f11_clarifying_questions.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

075 F12 conversation memory

prompts/F12-conversation-memory.md

F12 Conversation memory

GOAL

Implement the complete conversation memory behavior in the single-workflow Snowflake edition. Persist all messages and typed current state; compact context using retained structured facts; resume by enquiry/revision; never mix customer sessions.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F11. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f12_conversation_memory.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f12_conversation_memory/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f12.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f12.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f12_conversation_memory.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f12`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f12_list_messages(actor: ActorContext, enquiry_id: UUID, command: PageQuery) -> MessagePage
class MessagePage(Contract):
    items:list[MessageView]
    next_cursor:str | None

class PageQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F12; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Persist all messages and typed current state.

4. Compact context using retained structured facts.

5. Resume by enquiry/revision.

6. Never mix customer sessions.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/messages` → `f12_list_messages`; request `PageQuery` as validated query parameters; response `MessagePage`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Restart retains unresolved questions. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: old revision events are ignored and audited. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: foreign thread ID denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f12.py`. Frontend path: `apps/web/tests/features/f12.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Restart retains unresolved questions. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: old revision events are ignored and audited. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: foreign thread ID denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F12` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f12.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f12.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F12/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F12-conversation-memory.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f12): implement conversation memory`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F12 Conversation memory

Status: specified, not implemented.

Owning prompt: `prompts/F12-conversation-memory.md`.

Behavior: Persist all messages and typed current state; compact context using retained structured facts; resume by enquiry/revision; never mix customer sessions.

Failure criteria: Restart retains unresolved questions; old revision events are ignored and audited; foreign thread ID denied.

API operations: f12_list_messages

Files:

- `apps/api/partsiq/features/f12_conversation_memory.py`
- `apps/web/src/features/f12_conversation_memory/Panel.tsx`
- `tests/features/test_f12.py`
- `apps/web/tests/features/f12.test.tsx`
- `apps/api/partsiq/api/f12_conversation_memory.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

076 F13 private uploads

prompts/F13-private-uploads.md

F13 Private uploads

GOAL

Implement the complete private uploads behavior in the single-workflow Snowflake edition. Create pending attachment and unique object path; use signed upload; validate actual bytes and magic type after confirmation; strip EXIF and cap decoded pixels before OCR.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F12. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f13_private_uploads.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f13_private_uploads/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f13.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f13.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f13_private_uploads.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f13`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f13_create_upload(actor: ActorContext, enquiry_id: UUID, command: UploadRequest,
idempotency_key: UUID) -> UploadTicket
async def f13_list_attachments(actor: ActorContext, enquiry_id: UUID) -> AttachmentList
class AttachmentList(Contract):
    items:list[AttachmentView]

class UploadRequest(Contract):
    enquiry_id: UUID
    expected_revision: int = Field(ge=1)
    media_type: Literal['image/jpeg','image/png','image/webp']
```

```
    size_bytes: int = Field(gt=0,le=10485760)

class UploadTicket(Contract):
    attachment_id: UUID
    upload_url: str
    expires_at: datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F13; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Create pending attachment and unique object path.

4. Use signed upload.

5. Validate actual bytes and magic type after confirmation.

6. Strip EXIF and cap decoded pixels before OCR.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/uploads` → `f13_create_upload`; request `UploadRequest` as JSON; response `UploadTicket`, HTTP 200; access `owner`.

• `GET /v1/enquiries/{enquiry_id}/attachments` → `f13_list_attachments`; request `None` (no body); response `AttachmentList`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Wrong MIME, oversized content, image decompression bomb, duplicate confirmation and nonexistent object handled explicitly. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f13.py`. Frontend path: `apps/web/tests/features/f13.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Wrong MIME, oversized content, image decompression bomb, duplicate confirmation and nonexistent object handled explicitly. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F13` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f13.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f13.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F13/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F13-private-uploads.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f13): implement private uploads`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F13 Private uploads

Status: specified, not implemented.

Owning prompt: `prompts/F13-private-uploads.md`.

Behavior: Create pending attachment and unique object path; use signed upload; validate actual bytes and magic type after confirmation; strip EXIF and cap decoded pixels before OCR.

Failure criteria: Wrong MIME, oversized content, image decompression bomb, duplicate confirmation and nonexistent object handled explicitly.

API operations: f13_create_upload, f13_list_attachments

Files:

- `apps/api/partsiq/features/f13_private_uploads.py`
- `apps/web/src/features/f13_private_uploads/Panel.tsx`
- `tests/features/test_f13.py`
- `apps/web/tests/features/f13.test.tsx`
- `apps/api/partsiq/api/f13_private_uploads.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

077 F14 nameplate ocr

prompts/F14-nameplate-ocr.md

F14 Nameplate OCR

GOAL

Implement the complete nameplate ocr behavior in the single-workflow Snowflake edition. Read validated nameplate via OpenAI vision; return model_text, serial_text, issues and source attachment; persist result only with current lease and revision.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F13. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `worker` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f14_nameplate_ocr.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f14_nameplate_ocr/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f14.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f14.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f14_nameplate_ocr.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f14`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f14_confirm_upload(actor: ActorContext, attachment_id: UUID, command: RevisionOnly,
idempotency_key: UUID) -> JobView
class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None
```

```
class RevisionOnly(Contract):
    expected_revision:int = Field(ge=1)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F14; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Read validated nameplate via OpenAI vision.

4. Return model_text, serial_text, issues and source attachment.

5. Persist result only with current lease and revision.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/attachments/{attachment_id}/confirm-upload` → `f14_confirm_upload`; request `RevisionOnly` as JSON; response `JobView`, HTTP 202; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unreadable fields remain null. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: vision instructions in photo are untrusted. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: timeout is bounded and creates recoverable status. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f14.py`. Frontend path: `apps/web/tests/features/f14.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unreadable fields remain null. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: vision instructions in photo are untrusted. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: timeout is bounded and creates recoverable status. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F14` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f14.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f14.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F14/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F14-nameplate-ocr.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f14): implement nameplate ocr`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F14 Nameplate OCR

Status: specified, not implemented.

Owning prompt: `prompts/F14-nameplate-ocr.md`.

Behavior: Read validated nameplate via OpenAI vision; return model_text, serial_text, issues and source attachment; persist result only with current lease and revision.

Failure criteria: Unreadable fields remain null; vision instructions in photo are untrusted; timeout is bounded and creates recoverable status.

API operations: f14_confirm_upload

Files:

- `apps/api/partsiq/features/f14_nameplate_ocr.py`
- `apps/web/src/features/f14_nameplate_ocr/Panel.tsx`
- `tests/features/test_f14.py`
- `apps/web/tests/features/f14.test.tsx`
- `apps/api/partsiq/api/f14_nameplate_ocr.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

078 F15 ocr confirmation

prompts/F15-ocr-confirmation.md

F15 OCR confirmation

GOAL

Implement the complete ocr confirmation behavior in the single-workflow Snowflake edition. Show cropped or original nameplate beside extracted identifiers; require user confirm or correction; record actor and time; preserve leading zeros in serial text.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F14. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f15_ocr_confirmation.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f15_ocr_confirmation/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f15.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f15.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f15_ocr_confirmation.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f15`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f15_confirm_nameplate(actor: ActorContext, enquiry_id: UUID, command: ConfirmExtraction,
idempotency_key: UUID) -> EnquiryView
class ConfirmExtraction(Contract):
    extraction_id: UUID
    expected_revision: int = Field(ge=1)
    model_text: str = Field(min_length=1,max_length=120)
    serial_text: str | None = Field(default=None,max_length=120)

class EnquiryView(Contract):
    id: UUID
```

```
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
    catalogue_version_id: UUID | None
    updated_at: datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F15; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Show cropped or original nameplate beside extracted identifiers.

4. Require user confirm or correction.

5. Record actor and time.

6. Preserve leading zeros in serial text.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/confirm-nameplate` → `f15_confirm_nameplate`; request `ConfirmExtraction` as JSON; response `EnquiryView`, HTTP 202; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unconfirmed OCR never authorizes matching. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: blank corrected serial triggers missing-serial flow. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: stale confirmation returns 409. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f15.py`. Frontend path: `apps/web/tests/features/f15.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unconfirmed OCR never authorizes matching. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: blank corrected serial triggers missing-serial flow. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: stale confirmation returns 409. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F15` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f15.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f15.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F15/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F15-ocr-confirmation.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f15): implement ocr confirmation`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F15 OCR confirmation

Status: specified, not implemented.

Owning prompt: `prompts/F15-ocr-confirmation.md`.

Behavior: Show cropped or original nameplate beside extracted identifiers; require user confirm or correction; record actor and time; preserve leading zeros in serial text.

Failure criteria: Unconfirmed OCR never authorizes matching; blank corrected serial triggers missing-serial flow; stale confirmation returns 409.

API operations: f15_confirm_nameplate

Files:

- `apps/api/partsiq/features/f15_ocr_confirmation.py`
- `apps/web/src/features/f15_ocr_confirmation/Panel.tsx`
- `tests/features/test_f15.py`
- `apps/web/tests/features/f15.test.tsx`
- `apps/api/partsiq/api/f15_ocr_confirmation.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

079 F16 photo quality feedback

prompts/F16-photo-quality-feedback.md

F16 Photo quality feedback

GOAL

Implement the complete photo quality feedback behavior in the single-workflow Snowflake edition. Check decodability, dimensions and blur heuristic before paid call; return actionable retake instructions; mark heuristic as advisory; allow a readable image despite noncritical blur warning.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F15. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `worker` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f16_photo_quality_feedback.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f16_photo_quality_feedback/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f16.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f16.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f16_photo_quality_feedback.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f16`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f16_read_quality(actor: ActorContext, attachment_id: UUID) -> QualityView
class QualityView(Contract):
    attachment_id:UUID
    status:Literal['pending','valid','rejected']
    width:int | None
    height:int | None
    warnings:list[str]
    error_code:str | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F16; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Check decodability, dimensions and blur heuristic before paid call.

4. Return actionable retake instructions.

5. Mark heuristic as advisory.

6. Allow a readable image despite noncritical blur warning.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/attachments/{attachment_id}/quality` → `f16_read_quality`; request `None` (no body); response `QualityView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unsupported format rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: dark or low-resolution sample gets guidance. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: no fabricated OCR confidence percentage. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f16.py`. Frontend path: `apps/web/tests/features/f16.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unsupported format rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: dark or low-resolution sample gets guidance. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: no fabricated OCR confidence percentage. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F16` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f16.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f16.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F16/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F16-photo-quality-feedback.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f16): implement photo quality feedback`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F16 Photo quality feedback

Status: specified, not implemented.

Owning prompt: `prompts/F16-photo-quality-feedback.md`.

Behavior: Check decodability, dimensions and blur heuristic before paid call; return actionable retake instructions; mark heuristic as advisory; allow a readable image despite noncritical blur warning.

Failure criteria: Unsupported format rejected; dark or low-resolution sample gets guidance; no fabricated OCR confidence percentage.

API operations: f16_read_quality

Files:

- `apps/api/partsiq/features/f16_photo_quality_feedback.py`
- `apps/web/src/features/f16_photo_quality_feedback/Panel.tsx`
- `tests/features/test_f16.py`
- `apps/web/tests/features/f16.test.tsx`
- `apps/api/partsiq/api/f16_photo_quality_feedback.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

080 F17 multi part enquiries

prompts/F17-multi-part-enquiries.md

F17 Multi-part enquiries

GOAL

Implement the complete multi-part enquiries behavior in the single-workflow Snowflake edition. Track up to 20 items with independent quantity, machine, candidate selection and status; clarify individual items; verify aggregate allocation for repeated part requests.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F16. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f17_multi_part_enquiries.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f17_multi_part_enquiries/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f17.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f17.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f17_multi_part_enquiries.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f17`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f17_correct_items(actor: ActorContext, enquiry_id: UUID, command: ItemsEdit, idempotency_key:
 UUID) -> EnquiryView
class EnquiryView(Contract):
    id: UUID
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
```

```
    catalogue_version_id: UUID | None
    updated_at: datetime

class ItemsEdit(Contract):
    expected_revision:int = Field(ge=1)
    items:list[ItemCorrection] = Field(min_length=1,max_length=20)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F17; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Track up to 20 items with independent quantity, machine, candidate selection and status.

4. Clarify individual items.

5. Verify aggregate allocation for repeated part requests.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `PATCH /v1/enquiries/{enquiry_id}/items` → `f17_correct_items`; request `ItemsEdit` as JSON; response `EnquiryView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: One unresolved item blocks final all-items approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: allocations across duplicate parts cannot double count available stock. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: zero quantity rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f17.py`. Frontend path: `apps/web/tests/features/f17.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: One unresolved item blocks final all-items approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: allocations across duplicate parts cannot double count available stock. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: zero quantity rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F17` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f17.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f17.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F17/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F17-multi-part-enquiries.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f17): implement multi-part enquiries`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F17 Multi-part enquiries

Status: specified, not implemented.

Owning prompt: `prompts/F17-multi-part-enquiries.md`.

Behavior: Track up to 20 items with independent quantity, machine, candidate selection and status; clarify individual items; verify aggregate allocation for repeated part requests.

Failure criteria: One unresolved item blocks final all-items approval; allocations across duplicate parts cannot double count available stock; zero quantity rejected.

API operations: f17_correct_items

Files:

- `apps/api/partsiq/features/f17_multi_part_enquiries.py`
- `apps/web/src/features/f17_multi_part_enquiries/Panel.tsx`
- `tests/features/test_f17.py`
- `apps/web/tests/features/f17.test.tsx`
- `apps/api/partsiq/api/f17_multi_part_enquiries.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

081 F18 progress timeline

prompts/F18-progress-timeline.md

F18 Progress timeline

GOAL

Implement the complete progress timeline behavior in the single-workflow Snowflake edition. Show persisted stages, wait reason and last update; poll with bounded backoff; reconnect using server state; separate waiting for customer from waiting for staff.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F17. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f18_progress_timeline.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f18_progress_timeline/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f18.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f18.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f18_progress_timeline.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f18`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f18_read_progress(actor: ActorContext, enquiry_id: UUID) -> ProgressView
class ProgressView(Contract):
    enquiry_id:UUID
    revision:int
    step:WorkflowStep
    wait_reason:str | None
    updated_at:datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F18; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Show persisted stages, wait reason and last update.

4. Poll with bounded backoff.

5. Reconnect using server state.

6. Separate waiting for customer from waiting for staff.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/progress` → `f18_read_progress`; request `None` (no body); response `ProgressView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No fake completed progress. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: failed poll retains last state and offers retry. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: hidden tabs reduce polling. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but

stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f18.py`. Frontend path: `apps/web/tests/features/f18.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No fake completed progress. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: failed poll retains last state and offers retry. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: hidden tabs reduce polling. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F18` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f18.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f18.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F18/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F18-progress-timeline.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f18): implement progress timeline`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F18 Progress timeline

Status: specified, not implemented.

Owning prompt: `prompts/F18-progress-timeline.md`.

Behavior: Show persisted stages, wait reason and last update; poll with bounded backoff; reconnect using server state; separate waiting for customer from waiting for staff.

Failure criteria: No fake completed progress; failed poll retains last state and offers retry; hidden tabs reduce polling.

API operations: f18_read_progress

Files:

- `apps/api/partsiq/features/f18_progress_timeline.py`
- `apps/web/src/features/f18_progress_timeline/Panel.tsx`
- `tests/features/test_f18.py`
- `apps/web/tests/features/f18.test.tsx`
- `apps/api/partsiq/api/f18_progress_timeline.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

082 F19 enquiry history

prompts/F19-enquiry-history.md

F19 Enquiry history

GOAL

Implement the complete enquiry history behavior in the single-workflow Snowflake edition. Return cursor-paginated owned history with status and created time; filter active demo generation; link to full conversation; use stable created_at/id ordering.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F18. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f19_enquiry_history.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f19_enquiry_history/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f19.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f19.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f19_enquiry_history.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f19`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f19_list_enquiries(actor: ActorContext, command: HistoryQuery) -> EnquiryPage
class EnquiryPage(Contract):
    items:list[EnquiryView]
    next_cursor:str | None

class HistoryQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    status:str | None = None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F19; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Return cursor-paginated owned history with status and created time.

4. Filter active demo generation.

5. Link to full conversation.

6. Use stable created_at/id ordering.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries` → `f19_list_enquiries`; request `HistoryQuery` as validated query parameters; response `EnquiryPage`, HTTP 200; access `customer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Foreign owner ID ignored. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: invalid cursor returns 422. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: equal timestamps do not duplicate or skip records. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f19.py`. Frontend path: `apps/web/tests/features/f19.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Foreign owner ID ignored. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: invalid cursor returns 422. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: equal timestamps do not duplicate or skip records. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F19` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f19.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f19.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F19/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F19-enquiry-history.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f19): implement enquiry history`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F19 Enquiry history

Status: specified, not implemented.

Owning prompt: `prompts/F19-enquiry-history.md`.

Behavior: Return cursor-paginated owned history with status and created time; filter active demo generation; link to full conversation; use stable created_at/id ordering.

Failure criteria: Foreign owner ID ignored; invalid cursor returns 422; equal timestamps do not duplicate or skip records.

API operations: f19_list_enquiries

Files:

- `apps/api/partsiq/features/f19_enquiry_history.py`
- `apps/web/src/features/f19_enquiry_history/Panel.tsx`
- `tests/features/test_f19.py`
- `apps/web/tests/features/f19.test.tsx`
- `apps/api/partsiq/api/f19_enquiry_history.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

083 F20 machine identification

prompts/F20-machine-identification.md

F20 Machine identification

GOAL

Implement the complete machine identification behavior in the single-workflow Snowflake edition. Preserve identify_machine logic for model, serial and rating; return resolved configuration or explicit missing/ambiguous status; normalize numeric serial only after preserving input.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F19. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f20_machine_identification.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f20_machine_identification/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f20.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f20.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f20_machine_identification.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f20`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f20_identify_machine(actor: ActorContext, enquiry_id: UUID, item_id: UUID) -> MachineView
class MachineView(Contract):
    item_id:UUID
    revision:int
    state:Literal['resolved','needs_serial','ambiguous','mismatch','not_found']
    configuration_id:int | None
    model_text:str | None
    serial_text:str | None
    catalogue_version_id:UUID
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F20; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Preserve identify_machine logic for model, serial and rating.

4. Return resolved configuration or explicit missing/ambiguous status.

5. Normalize numeric serial only after preserving input.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/machine` → `f20_identify_machine`; request `None` (no body); response `MachineView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Boundary serials match exactly one configuration. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: overlapping ranges fail Snowflake publication validation and PostgreSQL serving-cache insertion. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: conflicting hints never silently override. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f20.py`. Frontend path: `apps/web/tests/features/f20.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Boundary serials match exactly one configuration. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: overlapping ranges fail Snowflake publication validation and PostgreSQL serving-cache insertion. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: conflicting hints never silently override. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F20` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f20.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f20.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F20/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F20-machine-identification.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f20): implement machine identification`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain

audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F20 Machine identification

Status: specified, not implemented.

Owning prompt: `prompts/F20-machine-identification.md`.

Behavior: Preserve identify_machine logic for model, serial and rating; return resolved configuration or explicit missing/ambiguous status; normalize numeric serial only after preserving input.

Failure criteria: Boundary serials match exactly one configuration; overlapping ranges fail Snowflake publication validation and PostgreSQL serving-cache insertion; conflicting hints never silently override.

API operations: f20_identify_machine

Files:

- `apps/api/partsiq/features/f20_machine_identification.py`
- `apps/web/src/features/f20_machine_identification/Panel.tsx`
- `tests/features/test_f20.py`
- `apps/web/tests/features/f20.test.tsx`
- `apps/api/partsiq/api/f20_machine_identification.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

084 F21 machine mismatch detection

prompts/F21-machine-mismatch-detection.md

F21 Machine mismatch detection

GOAL

Implement the complete machine mismatch detection behavior in the single-workflow Snowflake edition. Compare model and serial together; return mismatch reason with safe next question; prevent retrieval while identity unresolved.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F20. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f21_machine_mismatch_detection.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f21_machine_mismatch_detection/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f21.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f21.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/api/partsiq/api/f21_machine_mismatch_detection.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f21`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f21_check_hints(actor: ActorContext, enquiry_id: UUID, item_id: UUID) -> MismatchView
class MismatchView(Contract):
    item_id:UUID
    revision:int
    mismatch:bool
    reason_code:str | None
    question_en:str | None
    question_ar:str | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F21; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Compare model and serial together.

4. Return mismatch reason with safe next question.

5. Prevent retrieval while identity unresolved.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/mismatch` → `f21_check_hints`; request `None` (no body); response `MismatchView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Valid serial of a different model does not match requested model. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: nonnumeric serial handled without crashing. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f21.py`. Frontend path: `apps/web/tests/features/f21.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Valid serial of a different model does not match requested model. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: nonnumeric serial handled without crashing. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F21` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f21.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f21.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F21/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F21-machine-mismatch-detection.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f21): implement machine mismatch detection`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case

above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F21 Machine mismatch detection

Status: specified, not implemented.

Owning prompt: `prompts/F21-machine-mismatch-detection.md`.

Behavior: Compare model and serial together; return mismatch reason with safe next question; prevent retrieval while identity unresolved.

Failure criteria: Valid serial of a different model does not match requested model; nonnumeric serial handled without crashing.

API operations: f21_check_hints

Files:

- `apps/api/partsiq/features/f21_machine_mismatch_detection.py`
- `apps/web/src/features/f21_machine_mismatch_detection/Panel.tsx`
- `tests/features/test_f21.py`
- `apps/web/tests/features/f21.test.tsx`
- `apps/api/partsiq/api/f21_machine_mismatch_detection.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

085 F22 compatibility gate

prompts/F22-compatibility-gate.md

F22 Compatibility gate

GOAL

Implement the complete compatibility gate behavior in the single-workflow Snowflake edition. Fetch authoritative fitment set first; rank only members; carry source_reference and catalogue version through every candidate.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F21. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f22_compatibility_gate.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f22_compatibility_gate/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f22.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f22.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f22_compatibility_gate.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f22`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f22_compatible_set(actor: ActorContext, enquiry_id: UUID, item_id: UUID) -> CompatibilityView
class CompatibilityView(Contract):
    item_id:UUID
    revision:int
    configuration_id:int
    part_ids:list[int]
    catalogue_version_id:UUID
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F22; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Fetch authoritative fitment set first.

4. Rank only members.

5. Carry source_reference and catalogue version through every candidate.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/compatibility` → `f22_compatible_set`; request `None` (no body); response `CompatibilityView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: A perfect lexical match outside fitment is absent. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: empty fitment returns unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: blank provenance cannot be published. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f22.py`. Frontend path: `apps/web/tests/features/f22.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: A perfect lexical match outside fitment is absent. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: empty fitment returns unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: blank provenance cannot be published. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F22` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f22.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f22.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F22/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F22-compatibility-gate.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f22): implement compatibility gate`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F22 Compatibility gate

Status: specified, not implemented.

Owning prompt: `prompts/F22-compatibility-gate.md`.

Behavior: Fetch authoritative fitment set first; rank only members; carry source_reference and catalogue version through every candidate.

Failure criteria: A perfect lexical match outside fitment is absent; empty fitment returns unavailable; blank provenance cannot be published.

API operations: f22_compatible_set

Files:

- `apps/api/partsiq/features/f22_compatibility_gate.py`
- `apps/web/src/features/f22_compatibility_gate/Panel.tsx`
- `tests/features/test_f22.py`
- `apps/web/tests/features/f22.test.tsx`
- `apps/api/partsiq/api/f22_compatibility_gate.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

086 F23 aliases and typo matching

prompts/F23-aliases-and-typo-matching.md

F23 Aliases and typo matching

GOAL

Implement the complete aliases and typo matching behavior in the single-workflow Snowflake edition. Preserve Arabic normalization, part-number keys, phrase scoring and fuzzy token matching; remove machine terms using resolved model context.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F22. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f23_aliases_and_typo_matching.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f23_aliases_and_typo_matching/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f23.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f23.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f23_aliases_and_typo_matching.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f23`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f23_lexical_evidence(actor: ActorContext, enquiry_id: UUID, item_id: UUID) ->
MatchEvidenceView
class MatchEvidenceView(Contract):
    item_id:UUID
    revision:int
    normalized_query:str
    matched_aliases:list[str]
    removed_machine_terms:list[str]
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F23; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Preserve Arabic normalization, part-number keys, phrase scoring and fuzzy token matching.

4. Remove machine terms using resolved model context.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/match-evidence` → `f23_lexical_evidence`; request `None` (no body); response `MatchEvidenceView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Arabic part words are not removed as machine words. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: exact part number outranks unrelated fuzzy alias. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: normalization parity test matches SQL. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but

stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f23.py`. Frontend path: `apps/web/tests/features/f23.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Arabic part words are not removed as machine words. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: exact part number outranks unrelated fuzzy alias. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: normalization parity test matches SQL. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F23` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f23.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f23.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F23/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F23-aliases-and-typo-matching.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f23): implement aliases and typo matching`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F23 Aliases and typo matching

Status: specified, not implemented.

Owning prompt: `prompts/F23-aliases-and-typo-matching.md`.

Behavior: Preserve Arabic normalization, part-number keys, phrase scoring and fuzzy token matching; remove machine terms using resolved model context.

Failure criteria: Arabic part words are not removed as machine words; exact part number outranks unrelated fuzzy alias; normalization parity test matches SQL.

API operations: f23_lexical_evidence

Files:

- `apps/api/partsiq/features/f23_aliases_and_typo_matching.py`
- `apps/web/src/features/f23_aliases_and_typo_matching/Panel.tsx`
- `tests/features/test_f23.py`
- `apps/web/tests/features/f23.test.tsx`
- `apps/api/partsiq/api/f23_aliases_and_typo_matching.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

087 F24 semantic retrieval

prompts/F24-semantic-retrieval.md

F24 Semantic retrieval

GOAL

Implement the complete semantic retrieval behavior in the single-workflow Snowflake edition. Load one BGE-M3 encoder in the worker search module; use normalized 1024-d vectors; retain legacy weighted fusion; fall back to lexical on unavailable embedding path.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F23. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f24_semantic_retrieval.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f24_semantic_retrieval/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f24.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f24.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f24_semantic_retrieval.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f24`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f24_capabilities(actor: ActorContext) -> SearchCapabilities
class SearchCapabilities(Contract):
    lexical:bool
    semantic:bool
    embedding_model:str | None
    fallback_reason:str | None
    catalogue_source:Literal['snowflake','fixture']
    catalogue_version_id:UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F24; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Load one BGE-M3 encoder in the worker search module.

4. Use normalized 1024-d vectors.

5. Retain legacy weighted fusion.

6. Fall back to lexical on unavailable embedding path.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/search/capabilities` → `f24_capabilities`; request `None` (no body); response `SearchCapabilities`, HTTP 200; access `customer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Wrong dimension rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: no model loaded in API process. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: no semantic candidate outside compatible IDs. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f24.py`. Frontend path: `apps/web/tests/features/f24.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Wrong dimension rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: no model loaded in API process. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: no semantic candidate outside compatible IDs. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F24` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f24.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f24.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F24/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F24-semantic-retrieval.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f24): implement semantic retrieval`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F24 Semantic retrieval

Status: specified, not implemented.

Owning prompt: `prompts/F24-semantic-retrieval.md`.

Behavior: Load one BGE-M3 encoder in the worker search module; use normalized 1024-d vectors; retain legacy weighted fusion; fall back to lexical on unavailable embedding path.

Failure criteria: Wrong dimension rejected; no model loaded in API process; no semantic candidate outside compatible IDs.

API operations: f24_capabilities

Files:

- `apps/api/partsiq/features/f24_semantic_retrieval.py`
- `apps/web/src/features/f24_semantic_retrieval/Panel.tsx`
- `tests/features/test_f24.py`
- `apps/web/tests/features/f24.test.tsx`
- `apps/api/partsiq/api/f24_semantic_retrieval.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

088 F25 ranked candidates

prompts/F25-ranked-candidates.md

F25 Ranked candidates

GOAL

Implement the complete ranked candidates behavior in the single-workflow Snowflake edition. Return top candidates with lexical/semantic evidence and deterministic tie order; compute runner-up margin before applying response limit.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F24. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f25_ranked_candidates.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f25_ranked_candidates/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f25.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f25.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f25_ranked_candidates.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f25`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f25_rank_candidates(actor: ActorContext, enquiry_id: UUID, item_id: UUID) -> CandidateList
class CandidateList(Contract):
    item_id:UUID
    revision:int
    items:list[CandidateView]
    ambiguous:bool
    margin:float | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F25; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Return top candidates with lexical/semantic evidence and deterministic tie order.

4. Compute runner-up margin before applying response limit.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/candidates` → `f25_rank_candidates`; request `None` (no body); response `CandidateList`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Limit one still reports ambiguity. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: empty set returns no candidate. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: NaN scores rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f25.py`. Frontend path: `apps/web/tests/features/f25.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Limit one still reports ambiguity. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: empty set returns no candidate. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: NaN scores rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F25` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f25.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f25.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F25/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F25-ranked-candidates.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f25): implement ranked candidates`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F25 Ranked candidates

Status: specified, not implemented.

Owning prompt: `prompts/F25-ranked-candidates.md`.

Behavior: Return top candidates with lexical/semantic evidence and deterministic tie order; compute runner-up margin before applying response limit.

Failure criteria: Limit one still reports ambiguity; empty set returns no candidate; NaN scores rejected.

API operations: f25_rank_candidates

Files:

- `apps/api/partsiq/features/f25_ranked_candidates.py`
- `apps/web/src/features/f25_ranked_candidates/Panel.tsx`
- `tests/features/test_f25.py`
- `apps/web/tests/features/f25.test.tsx`
- `apps/api/partsiq/api/f25_ranked_candidates.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

089 F26 ambiguity handling

prompts/F26-ambiguity-handling.md

F26 Ambiguity handling

GOAL

Implement the complete ambiguity handling behavior in the single-workflow Snowflake edition. Use tested score and margin thresholds in code; ask customer to distinguish fuel filter versus separator; allow staff to select only verified candidates.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F25. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f26_ambiguity_handling.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f26_ambiguity_handling/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f26.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f26.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f26_ambiguity_handling.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f26`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f26_choose_candidate(actor: ActorContext, enquiry_id: UUID, item_id: UUID, command:
CandidateChoice, idempotency_key: UUID) -> EnquiryView
class CandidateChoice(Contract):
    expected_revision:int = Field(ge=1)
    part_id:int = Field(gt=0)

class EnquiryView(Contract):
    id: UUID
    revision: int
```

```
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
    catalogue_version_id: UUID | None
    updated_at: datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F26; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Use tested score and margin thresholds in code.

4. Ask customer to distinguish fuel filter versus separator.

5. Allow staff to select only verified candidates.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/items/{item_id}/choice` → `f26_choose_candidate`; request `CandidateChoice` as JSON; response `EnquiryView`, HTTP 202; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: LLM cannot override low confidence. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unavailable candidate choice returns 409. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: thresholds evaluated on labelled cases. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f26.py`. Frontend path: `apps/web/tests/features/f26.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: LLM cannot override low confidence. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unavailable candidate choice returns 409. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: thresholds evaluated on labelled cases. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F26` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f26.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f26.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F26/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F26-ambiguity-handling.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f26): implement ambiguity handling`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval

snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F26 Ambiguity handling

Status: specified, not implemented.

Owning prompt: `prompts/F26-ambiguity-handling.md`.

Behavior: Use tested score and margin thresholds in code; ask customer to distinguish fuel filter versus separator; allow staff to select only verified candidates.

Failure criteria: LLM cannot override low confidence; unavailable candidate choice returns 409; thresholds evaluated on labelled cases.

API operations: f26_choose_candidate

Files:

- `apps/api/partsiq/features/f26_ambiguity_handling.py`
- `apps/web/src/features/f26_ambiguity_handling/Panel.tsx`
- `tests/features/test_f26.py`
- `apps/web/tests/features/f26.test.tsx`
- `apps/api/partsiq/api/f26_ambiguity_handling.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

090 F27 match explanations

prompts/F27-match-explanations.md

F27 Match explanations

GOAL

Implement the complete match explanations behavior in the single-workflow Snowflake edition. Show actual alias, machine configuration, supersession chain and fitment reference; derive explanations from search evidence rather than invented model prose.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F26. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f27_match_explanations.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f27_match_explanations/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f27.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f27.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f27_match_explanations.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f27`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f27_explain_selection(actor: ActorContext, enquiry_id: UUID, item_id: UUID) ->
ExplanationView
class ExplanationView(Contract):
    item_id:UUID
    revision:int
    reasons:list[str]
    source_reference:str | None
    catalogue_version_id:UUID
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F27; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Show actual alias, machine configuration, supersession chain and fitment reference.

4. Derive explanations from search evidence rather than invented model prose.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/explanation` → `f27_explain_selection`; request `None` (no body); response `ExplanationView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Absent evidence shown as unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: user HTML is escaped. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: scores never labelled calibrated probability. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f27.py`. Frontend path: `apps/web/tests/features/f27.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Absent evidence shown as unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: user HTML is escaped. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: scores never labelled calibrated probability. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F27` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f27.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f27.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F27/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F27-match-explanations.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f27): implement match explanations`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F27 Match explanations

Status: specified, not implemented.

Owning prompt: `prompts/F27-match-explanations.md`.

Behavior: Show actual alias, machine configuration, supersession chain and fitment reference; derive explanations from search evidence rather than invented model prose.

Failure criteria: Absent evidence shown as unavailable; user HTML is escaped; scores never labelled calibrated probability.

API operations: f27_explain_selection

Files:

- `apps/api/partsiq/features/f27_match_explanations.py`
- `apps/web/src/features/f27_match_explanations/Panel.tsx`
- `tests/features/test_f27.py`
- `apps/web/tests/features/f27.test.tsx`
- `apps/api/partsiq/api/f27_match_explanations.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

091 F28 compatibility references

prompts/F28-compatibility-references.md

F28 Compatibility references

GOAL

Implement the complete compatibility references behavior in the single-workflow Snowflake edition. Resolve source_reference to version-specific document and page; retain citation with every candidate and approval snapshot.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F27. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f28_compatibility_references.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f28_compatibility_references/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f28.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f28.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f28_compatibility_references.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f28`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f28_list_references(actor: ActorContext, enquiry_id: UUID, item_id: UUID) -> ReferenceList
class ReferenceList(Contract):
    items:list[ReferenceView]
    revision:int
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal

feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F28; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Resolve source_reference to version-specific document and page.

4. Retain citation with every candidate and approval snapshot.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/references` → `f28_list_references`; request `None` (no body); response `ReferenceList`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Missing source file blocks evidence viewer and flags review. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: mismatched version fails verification. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f28.py`. Frontend path: `apps/web/tests/features/f28.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Missing source file blocks evidence viewer and flags review. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: mismatched version fails verification. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F28` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f28.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f28.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F28/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F28-compatibility-references.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f28): implement compatibility references`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F28 Compatibility references

Status: specified, not implemented.

Owning prompt: `prompts/F28-compatibility-references.md`.

Behavior: Resolve source_reference to version-specific document and page; retain citation with every candidate and approval snapshot.

Failure criteria: Missing source file blocks evidence viewer and flags review; mismatched version fails verification.

API operations: f28_list_references

Files:

- `apps/api/partsiq/features/f28_compatibility_references.py`
- `apps/web/src/features/f28_compatibility_references/Panel.tsx`
- `tests/features/test_f28.py`
- `apps/web/tests/features/f28.test.tsx`
- `apps/api/partsiq/api/f28_compatibility_references.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

092 F29 evidence page viewer

prompts/F29-evidence-page-viewer.md

F29 Evidence page viewer

GOAL

Implement the complete evidence page viewer behavior in the single-workflow Snowflake edition. Authorize assigned reviewer or own demo reviewer; request short-lived backend-signed PDF URL; open exact cited page; label synthetic document.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F28. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f29_evidence_page_viewer.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f29_evidence_page_viewer/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f29.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f29.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/api/partsiq/api/f29_evidence_page_viewer.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f29`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f29_sign_evidence(actor: ActorContext, enquiry_id: UUID, evidence_id: UUID, command:
RevisionOnly, idempotency_key: UUID) -> DownloadTicket
class DownloadTicket(Contract):
    url: str
    expires_at: datetime

class RevisionOnly(Contract):
    expected_revision:int = Field(ge=1)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F29; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Authorize assigned reviewer or own demo reviewer.

4. Request short-lived backend-signed PDF URL.

5. Open exact cited page.

6. Label synthetic document.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/evidence/{evidence_id}/download` → `f29_sign_evidence`; request `RevisionOnly` as JSON; response `DownloadTicket`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Customer receives reference only. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: arbitrary Storage key rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: expired link can be renewed after authorization. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f29.py`. Frontend path: `apps/web/tests/features/f29.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Customer receives reference only. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: arbitrary Storage key rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: expired link can be renewed after authorization. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F29` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f29.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f29.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F29/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F29-evidence-page-viewer.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f29): implement evidence page viewer`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F29 Evidence page viewer

Status: specified, not implemented.

Owning prompt: `prompts/F29-evidence-page-viewer.md`.

Behavior: Authorize assigned reviewer or own demo reviewer; request short-lived backend-signed PDF URL; open exact cited page; label synthetic document.

Failure criteria: Customer receives reference only; arbitrary Storage key rejected; expired link can be renewed after authorization.

API operations: f29_sign_evidence

Files:

- `apps/api/partsiq/features/f29_evidence_page_viewer.py`
- `apps/web/src/features/f29_evidence_page_viewer/Panel.tsx`
- `tests/features/test_f29.py`
- `apps/web/tests/features/f29.test.tsx`
- `apps/api/partsiq/api/f29_evidence_page_viewer.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

093 F30 old part numbers

prompts/F30-old-part-numbers.md

F30 Old part numbers

GOAL

Implement the complete old part numbers behavior in the single-workflow Snowflake edition. Follow supersession chain with cycle protection; display old and current numbers; verify current successor has explicit fitment for resolved configuration.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F29. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f30_old_part_numbers.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f30_old_part_numbers/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f30.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f30.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f30_old_part_numbers.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f30`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f30_resolve_supersession(actor: ActorContext, part_number: str) -> SupersessionView
class SupersessionView(Contract):
    requested_number:str
    chain:list[str]
    terminal_number:str | None
    active:bool
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F30; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Follow supersession chain with cycle protection.

4. Display old and current numbers.

5. Verify current successor has explicit fitment for resolved configuration.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/catalogue/parts/{part_number}/supersession` → `f30_resolve_supersession`; request `None` (no body); response `SupersessionView`, HTTP 200; access `customer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Cycles rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: inactive terminal part returns unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: inherited compatibility without source is rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f30.py`. Frontend path: `apps/web/tests/features/f30.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Cycles rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: inactive terminal part returns unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: inherited compatibility without source is rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F30` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f30.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f30.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F30/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F30-old-part-numbers.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f30): implement old part numbers`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F30 Old part numbers

Status: specified, not implemented.

Owning prompt: `prompts/F30-old-part-numbers.md`.

Behavior: Follow supersession chain with cycle protection; display old and current numbers; verify current successor has explicit fitment for resolved configuration.

Failure criteria: Cycles rejected; inactive terminal part returns unavailable; inherited compatibility without source is rejected.

API operations: f30_resolve_supersession

Files:

- `apps/api/partsiq/features/f30_old_part_numbers.py`
- `apps/web/src/features/f30_old_part_numbers/Panel.tsx`
- `tests/features/test_f30.py`
- `apps/web/tests/features/f30.test.tsx`
- `apps/api/partsiq/api/f30_old_part_numbers.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

094 F31 candidate comparison

prompts/F31-candidate-comparison.md

F31 Candidate comparison

GOAL

Implement the complete candidate comparison behavior in the single-workflow Snowflake edition. Compare two or three candidates from the same item/revision; show number, names, evidence and warehouse availability; selecting a candidate sends its ID, not browser-authored facts.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F30. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f31_candidate_comparison.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f31_candidate_comparison/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f31.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f31.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f31_candidate_comparison.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f31`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f31_compare_candidates(actor: ActorContext, enquiry_id: UUID, item_id: UUID, command:
ComparisonQuery) -> CandidateList
class CandidateList(Contract):
    item_id:UUID
    revision:int
    items:list[CandidateView]
    ambiguous:bool
    margin:float | None

class ComparisonQuery(Contract):
```

```
    part_ids:list[int] = Field(min_length=2,max_length=3)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F31; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Compare two or three candidates from the same item/revision.

4. Show number, names, evidence and warehouse availability.

5. Selecting a candidate sends its ID, not browser-authored facts.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/comparison` → `f31_compare_candidates`; request `ComparisonQuery` as validated query parameters; response `CandidateList`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Foreign candidate or stale revision rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing attribute shown as unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: keyboard navigation works. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f31.py`. Frontend path: `apps/web/tests/features/f31.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Foreign candidate or stale revision rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing attribute shown as unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: keyboard navigation works. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F31` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f31.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f31.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F31/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F31-candidate-comparison.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f31): implement candidate comparison`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F31 Candidate comparison

Status: specified, not implemented.

Owning prompt: `prompts/F31-candidate-comparison.md`.

Behavior: Compare two or three candidates from the same item/revision; show number, names, evidence and warehouse availability; selecting a candidate sends its ID, not browser-authored facts.

Failure criteria: Foreign candidate or stale revision rejected; missing attribute shown as unavailable; keyboard navigation works.

API operations: f31_compare_candidates

Files:

- `apps/api/partsiq/features/f31_candidate_comparison.py`
- `apps/web/src/features/f31_candidate_comparison/Panel.tsx`
- `tests/features/test_f31.py`
- `apps/web/tests/features/f31.test.tsx`
- `apps/api/partsiq/api/f31_candidate_comparison.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

095 F32 warehouse stock

prompts/F32-warehouse-stock.md

F32 Warehouse stock

GOAL

Implement the complete warehouse stock behavior in the single-workflow Snowflake edition. Show available quantity as on_hand minus reserved; include timestamp and warehouse name; keep display and planner using same snapshot.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F31. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f32_warehouse_stock.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f32_warehouse_stock/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f32.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f32.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f32_warehouse_stock.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f32`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f32_stock_for_selection(actor: ActorContext, enquiry_id: UUID, item_id: UUID) -> StockView
class StockView(Contract):
    item_id:UUID
    revision:int
    part_id:int
    items:list[StockRow]
    snapshot_id:UUID | None
    stale:bool
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F32; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Show available quantity as on_hand minus reserved.

4. Include timestamp and warehouse name.

5. Keep display and planner using same snapshot.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/items/{item_id}/stock` → `f32_stock_for_selection`; request `None` (no body); response `StockView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Negative stock blocked. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stock unavailable never displayed as zero without explanation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unknown warehouse rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f32.py`. Frontend path: `apps/web/tests/features/f32.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Negative stock blocked. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stock unavailable never displayed as zero without explanation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unknown warehouse rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F32` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f32.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f32.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F32/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F32-warehouse-stock.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f32): implement warehouse stock`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval

snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F32 Warehouse stock

Status: specified, not implemented.

Owning prompt: `prompts/F32-warehouse-stock.md`.

Behavior: Show available quantity as on_hand minus reserved; include timestamp and warehouse name; keep display and planner using same snapshot.

Failure criteria: Negative stock blocked; stock unavailable never displayed as zero without explanation; unknown warehouse rejected.

API operations: f32_stock_for_selection

Files:

- `apps/api/partsiq/features/f32_warehouse_stock.py`
- `apps/web/src/features/f32_warehouse_stock/Panel.tsx`
- `tests/features/test_f32.py`
- `apps/web/tests/features/f32.test.tsx`
- `apps/api/partsiq/api/f32_warehouse_stock.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

096 F33 fulfilment planning

prompts/F33-fulfilment-planning.md

F33 Fulfilment planning

GOAL

Implement the complete fulfilment planning behavior in the single-workflow Snowflake edition. Allocate local stock first then reachable warehouses deterministically; compute shortage; use quantity units; do not reserve stock or create orders.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F32. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f33_fulfilment_planning.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f33_fulfilment_planning/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f33.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f33.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f33_fulfilment_planning.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f33`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f33_plan_enquiry(actor: ActorContext, enquiry_id: UUID) -> PlanView
class PlanView(Contract):
    enquiry_id:UUID
    revision:int
    items:list[VerifiedItem]
    stock_snapshot_id:UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F33; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Allocate local stock first then reachable warehouses deterministically.

4. Compute shortage.

5. Use quantity units.

6. Do not reserve stock or create orders.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/plan` → `f33_plan_enquiry`; request `None` (no body); response `PlanView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Total allocations never exceed request or stock. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: fractional quantities respect unit policy. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unavailable transfer route cannot be invented. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but

stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f33.py`. Frontend path: `apps/web/tests/features/f33.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Total allocations never exceed request or stock. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: fractional quantities respect unit policy. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unavailable transfer route cannot be invented. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F33` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f33.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f33.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F33/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F33-fulfilment-planning.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f33): implement fulfilment planning`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F33 Fulfilment planning

Status: specified, not implemented.

Owning prompt: `prompts/F33-fulfilment-planning.md`.

Behavior: Allocate local stock first then reachable warehouses deterministically; compute shortage; use quantity units; do not reserve stock or create orders.

Failure criteria: Total allocations never exceed request or stock; fractional quantities respect unit policy; unavailable transfer route cannot be invented.

API operations: f33_plan_enquiry

Files:

- `apps/api/partsiq/features/f33_fulfilment_planning.py`
- `apps/web/src/features/f33_fulfilment_planning/Panel.tsx`
- `tests/features/test_f33.py`
- `apps/web/tests/features/f33.test.tsx`
- `apps/api/partsiq/api/f33_fulfilment_planning.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

097 F34 transfer estimates

prompts/F34-transfer-estimates.md

F34 Transfer estimates

GOAL

Implement the complete transfer estimates behavior in the single-workflow Snowflake edition. Carry integer transfer_days from approved route; show estimate and source warehouse; local availability means zero transfer days.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F33. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f34_transfer_estimates.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f34_transfer_estimates/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f34.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f34.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f34_transfer_estimates.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f34`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f34_transfer_evidence(actor: ActorContext, enquiry_id: UUID) -> TransferView
class TransferView(Contract):
    revision:int
    routes:list[TransferRow]
    estimate_only:Literal[True] = True
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F34; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Carry integer transfer_days from approved route.

4. Show estimate and source warehouse.

5. Local availability means zero transfer days.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/transfers` → `f34_transfer_evidence`; request `None` (no body); response `TransferView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No route gives unknown/unavailable plan rather than invented one-day promise. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: estimate labelled non-guaranteed. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f34.py`. Frontend path: `apps/web/tests/features/f34.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No route gives unknown/unavailable plan rather than invented one-day promise. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: estimate labelled non-guaranteed. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F34` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f34.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f34.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F34/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F34-transfer-estimates.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f34): implement transfer estimates`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F34 Transfer estimates

Status: specified, not implemented.

Owning prompt: `prompts/F34-transfer-estimates.md`.

Behavior: Carry integer transfer_days from approved route; show estimate and source warehouse; local availability means zero transfer days.

Failure criteria: No route gives unknown/unavailable plan rather than invented one-day promise; estimate labelled non-guaranteed.

API operations: f34_transfer_evidence

Files:

- `apps/api/partsiq/features/f34_transfer_estimates.py`
- `apps/web/src/features/f34_transfer_estimates/Panel.tsx`
- `tests/features/test_f34.py`
- `apps/web/tests/features/f34.test.tsx`
- `apps/api/partsiq/api/f34_transfer_estimates.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

098 F35 stock freshness

prompts/F35-stock-freshness.md

F35 Stock freshness

GOAL

Implement the complete stock freshness behavior in the single-workflow Snowflake edition. Compare stock timestamp with configurable freshness limit; mark stale data in review; re-read current stock during approval; invalidate plan when allocation no longer possible.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F34. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `search` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f35_stock_freshness.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f35_stock_freshness/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f35.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f35.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f35_stock_freshness.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f35`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f35_check_freshness(actor: ActorContext, enquiry_id: UUID) -> FreshnessView
class FreshnessView(Contract):
    revision:int
    fresh:bool
    observed_at:datetime | None
    max_age_seconds:int
    reason_code:str | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F35; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Compare stock timestamp with configurable freshness limit.

4. Mark stale data in review.

5. Re-read current stock during approval.

6. Invalidate plan when allocation no longer possible.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/stock-freshness` → `f35_check_freshness`; request `None` (no body); response `FreshnessView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Future timestamp flagged. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: changed stock returns 409 and reruns planning. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: stale stock never becomes a delivery guarantee. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f35.py`. Frontend path: `apps/web/tests/features/f35.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Future timestamp flagged. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: changed stock returns 409 and reruns planning. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: stale stock never becomes a delivery guarantee. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F35` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f35.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f35.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F35/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F35-stock-freshness.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f35): implement stock freshness`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F35 Stock freshness

Status: specified, not implemented.

Owning prompt: `prompts/F35-stock-freshness.md`.

Behavior: Compare stock timestamp with configurable freshness limit; mark stale data in review; re-read current stock during approval; invalidate plan when allocation no longer possible.

Failure criteria: Future timestamp flagged; changed stock returns 409 and reruns planning; stale stock never becomes a delivery guarantee.

API operations: f35_check_freshness

Files:

- `apps/api/partsiq/features/f35_stock_freshness.py`
- `apps/web/src/features/f35_stock_freshness/Panel.tsx`
- `tests/features/test_f35.py`
- `apps/web/tests/features/f35.test.tsx`
- `apps/api/partsiq/api/f35_stock_freshness.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

099 F36 staff inbox

prompts/F36-staff-inbox.md

F36 Staff inbox

GOAL

Implement the complete staff inbox behavior in the single-workflow Snowflake edition. List assigned operational enquiries and own active demo enquiries; filter status, priority, overdue and text; paginate stably; show counts in same access scope.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F35. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f36_staff_inbox.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f36_staff_inbox/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f36.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f36.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f36_staff_inbox.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f36`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f36_list_assigned(actor: ActorContext, command: InboxQuery) -> EnquiryPage
class EnquiryPage(Contract):
    items:list[EnquiryView]
    next_cursor:str | None

class InboxQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    status:str | None = None
    priority:Literal['normal','urgent'] | None = None
```

```
    overdue:bool = False
    query:str | None = Field(default=None,max_length=200)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F36; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. List assigned operational enquiries and own active demo enquiries.

4. Filter status, priority, overdue and text.

5. Paginate stably.

6. Show counts in same access scope.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries` → `f36_list_assigned`; request `InboxQuery` as validated query parameters; response `EnquiryPage`, HTTP 200; access `reviewer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unassigned real enquiries hidden from reviewer. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: admin can see all. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: filters do not leak global counts. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f36.py`. Frontend path: `apps/web/tests/features/f36.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unassigned real enquiries hidden from reviewer. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: admin can see all. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: filters do not leak global counts. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F36` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f36.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f36.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F36/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F36-staff-inbox.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f36): implement staff inbox`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval

snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F36 Staff inbox

Status: specified, not implemented.

Owning prompt: `prompts/F36-staff-inbox.md`.

Behavior: List assigned operational enquiries and own active demo enquiries; filter status, priority, overdue and text; paginate stably; show counts in same access scope.

Failure criteria: Unassigned real enquiries hidden from reviewer; admin can see all; filters do not leak global counts.

API operations: f36_list_assigned

Files:

- `apps/api/partsiq/features/f36_staff_inbox.py`
- `apps/web/src/features/f36_staff_inbox/Panel.tsx`
- `tests/features/test_f36.py`
- `apps/web/tests/features/f36.test.tsx`
- `apps/api/partsiq/api/f36_staff_inbox.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

100 F37 review workspace

prompts/F37-review-workspace.md

F37 Review workspace

GOAL

Implement the complete review workspace behavior in the single-workflow Snowflake edition. Assemble current extraction, items, candidates, evidence, stock, draft and timeline; show verification failures; separate staff-only notes from customer messages.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F36. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f37_review_workspace.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f37_review_workspace/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f37.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f37.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/api/partsiq/api/f37_review_workspace.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f37`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f37_get_workspace(actor: ActorContext, enquiry_id: UUID) -> ReviewView
class ReviewView(Contract):
    enquiry: EnquiryView
    facts: VerifiedFacts | None
    draft: ReplySlots | None
    verification: VerificationResult | None
    assigned_to: UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F37; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Assemble current extraction, items, candidates, evidence, stock, draft and timeline.

4. Show verification failures.

5. Separate staff-only notes from customer messages.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}` → `f37_get_workspace`; request `None` (no body); response `ReviewView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Missing panels show explicit empty states. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stale revision disables approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unassigned reviewer receives 404. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f37.py`. Frontend path: `apps/web/tests/features/f37.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Missing panels show explicit empty states. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stale revision disables approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unassigned reviewer receives 404. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F37` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f37.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f37.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F37/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F37-review-workspace.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f37): implement review workspace`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F37 Review workspace

Status: specified, not implemented.

Owning prompt: `prompts/F37-review-workspace.md`.

Behavior: Assemble current extraction, items, candidates, evidence, stock, draft and timeline; show verification failures; separate staff-only notes from customer messages.

Failure criteria: Missing panels show explicit empty states; stale revision disables approval; unassigned reviewer receives 404.

API operations: f37_get_workspace

Files:

- `apps/api/partsiq/features/f37_review_workspace.py`
- `apps/web/src/features/f37_review_workspace/Panel.tsx`
- `tests/features/test_f37.py`
- `apps/web/tests/features/f37.test.tsx`
- `apps/api/partsiq/api/f37_review_workspace.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

101 F38 correct extracted details

prompts/F38-correct-extracted-details.md

F38 Correct extracted details

GOAL

Implement the complete correct extracted details behavior in the single-workflow Snowflake edition. Lock enquiry; compare expected revision; apply typed allowed corrections; increment revision; invalidate derived selections/draft/approval; enqueue recomputation; record before and after.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F37. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f38_correct_extracted_details.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f38_correct_extracted_details/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f38.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f38.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f38_correct_extracted_details.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f38`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f38_correct_extraction(actor: ActorContext, enquiry_id: UUID, command: RevisionRequest,
idempotency_key: UUID) -> EnquiryView
class EnquiryView(Contract):
    id: UUID
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
```

```
    catalogue_version_id: UUID | None
    updated_at: datetime

class RevisionRequest(Contract):
    enquiry_id: UUID
    expected_revision: int
    corrections: list[ItemCorrection] = Field(min_length=1,max_length=20)
    reason: str = Field(min_length=1,max_length=1000)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F38; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Lock enquiry.

4. Compare expected revision.

5. Apply typed allowed corrections.

6. Increment revision.

7. Invalidate derived selections/draft/approval.

8. Enqueue recomputation.

9. Record before and after.

10. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

11. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

12. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

13. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `PATCH /v1/staff/enquiries/{enquiry_id}/extraction` → `f38_correct_extraction`; request `RevisionRequest` as JSON; response `EnquiryView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not

proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unknown field rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: old revision returns 409. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: correction cannot directly set approved status or invent candidate facts. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f38.py`. Frontend path: `apps/web/tests/features/f38.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unknown field rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: old revision returns 409. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: correction cannot directly set approved status or invent candidate facts. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F38` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f38.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f38.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F38/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F38-correct-extracted-details.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with

next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f38): implement correct extracted details`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F38 Correct extracted details

Status: specified, not implemented.

Owning prompt: `prompts/F38-correct-extracted-details.md`.

Behavior: Lock enquiry; compare expected revision; apply typed allowed corrections; increment revision; invalidate derived selections/draft/approval; enqueue recomputation; record before and after.

Failure criteria: Unknown field rejected; old revision returns 409; correction cannot directly set approved status or invent candidate facts.

API operations: f38_correct_extraction

Files:

- `apps/api/partsiq/features/f38_correct_extracted_details.py`
- `apps/web/src/features/f38_correct_extracted_details/Panel.tsx`
- `tests/features/test_f38.py`
- `apps/web/tests/features/f38.test.tsx`
- `apps/api/partsiq/api/f38_correct_extracted_details.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

102 F39 review actions

prompts/F39-review-actions.md

F39 Review actions

GOAL

Implement the complete review actions behavior in the single-workflow Snowflake edition. Approve only verified current draft; edit prose through controlled schema then reverify; reject with reason; record actor and idempotency key.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F38. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f39_review_actions.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f39_review_actions/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f39.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f39.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f39_review_actions.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f39`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f39_approve(actor: ActorContext, enquiry_id: UUID, command: ApprovalRequest, idempotency_key:
 UUID) -> ApprovalResult
async def f39_reject(actor: ActorContext, enquiry_id: UUID, command: RejectionRequest, idempotency_key:
 UUID) -> ReviewResult
class ApprovalRequest(Contract):
    enquiry_id: UUID
    expected_revision: int = Field(ge=1)
    draft_id: UUID
    edited_slots: ReplySlots | None = None
```

```
class ApprovalResult(Contract):
    decision_id: UUID
    message_id: UUID
    revision: int
    already_applied: bool

class RejectionRequest(Contract):
    enquiry_id: UUID
    expected_revision: int = Field(ge=1)
    reason: str = Field(min_length=1,max_length=1000)

class ReviewResult(Contract):
    decision_id: UUID
    revision: int
    status: Literal['approved','rejected']
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F39; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Approve only verified current draft.

4. Edit prose through controlled schema then reverify.

5. Reject with reason.

6. Record actor and idempotency key.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/staff/enquiries/{enquiry_id}/approve` → `f39_approve`; request `ApprovalRequest` as JSON; response `ApprovalResult`, HTTP 200; access `review`.

• `POST /v1/staff/enquiries/{enquiry_id}/reject` → `f39_reject`; request `RejectionRequest` as JSON; response `ReviewResult`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Double-click publishes once. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: concurrent edit defeats stale approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: failed verification blocks publication. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f39.py`. Frontend path: `apps/web/tests/features/f39.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Double-click publishes once. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: concurrent edit defeats stale approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: failed verification blocks publication. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F39` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f39.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f39.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F39/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F39-review-actions.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f39): implement review actions`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F39 Review actions

Status: specified, not implemented.

Owning prompt: `prompts/F39-review-actions.md`.

Behavior: Approve only verified current draft; edit prose through controlled schema then reverify; reject with reason; record actor and idempotency key.

Failure criteria: Double-click publishes once; concurrent edit defeats stale approval; failed verification blocks publication.

API operations: f39_approve, f39_reject

Files:

- `apps/api/partsiq/features/f39_review_actions.py`
- `apps/web/src/features/f39_review_actions/Panel.tsx`
- `tests/features/test_f39.py`
- `apps/web/tests/features/f39.test.tsx`
- `apps/api/partsiq/api/f39_review_actions.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

103 F40 approval invalidation

prompts/F40-approval-invalidation.md

F40 Approval invalidation

GOAL

Implement the complete approval invalidation behavior in the single-workflow Snowflake edition. Bind approval to revision and verified fact hash; any identity, quantity, selection or catalogue-sensitive change requires new draft and approval.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F39. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f40_approval_invalidation.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f40_approval_invalidation/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f40.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f40.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f40_approval_invalidation.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f40`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f40_read_binding(actor: ActorContext, enquiry_id: UUID) -> ApprovalState
class ApprovalState(Contract):
    revision:int
    approved:bool
    decision_id:UUID | None
    facts_sha256:str | None
    invalidated_reason:str | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F40; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Bind approval to revision and verified fact hash.

4. Any identity, quantity, selection or catalogue-sensitive change requires new draft and approval.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}/approval-state` → `f40_read_binding`; request `None` (no body); response `ApprovalState`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Old approval cannot authorize new revision. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: late workflow update cannot overwrite approved current state. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f40.py`. Frontend path: `apps/web/tests/features/f40.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Old approval cannot authorize new revision. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: late workflow update cannot overwrite approved current state. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F40` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f40.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f40.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F40/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F40-approval-invalidation.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f40): implement approval invalidation`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case

above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F40 Approval invalidation

Status: specified, not implemented.

Owning prompt: `prompts/F40-approval-invalidation.md`.

Behavior: Bind approval to revision and verified fact hash; any identity, quantity, selection or catalogue-sensitive change requires new draft and approval.

Failure criteria: Old approval cannot authorize new revision; late workflow update cannot overwrite approved current state.

API operations: f40_read_binding

Files:

- `apps/api/partsiq/features/f40_approval_invalidation.py`
- `apps/web/src/features/f40_approval_invalidation/Panel.tsx`
- `tests/features/test_f40.py`
- `apps/web/tests/features/f40.test.tsx`
- `apps/api/partsiq/api/f40_approval_invalidation.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

104 F41 internal notes

prompts/F41-internal-notes.md

F41 Internal notes

GOAL

Implement the complete internal notes behavior in the single-workflow Snowflake edition. Append reviewer notes with author and timestamp; show only in staff workspace; prohibit customer/PDF exposure; escape rendered content.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F40. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f41_internal_notes.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f41_internal_notes/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f41.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f41.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f41_internal_notes.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f41`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f41_append_note(actor: ActorContext, enquiry_id: UUID, command: NoteCreate, idempotency_key:
UUID) -> NoteView
async def f41_list_notes(actor: ActorContext, enquiry_id: UUID, command: PageQuery) -> NotePage
class NoteCreate(Contract):
    body:str = Field(min_length=1,max_length=4000)

class NotePage(Contract):
    items:list[NoteView]
    next_cursor:str | None
```

```
class NoteView(Contract):
    id:UUID
    author_id:UUID
    body:str
    created_at:datetime

class PageQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F41; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Append reviewer notes with author and timestamp.

4. Show only in staff workspace.

5. Prohibit customer/PDF exposure.

6. Escape rendered content.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/staff/enquiries/{enquiry_id}/notes` → `f41_append_note`; request `NoteCreate` as JSON; response `NoteView`, HTTP 201; access `review`.

• `GET /v1/staff/enquiries/{enquiry_id}/notes` → `f41_list_notes`; request `PageQuery` as validated query parameters; response `NotePage`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not

proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Empty or over-4000-character note rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: customer route never includes note text. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unassigned staff denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f41.py`. Frontend path: `apps/web/tests/features/f41.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Empty or over-4000-character note rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: customer route never includes note text. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unassigned staff denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F41` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f41.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f41.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F41/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F41-internal-notes.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together;

never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f41): implement internal notes`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F41 Internal notes

Status: specified, not implemented.

Owning prompt: `prompts/F41-internal-notes.md`.

Behavior: Append reviewer notes with author and timestamp; show only in staff workspace; prohibit customer/PDF exposure; escape rendered content.

Failure criteria: Empty or over-4000-character note rejected; customer route never includes note text; unassigned staff denied.

API operations: f41_append_note, f41_list_notes

Files:

- `apps/api/partsiq/features/f41_internal_notes.py`
- `apps/web/src/features/f41_internal_notes/Panel.tsx`
- `tests/features/test_f41.py`
- `apps/web/tests/features/f41.test.tsx`
- `apps/api/partsiq/api/f41_internal_notes.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

105 F42 assignments

prompts/F42-assignments.md

F42 Assignments

GOAL

Implement the complete assignments behavior in the single-workflow Snowflake edition. Admin assigns active reviewer or unassigns; persist assignment history atomically; access changes immediately; demo reviewer access remains session-scoped.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F41. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f42_assignments.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f42_assignments/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f42.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f42.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f42_assignments.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f42`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f42_assign(actor: ActorContext, enquiry_id: UUID, command: AssignRequest, idempotency_key:
UUID) -> AssignmentView
class AssignRequest(Contract):
    expected_revision:int = Field(ge=1)
    assignee_id:UUID | None

class AssignmentView(Contract):
    enquiry_id:UUID
    assigned_to:UUID | None
    changed_at:datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F42; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Admin assigns active reviewer or unassigns.

4. Persist assignment history atomically.

5. Access changes immediately.

6. Demo reviewer access remains session-scoped.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `PUT /v1/staff/enquiries/{enquiry_id}/assignment` → `f42_assign`; request `AssignRequest` as JSON; response `AssignmentView`, HTTP 200; access `admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Non-admin assignment denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: disabled/nonreviewer assignee rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: previous reviewer loses access on next request. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f42.py`. Frontend path: `apps/web/tests/features/f42.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Non-admin assignment denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: disabled/nonreviewer assignee rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: previous reviewer loses access on next request. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F42` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f42.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f42.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F42/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F42-assignments.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f42): implement assignments`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F42 Assignments

Status: specified, not implemented.

Owning prompt: `prompts/F42-assignments.md`.

Behavior: Admin assigns active reviewer or unassigns; persist assignment history atomically; access changes immediately; demo reviewer access remains session-scoped.

Failure criteria: Non-admin assignment denied; disabled/nonreviewer assignee rejected; previous reviewer loses access on next request.

API operations: f42_assign

Files:

- `apps/api/partsiq/features/f42_assignments.py`
- `apps/web/src/features/f42_assignments/Panel.tsx`
- `tests/features/test_f42.py`
- `apps/web/tests/features/f42.test.tsx`
- `apps/api/partsiq/api/f42_assignments.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

106 F43 overdue indicators

prompts/F43-overdue-indicators.md

F43 Overdue indicators

GOAL

Implement the complete overdue indicators behavior in the single-workflow Snowflake edition. Set due_at using configurable urgent/normal thresholds; display overdue only for unresolved states; sort overdue first when requested; store all times UTC.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F42. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f43_overdue_indicators.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f43_overdue_indicators/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f43.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f43.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f43_overdue_indicators.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f43`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f43_deadline(actor: ActorContext, enquiry_id: UUID) -> DeadlineView
class DeadlineView(Contract):
    due_at:datetime | None
    overdue:bool
    server_time:datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal

feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F43; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Set due_at using configurable urgent/normal thresholds.

4. Display overdue only for unresolved states.

5. Sort overdue first when requested.

6. Store all times UTC.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}/deadline` → `f43_deadline`; request `None` (no body); response `DeadlineView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Approved/rejected items never overdue. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: daylight-saving display does not change deadline. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: thresholds not described as contractual SLA. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only

denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f43.py`. Frontend path: `apps/web/tests/features/f43.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Approved/rejected items never overdue. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: daylight-saving display does not change deadline. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: thresholds not described as contractual SLA. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F43` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f43.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f43.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F43/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F43-overdue-indicators.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f43): implement overdue indicators`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F43 Overdue indicators

Status: specified, not implemented.

Owning prompt: `prompts/F43-overdue-indicators.md`.

Behavior: Set due_at using configurable urgent/normal thresholds; display overdue only for unresolved states; sort overdue first when requested; store all times UTC.

Failure criteria: Approved/rejected items never overdue; daylight-saving display does not change deadline; thresholds not described as contractual SLA.

API operations: f43_deadline

Files:

- `apps/api/partsiq/features/f43_overdue_indicators.py`
- `apps/web/src/features/f43_overdue_indicators/Panel.tsx`
- `tests/features/test_f43.py`
- `apps/web/tests/features/f43.test.tsx`
- `apps/api/partsiq/api/f43_overdue_indicators.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

107 F44 conversation summary

prompts/F44-conversation-summary.md

F44 Conversation summary

GOAL

Implement the complete conversation summary behavior in the single-workflow Snowflake edition. Produce staff-only concise summary from authorized messages and confirmed fields; store message IDs and revision; use shared budget wrapper; regenerate on relevant change.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F43. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f44_conversation_summary.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f44_conversation_summary/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f44.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f44.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f44_conversation_summary.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f44`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f44_enqueue_summary(actor: ActorContext, enquiry_id: UUID, command: RevisionOnly,
idempotency_key: UUID) -> JobView
async def f44_read_summary(actor: ActorContext, enquiry_id: UUID) -> SummaryView
class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
```

```
    resource_id:UUID | None

class RevisionOnly(Contract):
    expected_revision:int = Field(ge=1)

class SummaryView(Contract):
    revision:int
    summary:SummaryResult | None
    stale:bool
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F44; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Produce staff-only concise summary from authorized messages and confirmed fields.

4. Store message IDs and revision.

5. Use shared budget wrapper.

6. Regenerate on relevant change.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/staff/enquiries/{enquiry_id}/summary` → `f44_enqueue_summary`; request `RevisionOnly` as JSON; response `JobView`, HTTP 202; access `review`.

• `GET /v1/staff/enquiries/{enquiry_id}/summary` → `f44_read_summary`; request `None` (no body); response `SummaryView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not

proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Outdated summary labelled stale. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: instructions inside messages ignored. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: exhausted budget leaves full conversation available. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f44.py`. Frontend path: `apps/web/tests/features/f44.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Outdated summary labelled stale. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: instructions inside messages ignored. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: exhausted budget leaves full conversation available. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F44` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f44.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f44.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F44/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F44-conversation-summary.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update

docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f44): implement conversation summary`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F44 Conversation summary

Status: specified, not implemented.

Owning prompt: `prompts/F44-conversation-summary.md`.

Behavior: Produce staff-only concise summary from authorized messages and confirmed fields; store message IDs and revision; use shared budget wrapper; regenerate on relevant change.

Failure criteria: Outdated summary labelled stale; instructions inside messages ignored; exhausted budget leaves full conversation available.

API operations: f44_enqueue_summary, f44_read_summary

Files:

- `apps/api/partsiq/features/f44_conversation_summary.py`
- `apps/web/src/features/f44_conversation_summary/Panel.tsx`
- `tests/features/test_f44.py`
- `apps/web/tests/features/f44.test.tsx`
- `apps/api/partsiq/api/f44_conversation_summary.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

108 F45 reply drafting

prompts/F45-reply-drafting.md

F45 Reply drafting

GOAL

Implement the complete reply drafting behavior in the single-workflow Snowflake edition. Generate greeting, explanation and closing as structured slots; code inserts all numbers and allocations; choose Arabic or English from confirmed preference.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F44. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f45_reply_drafting.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f45_reply_drafting/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f45.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f45.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f45_reply_drafting.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f45`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f45_save_slots(actor: ActorContext, enquiry_id: UUID, command: DraftEdit, idempotency_key:
UUID) -> DraftView
class DraftEdit(Contract):
    expected_revision:int = Field(ge=1)
    slots:ReplySlots

class DraftView(Contract):
    revision:int
    slots:ReplySlots
```

```
    rendered_preview:str
    verification_passed:bool
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F45; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Generate greeting, explanation and closing as structured slots.

4. Code inserts all numbers and allocations.

5. Choose Arabic or English from confirmed preference.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `PUT /v1/staff/enquiries/{enquiry_id}/draft` → `f45_save_slots`; request `DraftEdit` as JSON; response `DraftView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unapproved critical facts never rendered. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unsupported slots rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: provider failure permits staff-authored controlled draft. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f45.py`. Frontend path: `apps/web/tests/features/f45.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unapproved critical facts never rendered. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unsupported slots rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: provider failure permits staff-authored controlled draft. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F45` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f45.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f45.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F45/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F45-reply-drafting.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f45): implement reply drafting`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used

by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F45 Reply drafting

Status: specified, not implemented.

Owning prompt: `prompts/F45-reply-drafting.md`.

Behavior: Generate greeting, explanation and closing as structured slots; code inserts all numbers and allocations; choose Arabic or English from confirmed preference.

Failure criteria: Unapproved critical facts never rendered; unsupported slots rejected; provider failure permits staff-authored controlled draft.

API operations: f45_save_slots

Files:

- `apps/api/partsiq/features/f45_reply_drafting.py`
- `apps/web/src/features/f45_reply_drafting/Panel.tsx`
- `tests/features/test_f45.py`
- `apps/web/tests/features/f45.test.tsx`
- `apps/api/partsiq/api/f45_reply_drafting.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

109 F46 verification gate

prompts/F46-verification-gate.md

F46 Verification gate

GOAL

Implement the complete verification gate behavior in the single-workflow Snowflake edition. Check item/configuration/candidate membership, provenance, catalogue version, selected quantities and allocation bounds; compute canonical fact hash; store named rule results.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F45. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f46_verification_gate.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f46_verification_gate/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f46.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f46.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f46_verification_gate.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f46`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f46_read_current(actor: ActorContext, enquiry_id: UUID) -> VerificationResult
class VerificationResult(Contract):
    passed: bool
    failures: list[VerificationFailure]
    facts_sha256: str
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F46; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Check item/configuration/candidate membership, provenance, catalogue version, selected quantities and allocation bounds.

4. Compute canonical fact hash.

5. Store named rule results.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}/verification` → `f46_read_current`; request `None` (no body); response `VerificationResult`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unknown part, incompatible part, stale catalogue, oversold plan and hidden extra item each fail a specific rule. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f46.py`. Frontend path: `apps/web/tests/features/f46.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unknown part, incompatible part, stale catalogue, oversold plan and hidden extra item each fail a specific rule. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F46` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f46.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f46.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F46/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F46-verification-gate.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f46): implement verification gate`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F46 Verification gate

Status: specified, not implemented.

Owning prompt: `prompts/F46-verification-gate.md`.

Behavior: Check item/configuration/candidate membership, provenance, catalogue version, selected quantities and allocation bounds; compute canonical fact hash; store named rule results.

Failure criteria: Unknown part, incompatible part, stale catalogue, oversold plan and hidden extra item each fail a specific rule.

API operations: f46_read_current

Files:

- `apps/api/partsiq/features/f46_verification_gate.py`
- `apps/web/src/features/f46_verification_gate/Panel.tsx`
- `tests/features/test_f46.py`
- `apps/web/tests/features/f46.test.tsx`
- `apps/api/partsiq/api/f46_verification_gate.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

110 F47 human publication gate

prompts/F47-human-publication-gate.md

F47 Human publication gate

GOAL

Implement the complete human publication gate behavior in the single-workflow Snowflake edition. Only approval service inserts recommendation messages; lock enquiry and recheck role, revision and verification; unique publication index prevents duplicates.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F46. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f47_human_publication_gate.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f47_human_publication_gate/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f47.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f47.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f47_human_publication_gate.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f47`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f47_read_approved(actor: ActorContext, enquiry_id: UUID) -> PublishedView
class PublishedView(Contract):
    message:MessageView | None
    decision_id:UUID | None
    approved_revision:int | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F47; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Only approval service inserts recommendation messages.

4. Lock enquiry and recheck role, revision and verification.

5. Unique publication index prevents duplicates.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/enquiries/{enquiry_id}/published-reply` → `f47_read_approved`; request `None` (no body); response `PublishedView`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No background job write tool can publish. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: database rejects recommendation lacking matching approved decision. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f47.py`. Frontend path: `apps/web/tests/features/f47.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No background job write tool can publish. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: database rejects recommendation lacking matching approved decision. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F47` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f47.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f47.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F47/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F47-human-publication-gate.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f47): implement human publication gate`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F47 Human publication gate

Status: specified, not implemented.

Owning prompt: `prompts/F47-human-publication-gate.md`.

Behavior: Only approval service inserts recommendation messages; lock enquiry and recheck role, revision and verification; unique publication index prevents duplicates.

Failure criteria: No background job write tool can publish; database rejects recommendation lacking matching approved decision.

API operations: f47_read_approved

Files:

- `apps/api/partsiq/features/f47_human_publication_gate.py`
- `apps/web/src/features/f47_human_publication_gate/Panel.tsx`
- `tests/features/test_f47.py`
- `apps/web/tests/features/f47.test.tsx`
- `apps/api/partsiq/api/f47_human_publication_gate.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

111 F48 input isolation

prompts/F48-input-isolation.md

F48 Input isolation

GOAL

Implement the complete input isolation behavior in the single-workflow Snowflake edition. Treat customer text, OCR and imported evidence as data; separate trusted instructions; bound inputs, transitions, steps and output size; never execute model-produced templates.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F47. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f48_input_isolation.py`: application facade connecting this feature to foundation domain functions.

• `tests/features/test_f48.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f48.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f48`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f48_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F48; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Treat customer text, OCR and imported evidence as data.

4. Separate trusted instructions.

5. Bound inputs, transitions, steps and output size.

6. Never execute model-produced templates.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Prompt injection cannot call privileged functions, change roles or reveal secrets. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: model result with rogue instructions remains inert. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f48.py`. Frontend path: `apps/web/tests/features/f48.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Prompt injection cannot call privileged functions, change roles or reveal secrets. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: model result with rogue instructions remains inert. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F48` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f48.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f48.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F48/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F48-input-isolation.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f48): implement input isolation`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F48 Input isolation

Status: specified, not implemented.

Owning prompt: `prompts/F48-input-isolation.md`.

Behavior: Treat customer text, OCR and imported evidence as data; separate trusted instructions; bound inputs, transitions, steps and output size; never execute model-produced templates.

Failure criteria: Prompt injection cannot call privileged functions, change roles or reveal secrets; model result with rogue instructions remains inert.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f48_input_isolation.py`
- `tests/features/test_f48.py`
- `apps/web/tests/features/f48.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

112 F49 audit trail

prompts/F49-audit-trail.md

F49 Audit trail

GOAL

Implement the complete audit trail behavior in the single-workflow Snowflake edition. Append actor, request, entity, revision and minimal before/after metadata for AI suggestions and human changes; redact secrets and full sensitive payloads.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F48. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `common` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f49_audit_trail.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f49_audit_trail/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f49.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f49.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f49_audit_trail.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f49`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f49_list_events(actor: ActorContext, enquiry_id: UUID, command: PageQuery) -> AuditPage
class AuditPage(Contract):
    items:list[AuditEvent]
    next_cursor:str | None

class PageQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F49; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Append actor, request, entity, revision and minimal before/after metadata for AI suggestions and human changes.

4. Redact secrets and full sensitive payloads.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}/audit` → `f49_list_events`; request `PageQuery` as validated query parameters; response `AuditPage`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Application UPDATE, DELETE, TRUNCATE fail. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: audit write failure rolls back approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: database-owner powers documented honestly. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but

stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f49.py`. Frontend path: `apps/web/tests/features/f49.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Application UPDATE, DELETE, TRUNCATE fail. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: audit write failure rolls back approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: database-owner powers documented honestly. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F49` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f49.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f49.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F49/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F49-audit-trail.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f49): implement audit trail`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F49 Audit trail

Status: specified, not implemented.

Owning prompt: `prompts/F49-audit-trail.md`.

Behavior: Append actor, request, entity, revision and minimal before/after metadata for AI suggestions and human changes; redact secrets and full sensitive payloads.

Failure criteria: Application UPDATE, DELETE, TRUNCATE fail; audit write failure rolls back approval; database-owner powers documented honestly.

API operations: f49_list_events

Files:

- `apps/api/partsiq/features/f49_audit_trail.py`
- `apps/web/src/features/f49_audit_trail/Panel.tsx`
- `tests/features/test_f49.py`
- `apps/web/tests/features/f49.test.tsx`
- `apps/api/partsiq/api/f49_audit_trail.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

113 F50 durable workflows

prompts/F50-durable-workflows.md

F50 Durable workflows

GOAL

Implement the complete durable workflows behavior in the single-workflow Snowflake edition. Persist workflow_runs and workflow_step_results in PostgreSQL; run a fixed Python transition table with durable jobs; serialize each enquiry and fence completion by revision and lease token; recover after restart without LangGraph.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F49. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `workflow` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f50_durable_workflows.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f50_durable_workflows/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f50.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f50.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f50_durable_workflows.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f50`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f50_inspect_run(actor: ActorContext, enquiry_id: UUID) -> WorkflowView
class WorkflowView(Contract):
    enquiry_id:UUID
    revision:int
    step:WorkflowStep
    completed_steps:list[str]
    pending_job_id:UUID | None
```

```
    error_code:str | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F50; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Persist workflow_runs and workflow_step_results in PostgreSQL.

4. Run a fixed Python transition table with durable jobs.

5. Serialize each enquiry and fence completion by revision and lease token.

6. Recover after restart without LangGraph.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}/workflow` → `f50_inspect_run`; request `None` (no body); response `WorkflowView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Duplicate jobs cannot repeat publication. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stale revisions are discarded. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: interrupted steps resume from committed results. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: no LLM chooses the next step. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f50.py`. Frontend path: `apps/web/tests/features/f50.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Duplicate jobs cannot repeat publication. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stale revisions are discarded. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: interrupted steps resume from committed results. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: no LLM chooses the next step. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F50` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f50.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f50.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F50/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F50-durable-workflows.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f50): implement durable workflows`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F50 Durable workflows

Status: specified, not implemented.

Owning prompt: `prompts/F50-durable-workflows.md`.

Behavior: Persist workflow_runs and workflow_step_results in PostgreSQL; run a fixed Python transition table with durable jobs; serialize each enquiry and fence completion by revision and lease token; recover after restart without LangGraph.

Failure criteria: Duplicate jobs cannot repeat publication; stale revisions are discarded; interrupted steps resume from committed results; no LLM chooses the next step.

API operations: f50_inspect_run

Files:

- `apps/api/partsiq/features/f50_durable_workflows.py`
- `apps/web/src/features/f50_durable_workflows/Panel.tsx`
- `tests/features/test_f50.py`
- `apps/web/tests/features/f50.test.tsx`
- `apps/api/partsiq/api/f50_durable_workflows.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

114 F51 background jobs

prompts/F51-background-jobs.md

F51 Background jobs

GOAL

Implement the complete background jobs behavior in the single-workflow Snowflake edition. Persist job before Redis dispatch; dispatcher claims by ID; sweeper redispatches pending/expired jobs; worker handles OCR, PDF, imports and cleanup.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F50. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `worker` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f51_background_jobs.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f51_background_jobs/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f51.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f51.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f51_background_jobs.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f51`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f51_get_status(actor: ActorContext, job_id: UUID) -> JobView
class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F51; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Persist job before Redis dispatch.

4. Dispatcher claims by ID.

5. Sweeper redispatches pending/expired jobs.

6. Worker handles OCR, PDF, imports and cleanup.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/jobs/{job_id}` → `f51_get_status`; request `None` (no body); response `JobView`, HTTP 200; access `scoped`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Redis loss does not lose DB job. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unavailable Redis rate limiter fails closed. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: heavy work never blocks HTTP intake. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f51.py`. Frontend path: `apps/web/tests/features/f51.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Redis loss does not lose DB job. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: unavailable Redis rate limiter fails closed. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: heavy work never blocks HTTP intake. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F51` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f51.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f51.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F51/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F51-background-jobs.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f51): implement background jobs`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F51 Background jobs

Status: specified, not implemented.

Owning prompt: `prompts/F51-background-jobs.md`.

Behavior: Persist job before Redis dispatch; dispatcher claims by ID; sweeper redispatches pending/expired jobs; worker handles OCR, PDF, imports and cleanup.

Failure criteria: Redis loss does not lose DB job; unavailable Redis rate limiter fails closed; heavy work never blocks HTTP intake.

API operations: f51_get_status

Files:

- `apps/api/partsiq/features/f51_background_jobs.py`
- `apps/web/src/features/f51_background_jobs/Panel.tsx`
- `tests/features/test_f51.py`
- `apps/web/tests/features/f51.test.tsx`
- `apps/api/partsiq/api/f51_background_jobs.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

115 F52 retry and deduplication

prompts/F52-retry-and-deduplication.md

F52 Retry and deduplication

GOAL

Implement the complete retry and deduplication behavior in the single-workflow Snowflake edition. Claim with FOR UPDATE SKIP LOCKED; assign lease token; heartbeat; retry transient errors with backoff up to max attempts; fence final writes by lease token.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F51. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `worker` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f52_retry_and_deduplication.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f52_retry_and_deduplication/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f52.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f52.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f52_retry_and_deduplication.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f52`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f52_retry_failed(actor: ActorContext, job_id: UUID, command: RetryRequest, idempotency_key:
UUID) -> JobView
class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None
```

```
class RetryRequest(Contract):
    expected_revision:int = Field(ge=1)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F52; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Claim with FOR UPDATE SKIP LOCKED.

4. Assign lease token.

5. Heartbeat.

6. Retry transient errors with backoff up to max attempts.

7. Fence final writes by lease token.

8. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

9. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

10. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

11. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/jobs/{job_id}/retry` → `f52_retry_failed`; request `RetryRequest` as JSON; response `JobView`, HTTP 202; access `scoped`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Expired worker cannot commit. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: permanent validation failure not retried. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: provider timeout marked uncertain and budget held conservatively. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f52.py`. Frontend path: `apps/web/tests/features/f52.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Expired worker cannot commit. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: permanent validation failure not retried. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: provider timeout marked uncertain and budget held conservatively. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F52` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f52.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f52.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F52/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F52-retry-and-deduplication.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f52): implement retry and deduplication`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F52 Retry and deduplication

Status: specified, not implemented.

Owning prompt: `prompts/F52-retry-and-deduplication.md`.

Behavior: Claim with FOR UPDATE SKIP LOCKED; assign lease token; heartbeat; retry transient errors with backoff up to max attempts; fence final writes by lease token.

Failure criteria: Expired worker cannot commit; permanent validation failure not retried; provider timeout marked uncertain and budget held conservatively.

API operations: f52_retry_failed

Files:

- `apps/api/partsiq/features/f52_retry_and_deduplication.py`
- `apps/web/src/features/f52_retry_and_deduplication/Panel.tsx`
- `tests/features/test_f52.py`
- `apps/web/tests/features/f52.test.tsx`
- `apps/api/partsiq/api/f52_retry_and_deduplication.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

116 F53 graceful failures

prompts/F53-graceful-failures.md

F53 Graceful failures

GOAL

Implement the complete graceful failures behavior in the single-workflow Snowflake edition. Map stable error codes to bilingual recovery guidance; distinguish validation, quota, service outage and staff-required states; preserve user input.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F52. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `web` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f53_graceful_failures.py`: application facade connecting this feature to foundation domain functions.

• `tests/features/test_f53.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f53.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f53`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f53_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F53; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Map stable error codes to bilingual recovery guidance.

4. Distinguish validation, quota, service outage and staff-required states.

5. Preserve user input.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No raw stack traces or keys. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: retry only safe idempotent action. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: search lexical fallback clearly reported. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f53.py`. Frontend path: `apps/web/tests/features/f53.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No raw stack traces or keys. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: retry only safe idempotent action. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: search lexical fallback clearly reported. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F53` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f53.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f53.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F53/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F53-graceful-failures.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f53): implement graceful failures`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F53 Graceful failures

Status: specified, not implemented.

Owning prompt: `prompts/F53-graceful-failures.md`.

Behavior: Map stable error codes to bilingual recovery guidance; distinguish validation, quota, service outage and staff-required states; preserve user input.

Failure criteria: No raw stack traces or keys; retry only safe idempotent action; search lexical fallback clearly reported.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f53_graceful_failures.py`
- `tests/features/test_f53.py`
- `apps/web/tests/features/f53.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

117 F54 access and permissions

prompts/F54-access-and-permissions.md

F54 Access and permissions

GOAL

Implement the complete access and permissions behavior in the single-workflow Snowflake edition. Verify Supabase issuer, audience, signature, expiry; derive identity server-side; enforce assigned-only review; keep credentials server-side; disable public real-client signup.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F53. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f54_access_and_permissions.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f54_access_and_permissions/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f54.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f54.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f54_access_and_permissions.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f54`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f54_identity(actor: ActorContext) -> IdentityView
class IdentityView(Contract):
    id:UUID
    roles:list[str]
    is_anonymous:bool
    demo_session_id:UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal

feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F54; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Verify Supabase issuer, audience, signature, expiry.

4. Derive identity server-side.

5. Enforce assigned-only review.

6. Keep credentials server-side.

7. Disable public real-client signup.

8. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

9. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

10. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

11. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/me` → `f54_identity`; request `None` (no body); response `IdentityView`, HTTP 200; access `customer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Forged role claim ignored. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing context denies rows. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: preview cannot reach prod database. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: demo cannot publish shared catalogue. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f54.py`. Frontend path: `apps/web/tests/features/f54.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Forged role claim ignored. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing context denies rows. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: preview cannot reach prod database. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: demo cannot publish shared catalogue. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F54` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f54.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f54.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F54/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F54-access-and-permissions.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f54): implement access and permissions`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain

audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F54 Access and permissions

Status: specified, not implemented.

Owning prompt: `prompts/F54-access-and-permissions.md`.

Behavior: Verify Supabase issuer, audience, signature, expiry; derive identity server-side; enforce assigned-only review; keep credentials server-side; disable public real-client signup.

Failure criteria: Forged role claim ignored; missing context denies rows; preview cannot reach prod database; demo cannot publish shared catalogue.

API operations: f54_identity

Files:

- `apps/api/partsiq/features/f54_access_and_permissions.py`
- `apps/web/src/features/f54_access_and_permissions/Panel.tsx`
- `tests/features/test_f54.py`
- `apps/web/tests/features/f54.test.tsx`
- `apps/api/partsiq/api/f54_access_and_permissions.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

118 F55 saved machines

prompts/F55-saved-machines.md

F55 Saved machines

GOAL

Implement the complete saved machines behavior in the single-workflow Snowflake edition. Save confirmed model and serial with user label; list own active entries; archive entry; prefill a new enquiry but re-identify against current catalogue.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F54. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f55_saved_machines.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f55_saved_machines/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f55.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f55.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/api/partsiq/api/f55_saved_machines.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f55`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f55_save(actor: ActorContext, command: MachineSave, idempotency_key: UUID) ->
SavedMachineView
async def f55_list_owned(actor: ActorContext, command: PageQuery) -> MachinePage
async def f55_archive(actor: ActorContext, machine_id: UUID) -> ArchiveResult
class ArchiveResult(Contract):
    id:UUID
    archived:Literal[True] = True

class MachinePage(Contract):
    items:list[SavedMachineView]
```

```
    next_cursor:str | None

class MachineSave(Contract):
    label:str = Field(min_length=1,max_length=120)
    model_text:str = Field(min_length=1,max_length=120)
    serial_text:str = Field(min_length=1,max_length=120)

class PageQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)

class SavedMachineView(Contract):
    id:UUID
    label:str
    model_text:str
    serial_text:str
    created_at:datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F55; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Save confirmed model and serial with user label.

4. List own active entries.

5. Archive entry.

6. Prefill a new enquiry but re-identify against current catalogue.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/machines` → `f55_save`; request `MachineSave` as JSON; response `SavedMachineView`, HTTP 201; access `customer`.

• `GET /v1/machines` → `f55_list_owned`; request `PageQuery` as validated query parameters; response `MachinePage`, HTTP 200; access `customer`.

• `DELETE /v1/machines/{machine_id}` → `f55_archive`; request `None` (no body); response `ArchiveResult`, HTTP 200; access `customer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202

means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Duplicate label permitted with distinct ID. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: foreign ID denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: stale configuration never reused as authoritative. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f55.py`. Frontend path: `apps/web/tests/features/f55.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Duplicate label permitted with distinct ID. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: foreign ID denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: stale configuration never reused as authoritative. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F55` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f55.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f55.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F55/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and

confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F55-saved-machines.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f55): implement saved machines`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F55 Saved machines

Status: specified, not implemented.

Owning prompt: `prompts/F55-saved-machines.md`.

Behavior: Save confirmed model and serial with user label; list own active entries; archive entry; prefill a new enquiry but re-identify against current catalogue.

Failure criteria: Duplicate label permitted with distinct ID; foreign ID denied; stale configuration never reused as authoritative.

API operations: f55_save, f55_list_owned, f55_archive

Files:

- `apps/api/partsiq/features/f55_saved_machines.py`
- `apps/web/src/features/f55_saved_machines/Panel.tsx`
- `tests/features/test_f55.py`
- `apps/web/tests/features/f55.test.tsx`
- `apps/api/partsiq/api/f55_saved_machines.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

119 F56 repeat enquiry

prompts/F56-repeat-enquiry.md

F56 Repeat enquiry

GOAL

Implement the complete repeat enquiry behavior in the single-workflow Snowflake edition. Copy customer request and confirmed hints into a new enquiry; issue new ID/revision; link origin in audit; run matching, stock and review again.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F55. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f56_repeat_enquiry.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f56_repeat_enquiry/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f56.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f56.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f56_repeat_enquiry.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f56`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f56_repeat_enquiry(actor: ActorContext, enquiry_id: UUID, command: RepeatRequest,
idempotency_key: UUID) -> EnquiryView
class EnquiryView(Contract):
    id: UUID
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
    catalogue_version_id: UUID | None
```

```
    updated_at: datetime

class RepeatRequest(Contract):
    locale:Locale
    destination_warehouse_id:int = Field(gt=0)
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F56; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Copy customer request and confirmed hints into a new enquiry.

4. Issue new ID/revision.

5. Link origin in audit.

6. Run matching, stock and review again.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/repeat` → `f56_repeat_enquiry`; request `RepeatRequest` as JSON; response `EnquiryView`, HTTP 202; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Never copy approval, allocation, draft or signed file URL. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: source ownership required. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: repeated submission deduplicated. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f56.py`. Frontend path: `apps/web/tests/features/f56.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Never copy approval, allocation, draft or signed file URL. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: source ownership required. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: repeated submission deduplicated. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F56` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f56.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f56.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F56/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F56-repeat-enquiry.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f56): implement repeat enquiry`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F56 Repeat enquiry

Status: specified, not implemented.

Owning prompt: `prompts/F56-repeat-enquiry.md`.

Behavior: Copy customer request and confirmed hints into a new enquiry; issue new ID/revision; link origin in audit; run matching, stock and review again.

Failure criteria: Never copy approval, allocation, draft or signed file URL; source ownership required; repeated submission deduplicated.

API operations: f56_repeat_enquiry

Files:

- `apps/api/partsiq/features/f56_repeat_enquiry.py`
- `apps/web/src/features/f56_repeat_enquiry/Panel.tsx`
- `tests/features/test_f56.py`
- `apps/web/tests/features/f56.test.tsx`
- `apps/api/partsiq/api/f56_repeat_enquiry.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

120 F57 catalogue browser

prompts/F57-catalogue-browser.md

F57 Catalogue browser

GOAL

Implement the complete catalogue browser behavior in the single-workflow Snowflake edition. Read published catalogue with model/category/text filters; page results; show part detail and supersession; staff can inspect fitment and sources.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F56. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

- `apps/api/partsiq/features/f57_catalogue_browser.py`: application facade connecting this feature to foundation domain functions.
- `apps/web/src/features/f57_catalogue_browser/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.
- `tests/features/test_f57.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/web/tests/features/f57.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.
- `apps/api/partsiq/api/f57_catalogue_browser.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f57`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f57_browse(actor: ActorContext, command: CatalogueQuery) -> CataloguePage
async def f57_get_part(actor: ActorContext, part_number: str) -> CataloguePart
class CataloguePage(Contract):
    items:list[CataloguePart]
    next_cursor:str | None

class CataloguePart(Contract):
    id:int
    part_number:str
    name_en:str
```

```
    name_ar:str
    category:str
    unit_of_measure:str
    is_active:bool
    catalogue_version_id:UUID

class CatalogueQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    query:str | None = Field(default=None,max_length=200)
    model_id:int | None = None
    category:str | None = None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F57; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Read published catalogue with model/category/text filters.

4. Page results.

5. Show part detail and supersession.

6. Staff can inspect fitment and sources.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/catalogue/parts` → `f57_browse`; request `CatalogueQuery` as validated query parameters; response `CataloguePage`, HTTP 200; access `customer`.

• `GET /v1/catalogue/parts/{part_number}` → `f57_get_part`; request `None` (no body); response `CataloguePart`, HTTP 200; access `customer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or

external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No unpublished import rows exposed. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing number returns 404. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: literal query cannot become SQL. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f57.py`. Frontend path: `apps/web/tests/features/f57.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No unpublished import rows exposed. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: missing number returns 404. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: literal query cannot become SQL. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F57` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f57.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f57.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F57/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F57-catalogue-browser.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript

together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f57): implement catalogue browser`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F57 Catalogue browser

Status: specified, not implemented.

Owning prompt: `prompts/F57-catalogue-browser.md`.

Behavior: Read published catalogue with model/category/text filters; page results; show part detail and supersession; staff can inspect fitment and sources.

Failure criteria: No unpublished import rows exposed; missing number returns 404; literal query cannot become SQL.

API operations: f57_browse, f57_get_part

Files:

- `apps/api/partsiq/features/f57_catalogue_browser.py`
- `apps/web/src/features/f57_catalogue_browser/Panel.tsx`
- `tests/features/test_f57.py`
- `apps/web/tests/features/f57.test.tsx`
- `apps/api/partsiq/api/f57_catalogue_browser.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

121 F58 catalogue imports

prompts/F58-catalogue-imports.md

F58 Catalogue imports

GOAL

Implement the complete catalogue imports behavior in the single-workflow Snowflake edition. Read CSV UTF-8 or XLSX without evaluating formulas; validate row schema, references, serial overlap, supersession cycles and source evidence; show dry-run report; explicit admin publish. Stage a version in Snowflake, validate all relationships explicitly and only then activate a validated PostgreSQL serving snapshot.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F57. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `worker` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f58_catalogue_imports.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f58_catalogue_imports/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f58.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f58.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f58_catalogue_imports.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f58`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f58_stage_import(actor: ActorContext, command: ImportCreate, idempotency_key: UUID) ->
ImportView
async def f58_read_import(actor: ActorContext, import_id: UUID) -> ImportView
async def f58_sign_import_upload(actor: ActorContext, import_id: UUID, command: ImportUpload,
idempotency_key: UUID) -> UploadTicket
async def f58_validate_import(actor: ActorContext, import_id: UUID, idempotency_key: UUID) -> JobView
class ImportCreate(Contract):
```

```
    filename:str = Field(min_length=1,max_length=160)
    format:Literal['csv','xlsx']
    size_bytes:int = Field(gt=0,le=10485760)

class ImportUpload(Contract):
    media_type:Literal['text/csv','application/vnd.openxmlformats-officedocument.spreadsheetml.sheet']

class ImportView(Contract):
    id:UUID
    state:str
    row_count:int
    issues:list[ImportIssue]
    version_id:UUID | None
    content_sha256:str | None

class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None

class UploadTicket(Contract):
    attachment_id: UUID
    upload_url: str
    expires_at: datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F58; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Read CSV UTF-8 or XLSX without evaluating formulas.

4. Validate row schema, references, serial overlap, supersession cycles and source evidence.

5. Show dry-run report.

6. Explicit admin publish. Stage a version in Snowflake, validate all relationships explicitly and only then activate a validated PostgreSQL serving snapshot.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/admin/imports` → `f58_stage_import`; request `ImportCreate` as JSON; response `ImportView`, HTTP 201; access `catalogue_admin`.

• `GET /v1/admin/imports/{import_id}` → `f58_read_import`; request `None` (no body); response `ImportView`, HTTP 200; access `catalogue_admin`.

• `POST /v1/admin/imports/{import_id}/upload` → `f58_sign_import_upload`; request `ImportUpload` as JSON; response `UploadTicket`, HTTP 200; access `catalogue_admin`.

• `POST /v1/admin/imports/{import_id}/validate` → `f58_validate_import`; request `None` (no body); response `JobView`, HTTP 202; access `catalogue_admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Invalid batch changes no published data. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: file with macros/formulas rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: 50000-row and 10MiB limits enforced. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: public demo cannot publish. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f58.py`. Frontend path: `apps/web/tests/features/f58.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Invalid batch changes no published data. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: file with macros/formulas rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: 50000-row and 10MiB limits enforced. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: public demo cannot publish. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F58` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f58.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f58.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F58/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F58-catalogue-imports.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f58): implement catalogue imports`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F58 Catalogue imports

Status: specified, not implemented.

Owning prompt: `prompts/F58-catalogue-imports.md`.

Behavior: Read CSV UTF-8 or XLSX without evaluating formulas; validate row schema, references, serial overlap, supersession cycles and source evidence; show dry-run report; explicit admin publish. Stage a version in Snowflake, validate all relationships explicitly and only then activate a validated PostgreSQL serving snapshot.

Failure criteria: Invalid batch changes no published data; file with macros/formulas rejected; 50000-row and 10MiB limits enforced; public demo cannot publish.

API operations: f58_stage_import, f58_read_import, f58_sign_import_upload, f58_validate_import

Files:

- `apps/api/partsiq/features/f58_catalogue_imports.py`
- `apps/web/src/features/f58_catalogue_imports/Panel.tsx`
- `tests/features/test_f58.py`

- `apps/web/tests/features/f58.test.tsx`
- `apps/api/partsiq/api/f58_catalogue_imports.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

122 F59 catalogue versioning

prompts/F59-catalogue-versioning.md

F59 Catalogue versioning

GOAL

Implement the complete catalogue versioning behavior in the single-workflow Snowflake edition. Store immutable versioned catalogue records in Snowflake; verify record counts and canonical SHA256; stage the PostgreSQL serving snapshot; atomically switch the active version in a PostgreSQL transaction; retain published historical records; reconcile partial remote writes.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F58. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `catalog` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f59_catalogue_versioning.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f59_catalogue_versioning/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f59.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f59.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f59_catalogue_versioning.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f59`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f59_request_publish(actor: ActorContext, import_id: UUID, command: PublishRequest,
idempotency_key: UUID) -> JobView
async def f59_list_versions(actor: ActorContext) -> VersionList
class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
```

```
    attempts:int
    error_code:str | None
    resource_id:UUID | None

class PublishRequest(Contract):
    content_sha256:str = Field(pattern='^[a-f0-9]{64}$')
    expected_active_version_id:UUID | None

class VersionList(Contract):
    items:list[VersionView]
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F59; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Store immutable versioned catalogue records in Snowflake.

4. Verify record counts and canonical SHA256.

5. Stage the PostgreSQL serving snapshot.

6. Atomically switch the active version in a PostgreSQL transaction.

7. Retain published historical records.

8. Reconcile partial remote writes.

9. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

10. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

11. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

12. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/admin/imports/{import_id}/publish` → `f59_request_publish`; request `PublishRequest` as JSON; response `JobView`, HTTP 202; access `catalogue_admin`.

• `GET /v1/admin/catalogue/versions` → `f59_list_versions`; request `None` (no body); response `VersionList`, HTTP 200; access `catalogue_admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Two publishers cannot leave two active versions. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: failed publish rolls back. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: old approval retains immutable evidence snapshot. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f59.py`. Frontend path: `apps/web/tests/features/f59.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Two publishers cannot leave two active versions. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: failed publish rolls back. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: old approval retains immutable evidence snapshot. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F59` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f59.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f59.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F59/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F59-catalogue-versioning.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f59): implement catalogue versioning`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F59 Catalogue versioning

Status: specified, not implemented.

Owning prompt: `prompts/F59-catalogue-versioning.md`.

Behavior: Store immutable versioned catalogue records in Snowflake; verify record counts and canonical SHA256; stage the PostgreSQL serving snapshot; atomically switch the active version in a PostgreSQL transaction; retain published historical records; reconcile partial remote writes.

Failure criteria: Two publishers cannot leave two active versions; failed publish rolls back; old approval retains immutable evidence snapshot.

API operations: f59_request_publish, f59_list_versions

Files:

- `apps/api/partsiq/features/f59_catalogue_versioning.py`
- `apps/web/src/features/f59_catalogue_versioning/Panel.tsx`
- `tests/features/test_f59.py`
- `apps/web/tests/features/f59.test.tsx`
- `apps/api/partsiq/api/f59_catalogue_versioning.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

123 F60 missing parts queue

prompts/F60-missing-parts-queue.md

F60 Missing-parts queue

GOAL

Implement the complete missing-parts queue behavior in the single-workflow Snowflake edition. Record unresolved item once; allow assigned staff to mark investigating, resolved or unavailable with note; link back to enquiry; no automatic catalogue creation.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F59. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f60_missing_parts_queue.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f60_missing_parts_queue/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f60.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f60.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f60_missing_parts_queue.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f60`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f60_update_missing(actor: ActorContext, item_id: UUID, command: MissingUpdate,
idempotency_key: UUID) -> MissingView
async def f60_list_missing(actor: ActorContext, command: InboxQuery) -> MissingPage
class InboxQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    status:str | None = None
    priority:Literal['normal','urgent'] | None = None
    overdue:bool = False
    query:str | None = Field(default=None,max_length=200)
```

```
class MissingPage(Contract):
    items:list[MissingView]
    next_cursor:str | None

class MissingUpdate(Contract):
    expected_revision:int = Field(ge=1)
    state:Literal['investigating','resolved','unavailable']
    note:str = Field(min_length=1,max_length=2000)

class MissingView(Contract):
    item_id:UUID
    enquiry_id:UUID
    state:str
    note:str | None
    updated_at:datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F60; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Record unresolved item once.

4. Allow assigned staff to mark investigating, resolved or unavailable with note.

5. Link back to enquiry.

6. No automatic catalogue creation.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `PATCH /v1/staff/missing-parts/{item_id}` → `f60_update_missing`; request `MissingUpdate` as JSON; response `MissingView`, HTTP 200; access `review`.

• `GET /v1/staff/missing-parts` → `f60_list_missing`; request `InboxQuery` as validated query parameters; response `MissingPage`, HTTP 200; access `reviewer`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Resolved status requires staff explanation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: foreign item denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: empty search result is not fabricated stock. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f60.py`. Frontend path: `apps/web/tests/features/f60.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Resolved status requires staff explanation. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: foreign item denied. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: empty search result is not fabricated stock. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F60` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f60.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f60.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F60/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F60-missing-parts-queue.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f60): implement missing-parts queue`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F60 Missing-parts queue

Status: specified, not implemented.

Owning prompt: `prompts/F60-missing-parts-queue.md`.

Behavior: Record unresolved item once; allow assigned staff to mark investigating, resolved or unavailable with note; link back to enquiry; no automatic catalogue creation.

Failure criteria: Resolved status requires staff explanation; foreign item denied; empty search result is not fabricated stock.

API operations: f60_update_missing, f60_list_missing

Files:

- `apps/api/partsiq/features/f60_missing_parts_queue.py`
- `apps/web/src/features/f60_missing_parts_queue/Panel.tsx`
- `tests/features/test_f60.py`
- `apps/web/tests/features/f60.test.tsx`
- `apps/api/partsiq/api/f60_missing_parts_queue.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

124 F61 enquiry pdf

prompts/F61-enquiry-pdf.md

F61 Enquiry PDF

GOAL

Implement the complete enquiry pdf behavior in the single-workflow Snowflake edition. Export approved revision only; render logo, bilingual customer text, verified selections, estimates and references; embed Arabic fonts; store private object and signed download.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F60. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `worker` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f61_enquiry_pdf.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f61_enquiry_pdf/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f61.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f61.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f61_enquiry_pdf.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f61`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f61_request_pdf(actor: ActorContext, enquiry_id: UUID, command: ExportRequest,
idempotency_key: UUID) -> JobView
async def f61_sign_download(actor: ActorContext, export_id: UUID) -> DownloadTicket
class DownloadTicket(Contract):
    url: str
    expires_at: datetime

class ExportRequest(Contract):
    enquiry_id: UUID
```

```
    revision: int
    decision_id: UUID

class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F61; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Export approved revision only.

4. Render logo, bilingual customer text, verified selections, estimates and references.

5. Embed Arabic fonts.

6. Store private object and signed download.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/enquiries/{enquiry_id}/exports` → `f61_request_pdf`; request `ExportRequest` as JSON; response `JobView`, HTTP 202; access `owner`.

• `GET /v1/exports/{export_id}/download` → `f61_sign_download`; request `None` (no body); response `DownloadTicket`, HTTP 200; access `owner`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not

proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Exclude notes and secrets. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stale or unapproved revision rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: long Arabic names wrap without clipping. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: no prices or quotation title. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f61.py`. Frontend path: `apps/web/tests/features/f61.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Exclude notes and secrets. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: stale or unapproved revision rejected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: long Arabic names wrap without clipping. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: no prices or quotation title. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F61` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f61.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f61.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F61/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F61-enquiry-pdf.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f61): implement enquiry pdf`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F61 Enquiry PDF

Status: specified, not implemented.

Owning prompt: `prompts/F61-enquiry-pdf.md`.

Behavior: Export approved revision only; render logo, bilingual customer text, verified selections, estimates and references; embed Arabic fonts; store private object and signed download.

Failure criteria: Exclude notes and secrets; stale or unapproved revision rejected; long Arabic names wrap without clipping; no prices or quotation title.

API operations: f61_request_pdf, f61_sign_download

Files:

- `apps/api/partsiq/features/f61_enquiry_pdf.py`
- `apps/web/src/features/f61_enquiry_pdf/Panel.tsx`
- `tests/features/test_f61.py`
- `apps/web/tests/features/f61.test.tsx`
- `apps/api/partsiq/api/f61_enquiry_pdf.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

125 F62 labelled evaluations

prompts/F62-labelled-evaluations.md

F62 Labelled evaluations

GOAL

Implement the complete labelled evaluations behavior in the single-workflow Snowflake edition. Include four demos plus Arabic, typo, serial-boundary, ambiguity, stock and injection cases; record expected compatible set and clarification outcome; run deterministic suite without API calls.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F61. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `evals` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f62_labelled_evaluations.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f62_labelled_evaluations/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f62.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f62.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f62_labelled_evaluations.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f62`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f62_enqueue_run(actor: ActorContext, command: EvalRequest, idempotency_key: UUID) -> JobView
async def f62_read_run(actor: ActorContext, run_id: UUID) -> EvaluationView
class EvalRequest(Contract):
    suite:Literal['deterministic','live']
    include_demo:bool = True

class EvaluationView(Contract):
    run_id:UUID
    state:str
```

```
    total:int
    passed:int
    failed:int
    results:list[EvalResult]

class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F62; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Include four demos plus Arabic, typo, serial-boundary, ambiguity, stock and injection cases.

4. Record expected compatible set and clarification outcome.

5. Run deterministic suite without API calls.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/admin/evaluations` → `f62_enqueue_run`; request `EvalRequest` as JSON; response `JobView`, HTTP 202; access `admin`.

• `GET /v1/admin/evaluations/{run_id}` → `f62_read_run`; request `None` (no body); response `EvaluationView`, HTTP 200; access `admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not

proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No self-reported model confidence as accuracy. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: evaluation fixture disjoint from examples used in prompts. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: failures keep actual output. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f62.py`. Frontend path: `apps/web/tests/features/f62.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No self-reported model confidence as accuracy. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: evaluation fixture disjoint from examples used in prompts. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: failures keep actual output. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F62` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f62.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f62.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F62/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F62-labelled-evaluations.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next

step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f62): implement labelled evaluations`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F62 Labelled evaluations

Status: specified, not implemented.

Owning prompt: `prompts/F62-labelled-evaluations.md`.

Behavior: Include four demos plus Arabic, typo, serial-boundary, ambiguity, stock and injection cases; record expected compatible set and clarification outcome; run deterministic suite without API calls.

Failure criteria: No self-reported model confidence as accuracy; evaluation fixture disjoint from examples used in prompts; failures keep actual output.

API operations: f62_enqueue_run, f62_read_run

Files:

- `apps/api/partsiq/features/f62_labelled_evaluations.py`
- `apps/web/src/features/f62_labelled_evaluations/Panel.tsx`
- `tests/features/test_f62.py`
- `apps/web/tests/features/f62.test.tsx`
- `apps/api/partsiq/api/f62_labelled_evaluations.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

126 F63 metrics dashboard

prompts/F63-metrics-dashboard.md

F63 Metrics dashboard

GOAL

Implement the complete metrics dashboard behavior in the single-workflow Snowflake edition. Show volume, status, review time, clarification and labelled accuracy with denominators; separate demo traffic; filter time window; scope reviewer data to assignments. Export redacted aggregate events to Snowflake through an outbox and show reporting freshness separately from live operational counts.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F62. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f63_metrics_dashboard.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f63_metrics_dashboard/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f63.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f63.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f63_metrics_dashboard.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f63`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f63_metrics(actor: ActorContext, command: MetricsQuery) -> MetricsView
async def f63_read_sync(actor: ActorContext) -> AnalyticsSyncView
class AnalyticsSyncView(Contract):
    last_success_at:datetime | None
    pending_events:int
    source:Literal['snowflake','fixture']
    stale:bool

class MetricsQuery(Contract):
```

```
    start: datetime
    end: datetime
    include_demo: bool = False

class MetricsView(Contract):
    enquiry_count: int
    approved_count: int
    clarification_count: int
    evaluated_count: int
    correct_count: int
    median_review_seconds: float | None
    p95_latency_ms: float | None
    estimated_cost_usd: Decimal
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F63; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Show volume, status, review time, clarification and labelled accuracy with denominators.

4. Separate demo traffic.

5. Filter time window.

6. Scope reviewer data to assignments. Export redacted aggregate events to Snowflake through an outbox and show reporting freshness separately from live operational counts.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/metrics` → `f63_metrics`; request `MetricsQuery` as validated query parameters; response `MetricsView`, HTTP 200; access `reviewer`.

• `GET /v1/admin/analytics/sync` → `f63_read_sync`; request `None` (no body); response `AnalyticsSyncView`, HTTP 200; access `admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Zero denominator displays unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: no fabricated percentage. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: demo traffic never silently improves operational metrics. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f63.py`. Frontend path: `apps/web/tests/features/f63.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Zero denominator displays unavailable. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: no fabricated percentage. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: demo traffic never silently improves operational metrics. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F63` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f63.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f63.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F63/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and

confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F63-metrics-dashboard.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f63): implement metrics dashboard`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F63 Metrics dashboard

Status: specified, not implemented.

Owning prompt: `prompts/F63-metrics-dashboard.md`.

Behavior: Show volume, status, review time, clarification and labelled accuracy with denominators; separate demo traffic; filter time window; scope reviewer data to assignments. Export redacted aggregate events to Snowflake through an outbox and show reporting freshness separately from live operational counts.

Failure criteria: Zero denominator displays unavailable; no fabricated percentage; demo traffic never silently improves operational metrics.

API operations: f63_metrics, f63_read_sync

Files:

- `apps/api/partsiq/features/f63_metrics_dashboard.py`
- `apps/web/src/features/f63_metrics_dashboard/Panel.tsx`
- `tests/features/test_f63.py`
- `apps/web/tests/features/f63.test.tsx`
- `apps/api/partsiq/api/f63_metrics_dashboard.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

127 F64 latency and ai cost

prompts/F64-latency-and-ai-cost.md

F64 Latency and AI cost

GOAL

Implement the complete latency and ai cost behavior in the single-workflow Snowflake edition. Measure stages with monotonic clock; log model/token usage and estimated cost; reserve confirmed free credit before each attempt; expose consumed/reserved allowance.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F63. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `common` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f64_latency_and_ai_cost.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f64_latency_and_ai_cost/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f64.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f64.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f64_latency_and_ai_cost.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f64`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f64_read_allowance(actor: ActorContext) -> AllowanceView
async def f64_configure_allowance(actor: ActorContext, command: AllowanceSet, idempotency_key: UUID) ->
 AllowanceView
class AllowanceSet(Contract):
    confirmed_usd:Decimal = Field(ge=0)
    expires_at:datetime
    model:str = Field(min_length=1,max_length=120)
    input_usd_per_million:Decimal = Field(ge=0)
    output_usd_per_million:Decimal = Field(ge=0)
```

```
class AllowanceView(Contract):
    enabled:bool
    confirmed_usd:Decimal
    consumed_usd:Decimal
    reserved_usd:Decimal
    expires_at:datetime | None
    model:str | None
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F64; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Measure stages with monotonic clock.

4. Log model/token usage and estimated cost.

5. Reserve confirmed free credit before each attempt.

6. Expose consumed/reserved allowance.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/admin/ai-allowance` → `f64_read_allowance`; request `None` (no body); response `AllowanceView`, HTTP 200; access `admin`.

• `PUT /v1/admin/ai-allowance` → `f64_configure_allowance`; request `AllowanceSet` as JSON; response `AllowanceView`, HTTP 200; access `admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Unknown price or expired credit blocks calls. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: concurrent reservations cannot exceed cap. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: API key validity never implies free credit. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f64.py`. Frontend path: `apps/web/tests/features/f64.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Unknown price or expired credit blocks calls. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: concurrent reservations cannot exceed cap. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: API key validity never implies free credit. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F64` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f64.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f64.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F64/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F64-latency-and-ai-cost.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f64): implement latency and ai cost`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F64 Latency and AI cost

Status: specified, not implemented.

Owning prompt: `prompts/F64-latency-and-ai-cost.md`.

Behavior: Measure stages with monotonic clock; log model/token usage and estimated cost; reserve confirmed free credit before each attempt; expose consumed/reserved allowance.

Failure criteria: Unknown price or expired credit blocks calls; concurrent reservations cannot exceed cap; API key validity never implies free credit.

API operations: f64_read_allowance, f64_configure_allowance

Files:

- `apps/api/partsiq/features/f64_latency_and_ai_cost.py`
- `apps/web/src/features/f64_latency_and_ai_cost/Panel.tsx`
- `tests/features/test_f64.py`
- `apps/web/tests/features/f64.test.tsx`
- `apps/api/partsiq/api/f64_latency_and_ai_cost.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

128 F65 correction feedback

prompts/F65-correction-feedback.md

F65 Correction feedback

GOAL

Implement the complete correction feedback behavior in the single-workflow Snowflake edition. List authorized before/after corrections; let admin curate anonymized labelled eval cases; record reviewer and source revision; never auto-train on unreviewed edits.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F64. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `api` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f65_correction_feedback.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f65_correction_feedback/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f65.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f65.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f65_correction_feedback.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f65`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f65_curate_correction(actor: ActorContext, correction_id: UUID, command: CurateRequest,
idempotency_key: UUID) -> CuratedView
class CurateRequest(Contract):
    case_id:str = Field(min_length=1,max_length=80)
    anonymized_text:str = Field(min_length=1,max_length=12000)
    expected_status:str
    expected_part_ids:list[int]

class CuratedView(Contract):
    case_id:str
```

```
    source_correction_id:UUID
    approved_by:UUID
    created_at:datetime
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F65; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. List authorized before/after corrections.

4. Let admin curate anonymized labelled eval cases.

5. Record reviewer and source revision.

6. Never auto-train on unreviewed edits.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `POST /v1/admin/corrections/{correction_id}/evaluation-case` → `f65_curate_correction`; request `CurateRequest` as JSON; response `CuratedView`, HTTP 201; access `admin`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Private text excluded from exported fixtures. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: duplicate case source detected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unreviewed correction cannot become ground truth. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f65.py`. Frontend path: `apps/web/tests/features/f65.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Private text excluded from exported fixtures. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: duplicate case source detected. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: unreviewed correction cannot become ground truth. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F65` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f65.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f65.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F65/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F65-correction-feedback.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f65): implement correction feedback`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval

snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE
Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F65 Correction feedback

Status: specified, not implemented.

Owning prompt: `prompts/F65-correction-feedback.md`.

Behavior: List authorized before/after corrections; let admin curate anonymized labelled eval cases; record reviewer and source revision; never auto-train on unreviewed edits.

Failure criteria: Private text excluded from exported fixtures; duplicate case source detected; unreviewed correction cannot become ground truth.

API operations: f65_curate_correction

Files:

- `apps/api/partsiq/features/f65_correction_feedback.py`
- `apps/web/src/features/f65_correction_feedback/Panel.tsx`
- `tests/features/test_f65.py`
- `apps/web/tests/features/f65.test.tsx`
- `apps/api/partsiq/api/f65_correction_feedback.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

129 F66 tests and ci

prompts/F66-tests-and-ci.md

F66 Tests and CI

GOAL

Implement the complete tests and ci behavior in the single-workflow Snowflake edition. Run service-isolated Python tests, frontend checks, RLS integration and deterministic evals; build images; require explicit environment approval for deployment; pin action SHAs.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F65. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `infra` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f66_tests_and_ci.py`: application facade connecting this feature to foundation domain functions.

• `tests/features/test_f66.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f66.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f66`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f66_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F66; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Run service-isolated Python tests, frontend checks, RLS integration and deterministic evals.

4. Build images.

5. Require explicit environment approval for deployment.

6. Pin action SHAs.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No prod DB in tests. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: no secret available to untrusted fork jobs. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: failed gate blocks deployment. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f66.py`. Frontend path: `apps/web/tests/features/f66.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No prod DB in tests. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: no secret available to untrusted fork jobs. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: failed gate blocks deployment. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F66` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f66.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f66.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F66/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F66-tests-and-ci.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f66): implement tests and ci`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F66 Tests and CI

Status: specified, not implemented.

Owning prompt: `prompts/F66-tests-and-ci.md`.

Behavior: Run service-isolated Python tests, frontend checks, RLS integration and deterministic evals; build images; require explicit environment approval for deployment; pin action SHAs.

Failure criteria: No prod DB in tests; no secret available to untrusted fork jobs; failed gate blocks deployment.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f66_tests_and_ci.py`
- `tests/features/test_f66.py`
- `apps/web/tests/features/f66.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

130 F67 traces

prompts/F67-traces.md

F67 Traces

GOAL

Implement the complete traces behavior in the single-workflow Snowflake edition. Propagate request/enquiry/job IDs; capture stage timing and safe status; export optional Langfuse traces with payload redaction; operate normally when tracing unavailable.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F66. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `common` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f67_traces.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f67_traces/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f67.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f67.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/api/partsiq/api/f67_traces.py`: FastAPI route adapter with declared authorization and response model.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f67`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
async def f67_list_stages(actor: ActorContext, enquiry_id: UUID) -> TraceView
class TraceView(Contract):
    enquiry_id:UUID
    revision:int
    stages:list[TraceStage]
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props:`

`{enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F67; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Propagate request/enquiry/job IDs.

4. Capture stage timing and safe status.

5. Export optional Langfuse traces with payload redaction.

6. Operate normally when tracing unavailable.

7. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

8. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

9. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

10. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

• `GET /v1/staff/enquiries/{enquiry_id}/traces` → `f67_list_stages`; request `None` (no body); response `TraceView`, HTTP 200; access `review`.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No raw key or full photo logged. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: exporter outage never breaks approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: budget accounting remains local and authoritative. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but

stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f67.py`. Frontend path: `apps/web/tests/features/f67.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No raw key or full photo logged. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: exporter outage never breaks approval. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: budget accounting remains local and authoritative. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F67` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f67.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f67.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F67/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F67-traces.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f67): implement traces`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F67 Traces

Status: specified, not implemented.

Owning prompt: `prompts/F67-traces.md`.

Behavior: Propagate request/enquiry/job IDs; capture stage timing and safe status; export optional Langfuse traces with payload redaction; operate normally when tracing unavailable.

Failure criteria: No raw key or full photo logged; exporter outage never breaks approval; budget accounting remains local and authoritative.

API operations: f67_list_stages

Files:

- `apps/api/partsiq/features/f67_traces.py`
- `apps/web/src/features/f67_traces/Panel.tsx`
- `tests/features/test_f67.py`
- `apps/web/tests/features/f67.test.tsx`
- `apps/api/partsiq/api/f67_traces.py`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

131 F68 recruiter documentation

prompts/F68-recruiter-documentation.md

F68 Recruiter documentation

GOAL

Implement the complete recruiter documentation behavior in the single-workflow Snowflake edition. Write concise overview, real architecture, quick start, demo paths, tests, trade-offs and limitations; link measured evidence; explain deterministic decisions and ordinary Python functions.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F67. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `docs` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f68_recruiter_documentation.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f68_recruiter_documentation/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f68.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f68.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f68`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f68_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F68; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Write concise overview, real architecture, quick start, demo paths, tests, trade-offs and limitations.

4. Link measured evidence.

5. Explain deterministic decisions and ordinary Python functions.

6. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

7. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

8. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

9. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: No invented accuracy, uptime, client relationship or production adoption. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: distinguish implementation status from specification. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f68.py`. Frontend path: `apps/web/tests/features/f68.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: No invented accuracy, uptime, client relationship or production adoption. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: distinguish implementation status from specification. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F68` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f68.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f68.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F68/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F68-recruiter-documentation.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f68): implement recruiter documentation`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F68 Recruiter documentation

Status: specified, not implemented.

Owning prompt: `prompts/F68-recruiter-documentation.md`.

Behavior: Write concise overview, real architecture, quick start, demo paths, tests, trade-offs and limitations; link measured evidence; explain deterministic decisions and ordinary Python functions.

Failure criteria: No invented accuracy, uptime, client relationship or production adoption; distinguish implementation status from specification.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f68_recruiter_documentation.py`
- `apps/web/src/features/f68_recruiter_documentation/Panel.tsx`
- `tests/features/test_f68.py`
- `apps/web/tests/features/f68.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

132 F69 demo walkthrough

prompts/F69-demo-walkthrough.md

F69 Demo walkthrough

GOAL

Implement the complete demo walkthrough behavior in the single-workflow Snowflake edition. Create 3-minute and 8-minute scripts covering happy path, missing serial, incompatibility, stock split and human review; include recording checklist and captions.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

This feature extends foundations00–20 and follows F68. Read docs/PRD.md for the user outcome, docs/REST_API.md for transport rules, docs/DATA_OWNERSHIP.md for persistence, docs/WORKFLOW.md for state changes, and docs/SECURITY.md for access. Read the exact models below before implementing UI assumptions. This feature belongs to `docs` at product level but runs inside the common application; ownership does not imply a separate service or agent. Preserve the behavior of all previously completed features.

PACKAGES AND INSTALLATION

Reuse foundation-installed packages; this feature introduces no floating dependency. Backend: FastAPI==0.135.1 with APIRouter, Depends, HTTPException and response_model; Pydantic==2.12.5 with Field and model_validate_json; asyncpg==0.31.0 with parameterized queries and explicit transactions; httpx==0.28.1 for bounded dependency calls. Tests: pytest==9.0.2 and pytest-asyncio==1.3.0. Frontend: the exact Next.js, React, Zod, next-intl, Vitest and Testing Library versions in spec/web-package.json. Run `uv pip sync spec/python-requirements.lock` and `corepack pnpm --dir apps/web install --frozen-lockfile` only if dependencies are not already synchronized. Snowflake access belongs to the catalogue/reporting adapter, not this browser feature. No extra agent library is allowed.

FILES AND OWNERSHIP

• `apps/api/partsiq/features/f69_demo_walkthrough.py`: application facade connecting this feature to foundation domain functions.

• `apps/web/src/features/f69_demo_walkthrough/Panel.tsx`: accessible bilingual feature panel and its loading/error/empty states.

• `tests/features/test_f69.py`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

• `apps/web/tests/features/f69.test.tsx`: feature acceptance tests; exercise the real public entry point with isolated fixtures.

Register its router in `apps/api/partsiq/api/router.py` after the new module exists. Mount the panel in CustomerShell, StaffShell or the catalogue/admin page according to the access level. Extend existing en/ar dictionaries with namespaced keys `f69`; do not duplicate whole dictionaries. Record modifications to shared files in the handover. Add no second implementation of compatibility, approval, budget accounting or Snowflake access.

TYPED CONTRACTS

```
def f69_acceptance_state(context: FeatureContext) -> FeatureOutcome
```

Shared nested models are fully defined in spec/contracts.py and spec/api_models.py; copy their types rather than weakening them to unvalidated dictionaries. For a UI panel define `export function Panel(props: {enquiryId?: string; locale: 'en' | 'ar'; readOnly: boolean}): React.JSX.Element`. For a pure internal feature use `FeatureContext` with request_id, actor, enquiry_id, revision and `FeatureOutcome` with passed, code, details, as defined in spec/api_models.py. ReadOnly is a UI affordance; backend authorization remains mandatory. All serials and part numbers remain strings. Serialize Decimal as strings and UUID/datetime through the shared models.

IMPLEMENTATION STEPS

1. Read the existing domain implementation and its tests. Write down the current caller and the expected persisted outcome for F69; use the canonical endpoint owner rather than adding another path.

2. Implement the facade signature above. Derive actor identity from the verified dependency, load the scoped enquiry/item when relevant, and reject a path/body ID mismatch before querying private data. A route must never accept an arbitrary actor, role, table name or Storage path from JSON.

3. Create 3-minute and 8-minute scripts covering happy path, missing serial, incompatibility, stock split and human review.

4. Include recording checklist and captions.

5. Build the response from persisted current-revision records. Avoid returning internal notes, raw provider payloads, keys or object credentials. Use response_model filtering as an additional check, not as the only authorization boundary.

6. Wire the frontend through apiFetch and the exact response schema. Implement loading, empty, success, validation error, stale revision, forbidden and dependency-unavailable states where they apply. Disable duplicate submit while pending and keep a stable idempotency key for an explicit retry of the same operation.

7. Re-fetch the authoritative resource after a successful mutation. Do not optimistically display approval, stock availability or completed OCR before the server confirms it. On409 preserve the user input, show that data changed, and require a fresh intentional action using the new revision.

8. Attach request ID to logs and append an audit event for business changes inside the transaction. Measurements use monotonic durations and UTC event timestamps. Any reporting export is redacted and goes through analytics_outbox rather than a second uncoordinated remote write.

REST API AND DATA FLOW

No new public REST operation belongs to this feature. Integrate it through the existing shell or domain hook; do not create a redundant endpoint.

API module calls this feature facade, which uses the foundations' domain service/repository. Customer-visible reads require ownership or active own-demo scope; staff detail reads require assignment or admin. Backend search uses the validated PostgreSQL serving version originally loaded from Snowflake. Never issue one Snowflake query per render, poll or keystroke. If the feature starts background work, commit its job and return the stored resource ID; the worker runs fixed Python steps and the client polls progress. HTTP 202 means accepted, not successful completion. For an idempotent replay, return the original status and response body. All errors follow Problem and the mapping in docs/REST_API.md.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

• Case 1: Do not claim video exists until recorded. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: use synthetic data. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: show failure handling and reset. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: verify links before sending recruiters. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

Check string lengths, list bounds, UUID validity, enum values and timestamps at the boundary. Pydantic forbids unknown keys. Ownership checks precede expensive or provider operations. A syntactically valid but stale revision is409, not422. Foreign scoped records are404; unauthenticated requests401; role-only denial403. Dependency failure must preserve the previous usable state and return a recoverable code when retry is safe. Never convert missing stock to zero or a retrieval outage to no compatible parts.

TESTS

Backend path: `tests/features/test_f69.py`. Frontend path: `apps/web/tests/features/f69.test.tsx`.

• Happy path: arrange the relevant synthetic catalogue, actor and enquiry state; execute the contract above; assert the feature-specific behavior stated in GOAL, exact status and typed response. For writes, reload PostgreSQL and inspect the audit and current revision.

• Case 1: Do not claim video exists until recorded. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 2: use synthetic data. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 3: show failure handling and reset. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Case 4: verify links before sending recruiters. Arrange a fixture that reaches this boundary, call the production entry point, and assert both the response and absence of unauthorized or partial writes.

• Scope isolation: repeat with another customer/session or an unassigned reviewer; assert404 or403 according to the access table, and no leaked response fields or writes.

• Concurrency: where this feature mutates state, send two matching idempotency keys concurrently and assert one effect; send a stale expected_revision and assert409. A read-only feature instead tests stable ordering and a changed resource between polls.

• Dependency boundary: replace the relevant adapter with a timeout/failure and assert explicit degraded behavior with no publication. Unit tests must not call a real paid API.

• UI: render English and Arabic, simulate keyboard interaction, check the named error and empty states, and assert that displayed part numbers/quantities come from the response. Verify no page-wide overflow at 360px and no inaccessible primary action.

COMMANDS AND EXPECTED RESULTS

`python tools/run_checks.py --prompt F69` must discover this feature's exact backend/UI tests and exit0 only after they pass. Run `uv run pytest tests/features/test_f69.py -q` and `corepack pnpm --dir apps/web exec vitest run tests/features/f69.test.tsx` for focused diagnosis. Run `python tools/check_contract_drift.py` after adding routes; generated OpenAPI must match the owned operations in spec/openapi.json. During incremental builds compare only completed owners; the final gate requires every operation. Expected output is real pass/fail reporting with no skipped required scenario; do not fabricate a number. Save redacted logs in `evidence/F69/`.

MANUAL CHECK

Open the relevant page using the supplied synthetic scenario and confirm the behavior in GOAL. Switch language, resize to360px, and repeat with keyboard only. Verify current revision, source catalogue version and approved/unapproved status remain visible where meaningful. Then reproduce one listed failure and confirm that user input is preserved and no unauthorized final recommendation appears. Internal-only features are checked through their calling feature plus their CLI/test receipt rather than a fabricated screen.

DOCUMENTATION UPDATES

Update `features/F69-demo-walkthrough.md` with actual paths, commands, results, screenshots if useful, commit and remaining issues. Update docs/HANDOVER.md and append docs/SESSION_LOG.md with next step. If an API model changes, update spec/api_models.py, spec/openapi.json and generated TypeScript together; never let examples diverge. If Snowflake/SQL behavior changes, update docs/DATA_OWNERSHIP.md and migration tests. Mark the feature implemented only after its own evidence exists.

ROLLBACK AND COMMIT

Commit as `feat(f69): implement demo walkthrough`. Revert this exact commit after checking dependants; do not reset the database or revert unrelated work. Disable the panel/route before reverting a prerequisite used by other features. Any applied schema change uses a compensating migration. Retain audit, approval snapshots and immutable catalogue versions. A rollback never disables compatibility or human approval to make the demo work.

COMPLETION EVIDENCE

Return the implemented behavior, signatures, API methods, tests with actual outcomes, and commit SHA. Explicitly separate offline tests from live Snowflake/OpenAI/hosted checks. Confirm that every failure case

above was exercised and that no features were silently removed. If a provider account is unavailable, the deterministic fixture tests still run and live verification remains blocked rather than falsely passed.

Feature implementation trace

F69 Demo walkthrough

Status: specified, not implemented.

Owning prompt: `prompts/F69-demo-walkthrough.md`.

Behavior: Create 3-minute and 8-minute scripts covering happy path, missing serial, incompatibility, stock split and human review; include recording checklist and captions.

Failure criteria: Do not claim video exists until recorded; use synthetic data; show failure handling and reset; verify links before sending recruiters.

API operations: No independent endpoint

Files:

- `apps/api/partsiq/features/f69_demo_walkthrough.py`
- `apps/web/src/features/f69_demo_walkthrough/Panel.tsx`
- `tests/features/test_f69.py`
- `apps/web/tests/features/f69.test.tsx`

Execution receipt: no application tests have been executed during kit authoring. The implementation agent must append command, actual result, commit and remaining blocker here.

133 22 security

prompts/22-security.md

22 Security concurrency and failure recovery acceptance

GOAL

Run negative RLS cases for anonymous visitors, foreign customers, unassigned reviewers, disabled staff and stale demo generations.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors F69 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `tests/acceptance/test_security.py`: executable acceptance coverage for this step.
- `tests/acceptance/test_recovery.py`: executable acceptance coverage for this step.
- `tests/acceptance/test_concurrency.py`: executable acceptance coverage for this step.
- `tools/release_gate.py`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_22.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def verify_release_gates(receipts: list[TestReceipt]) -> GateResult
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Run negative RLS cases for anonymous visitors, foreign customers, unassigned reviewers, disabled staff and stale demo generations.

2. Kill the worker between provider completion and database commit; confirm lease fencing and conservative budget accounting.

3. Race two approvals, two corrections and two catalogue publishers.

4. Inject Redis outage, Snowflake timeout, invalid catalogue checksum, Storage 404 and OpenAI 429.

5. Verify availability is degraded explicitly without bypassing approval.

6. Feed instruction injection in chat, nameplate and imported evidence; no model-produced command can invoke privileged code.

7. Verify no logs contain signed URLs, keys or raw images.

8. Ensure production URLs are rejected by test configuration.

9. Application correctness requires execution; document blocked hosted checks honestly when accounts are absent.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_22.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 22`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/22/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable

dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step22, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(security): implement foundation 22 security concurrency and failure recovery acceptance`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

134 26 deployment

prompts/26-deployment.md

26 AWS and Snowflake account setup and deployment

GOAL

Confirm account-specific AWS eligibility and Snowflake credit balance/expiry before creating compute.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 22, 25 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `infra/deploy.sh`: implement the responsibility named by this path and the ordered instructions below.
- `infra/rollback.sh`: implement the responsibility named by this path and the ordered instructions below.
- `tools/account_preflight.py`: implement the responsibility named by this path and the ordered instructions below.
- `docs/DEPLOYMENT_RECEIPT.md`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_26.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
def deployment_preflight(settings: Settings, receipts: AccountReceipt) -> DeploymentDecision
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Confirm account-specific AWS eligibility and Snowflake credit balance/expiry before creating compute.
2. No paid resource activation is authorized.
3. For Snowflake create dev database and roles with the supplied DDL, register key-pair service users, then provision one X-Small warehouse only if covered by confirmed credits.

4. Set auto-suspend60seconds, auto-resume and bounded statement timeouts; configure a resource monitor using the confirmed budget.

5. Monitors are not hard zero-spend guarantees and do not cover all serverless/cloud services; disable those features.

6. Keep production live mode disabled until its own approval.

7. EC2 hosts Caddy, API, worker and Redis with Docker Compose.

8. Use a domain the owner already controls or an explicitly supported HTTPS setup; do not invent free domains.

9. Document local fixture mode as the durable no-account demo path.

10. Restore and rollback are rehearsed against dev before any production rollout.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_26.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 26`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/26/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step26, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(deployment): implement foundation 26 aws and snowflake account setup and deployment`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

135 27 hosted

prompts/27-hosted.md

27 Vercel configuration and complete hosted walkthrough

GOAL

Deploy the Next.js frontend to the allowed Vercel plan only after checking current account terms.

CONTEXT AND PREREQUISITES

The product is a bilingual spare-parts enquiry and human-review application. Snowflake stores versioned source catalogue data and redacted reporting events; Supabase supplies authentication, private files and PostgreSQL operational records. A validated PostgreSQL catalogue mirror supports compatibility checks and fast interactive reads. One FastAPI package and one background worker run the application. OpenAI extracts text/photos and writes constrained prose; ordinary Python controls all decisions. No multi-agent framework is part of this build.

Read AGENTS.md, docs/PRD.md, docs/ARCHITECTURE.md, docs/CONSTRAINTS.md, docs/TECH_SPEC.md, docs/REST_API.md and docs/WORKFLOW.md. Execute the predecessors 26 before this prompt. Read the feature matrix for downstream callers. Presence of a prior prompt file does not mean its work is implemented.

PACKAGES AND INSTALLATION

Use the exact pins in spec/python-requirements.in and spec/web-package.json. The resolved reference locks are spec/python-requirements.lock and spec/pnpm-lock.yaml. Foundation 00 creates the development environment with `uv venv --python 3.12`; then run `uv pip sync spec/python-requirements.lock`. Foundation 18 copies the supplied web manifest and lock before `corepack pnpm --dir apps/web install --frozen-lockfile`. Do not install LangGraph, LangChain, CrewAI or an agent SDK. Snowflake uses snowflake-connector-python==4.8.0 with fixed SQL and bound parameters. OpenAI uses openai==3.19.2; FastAPI==0.135.1, Pydantic==2.12.5, asyncpg==0.31.0 and httpx==0.28.1 supply the application boundaries. Optional sentence-transformers==5.2.3 and the explicit CPU torch wheel are loaded only by the worker. Full package purpose and installation ownership are in docs/DEPENDENCIES.md.

FILES AND OWNERSHIP

- `tests/hosted/test_walkthrough.py`: executable acceptance coverage for this step.
- `docs/HOSTED_ACCEPTANCE.md`: implement the responsibility named by this path and the ordered instructions below.
- `docs/RECRUITER_GUIDE.md`: implement the responsibility named by this path and the ordered instructions below.
- `docs/DEMO_SCRIPT.md`: implement the responsibility named by this path and the ordered instructions below.
- `tests/foundations/test_27.py`: executable acceptance coverage for this step.

Keep this step limited to its owned files. Connect additions in the already-created application router, settings and dependency container only when they exist. Add each such modification to the completion receipt. Preserve legacy source in references/legacy-backend.zip; application paths use the single partsiq package.

TYPED CONTRACTS

```
async def run_hosted_smoke(base_url: str, api_url: str) -> HostedReceipt
```

Contract models are defined in spec/contracts.py and spec/api_models.py. Legacy IdentifyRequest, IdentifyResponse, PartSearchRequest and PartSearchResponse remain in partsiq.search.schemas from the characterized legacy code. Path IDs are UUID except numeric catalogue IDs. Monetary usage and quantities use Decimal; timestamps are timezone-aware UTC. Settings is a pydantic-settings BaseSettings class generated from spec/environment.json. Return domain results; convert them to HTTP only in a route adapter. No domain function accepts an unverified actor supplied in a request body.

IMPLEMENTATION STEPS

1. Deploy the Next.js frontend to the allowed Vercel plan only after checking current account terms.
2. Configure public Supabase URL/anon key and API origin; never expose database or Snowflake credentials.
3. Preview points exclusively to dev and production to its separate Supabase project.
4. Verify CORS, JWT issuer, HTTPS, upload paths and private downloads.

5. Execute all four demos through the actual browser, including human approval and an unauthorized foreign-session attempt.

6. Record observed latency, host sizing and credit consumption rather than promised values.

7. Show Snowflake source provenance and last-sync timestamps in the technical walkthrough, with visible fixture mode when offline.

8. Recruiter docs distinguish implemented code, tested behavior and remaining deployment constraints.

9. Do not claim official TAMGO affiliation or a recorded video before it exists.

Inject provider and database adapters so unit tests use deterministic fakes and integration tests use disposable services. Do not make network calls while holding an enquiry row lock. Separate external work from final fenced persistence. Every irreversible business mutation needs an audit record in the same transaction. Search/read operations must use the validated active catalogue version, never an arbitrary latest Snowflake row. Add explicit bounded timeouts for every dependency and map known failures to stable error codes.

REST API AND DATA FLOW

The only public API is FastAPI under `/v1`. `apps/api/partsiq/api/router.py` includes feature routers created later. This foundation supplies domain functions; it must not invent additional competing endpoints. Read spec/endpoints.json for exact methods, resource paths, models and owners. Browser → verified route dependency → feature application function → this domain module → repository. Slow work is represented by a committed jobs row and a202 response. Worker → fixed transition table → domain function → lease-fenced PostgreSQL update. Snowflake synchronization is a separate job; no distributed transaction spans Snowflake and Supabase.

VALIDATION AND FAILURES

Never remove compatible-set filtering, provenance, revision checks, assigned-only real review or own-session demo isolation. Never make an LLM choose a machine, authorize a part, approve, publish, execute SQL or invoke a write tool. Do not add voice, quotation builder, order creation, inventory reservation or external-message sending. A known part number is not sufficient evidence of fitment. Free keys are not proof of available credit. No paid fallback or account upgrade is authorized. Keep live providers disabled until their dedicated allowance and expiry have been confirmed.

Missing auth is401; insufficient role is403; a foreign scoped resource is404; stale revision, changed idempotency payload or stale catalogue is409; invalid typed input is422; request quota is429; a required unavailable dependency is503. Use Problem.code for machine-readable behavior and request_id for diagnosis. Do not expose database errors, secrets, private object paths or provider responses. A failed dependency must not be reported as an empty successful result.

TESTS

Create `tests/foundations/test_27.py` with fixtures for the exact contracts above. Test the normal algorithm described in IMPLEMENTATION STEPS, each rejected boundary described there, and dependency failure without partial business writes. For data mutations use a rollback assertion and verify the audit event. For async work simulate an expired lease and a new revision before completion. For schema/infrastructure steps assert the actual SQL, configuration and installed versions rather than mocking the check itself. Keep network tests explicitly marked and never run them against production. Record separate unit, integration and hosted outcomes; no skipped required case may be called passed.

COMMANDS AND EXPECTED RESULTS

Run `python tools/run_checks.py --prompt 27`. Expected: all named files exist, the selected tests execute, and exit status0 follows only when every required assertion passes. Run `uv run ruff check apps/api tests` and `uv run mypy apps/api/partsiq` when the package exists; fix new errors before continuing. For frontend changes run `corepack pnpm --dir apps/web test --run` and `corepack pnpm --dir apps/web typecheck`. Do not assert a fixed passing test count before executing tests. Save stdout/stderr with credentials redacted in `evidence/27/`.

MANUAL CHECK

Exercise this module through the nearest available caller using the synthetic fixture. Inspect the input, typed output, persisted revision, source version and request ID. Repeat with one invalid input and one unavailable dependency. If this is an infrastructure prompt, verify the real account/host receipt; lack of account access is blocked, not passed. UI checks include English, Arabic, 360px width, keyboard navigation and200 percent zoom. Do not display test-only data as real client inventory.

DOCUMENTATION UPDATES

Update docs/HANDOVER.md with step27, actual changed paths, commands, results and the next build-order ID. Append docs/SESSION_LOG.md rather than replacing previous failures. Record source/schema changes in docs/TECH_SPEC.md and the relevant ADR. Update corresponding feature traces only after their acceptance tests pass. Add runtime API changes to the canonical specification and prove contract parity. Keep all claims about accuracy, cost and latency tied to measured evidence.

ROLLBACK AND COMMIT

Create a clean checkpoint commit before this step. Commit only this prompt as `feat(hosted): implement foundation 27 vercel configuration and complete hosted walkthrough`. Revert that exact commit after checking for dependent later changes. Applied database schema changes require a forward compensating migration; never reset shared Supabase or delete audit/history. Restore the previous active catalogue pointer only after validating its snapshot and stock freshness. Remote orphaned Snowflake versions stay unactivated for reconciliation. Rollback provider configuration without enabling a paid fallback.

COMPLETION EVIDENCE

Report implemented signatures, caller/callee wiring, test commands, actual pass/fail output, manual observations, commit SHA and unresolved blockers. The coding agent must inspect its own diff and confirm no scope was silently removed. No runtime behavior is complete merely because this specification exists. Continue only when its dependencies are usable or clearly report the narrow blocker.

136 contracts

spec/contracts.py

```
"""Executable contract specification copied by foundation 01; no provider or database side effects."""
from datetime import datetime
from decimal import Decimal
from enum import StrEnum
from typing import Literal
from uuid import UUID
from pydantic import BaseModel, ConfigDict, Field

class Contract(BaseModel):
    model_config = ConfigDict(extra='forbid', strict=True)

Locale = Literal['en', 'ar']
class AccessAction(StrEnum):
    READ='read'
    REVIEW='review'
    ASSIGN='assign'
    IMPORT='import'
class ActorContext(Contract):
    user_id: UUID
    request_id: UUID
    roles: list[Literal['reviewer','catalogue_admin','admin']]
    session_id: UUID | None = None
    session_generation: int | None = None
class DemoSession(Contract):
    id: UUID
    generation: int = Field(ge=1)
    expires_at: datetime
class ItemInput(Contract):
    description: str = Field(min_length=1,max_length=1000)
    quantity: Decimal | None = Field(default=None,gt=0,le=100000)
    model_text: str | None = None
    serial_text: str | None = None
    source_message_id: UUID
    source_start: int = Field(ge=0)
    source_end: int = Field(ge=0)
class IntakeResult(Contract):
    language: Literal['en','ar','mixed']
    preferred_locale: Locale
    urgency: Literal['normal','urgent']
    items: list[ItemInput] = Field(min_length=1,max_length=20)
class EnquiryCreate(Contract):
    text: str = Field(min_length=1,max_length=12000)
    locale: Locale
    destination_warehouse_id: int = Field(gt=0)
    session_id: UUID | None = None
class MessageCreate(Contract):
    text: str = Field(min_length=1,max_length=12000)
    expected_revision: int = Field(ge=1)
    item_id: UUID | None = None
class MessageView(Contract):
    id: UUID
    enquiry_id: UUID
    revision: int
    kind: Literal['customer','clarification','status','recommendation']
    body: str
    created_at: datetime
class EnquiryView(Contract):
    id: UUID
    revision: int
    status:
Literal['received','processing','needs_information','awaiting_review','approved','rejected','failed']
    messages: list[MessageView]
    item_ids: list[UUID]
    catalogue_version_id: UUID | None
    updated_at: datetime
class UploadRequest(Contract):
    enquiry_id: UUID
    expected_revision: int = Field(ge=1)
    media_type: Literal['image/jpeg','image/png','image/webp']
    size_bytes: int = Field(gt=0,le=10485760)
class UploadTicket(Contract):
    attachment_id: UUID
    upload_url: str
    expires_at: datetime
class AttachmentView(Contract):
    id: UUID
    status: Literal['pending','validated','rejected','deleted']
    error_code: str | None = None
class DownloadRequest(Contract):
    actor: ActorContext
    resource_type: Literal['evidence','export','attachment']
    resource_id: UUID
class DownloadTicket(Contract):
    url: str
    expires_at: datetime
class ImageInfo(Contract):
```

```
    width: int = Field(gt=0,le=12000)
    height: int = Field(gt=0,le=12000)
    media_type: str
    sha256: str = Field(pattern='^[a-f0-9]{64}$')
class NameplateResult(Contract):
    attachment_id: UUID
    model_text: str | None
    serial_text: str | None
    issues: list[Literal['unreadable','blur','low_resolution','glare','partial_plate']]
    confirmation_required: Literal[True] = True
class ConfirmExtraction(Contract):
    extraction_id: UUID
    expected_revision: int = Field(ge=1)
    model_text: str = Field(min_length=1,max_length=120)
    serial_text: str | None = Field(default=None,max_length=120)
class Allocation(Contract):
    warehouse_id: int = Field(gt=0)
    quantity: Decimal = Field(gt=0)
    transfer_days: int = Field(ge=0,le=60)
    stock_updated_at: datetime
class VerifiedItem(Contract):
    item_id: UUID
    configuration_id: int
    part_id: int
    part_number: str
    name_en: str
    name_ar: str
    requested_quantity: Decimal = Field(gt=0)
    allocations: list[Allocation]
    shortage: Decimal = Field(ge=0)
    source_reference: str
    evidence_id: UUID
class VerifiedFacts(Contract):
    enquiry_id: UUID
    revision: int
    catalogue_version_id: UUID
    items: list[VerifiedItem] = Field(min_length=1,max_length=20)
class CandidateEvidence(Contract):
    item_id: UUID
    part_id: int
    configuration_id: int
    evidence_id: UUID
    source_reference: str
class StockEvidence(Contract):
    part_id: int
    warehouse_id: int
    available: Decimal = Field(ge=0)
    updated_at: datetime
class VerificationEvidence(Contract):
    current_revision: int
    active_catalogue_version_id: UUID
    candidates: list[CandidateEvidence]
    stock: list[StockEvidence]
class VerificationFailure(Contract):
    code: Literal['STALE_REVISION','STALE_CATALOGUE','INCOMPATIBLE','MISSING_EVIDENCE','INVALID_QUANTIT
Y','OVERALLOCATION','STALE_STOCK','UNRESOLVED_ITEM','UNSAFE_PROSE']
    item_id: UUID | None
    detail: str
class VerificationResult(Contract):
    passed: bool
    failures: list[VerificationFailure]
    facts_sha256: str
class ReplySlots(Contract):
    greeting: str = Field(max_length=200)
    explanation: str = Field(max_length=1500)
    closing: str = Field(max_length=300)
class ApprovalRequest(Contract):
    enquiry_id: UUID
    expected_revision: int = Field(ge=1)
    draft_id: UUID
    edited_slots: ReplySlots | None = None
class RejectionRequest(Contract):
    enquiry_id: UUID
    expected_revision: int = Field(ge=1)
    reason: str = Field(min_length=1,max_length=1000)
class ApprovalResult(Contract):
    decision_id: UUID
    message_id: UUID
    revision: int
    already_applied: bool
class ReviewResult(Contract):
    decision_id: UUID
    revision: int
    status: Literal['approved','rejected']
class ItemCorrection(Contract):
    item_id: UUID
    description: str | None = None
    quantity: Decimal | None = Field(default=None,gt=0,le=100000)
    model_text: str | None = None
    serial_text: str | None = None
class RevisionRequest(Contract):
```

```
    enquiry_id: UUID
    expected_revision: int
    corrections: list[ItemCorrection] = Field(min_length=1,max_length=20)
    reason: str = Field(min_length=1,max_length=1000)
class JobLease(Contract):
    id: UUID
    kind: Literal['nameplate','export_pdf','catalog_import','catalog_publish','storage_cleanup','workfl
ow','analytics_sync','catalog_sync']
    enquiry_id: UUID | None
    revision: int | None
    session_generation: int | None
    lease_token: UUID
    expires_at: datetime
    attempt: int
    payload: dict[str,str | int | bool | None]
class JobResult(Contract):
    status: Literal['succeeded','failed','cancelled']
    result_id: UUID | None
    error_code: str | None
class ExportRequest(Contract):
    enquiry_id: UUID
    revision: int
    decision_id: UUID
class ExportResult(Contract):
    export_id: UUID
    storage_key: str
    sha256: str
class ApprovedSnapshot(Contract):
    decision_id: UUID
    approved_at: datetime
    locale: Locale
    facts: VerifiedFacts
    reply: str
    synthetic: bool
class AuditEvent(Contract):
    actor_type: Literal['system','ai','user']
    actor_id: str
    action: str
    entity_type: str
    entity_id: str
    request_id: UUID
    enquiry_id: UUID | None
    revision: int | None
    payload: dict[str,str | int | bool | None]
class AIRequest(Contract):
    request_key: UUID
    enquiry_id: UUID | None
    stage: Literal['intake','nameplate','parts','reply','summary']
    model: str
    trusted_instructions: str
    untrusted_text: str
    image_base64: str | None = None
    output_schema_name:
Literal['IntakeResult','NameplateResult','PartsProposal','ReplySlots','SummaryResult']
    max_output_tokens: int = Field(ge=1,le=2048)
class AIResult(Contract):
    output_json: str
    input_tokens: int
    output_tokens: int
    provider_request_id: str
class BudgetReservation(Contract):
    id: UUID
    account_id: str
    reserved_usd: Decimal
class CreditAllowance(Contract):
    amount_usd: Decimal = Field(ge=0)
    expires_at: datetime
    paid_budget_usd: Decimal = Field(ge=0,le=0)
class ItemState(Contract):
    id: UUID
    request: ItemInput
    configuration_id: int | None
    candidate_part_ids: list[int]
    selected_part_id: int | None
    status: Literal['unresolved','needs_information','ready','unavailable']
class EnquiryState(Contract):
    enquiry_id: UUID
    revision: int
    session_generation: int | None
    catalogue_version_id: UUID | None
    items: list[ItemState]
    step_count: int = Field(ge=0)
    pending_question: str | None
    draft_id: UUID | None
    verification_id: UUID | None
    phase: str
class SearchContext(Contract):
    enquiry_id: UUID
    revision: int
    catalogue_version_id: UUID
    item_id: UUID
```

```
    max_calls: int = Field(ge=0,le=4)
class PartsProposal(Contract):
    explanation: str
    clarification_code: Literal['needs_serial','needs_model','ambiguous_part','mismatch','none']
    candidate_part_ids: list[int]
class SummaryResult(Contract):
    summary: str = Field(max_length=2000)
    source_message_ids: list[UUID]
class MetricsQuery(Contract):
    start: datetime
    end: datetime
    include_demo: bool = False
class MetricsView(Contract):
    enquiry_count: int
    approved_count: int
    clarification_count: int
    evaluated_count: int
    correct_count: int
    median_review_seconds: float | None
    p95_latency_ms: float | None
    estimated_cost_usd: Decimal
class ReviewView(Contract):
    enquiry: EnquiryView
    facts: VerifiedFacts | None
    draft: ReplySlots | None
    verification: VerificationResult | None
    assigned_to: UUID | None
class DemoScenario(Contract):
    id: Literal['mixed_photo','missing_serial','incompatible_number','local_shortage']
    text: str
    attachment_path: str | None
    expected_status: str
    expected_configuration_id: int | None
    expected_part_ids: list[int]
class WorkflowStep(StrEnum):
    INTAKE='intake'
    AWAIT_OCR='await_ocr'
    CONFIRM='confirm'
    IDENTIFY='identify'
    CLARIFY='clarify'
    SEARCH='search'
    CHOOSE='choose'
    PLAN='plan'
    DRAFT='draft'
    VERIFY='verify'
    REVIEW='review'
    COMPLETE='complete'
    NEEDS_STAFF='needs_staff'
class EmbeddingReceipt(Contract):
    version_id: UUID
    embedded_count: int
    model_revision: str
class EventOutcome(Contract):
    event_id: UUID
    state: Literal['processed','obsolete','retry','failed']
class EvalCase(Contract):
    id: str
    scenario: str
    input_text: str
    expected_status: str
    expected_part_ids: list[int]
class EvalOutput(Contract):
    status: str
    part_ids: list[int]
    clarification_code: str | None
class EvalResult(Contract):
    case_id: str
    passed: bool
    actual: EvalOutput
class EvalRun(Contract):
    id: UUID
    dataset_sha256: str
    results: list[EvalResult]
class SchemaReceipt(Contract):
    postgres_version: str
    extensions: dict[str,str]
    columns: dict[str,list[str]]
    indexes: dict[str,list[str]]
    policies: dict[str,list[str]]
class TestConfig(Contract):
    app_env: Literal['local','test']
    database_url: str
    production_project_ref: str
class ReadinessReport(Contract):
    database: bool
    redis: bool
    search_lexical: bool
    search_semantic: bool
    ai_credit_enabled: bool
    storage: bool
class AccountReceipt(Contract):
```

```
    account_id: str
    plan: Literal['free']
    region: str
    credit_usd: Decimal
    expires_at: datetime
class SizingReceipt(Contract):
    instance_type: str
    peak_memory_mib: int
    instance_memory_mib: int
    measured_at: datetime
class DeploymentDecision(Contract):
    allowed: bool
    reasons: list[str]
class HostedReceipt(Contract):
    base_url: str
    api_url: str
    scenarios_passed: list[str]
    scenarios_failed: list[str]
    checked_at: datetime
```

137 api models

spec/api_models.py

```
"""REST contract specification; copy to partsiq.core.api_models during foundation01."""
from datetime import datetime
from decimal import Decimal
from typing import Literal
from uuid import UUID
from pydantic import BaseModel,Field,ConfigDict
try:
    from .contracts import *
except ImportError:
    from contracts import *

class RevisionOnly(Contract):
    expected_revision:int = Field(ge=1)

class PageQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)

class HistoryQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    status:str | None = None

class InboxQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    status:str | None = None
    priority:Literal['normal','urgent'] | None = None
    overdue:bool = False
    query:str | None = Field(default=None,max_length=200)

class ScenarioList(Contract):
    items:list[DemoScenario]

class ExtractionView(Contract):
    revision:int
    intake:IntakeResult | None
    nameplate:NameplateResult | None
    confirmed:bool

class MessagePage(Contract):
    items:list[MessageView]
    next_cursor:str | None

class EnquiryPage(Contract):
    items:list[EnquiryView]
    next_cursor:str | None

class QualityView(Contract):
    attachment_id:UUID
    status:Literal['pending','valid','rejected']
    width:int | None
    height:int | None
    warnings:list[str]
    error_code:str | None

class ItemsEdit(Contract):
    expected_revision:int = Field(ge=1)
    items:list[ItemCorrection] = Field(min_length=1,max_length=20)

class ProgressView(Contract):
    enquiry_id:UUID
    revision:int
    step:WorkflowStep
    wait_reason:str | None
    updated_at:datetime

class MachineView(Contract):
    item_id:UUID
    revision:int
    state:Literal['resolved','needs_serial','ambiguous','mismatch','not_found']
    configuration_id:int | None
    model_text:str | None
    serial_text:str | None
    catalogue_version_id:UUID

class MismatchView(Contract):
    item_id:UUID
    revision:int
    mismatch:bool
    reason_code:str | None
    question_en:str | None
    question_ar:str | None

class CompatibilityView(Contract):
    item_id:UUID
```

```
    revision:int
    configuration_id:int
    part_ids:list[int]
    catalogue_version_id:UUID

class MatchEvidenceView(Contract):
    item_id:UUID
    revision:int
    normalized_query:str
    matched_aliases:list[str]
    removed_machine_terms:list[str]

class SearchCapabilities(Contract):
    lexical:bool
    semantic:bool
    embedding_model:str | None
    fallback_reason:str | None
    catalogue_source:Literal['snowflake','fixture']
    catalogue_version_id:UUID | None

class CandidateView(Contract):
    part_id:int
    part_number:str
    name_en:str
    name_ar:str
    score:float = Field(ge=0,le=1)
    reasons:list[str]
    source_reference:str
    catalogue_version_id:UUID

class CandidateList(Contract):
    item_id:UUID
    revision:int
    items:list[CandidateView]
    ambiguous:bool
    margin:float | None

class CandidateChoice(Contract):
    expected_revision:int = Field(ge=1)
    part_id:int = Field(gt=0)

class ExplanationView(Contract):
    item_id:UUID
    revision:int
    reasons:list[str]
    source_reference:str | None
    catalogue_version_id:UUID

class ReferenceView(Contract):
    evidence_id:UUID
    source_reference:str
    page_number:int
    synthetic:bool

class ReferenceList(Contract):
    items:list[ReferenceView]
    revision:int

class SupersessionView(Contract):
    requested_number:str
    chain:list[str]
    terminal_number:str | None
    active:bool

class ComparisonQuery(Contract):
    part_ids:list[int] = Field(min_length=2,max_length=3)

class StockRow(Contract):
    warehouse_id:int
    warehouse_name:str
    available:Decimal
    observed_at:datetime

class StockView(Contract):
    item_id:UUID
    revision:int
    part_id:int
    items:list[StockRow]
    snapshot_id:UUID | None
    stale:bool

class PlanView(Contract):
    enquiry_id:UUID
    revision:int
    items:list[VerifiedItem]
    stock_snapshot_id:UUID | None

class TransferRow(Contract):
    source_warehouse_id:int
    destination_warehouse_id:int
    transfer_days:int | None
```

```
class TransferView(Contract):
    revision:int
    routes:list[TransferRow]
    estimate_only:Literal[True] = True

class FreshnessView(Contract):
    revision:int
    fresh:bool
    observed_at:datetime | None
    max_age_seconds:int
    reason_code:str | None

class ApprovalState(Contract):
    revision:int
    approved:bool
    decision_id:UUID | None
    facts_sha256:str | None
    invalidated_reason:str | None

class NoteCreate(Contract):
    body:str = Field(min_length=1,max_length=4000)

class NoteView(Contract):
    id:UUID
    author_id:UUID
    body:str
    created_at:datetime

class NotePage(Contract):
    items:list[NoteView]
    next_cursor:str | None

class AssignRequest(Contract):
    expected_revision:int = Field(ge=1)
    assignee_id:UUID | None

class AssignmentView(Contract):
    enquiry_id:UUID
    assigned_to:UUID | None
    changed_at:datetime

class DeadlineView(Contract):
    due_at:datetime | None
    overdue:bool
    server_time:datetime

class SummaryView(Contract):
    revision:int
    summary:SummaryResult | None
    stale:bool

class DraftEdit(Contract):
    expected_revision:int = Field(ge=1)
    slots:ReplySlots

class DraftView(Contract):
    revision:int
    slots:ReplySlots
    rendered_preview:str
    verification_passed:bool

class PublishedView(Contract):
    message:MessageView | None
    decision_id:UUID | None
    approved_revision:int | None

class AuditPage(Contract):
    items:list[AuditEvent]
    next_cursor:str | None

class WorkflowView(Contract):
    enquiry_id:UUID
    revision:int
    step:WorkflowStep
    completed_steps:list[str]
    pending_job_id:UUID | None
    error_code:str | None

class JobView(Contract):
    id:UUID
    status:Literal['pending','running','succeeded','failed','cancelled']
    kind:str
    attempts:int
    error_code:str | None
    resource_id:UUID | None

class RetryRequest(Contract):
    expected_revision:int = Field(ge=1)

class IdentityView(Contract):
```

```
    id:UUID
    roles:list[str]
    is_anonymous:bool
    demo_session_id:UUID | None

class MachineSave(Contract):
    label:str = Field(min_length=1,max_length=120)
    model_text:str = Field(min_length=1,max_length=120)
    serial_text:str = Field(min_length=1,max_length=120)

class SavedMachineView(Contract):
    id:UUID
    label:str
    model_text:str
    serial_text:str
    created_at:datetime

class MachinePage(Contract):
    items:list[SavedMachineView]
    next_cursor:str | None

class ArchiveResult(Contract):
    id:UUID
    archived:Literal[True] = True

class RepeatRequest(Contract):
    locale:Locale
    destination_warehouse_id:int = Field(gt=0)

class CatalogueQuery(Contract):
    cursor:str | None = None
    limit:int = Field(default=25,ge=1,le=100)
    query:str | None = Field(default=None,max_length=200)
    model_id:int | None = None
    category:str | None = None

class CataloguePart(Contract):
    id:int
    part_number:str
    name_en:str
    name_ar:str
    category:str
    unit_of_measure:str
    is_active:bool
    catalogue_version_id:UUID

class CataloguePage(Contract):
    items:list[CataloguePart]
    next_cursor:str | None

class ImportCreate(Contract):
    filename:str = Field(min_length=1,max_length=160)
    format:Literal['csv','xlsx']
    size_bytes:int = Field(gt=0,le=10485760)

class ImportUpload(Contract):
    media_type:Literal['text/csv','application/vnd.openxmlformats-officedocument.spreadsheetml.sheet']

class ImportIssue(Contract):
    row_number:int
    field:str
    code:str
    message:str

class ImportView(Contract):
    id:UUID
    state:str
    row_count:int
    issues:list[ImportIssue]
    version_id:UUID | None
    content_sha256:str | None

class PublishRequest(Contract):
    content_sha256:str = Field(pattern='^[a-f0-9]{64}$')
    expected_active_version_id:UUID | None

class VersionView(Contract):
    id:UUID
    label:str
    state:str
    content_sha256:str
    published_at:datetime | None
    source:Literal['snowflake','fixture']

class VersionList(Contract):
    items:list[VersionView]

class MissingUpdate(Contract):
    expected_revision:int = Field(ge=1)
    state:Literal['investigating','resolved','unavailable']
    note:str = Field(min_length=1,max_length=2000)
```

```
class MissingView(Contract):
    item_id:UUID
    enquiry_id:UUID
    state:str
    note:str | None
    updated_at:datetime

class MissingPage(Contract):
    items:list[MissingView]
    next_cursor:str | None

class EvalRequest(Contract):
    suite:Literal['deterministic','live']
    include_demo:bool = True

class EvaluationView(Contract):
    run_id:UUID
    state:str
    total:int
    passed:int
    failed:int
    results:list[EvalResult]

class AllowanceView(Contract):
    enabled:bool
    confirmed_usd:Decimal
    consumed_usd:Decimal
    reserved_usd:Decimal
    expires_at:datetime | None
    model:str | None

class AllowanceSet(Contract):
    confirmed_usd:Decimal = Field(ge=0)
    expires_at:datetime
    model:str = Field(min_length=1,max_length=120)
    input_usd_per_million:Decimal = Field(ge=0)
    output_usd_per_million:Decimal = Field(ge=0)

class CurateRequest(Contract):
    case_id:str = Field(min_length=1,max_length=80)
    anonymized_text:str = Field(min_length=1,max_length=12000)
    expected_status:str
    expected_part_ids:list[int]

class CuratedView(Contract):
    case_id:str
    source_correction_id:UUID
    approved_by:UUID
    created_at:datetime

class TraceStage(Contract):
    stage:str
    started_at:datetime
    duration_ms:float | None
    status:str
    error_code:str | None

class TraceView(Contract):
    enquiry_id:UUID
    revision:int
    stages:list[TraceStage]

class AttachmentList(Contract):
    items:list[AttachmentView]

class AnalyticsSyncView(Contract):
    last_success_at:datetime | None
    pending_events:int
    source:Literal['snowflake','fixture']
    stale:bool

class Problem(Contract):
    type:str
    title:str
    status:int
    code:str
    detail:str
    request_id:UUID
    retryable:bool

class CatalogueRecord(Contract):
    entity_type: str
    entity_key: str
    payload: dict[str, object]
class CatalogueSnapshot(Contract):
    version_id: UUID
    source: Literal['snowflake','fixture']
    content_sha256: str
    records: list[CatalogueRecord]
class ValidationIssue(Contract):
```

```
    entity_type: str
    entity_key: str
    field: str
    code: str
    message: str
class SyncReceipt(Contract):
    version_id: UUID
    state: Literal['requested','remote_written','validated','activated','failed']
    record_count: int
    content_sha256: str
    query_id: str | None
class StepOutcome(Contract):
    step: WorkflowStep
    reason_code: str | None
    data: dict[str, object]
class ResolutionOutcome(Contract):
    items: list[MachineView]
    next_step: WorkflowStep
class ExtractionContext(Contract):
    enquiry_id: UUID
    revision: int
    messages: list[MessageView]
    confirmed_model: str | None
    confirmed_serial: str | None
class TestReceipt(Contract):
    name: str
    passed: bool
    command: str
    evidence_path: str
class GateResult(Contract):
    passed: bool
    blocked: list[str]
class AnalyticsSyncReceipt(Contract):
    run_id: UUID
    row_count: int
    query_id: str | None
    succeeded: bool

class FeatureContext(Contract):
    request_id: UUID
    actor: ActorContext
    enquiry_id: UUID | None
    revision: int | None
class FeatureOutcome(Contract):
    passed: bool
    code: str
    details: list[str]
```